

# 프로젝트로 배우는 Git & GitHub 협업

---

## 목차

---

### 1장. Git 시작하기: 버전관리의 이유와 설치·초기 설정 [입문]

- 도입: '과제\_최종\_진짜최종'의 비극과 버전관리의 필요성
- Git이란 무엇인가 — 분산 버전관리와 스냅샷
- Git과 GitHub는 어떻게 다른가
- 설치와 확인: `git --version`
- worked example: `git config`로 이름·이메일·main 설정하기
- 설정 범위: `system·global·local`과 `--show-origin`으로 진단
- 흔한 문제·해결: 'Author identity unknown'과 `master/main` 혼선
- 즉시 연습(2종 1세트): 내 환경 설정·확인 채우기
- 오개념 교정: Git=GitHub 오해·`config`는 계정이 아니다

### 2장. 저장소 만들기: `git init`과 상태 읽기(`status`) [입문]

- 도입: 평범한 폴더를 저장소로 바꾸기
- `git init`과 숨은 `.git` 폴더의 정체
- worked example: 빈 프로젝트에서 `init` → `status` 읽기
- `status` 출력 해부: On branch·No commits yet·Untracked files
- `git status -s` 짧은 형식과 상태 기호
- 흔한 문제·해결: 'not a git repository'와 엉뚱한 폴더 `init`
- 즉시 연습(2종 1세트): 저장소 만들고 상태 진단 채우기
- 오개념 교정: `init`은 저장이 아니다·`status`는 안전한 읽기

### 3장. 변경 기록하기: `add·commit`과 3영역, `.gitignore` [입문]

- 도입: 변경은 어떻게 '기록'이 되는가
- 동작 원리: 작업트리 → 스테이징 → 저장소 3영역
- `git add`: 이번 커밋에 담을 것 고르기(., -p)
- worked example: 샘플 페이지 첫 커밋 만들기
- 좋은 커밋 메시지와 커밋 단위, `--amend`로 직전 수정

- .gitignore로 로그·env·빌드 산출물 제외하기
- 흔한 문제·해결: 비밀키 커밋 → git rm --cached로 빼내기
- 즉시 연습(2종 1세트): add-commit-gitignore 채우기
- 오개념 교정: add 안 한 변경은 빠진다·이미 추적된 파일과 gitignore
- 단원 미니 프로젝트: 내 첫 Git 저장소(2종 1세트)

#### 4장. 이력 읽기: git log와 git diff로 진단하기 [기초]

- 도입: 이력을 읽는 힘이 모든 복구의 출발점
- git log 기본과 --oneline---graph---all
- 커밋 해시(SHA)·HEAD·부모 커밋 이해하기
- worked example: log로 이력 훑고 show로 한 커밋 들여다보기
- git diff(작업트리 vs 스테이징)와 --staged
- 범위 비교: git diff / main..feature
- 흔한 문제·해결: 고쳤는데 diff에 안 보임·log 페이지저 멈춤
- 즉시 연습(2종 1세트): log-diff로 진단 채우기
- 오개념 교정: diff 기본 대상·페이지 q·해시는 지문

#### 5장. 분기 만들기: git branch와 git switch [기초]

- 도입: main을 지키며 따로 실험하기
- 동작 원리: 브랜치=포인터, HEAD=현재 브랜치
- git branch: 생성·목록·삭제(-d)
- worked example: switch -c로 기능 브랜치 파고 돌아오기
- git switch와 옛 checkout -b의 대응(두 역할 분리)
- 기능 브랜치 워크플로우와 이름 짓기 관례
- 흔한 문제·해결: 미커밋 변경으로 전환 거부 → 커밋/stash
- 즉시 연습(2종 1세트): 브랜치 생성·전환·비교 채우기
- 오개념 교정: 폴더가 복사되지 않는다·branch는 이동 아님
- 단원 미니 프로젝트: 기능 브랜치로 실험하기(2종 1세트)

#### 6장. 갈래 합치기: git merge(fast-forward vs 3-way) [기초]

- 도입: 따로 만든 기능을 본류로 합치기
- 동작 원리: fast-forward vs 3-way merge
- 공통 조상(merge base)과 두 부모를 가진 병합 커밋
- worked example: main으로 switch 후 merge feature
- git merge --no-ff와 이력 가독성
- 병합 후 정리: git branch -d, merge --abort

- 흔한 문제·해결: 병합 방향 혼동·사라진 변경 진단
- 즉시 연습(2종 1세트): ff-3-way 재현 채우기
- 오개념 교정: 합치는 방향·ff와 이력·병합은 둘 다 살림

## 7장. 충돌 해결: merge conflict 진단부터 마무리까지 [기초]

- 도입: 충돌은 실패가 아니라 정상 과정
- 동작 원리: 충돌 표시 <<<<<< ===== >>>>>> 읽기
- git status로 Unmerged paths 진단하기
- worked example: 같은 줄 충돌을 편집·add·commit으로 해결
- git merge --abort로 충돌 전으로 안전 복귀
- 충돌을 줄이는 습관(작은 커밋·작은 통합)
- 흔한 문제·해결: 표시 잔존 커밋·어느 쪽 선택 막힘
- 즉시 연습(2종 1세트): 충돌 최종본 직접 작성
- 오개념 교정: 충돌은 정상·표시 제거 필수·add로 해결 표시
- 단원 미니 프로젝트: 두 기능 병합·충돌 해결(2종 1세트)

## 8장. 되돌리기: restore·reset·revert 구분과 reflog 안전망 [중급]

- 도입: 실수는 반드시 일어난다 — 복구라는 자신감
- git restore: 작업트리·스테이징 되돌리기(--staged)
- 동작 원리: reset --soft/--mixed/--hard와 3영역
- worked example: 잘못 add·잘못 커밋을 restore·reset으로 복구
- git revert: 공유 이력을 안전하게 되돌리는 새 커밋
- reset vs revert: 비공유 정리 vs 공유 취소
- worked example: reset --hard 사고를 git reflog로 복구
- 흔한 문제·해결: 미커밋 변경 소실·공유 이력 어긋남
- 즉시 연습(2종 1세트): 상황별 복구 명령 고르기
- 오개념 교정: hard 위험·공유엔 revert·restore≠reset

## 9장. 임시 저장: git stash로 하던 일 미뤄 두기 [중급]

- 도입: 커밋하기 애매한 변경, 잠시 치워 두기
- 동작 원리: 스테시 선반(스택) 구조
- git stash로 치우고 작업트리 정리하기
- worked example: 작업 중 핫픽스 전환 → 돌아와 pop
- stash list·pop·apply·drop과 -u 옵션
- 흔한 문제·해결: pop 충돌·untracked 미포함
- 즉시 연습(2종 1세트): 핫픽스 전환 흐름 채우기

- 오개념 교정: stash는 커밋 아님·pop≠apply·-u 필요
- 단원 미니 프로젝트: 잘못된 커밋 복구 훈련소(2종 1세트)

## 10장. 원격 저장소: git remote와 git clone으로 GitHub 잇기 [실무]

- 도입: 혼자 쓰던 Git을 공유의 세계로
- 동작 원리: 원격(origin) ↔ 로컬 저장소
- git clone으로 이력재 복제하기
- worked example: GitHub 빈 저장소 만들고 remote add
- git remote -v와 HTTPS·SSH 인증 개요
- 흔한 문제·해결: 원격 주소 오타 → remote set-url
- 즉시 연습(2종 1세트): 원격 연결·확인 채우기
- 오개념 교정: clone은 다운로드 이상·origin은 관례·자동 업로드 아님

## 11장. 주고받기: git push·pull·fetch로 동기화하기 [실무]

- 도입: 올리고 받고 합치는 일상 동기화
- 동작 원리: push·fetch·pull과 origin/main 추적 브랜치
- git push와 -u origin (upstream)
- worked example: 첫 push와 이후 간단 동기화
- git fetch vs git pull(=fetch+merge), --rebase 개요
- 흔한 문제·해결: non-fast-forward 거부 → pull 후 push
- 즉시 연습(2종 1세트): push·pull 동기화 채우기
- 오개념 교정: pull은 합치기·거부는 신호·fetch는 안전

## 12장. 태그와 릴리스: git tag로 버전 표시하기 [실무]

- 도입: '이 지점이 배포 버전'을 표시하기
- 동작 원리: 커밋에 붙는 고정 북마크 태그
- lightweight vs annotated(-a, -m) 태그
- worked example: v1.0.0 태그 달고 show로 확인·push
- 유의적 버전(SemVer)과 GitHub Releases 개요
- 흔한 문제·해결: 태그가 원격에 안 보임 → push --tags
- 즉시 연습(2종 1세트): 태그·릴리스 채우기
- 오개념 교정: 태그 자동 전송 아님·고정 북마크·릴리스 층 구분
- 단원 미니 프로젝트: GitHub에 올리고 v1.0.0 릴리스(2종 1세트)

## 13장. GitHub 협업: PR·코드 리뷰·이슈와 gh CLI [실무]

- 도입: 변경을 제안하고 함께 검토하는 관문, PR

- 동작 원리: 브랜치 push → PR → 리뷰 → 머지 흐름
- 이슈로 할 일·버그 추적과 할당·라벨
- worked example: 이슈 → 브랜치 → gh pr create → 머지
- 코드 리뷰: 라인 코멘트·Approve·Request changes
- 반려 후 같은 브랜치 수정 push로 PR 갱신, 머지 방식 개요
- 흔한 문제·해결: PR 충돌로 머지 막힘 → 로컬 병합·해결·push
- 즉시 연습(2종 1세트): PR 한 바퀴 채우기
- 오개념 교정: PR은 GitHub 기능·반려는 과정·머지 후 정리

#### 14장. 이력 정리: git rebase와 황금률 [실무]

- 도입: 어수선한 커밋을 한 줄로 정리하기
- 동작 원리: merge vs rebase(병합 커밋 vs 직선 이력)
- git rebase main으로 기능 브랜치 최신화
- worked example: rebase -i로 wip·오타 커밋 squash
- rebase 충돌과 --continue / --abort
- 황금률: 공유한 이력은 rebase-force push 금물
- 흔한 문제·해결: 공유 브랜치 rebase 후 push 거부
- 즉시 연습(2종 1세트): 커밋 정리 채우기
- 오개념 교정: 공유 이력 금지·rebase가 만능 아님·충돌 후 continue
- 단원 미니 프로젝트: PR 하나를 끝까지(2종 1세트)

#### 15장. 종합 프로젝트: 팀 협업 시뮬레이션 — 분석(요구사항·범위) [실무]

- 드라이빙 퀘스천과 POV 다시 보기
- 기능 요구사항(FR): 협업·충돌·사고 복구·릴리스
- 비기능 요구사항(NFR): 결정론적 재현·황금률·진단 단계
- 협업 사용자 스토리와 유스케이스
- 범위 In/Out 확정(Actions는 다음 학습)
- 추적성: 요구사항 ↔ 단원 개념 매핑
- 작성형 연습(2종 1세트): 요구사항 표 채우기·자가체크

#### 16장. 종합 프로젝트: 팀 협업 시뮬레이션 — 설계(저장소·워크플로우·ADR) [실무]

- 설계 개요: 무엇을 어떻게 협업으로 나눌까
- 저장소 모델: 단일 origin + 두 로컬 클론
- 브랜치 전략과 협업 워크플로우 분해
- 협업 정책 결정 기록: ADR 5건
- 사고 복구 분기도(revert-reflog·충돌)

- 작성형 연습(2종 1세트): ADR 작성·자가체크

## 17장. 종합 프로젝트: 팀 협업 시뮬레이션 — 구현(협업·충돌·사고·릴리스) [실무]

- 구현 전략과 마일스톤 개요
- M1 협업 환경 구성(원격·두 클론·브랜치 전략)
- M2 정상 협업 한 바퀴(이슈→PR→리뷰→머지)
- M3 동시 수정 충돌 발생·진단·해결·검증
- M4 사고 복구: 잘못된 머지 revert·force push reflog
- M5 핫픽스·rebase 정리·v1.0.0 릴리스
- 통합 시나리오 실행과 결과 확인
- 작성형 연습(2종 1세트): 사고 복구 단계 채우기·자가체크

## 18장. 종합 프로젝트: 팀 협업 시뮬레이션 — 발표·평가·성찰 [실무]

- 산출물 시연: 협업 이력·머지된 PR·릴리스 데모
  - 평가 루브릭
  - 동료 피드백(갤러리 워크)
  - 성찰 회고: 명령 ↔ 사고 복구의 연결
  - 작성형 연습(2종 1세트): 성찰 템플릿·자가체크
-



# 1장 Git 시작하기: 버전관리의 이유와 설치·초기 설정

## 이 장의 학습 목표

- 버전관리가 해결하는 문제와 분산 버전관리(distributed version control)로서의 Git 개념을 설명할 수 있습니다.
- Git을 설치하고 `git --version` 으로 설치를 확인할 수 있습니다.
- `git config --global` 로 `user.name` · `user.email` 과 기본 브랜치 이름(main)을 설정할 수 있습니다.
- `--global` 과 `--local` 설정 범위(scope)의 차이를 구분하고 `--show-origin` 으로 진단할 수 있습니다.

이 장은 책 전체의 토대입니다. 여기서 다루는 개념은 **c01 Git 소개와 설치·초기 설정**입니다. 선수 개념은 없으며, 컴퓨터에서 폴더와 파일을 다뤄 본 경험만 있으면 충분합니다. 이 책의 모든 실습은 작은 웹 프로젝트(간단한 HTML/JS To-Do 페이지) 저장소를 소재로 삼아, 마지막 종합 프로젝트인 "팀 협업 To-Do 웹앱"으로 자연스럽게 이어집니다.

## 1.1 도입: '과제\_최종\_진짜최종'의 비극과 버전관리의 필요성

발표 자료나 코드를 만들어 본 사람이라면 누구나 다음과 같은 폴더를 가지고 있습니다.

```
todo_app/  
todo_app_최종/  
todo_app_최종_2/  
todo_app_진짜최종/  
todo_app_진짜최종_제출본/  
todo_app_진짜최종_제출본_수정.zip
```



이 방식은 처음에는 안전해 보이지만 곧 무너집니다. 다음과 같은 질문에 답할 수 없기 때문입니다.

- "어제 잘 되던 코드가 오늘 안 됩니다. 어제와 무엇이 달라졌습니까?"
- "진짜최종 과 진짜최종\_2 중 어느 것이 실제로 제출한 버전입니까?"
- "이 한 줄을 누가, 언제, 왜 바꿨습니까?"
- "둘이서 같은 파일을 동시에 고쳤는데, 둘의 변경을 어떻게 합칩니까?"

폴더 복사본은 **변경의 이유도, 변경의 순서도, 변경한 사람도** 기록하지 못합니다. 용량은 폭발하고, 어느 것이 진짜인지 알 수 없게 됩니다. 이 문제를 정면으로 푸는 도구가 바로 **버전관리 시스템(version control system, VCS)**입니다.

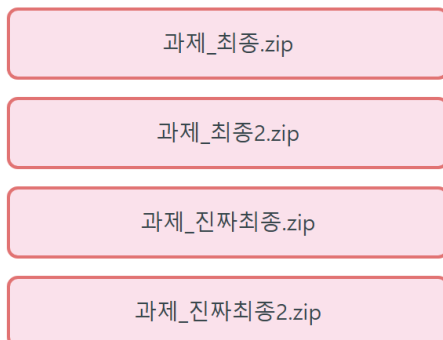
버전관리 시스템은 파일의 변경을 시간 순서대로 기록하고, 각 변경에 "누가·언제·무엇을·왜"라는 라벨을 붙입니다. 그 결과 우리는 폴더를 복사하지 않고도 과거의 어느 시점으로든 돌아갈 수 있고, 두 사람의 변경을 안전하게 합칠 수 있으며, "어제와 무엇이 달라졌는지"를 한 줄 단위로 비교할 수 있게 됩니다.

**핵심 한 줄.** 버전관리는 "폴더를 복사해서 보관하는 일"을 자동화·체계화한 것이 아니라, **변경의 역사 그 자체를 기록하는 일**입니다. 우리는 더 이상 \_진짜최종\_을 만들지 않습니다.

## 1.2 Git이란 무엇인가 — 분산 버전관리와 스냅샷

### 버전관리 방식 비교

#### 폴더 복사 방식 (혼란)



어느 파일이 최신인지 알 수 없음  
변경 이력 추적 불가

#### Git 스냅샷 방식 (명확)



각 시점 변경이 스냅샷으로 기록  
언제든지 과거 시점으로 복원 가능

분산(distributed): 모든 이력이 내 PC에도 완전히 존재

## 도입·필요성

버전관리의 필요성을 느꼈다면, 이제 그 일을 실제로 해 주는 도구가 필요합니다. 오늘날 사실상의 표준은 **Git**입니다. 리눅스 커널 개발을 위해 2005년에 만들어졌고, 지금은 거의 모든 소프트웨어 프로젝트가 Git을 사용합니다.

## 정의

**Git(깃)**은 파일의 변경 이력을 **스냅샷(snapshot)**으로 기록하는 **분산 버전관리 시스템(distributed version control system, DVCS)**입니다.

용어를 풀어 봅니다.

- **분산(distributed)**: 변경 이력 전체가 중앙 서버 한 곳에만 있는 것이 아니라, 작업하는 모든 사람의 컴퓨터에 **완전한 복사본**으로 들어 있습니다. 인터넷이 끊겨도 내 컴퓨터에서 이력을 보고 새 버전을 기록할 수 있습니다.
- **스냅샷(snapshot)**: Git은 변경을 저장할 때 "무엇이 바뀌었는지(차이)"만 저장하는 것이 아니라, 그 순간 프로젝트 전체의 **사진**을 찍어 둡니다. 이 한 장의 사진을 **커밋(commit)**이라고 부르며, 커밋들이 시간 순서로 한 줄로 이어져 프로젝트의 역사가 됩니다.

## 동작 원리

폴더 복사 방식과 Git 방식을 비교하면 동작 원리가 분명해집니다.

| 비교 항목     | 폴더 복사본 방식         | Git(스냅샷) 방식         |
|-----------|-------------------|---------------------|
| 버전 저장 단위  | 폴더 전체를 통째로 복사     | 변경 시점마다 커밋(스냅샷) 하나  |
| 변경 이유 기록  | 없음(폴더 이름에 의존)     | 커밋 메시지로 "왜"를 기록     |
| 누가 바꿨는가   | 알 수 없음            | 커밋마다 작성자(author) 기록 |
| 과거로 되돌리기  | 옛 폴더를 찾아 수동 교체    | 명령 한 줄로 특정 커밋으로 이동  |
| 동시 작업 합치기 | 수동으로 비교·복사(사고 잦음) | 브랜치·병합으로 안전하게 통합    |
| 용량        | 복사할수록 선형 증가       | 변경분만 효율적으로 보관       |

Git은 우리가 작업하는 폴더 안에 `.git` 이라는 숨은 폴더를 하나 만들고, 그 안에 모든 스냅샷(커밋)과 이력을 차곡차곡 쌓습니다. 우리가 보는 파일은 "지금 펼쳐 둔 한 장의 사진"이고, 과거의

모든 사진은 `.git` 안에 안전하게 보관됩니다. 이 구조 덕분에 우리는 언제든지 과거 사진을 꺼내 볼 수 있습니다.

**비유.** Git을 "프로젝트 전용 타임머신"이라고 생각하면 좋습니다. 커밋은 타임머신이 기억하는 한 시점이고, 우리는 그 어느 시점으로든 안전하게 다녀올 수 있습니다.

## 1.3 Git과 GitHub는 어떻게 다른가

입문자가 가장 많이 헷갈리는 부분입니다. **Git과 GitHub는 같은 것이 아닙니다.**

- **Git은 내 컴퓨터에서 도는 프로그램**입니다. 인터넷 없이도 동작하는 버전관리 도구입니다.
- **GitHub(깃허브)**는 **Git 저장소를 인터넷에 올려 두는 웹 호스팅 서비스**입니다. 비슷한 서비스로 GitLab, Bitbucket 등이 있습니다.

비유하자면 Git은 "사진을 찍고 정리하는 카메라·앨범 도구"이고, GitHub는 "그 앨범을 올려 두고 친구와 함께 보는 클라우드 서비스"입니다. 카메라(Git)는 클라우드(GitHub) 없이도 사진을 찍을 수 있습니다. 우리는 이 책의 1장부터 9장까지는 주로 내 컴퓨터의 Git만으로 학습하고, 10장부터 GitHub와 연결합니다.

| 구분     | Git          | GitHub              |
|--------|--------------|---------------------|
| 정체     | 버전관리 프로그램    | 웹 호스팅 서비스           |
| 실행 위치  | 내 컴퓨터(로컬)    | 인터넷 서버(원격)          |
| 인터넷 필요 | 불필요          | 필요                  |
| 제공자    | 오픈소스(여러 기여자) | Microsoft 소유 회사     |
| 대체재    | (사실상 표준)     | GitLab, Bitbucket 등 |

## 1.4 설치와 확인: `git --version`

### 도입

Git을 쓰려면 먼저 내 컴퓨터에 Git을 설치해야 합니다. 이 책은 **Windows 11** 환경을 기준으로 설명하지만, 설치 후의 명령은 운영체제와 무관하게 동일합니다.

## Windows에 Git 설치하기

1. 웹 브라우저에서 공식 사이트 `https://git-scm.com` 으로 이동합니다.
2. "Download for Windows"를 눌러 설치 파일을 받습니다.
3. 설치 마법사를 진행합니다. 대부분 기본값을 그대로 두면 되며, 이때 함께 설치되는 **Git Bash**라는 터미널을 이 책의 실습에서 사용합니다.

설치가 끝나면 Windows에는 두 가지 방법으로 Git 명령을 쓸 수 있습니다.

- **Git Bash**: 유닉스 계열 명령(`ls`, `pwd` 등)을 함께 쓸 수 있는 터미널. 이 책의 권장 환경입니다.
- **PowerShell** 또는 명령 프롬프트: Windows 기본 터미널. `git` 명령 자체는 동일하게 동작합니다.

## 설치 확인: `git --version`

설치가 제대로 됐는지 확인하는 명령은 다음과 같습니다.

```
git --version
```

정상적으로 설치됐다면 다음과 비슷한 출력이 나옵니다(숫자는 설치한 버전에 따라 다릅니다).

```
$ git --version
git version 2.45.2.windows.1
```

이 출력을 읽는 법은 다음과 같습니다.

- `git version` 다음의 숫자가 설치된 Git의 버전입니다. 이 책은 **2.45 이상**을 기준으로 하며, 2.23 이후 버전이면 이 책의 모든 명령(특히 `git switch`, `git restore`)을 그대로 쓸 수 있습니다.
- `.windows.1`은 Windows용 배포판이라는 표시입니다. macOS나 Linux에서는 이 접미사가 없습니다.

만약 `git: command not found` 나 '`git`'은(는) ... 인식할 수 없습니다 라는 메시지가 나오면, Git이 설치되지 않았거나 설치 후 터미널을 다시 열지 않은 것입니다. 터미널을 닫았다가 새로 열고 다시 시도합니다.

## 1.5 worked example: git config로 이름·이메일·main 설정하기

이제 첫 worked example입니다. Git을 설치한 직후, 가장 먼저 해야 할 일은 **나를 소개하는 설정**입니다. Git은 커밋(스냅샷)을 만들 때마다 "누가 만들었는가"를 기록하는데, 그 이름과 이메일을 한 번 알려 줘야 하기 때문입니다.

### 설정을 안 하면 어떻게 되는가

설정 없이 커밋을 시도하면 Git은 다음과 같이 거부하거나 경고합니다(이 메시지는 자주 만나므로 1.7에서 다시 다룹니다).

```
$ git commit -m "first commit"
Author identity unknown

*** Please tell me who you are.

Run

    git config --global user.email "you@example.com"
    git config --global user.name "Your Name"

to set your account's default identity.

fatal: unable to auto-detect email address
```

Git이 친절하게 해결 명령까지 알려 줍니다. 그대로 따라 하면 됩니다.

### worked example: 이름·이메일·기본 브랜치 설정하기

```
git config --global user.name "Hong Gildong"
git config --global user.email "gildong@example.com"
git config --global init.defaultBranch main
git config --list
```

한 줄씩 읽기.

- 1번째 줄 `git config --global user.name "Hong Gildong"` — 앞으로 만들 모든 커밋에 박힐 **작성자 이름**을 설정합니다. `--global`은 "이 컴퓨터의 모든 저장소에 적용"이라는 뜻입니다(범위는 1.6에서 자세히 다룹니다). 이름에 공백이 있으므로 큰따옴표로 감쌌습니다.

- 2번째 줄 `git config --global user.email "gildong@example.com"` — 커밋에 박힐 **작성자 이메일**을 설정합니다. 이 값은 GitHub에서 커밋을 내 계정과 연결하는 데에도 쓰이므로, 나중에 GitHub에 가입할 이메일과 맞춰 두면 좋습니다.
- 3번째 줄 `git config --global init.defaultBranch main` — 앞으로 `git init`으로 저장소를 만들 때 **기본 브랜치 이름을 main**으로 정합니다. 과거에는 기본 이름이 `master` 였으나, 현재 GitHub를 비롯한 업계 표준은 `main` 입니다. 이 한 줄로 `master/main` 혼선을 예방합니다.
- 4번째 줄 `git config --list` — 지금까지 설정된 값을 모두 출력해 **확인**합니다.

## 실행 결과

`git config --list`의 출력은 다음과 같습니다(환경에 따라 줄 수는 다릅니다).

```
$ git config --list
user.name=Hong Gildong
user.email=gildong@example.com
init.defaultbranch=main
core.autocrlf=true
```

이 출력을 읽는 법은 다음과 같습니다.

- `key=value` 형태로, 왼쪽이 설정 항목, 오른쪽이 값입니다.
- `user.name` 과 `user.email` 이 방금 설정한 값으로 보이면 성공입니다.
- `init.defaultbranch` 가 소문자로 보이는 것은 Git이 설정 키를 대소문자 구분 없이 다루기 때문이며 정상입니다.
- `core.autocrlf=true` 는 Windows에서 설치 시 자동으로 잡히는 줄바꿈 처리 설정입니다 (Windows의 CRLF 줄바꿈을 저장소에서는 LF로 다루도록 변환). 지금은 신경 쓰지 않아도 됩니다.

이제 이 컴퓨터의 Git은 "내가 누구인지"를 알게 됐고, 새 저장소의 기본 브랜치가 `main`이 되도록 준비됐습니다.

## 1.6 설정 범위: `system·global·local`과 `--show-origin`으로 진단

`git config`에는 설정이 적용되는 **범위(scope)**가 세 단계 있습니다. 이 구조를 이해하면 "회사 저장소에서는 회사 이메일, 개인 저장소에서는 개인 이메일"처럼 저장소별로 다른 설정을 줄 수

있습니다.

| 범위     | 옵션                    | 적용 대상          | 저장 위치(개념)                         | 우선순<br>위  |
|--------|-----------------------|----------------|-----------------------------------|-----------|
| system | <code>--system</code> | 컴퓨터의 모든 사용자    | Git 설치 폴더의 config                 | 가장 낮<br>음 |
| global | <code>--global</code> | 현재 사용자의 모든 저장소 | 사용자 홈 폴더의 <code>.gitconfig</code> | 중간        |
| local  | <code>--local</code>  | 지금 이 저장소 하나    | 저장소 안 <code>.git/config</code>    | 가장 높<br>음 |

핵심 규칙은 "좁은 범위가 넓은 범위를 덮어쓴다"입니다. 즉 같은 항목이 여러 범위에 설정돼 있으면 local이 global을, global이 system을 이깁니다. 그래서 평소에는 `--global` 로 한 번 설정해 두고, 특정 저장소에서만 다르게 하고 싶을 때 그 저장소 안에서 `--local` 로 덮어씁니다.

## 저장소별로 이메일을 다르게 설정하기

```
git config --local user.email "gildong@company.com"
```

이 명령은 **현재 폴더가 Git 저장소일 때만** 동작하며(2장에서 만드는 `.git/config` 에 기록), 그 저장소의 커밋에는 회사 이메일이 박힙니다. 다른 저장소에는 영향을 주지 않습니다.

## --show-origin으로 "이 값이 어디서 왔는가" 진단하기

설정이 꼬여서 "왜 이 이메일이 박히지?" 싶을 때, 값의 **출처 파일**을 함께 보여 주는 진단 옵션이 `--show-origin` 입니다.

```
git config --list --show-origin
```

## 실행 결과

```
$ git config --list --show-origin
file:C:/Program Files/Git/etc/gitconfig core.autocrlf=true
file:C:/Users/gildong/.gitconfig user.name=Hong Gildong
file:C:/Users/gildong/.gitconfig user.email=gildong@example.com
file:.git/config user.email=gildong@company.com
```

이 출력을 읽는 법은 다음과 같습니다.

- 각 줄 앞의 `file:...` 이 그 설정이 어느 파일에서 왔는지를 알려 줍니다.
- 홈 폴더의 `.gitconfig` 는 `--global` 설정, `.git/config` 는 `--local` 설정입니다.
- `user.email` 이 두 번 보입니다. 위는 `global`(개인), 아래는 `local`(회사)입니다. 우선순위 규칙에 따라 이 저장소에서는 아래쪽 `local` 값(회사 이메일)이 실제로 적용됩니다.

이처럼 `--show-origin` 은 "지금 적용되는 값이 어느 파일에서 왔는가"를 눈으로 확인하게 해 주는 강력한 진단 도구입니다.

## 더 알아두면 좋은 핵심 설정: `core.editor`와 `core.autocrlf`

`user.name` · `user.email` · `init.defaultBranch` 다음으로 자주 만나는 설정 두 가지를 미리 알아 둡니다. 지금 당장 바꾸지 않아도 되지만, 나중에 "왜 이상한 편집기가 뜨지?", "왜 줄바꿈이 전부 바뀐 것처럼 보이지?" 같은 의문을 풀어 줍니다.

**core.editor** — 커밋 메시지를 길게 쓸 때 열리는 편집기. `git commit` 을 `-m` 없이 실행하면 Git 이 편집기를 띄워 메시지를 받습니다. 기본 편집기가 익숙지 않은 Vim이라 당황하는 입문자가 많습니다. VS Code를 쓴다면 다음처럼 바꿀 수 있습니다.

```
git config --global core.editor "code --wait"
```

`code --wait` 는 "VS Code 창이 닫힐 때까지 기다렸다가 그 내용을 커밋 메시지로 쓰라"는 뜻입니다.

**core.autocrlf** — 운영체제마다 다른 줄바꿈 처리. Windows는 줄바꿈을 CRLF로, macOS·Linux는 LF로 저장합니다. 협업 시 이 차이로 "한 줄도 안 고쳤는데 파일 전체가 바뀐 것처럼" 보일 수 있습니다. Windows용 Git은 설치 시 보통 `core.autocrlf=true` 로 잡아 두는데, 이는 "받을 때 LF, 저장할 때 CRLF로 변환"해 차이를 흡수한다는 뜻입니다. 다음으로 현재 값을 확인할 수 있습니다.

```
git config --show-origin core.autocrlf
```

## 실행 결과



```
$ git config --show-origin core.autocrlf
file:C:/Program Files/Git/etc/gitconfig      true
```

file:...gitconfig (system 범위)에서 true 로 왔음을 알 수 있습니다. Windows에서는 이 기본 값을 그대로 두는 것을 권장합니다.

## 1.7 흔한 문제·해결: 'Author identity unknown'과 master/main 혼선

이 절은 실제로 입문자가 가장 자주 막히는 두 상황을, 상황 → 명령 → 문제 발생 → 진단 → 해결 흐름으로 다룹니다.

### 문제 1: Author identity unknown

**상황.** Git을 막 설치하고 첫 커밋을 시도합니다.

```
git commit -m "프로젝트 시작"
```

**문제 발생.** 다음과 같이 커밋이 거부됩니다.

```
Author identity unknown

*** Please tell me who you are.

Run

  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"

fatal: unable to auto-detect email address (got 'gildong@DESKTOP.(none)')
```

**진단.** 작성자 정보(user.name, user.email)가 설정되지 않았다는 신호입니다. 다음 명령으로 누락을 확인합니다.

```
git config --list
```

출력에 user.name 과 user.email 줄이 보이지 않으면 설정이 빠진 것입니다.

**해결.** 1.5에서 배운 대로 두 값을 설정한 뒤 다시 커밋합니다.

```
git config --global user.name "Hong Gildong"
git config --global user.email "gildong@example.com"
git commit -m "프로젝트 시작"
```

이제 커밋이 정상적으로 만들어집니다.

## 문제 2: master/main 혼선

**상황.** 오래된 환경이나 기본 브랜치를 설정하지 않은 상태에서 `git init` 을 했습니다.

**문제 발생.** `git status` 를 보니 브랜치 이름이 `main` 이 아니라 `master` 로 나옵니다. 협업하는 GitHub 저장소는 `main` 을 쓰는데, 내 로컬은 `master` 라서 헷갈립니다.

```
$ git status
On branch master

No commits yet
```

**진단.** 기본 브랜치 이름이 어디서 왔는지 확인합니다.

```
git config --show-origin init.defaultBranch
```

아무 값도 안 나오면 기본값(`master`)이 쓰인 것이고, system이나 global에 `master` 로 설정돼 있을 수도 있습니다.

**해결.** 두 가지 방법이 있습니다.

1. 앞으로 만들 저장소를 위해 기본값을 `main`으로 바꿉니다(한 번만).

```
git config --global init.defaultBranch main
```

1. 이미 만든 저장소(아직 커밋 전)의 현재 브랜치 이름을 `main`으로 바꿉니다.

```
git branch -m master main
```

`git branch -m` 은 현재 브랜치의 이름을 바꾸는(move) 명령입니다. 이후 `git status` 를 다시 보면 `On branch main` 으로 바뀌어 있습니다.

**주의.** 기본 브랜치를 main으로 바꾸는 설정( `init.defaultBranch` )은 앞으로 만들 저장소에만 적용됩니다. 이미 만든 저장소의 이름까지 자동으로 바뀌지는 않으므로, 기존 저장소는 `git branch -m` 으로 따로 바꿔야 합니다.

## 1.8 즉시 연습(2종 1세트): 내 환경 설정·확인 채우기

배운 것을 직접 손으로 해 봅니다. 먼저 **모범 답안**을 보고, 그다음 **빈 템플릿**을 직접 채운 뒤 **자가체크**로 점검합니다.

### 모범 답안

새 컴퓨터에서 Git을 처음 설정하고 확인하는 표준 흐름입니다.

```
git --version
git config --global user.name "Hong Gildong"
git config --global user.email "gildong@example.com"
git config --global init.defaultBranch main
git config --list --show-origin
```

### 실행 결과(요약)

```
$ git --version
git version 2.45.2.windows.1
$ git config --list --show-origin
file:C:/Users/gildong/.gitconfig user.name=Hong Gildong
file:C:/Users/gildong/.gitconfig user.email=gildong@example.com
file:C:/Users/gildong/.gitconfig init.defaultbranch=main
```

### 직접 작성: 내 환경으로 채우기

아래 빈칸을 **여러분 자신의 이름·이메일**로 채워 실제 터미널에서 실행해 보십시오. 이메일은 나중에 GitHub에 가입할 이메일로 맞춰 두는 것을 권장합니다.

```
# 1) 설치 확인 — 버전 숫자가 나오는지 확인
git _____

# 2) 내 이름 설정(공백 있으면 큰따옴표)
git config --global user.name "_____"

# 3) 내 이메일 설정
git config --global user.email "_____"

# 4) 기본 브랜치를 main으로
git config --global init.defaultBranch _____

# 5) 설정과 출처를 함께 확인
git config --list _____
```

**도전 과제.** 이 책의 실습용 저장소(2장에서 만들 예정)에서만 회사용 이메일을 쓰고 싶다면, 어떤 옵션으로 어떻게 설정해야 할지 한 줄로 적어 보십시오.

## 자가체크 체크리스트

- ☐ `git --version` 이 2.23 이상의 버전 숫자를 출력합니까?
- ☐ `git config --list` 에 `user.name` 과 `user.email` 이 내 값으로 보입니까?
- ☐ `init.defaultbranch` 가 `main` 으로 설정돼 있습니까?
- ☐ `--show-origin` 으로 각 값이 `.gitconfig (global)`에서 왔음을 확인했습니까?
- ☐ 도전 과제의 답이 `git config --local user.email "..."` 형태입니까?

## 1.8+ 한 걸음 더: 저장소별 설정 분리 실습(2종 1세트)

1.6에서 설정 범위(system/global/local)를 배웠습니다. 이 개념을 실제로 손에 익히는 작은 실습을 하나 더 합니다. 회사 프로젝트와 개인 프로젝트에서 커밋 작성자 이메일을 다르게 두는 흔한 실무 상황을 재현합니다.

### 모범 답안

먼저 컴퓨터 전체 기본값(global)을 개인 이메일로 두고, 특정 저장소에서만 회사 이메일로 덮어 쓴 뒤, `--show-origin` 으로 실제 적용 값을 확인하는 흐름입니다.

```
git config --global user.email "gildong.personal@example.com"
git init company-todo
cd company-todo
git config --local user.email "gildong@company.com"
git config user.email
git config --show-origin user.email
```

### 한 줄씩 읽기.

- 1번째 줄 — 컴퓨터 전체 기본 이메일을 개인 이메일로 둡니다(global).
- 2~3번째 줄 — `company-todo` 라는 저장소를 만들고 그 안으로 들어갑니다(`git init <폴더명>` 은 그 폴더를 만들면서 저장소로 만듭니다).
- 4번째 줄 — **이 저장소에서만** 회사 이메일로 덮어씁니다(local).
- 5번째 줄 — 범위를 안 붙인 `git config user.email` 은 "지금 실제로 적용되는 값"을 보여줍니다.
- 6번째 줄 — 그 값이 어느 파일에서 왔는지(`.git/config` = local) 진단합니다.

### 실행 결과

```
$ git config user.email
gildong@company.com
$ git config --show-origin user.email
file:.git/config      gildong@company.com
```

이 저장소 안에서는 local(회사) 값이 global(개인) 값을 이깁니다. 좁은 범위가 우선한다는 1.6의 규칙을 눈으로 확인할 수 있습니다.

## 직접 작성: 내 저장소별 설정 분리하기

아래 빈칸을 채워 실제로 "개인 기본값 + 특정 저장소만 다른 값"을 만들어 보십시오.

```
# 1) 컴퓨터 전체 기본 이메일(개인)
git config --global user.email "내개인@example.com"

# 2) 실습 저장소 만들고 들어가기
git init scope-practice
cd scope-practice

# 3) 이 저장소에서만 다른 이메일로 덮어쓰기
git config --local user.email "이저장소용@example.com"

# 4) 지금 실제 적용되는 값 확인
git config --local user.email

# 5) 그 값이 어느 파일에서 왔는지 진단
git config --local user.email
```

## 자가체크 체크리스트

- ☐ 1번에 `global`, 3번에 `local` 을 넣었습니까?
- ☐ 4번의 `git config user.email` 결과가 3번에서 넣은 저장소용 값입니까?(`local`이 `global` 을 이김)
- ☐ 5번의 `--show-origin` 결과가 `.git/config` 를 가리킵니까?
- ☐ 저장소 바깥(다른 폴더)에서 `git config user.email` 을 치면 개인 기본값이 나오는지 확인했습니까?

## FAQ: 1장에서 자주 나오는 질문

**Q. `git config --global` 을 하면 비밀번호처럼 어딘가에 로그인되는 건가요?** 아닙니다.

`user.name` · `user.email` 은 커밋에 붙는 라벨일 뿐이고 로그인과 무관합니다. GitHub 로그인(인증)은 10장에서 따로 다룹니다.

**Q. 이름·이메일을 나중에 바꿀 수 있나요?** 네. 같은 명령(`git config --global user.email "새 값"`)을 다시 실행하면 값이 갱신됩니다. 다만 이미 만들어진 과거 커밋의 작성자 정보는 자동으로 바뀌지 않습니다.

**Q. `git --version` 이 2.20처럼 좀 낮게 나옵니다. 그대로 써도 되나요?** `git switch` · `git restore` 는 2.23부터 들어왔으므로, 2.23 미만이면 이 책의 일부 명령이 동작하지 않습니다. 공식 사이트에서 최신 버전으로 다시 설치하기를 권장합니다.

**Q. Windows에서 Git Bash와 PowerShell 중 무엇을 써야 하나요?** git 명령 자체는 둘 다 동일하게 동작합니다. 이 책은 유닉스 명령( `ls` · `pwd` )을 함께 쓰는 Git Bash를 권장하지만, PowerShell을 써도 학습에 지장이 없습니다.

## 1.9 오개념 교정: Git=GitHub 오해·config는 계정이 아니다

### 흔한 오개념과 바로잡기

**오개념 1: "Git이 곧 GitHub다."** 아닙니다. Git은 내 컴퓨터에서 도는 버전관리 프로그램이고, GitHub는 그 결과물을 올려 두는 웹 서비스입니다. 인터넷이 없어도 Git으로 커밋을 쌓을 수 있습니다. GitHub 계정이 없어도 1장부터 9장까지 학습에는 아무 문제가 없습니다.

**오개념 2: " `user.name` · `user.email` 은 로그인 계정이다."** 아닙니다. 이 값은 로그인에 쓰는 비밀번호 같은 것이 아니라, 커밋에 박히는 '작성자 라벨'일 뿐입니다. 그래서 누구나 임의의 이름·이메일을 적을 수 있습니다. 틀리게 적으면 이후 모든 커밋의 작성자 정보가 잘못 기록되므로, 처음에 정확히 설정하는 것이 중요합니다.

**오개념 3: " `--global` 설정은 모든 저장소에 강제로 똑같이 적용되어 바꿀 수 없다."** 아닙니다. `--global`은 "기본값"일 뿐이고, 특정 저장소에서 `--local`로 덮어쓰면 그 저장소에서는 local 값이 우선합니다(좁은 범위가 이깁니다). `--show-origin`으로 어느 값이 적용되는지 언제든지 진단할 수 있습니다.

**오개념 4: "기본 브랜치는 원래 master이고 main은 GitHub만의 것이다."** 이름은 단지 관례일 뿐입니다. 과거 기본값이 `master` 였으나 현재 표준은 `main`이며, `init.defaultBranch` 설정으로 내가 정합니다. 둘 다 그냥 첫 브랜치의 이름일 뿐, 기능적 차이는 없습니다.

## 한눈에 보는 1장 명령 요약

이 장에서 익힌 명령을 한 표로 모았습니다. 손에 익을 때까지 옆에 두고 참고하십시오.

| 명령                                                    | 하는 일              | 자주 쓰는 형태                                         |
|-------------------------------------------------------|-------------------|--------------------------------------------------|
| <code>git --version</code>                            | 설치·버전 확인          | <code>git --version</code>                       |
| <code>git config --global user.name</code>            | 작성자 이름 설정(컴퓨터 전체) | <code>git config --global user.name "이름"</code>  |
| <code>git config --global user.email</code>           | 작성자 이메일 설정        | <code>git config --global user.email "메일"</code> |
| <code>git config --global init.defaultBranch</code>   | 기본 브랜치 이름 설정      | <code>... init.defaultBranch main</code>         |
| <code>git config --local &lt;key&gt; &lt;값&gt;</code> | 이 저장소에서만 설정(덮어쓰기) | <code>git config --local user.email "..."</code> |
| <code>git config --list</code>                        | 설정 전체 보기          | <code>git config --list</code>                   |
| <code>git config --show-origin &lt;key&gt;</code>     | 설정 값의 출처 진단       | <code>git config --show-origin user.email</code> |
| <code>git branch -m &lt;옛이름&gt; &lt;새이름&gt;</code>    | 현재 브랜치 이름 변경      | <code>git branch -m master main</code>           |

설정의 우선순위는 **local > global > system**이며, 좁은 범위가 넓은 범위를 덮어씁니다. 어떤 값이 실제로 적용되는지 헛갈리면 언제나 `--show-origin`으로 출처를 확인하면 됩니다.

## 1장 정리

- 폴더 복사본(\_진짜최종)은 변경의 이유·순서·작성자를 기록하지 못합니다. 이를 푸는 것이 버전관리입니다.
- **Git**은 변경을 스냅샷(커밋)으로 기록하는 분산 버전관리 시스템입니다. **GitHub**는 그 저장소를 올려 두는 웹 서비스로, 둘은 다릅니다.
- 설치는 `git --version`으로 확인합니다(2.23 이상 권장).
- 가장 먼저 `git config --global`로 `user.name`·`user.email`·`init.defaultBranch main`을 설정합니다.



- 설정 범위는 system < global < local 순으로 좁은 범위가 우선하며, `--show-origin` 으로 출처를 진단합니다.
- 'Author identity unknown'은 작성자 미설정 신호이고, master/main 혼선은 `init.defaultBranch` 와 `git branch -m` 으로 해결합니다.

이제 우리 컴퓨터의 Git은 "내가 누구인지"를 알고, 새 저장소를 main 브랜치로 시작할 준비가 됐습니다. 설정은 한 번만 해 두면 이후 모든 저장소에 적용되므로, 새 컴퓨터를 쓸 때만 다시 하면 됩니다.

다음 장에서는 실제로 폴더 하나를 `git init` 으로 저장소로 바꾸고, `git status` 로 그 상태를 읽는 법을 배웁니다. 이 책의 소재인 작은 To-Do 웹 페이지 프로젝트를 직접 저장소로 만들어 보며, 본격적인 버전관리의 첫걸음을 뚝니다.



## 2장 저장소 만들기: git init과 상태 읽기 (status)

### 이 장의 학습 목표

- `git init` 으로 평범한 폴더를 저장소(repository)로 만들고 `.git` 디렉터리의 역할을 설명할 수 있습니다.
- `git status` 의 출력(현재 브랜치·커밋 없음·추적 안 됨 등)을 정확히 읽을 수 있습니다.
- `git status -s` (짧은 형식)의 상태 기호(??·M·A)를 해석할 수 있습니다.
- 추적 안 됨(untracked)과 추적됨(tracked)의 차이를 구분할 수 있습니다.

이 장에서 다루는 개념은 **c02 저장소 만들기**와 **상태 확인**입니다. 선수 개념은 **c01 Git 소개와 설치·초기 설정**입니다. 1장에서 `git config` 로 이름·이메일·기본 브랜치(main)를 설정했다는 것을 전제로 합니다. 아직 설정하지 않았다면 1장의 1.5절을 먼저 마치고 돌아오십시오. 이 장 역시 작은 To-Do 웹 페이지 프로젝트를 소재로 진행합니다.

### 2.1 도입: 평범한 폴더를 저장소로 바꾸기

1장에서 Git을 설치하고 나를 소개했습니다. 하지만 아직 Git은 **어떤 폴더도 추적하고 있지 않습니다**. Git은 우리가 명시적으로 "이 폴더를 추적해 줘"라고 말하기 전까지는 우리의 파일에 손대지 않습니다.

그 "추적해 줘"라는 한마디가 바로 `git init` 입니다. 이 명령 하나로 평범한 폴더가 변경 이력을 기록할 수 있는 **저장소(repository, 줄여서 repo)**로 바뀝니다. 이번 장의 실습 폴더는 작은 To-Do 웹 페이지 프로젝트가 될 것입니다.

생각해 보면 흐름은 단순합니다.

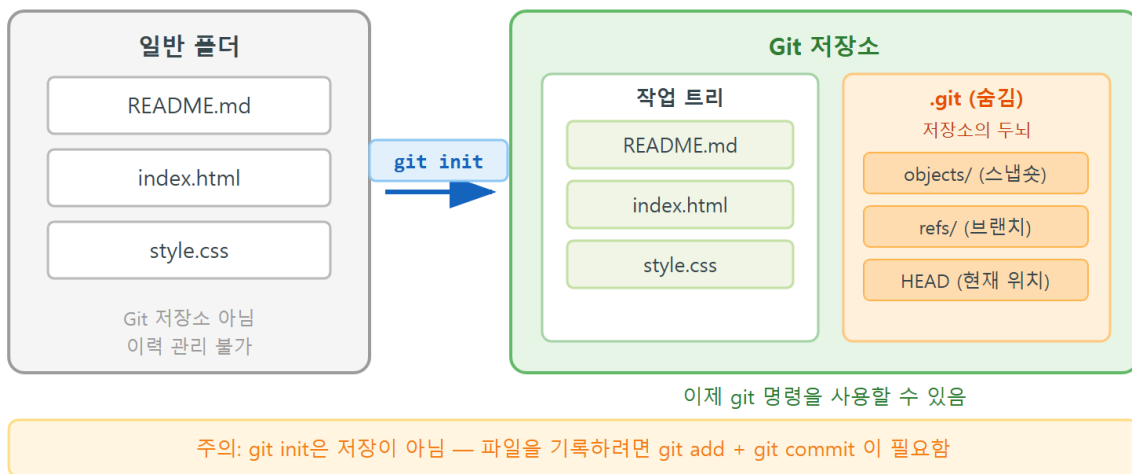
1. 작업할 폴더를 만듭니다.
2. 그 폴더에서 `git init` 을 실행해 저장소로 만듭니다.
3. `git status` 로 "지금 무슨 상태인가"를 확인하며 작업을 이어 갑니다.

이 장은 이 가운데 1~3을 다루고, 실제로 변경을 기록(add·commit)하는 일은 3장에서 다룹니다.

왜 `git status` 에 한 장의 절반을 쓸 만큼 공을 들일까요? Git을 쓰다 보면 충돌·되돌리기·병합처럼 복잡한 상황을 반드시 만나게 되는데, 그때마다 길을 알려 주는 나침반이 바로 `git status` 이기 때문입니다. 베테랑 개발자일수록 명령을 치기 전에 `git status` 로 "지금 내가 어디에 있고, 무엇이 기록 대기 중인가"를 먼저 확인합니다. 반대로 입문자가 사고를 내는 가장 흔한 원인은 상태를 확인하지 않고 명령부터 치는 것입니다. 그래서 이 장에서 status를 읽는 눈을 제대로 길러 두면, 앞으로 등장할 모든 장이 훨씬 수월해집니다. status는 외우는 명령이 아니라 **읽는 명령**이라는 점을 기억하십시오.

## 2.2 git init과 숨은 .git 폴더의 정체

### git init — 폴더에서 저장소로



### 도입·필요성

저장소를 만든다는 것이 정확히 무엇을 의미하는지 알아야, 나중에 "왜 이 폴더는 Git이 모를까", "저장소를 잘못 만들었는데 어떻게 되돌리지" 같은 문제를 스스로 진단할 수 있습니다.

### 정의

`git init` 은 현재 폴더 안에 숨은 `.git` 디렉터리를 만들어, 그 폴더를 Git이 추적하는 **저장소 (repository)**로 바꾸는 명령입니다.

## 동작 원리

`git init` 을 실행하면 정확히 다음 일이 일어납니다.

1. 현재 폴더 안에 `.git` 이라는 이름의 **숨은 폴더**가 하나 생깁니다(이름이 점으로 시작하면 숨김 처리됩니다).
2. 그 `.git` 안에 Git이 이력을 관리하는 데 필요한 모든 것(설정, 커밋 저장소, 브랜치 정보 등)이 들어갑니다.
3. 이 순간부터 그 폴더는 "저장소"가 되며, Git 명령을 그 안에서 쓸 수 있게 됩니다.

`.git` 폴더는 말하자면 **저장소의 두뇌**입니다. 우리가 만드는 모든 커밋(스냅샷)과 변경 이력이 전부 이 `.git` 안에 보관됩니다. 우리가 평소에 보는 파일들( `index.html` 등)은 "작업 중인 펼쳐둔 사본"이고, 진짜 역사는 `.git` 안에 있습니다.

| 구분                         | 일반 폴더                                    | <code>git init</code> 한 저장소 |
|----------------------------|------------------------------------------|-----------------------------|
| <code>.git</code> 폴더       | 없음                                       | 있음(숨김)                      |
| 변경 이력 기록                   | 불가능                                      | 가능(커밋으로)                    |
| <code>git status</code> 실행 | <code>fatal: not a git repository</code> | 정상 동작                       |
| 과거로 되돌리기                   | 불가능                                      | 가능                          |

**주의.** `.git` 폴더를 사람이 직접 열어 파일을 손으로 고치거나 지우면 저장소가 망가질 수 있습니다. 반대로, 저장소를 통째로 "그냥 폴더"로 되돌리고 싶다면 `.git` 폴더만 삭제하면 됩니다(이력은 사라지지만 작업 파일은 남습니다). 이 성질은 2.6의 "엉뚱한 폴더 init" 복구에서 다시 활용합니다.

## 2.3 worked example: 빈 프로젝트에서 init → status 읽기

첫 worked example입니다. To-Do 웹 페이지 프로젝트 폴더를 만들고, 저장소로 만든 뒤, 상태를 읽는 전체 흐름을 따라갑니다. (아래 `mkdir · cd` 는 운영체제 터미널 명령이며, Git Bash 기준입니다.)

```
mkdir todo-app
cd todo-app
git init
git status
```

### 한 줄씩 읽기.

- 1번째 줄 `mkdir todo-app` — `todo-app` 이라는 새 폴더를 만듭니다. 아직은 그냥 평범한 폴더입니다.
- 2번째 줄 `cd todo-app` — 그 폴더 안으로 이동합니다. **Git 명령은 반드시 저장소 폴더 안에서 실행해야** 하므로 이 이동이 중요합니다.
- 3번째 줄 `git init` — 현재 폴더(`todo-app`)를 저장소로 만듭니다. 이때 `.git` 이 생깁니다.
- 4번째 줄 `git status` — 저장소의 현재 상태를 읽습니다. 가장 자주 쓰게 될 명령입니다.

### 실행 결과

```
$ git init
Initialized empty Git repository in C:/Users/gildong/todo-app/.git/

$ git status
On branch main

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

이 출력을 읽는 법은 다음과 같습니다.

- `Initialized empty Git repository in .../todo-app/.git/` — 저장소가 만들어졌고, 그 두뇌인 `.git` 이 어느 경로에 생겼는지 알려 줍니다. `empty` 는 "아직 커밋이 하나도 없는 빈 저장소"라는 뜻입니다.
- `On branch main` — 현재 `main` 브랜치 위에 있습니다(1장에서 `init.defaultBranch main` 을 설정해 둔 덕분입니다). 설정을 안 했다면 `On branch master` 로 나옵니다.
- `No commits yet` — 아직 스냅샷(커밋)을 하나도 만들지 않았습니다.
- `nothing to commit` — 기록할 변경이 없습니다. 폴더가 비어 있으니 당연합니다.

## 파일을 하나 넣어 보기

이제 To-Do 페이지의 첫 파일을 만들고 상태가 어떻게 바뀌는지 봅시다. (`echo ... > index.html` 은 파일을 만드는 터미널 명령입니다.)

```
echo "<h1>My To-Do</h1>" > index.html
git status
```

## 실행 결과

```
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    index.html

nothing added to commit but untracked files present (use "git add" to track)
```

방금 만든 `index.html` 이 **Untracked files** 목록에 나타났습니다. 이것이 다음 절에서 자세히 해 부할 핵심입니다.

여기서 한 가지를 분명히 짚고 갑니다. 우리는 분명히 `git init` 을 했고 폴더 안에 파일도 생겼는데, Git은 그 파일을 "추적 안 됨"이라고 말합니다. 이는 오류가 아니라 Git의 **의도된 동작**입니다. Git은 사용자가 명시적으로 "이 파일을 이력에 넣어 줘"라고 지시하기 전까지는 어떤 파일도 멋대로 기록하지 않습니다. 그 지시가 바로 3장에서 배울 `git add` 이며, 지금은 `git status` 가 그 경계선("추적함" vs "추적 안 함")을 어떻게 보여 주는지를 정확히 읽는 데 집중합니다. 이 경계선을 또렷이 구분하는 능력이 이후 모든 장의 토대가 됩니다.

## 2.4 status 출력 해부: On branch·No commits yet·Untracked files

`git status` 는 이 책에서 가장 자주 쓰게 될 명령입니다. 무슨 일을 하든 헛갈리면 `git status` 부터 칩니다. 출력의 각 부분을 정확히 읽는 연습을 합니다.

방금 본 출력을 다시 보며 줄마다 해석합니다.

```

On branch main      <- (1) 지금 어느 브랜치에 있는가

No commits yet      <- (2) 아직 커밋이 없는가

Untracked files:    <- (3) Git이 아직 추적하지 않는 파일 목록
  (use "git add ...") <- 다음에 할 일 안내
    index.html      <- 추적 안 되는 파일 이름

```

- **(1) On branch main**: 현재 위치한 브랜치 이름입니다. 브랜치는 5장에서 본격적으로 배웁니다. 지금은 "기본 작업 줄"이라고만 알면 됩니다.
- **(2) No commits yet**: 이 저장소에 아직 스냅샷이 하나도 없다는 뜻입니다. 첫 커밋을 만들면 이 줄은 사라집니다.
- **(3) Untracked files**: Git이 존재는 하는데 아직 이력에 포함하지 않는 파일입니다. `git init` 직후에는 폴더 안 모든 파일이 untracked로 시작합니다(자동으로 추적되지 않습니다).

## 추적 안 됨(untracked) vs 추적됨(tracked)

이 구분이 2장의 가장 중요한 개념입니다.

| 상태                    | 의미                                              | status에 보이는 곳                                                 |
|-----------------------|-------------------------------------------------|---------------------------------------------------------------|
| 추적 안 됨<br>(untracked) | Git이 아직 이력에 넣지 않은 새 파일                          | Untracked files: 아래                                           |
| 추적됨(tracked)          | 한 번이라도 <code>git add</code> / 커밋되어 Git이 관리하는 파일 | Changes to be committed: 또는 Changes not staged for commit: 아래 |

`git status` 는 늘 "다음에 무엇을 하면 되는지"를 괄호 안내문으로 알려 줍니다. `use "git add <file>..."` 라는 안내는 "이 파일을 이력에 넣으려면 `git add` 를 쓰라"는 뜻입니다. 실제 `git add` 커밋은 3장에서 다룹니다.

한 가지 비유를 더 들면, `git status` 는 내비게이션의 "현재 위치" 화면과 같습니다. 목적지로 가는 길을 정하기 전에 먼저 "나는 지금 어디에 있는가"를 확인해야 하듯, Git에서도 다음 명령을 정하기 전에 status로 현재 위치(브랜치·커밋·파일 상태)를 확인합니다. 이 작은 습관 하나가 입문자와 숙련자를 가르는 가장 큰 차이입니다.

**핵심.** `git status` 를 읽는 능력이 모든 진단의 출발점입니다. "지금 어느 브랜치인가 / 커밋이 있는가 / 어떤 파일이 어느 상태인가" 세 가지를 항상 이 명령으로 확인하는 습관을 들입니다.



## .git 폴더 안에는 무엇이 있는가(살짝 들여다보기)

.git 을 사람이 직접 고칠 일은 없지만, 안에 무엇이 들어 있는지 한 번 보면 "저장소의 두뇌"라는 표현이 와닿습니다. Git Bash에서 `ls` 로 들여다볼 수 있습니다.

```
ls .git
```

### 실행 결과

```
$ ls .git
HEAD  config  description  hooks  info  objects  refs
```

각 항목을 가볍게 읽어 봅니다(외울 필요는 없습니다).

| 항목      | 역할(개념)                                             |
|---------|----------------------------------------------------|
| HEAD    | "지금 어느 브랜치에 있는가"를 가리키는 포인터(5장에서 자세히)               |
| config  | 이 저장소의 local 설정(1장의 <code>--local</code> 이 여기에 기록) |
| objects | 커밋·파일 스냅샷의 실제 데이터가 저장되는 곳                          |
| refs    | 브랜치·태그가 어느 커밋을 가리키는지 기록                            |
| hooks   | 특정 시점에 자동 실행할 스크립트(고급)                             |

지금 단계에서 기억할 것은 단 하나입니다. 우리가 만드는 모든 이력은 이 `.git` 안에 들어가고, `.git` 을 지우면 그 폴더는 다시 평범한 폴더로 돌아간다는 것입니다.

## 2.5 git status -s 짧은 형식과 상태 기호

`git status` 의 기본 출력은 친절하지만 길입니다. 파일이 많아지면 한눈에 보기 어렵습니다. 이때 짧은 형식(short format)인 `-s` (또는 `--short`) 를 씁니다.

```
git status -s
```

## 상태 기호 읽기

짧은 형식은 각 파일 앞에 **두 칸의 기호**로 상태를 압축해 보여 줍니다. 자주 만나는 기호는 다음과 같습니다.

| 기호 | 의미                 | 언제 보이는가               |
|----|--------------------|-----------------------|
| ?? | 추적 안 됨(untracked)  | 새로 만든, 아직 add 안 한 파일  |
| A  | 추가됨(added, staged) | git add 로 새로 스테이징한 파일 |
| M  | 수정됨(modified)      | 추적 중인 파일을 고친 경우       |
| D  | 삭제됨(deleted)       | 추적 중인 파일을 지운 경우       |

기호는 **두 자리**인데, 왼쪽 칸은 "스테이징 영역의 상태", 오른쪽 칸은 "작업트리의 상태"를 나타냅니다(스테이징은 3장에서 자세히 배웁니다). 지금은 대표적인 패턴만 익힙니다.

### 실행 결과

To-Do 프로젝트에 파일 몇 개가 섞여 있을 때의 예시입니다.

```
$ git status -s
A  index.html
?? style.css
M  README.md
```

이 출력을 읽는 법은 다음과 같습니다.

- **A index.html** — index.html 은 git add 로 스테이징되어 다음 커밋에 들어갈 준비가 됐습니다(왼쪽 칸 A).
- **?? style.css** — style.css 는 아직 Git이 추적하지 않는 새 파일입니다.
- **M README.md** — README.md 는 추적 중인 파일인데 작업트리에서 수정됐고, 아직 add 하지 않았습니다(오른쪽 칸 M, 왼쪽은 공백).

기본 형식과 짧은 형식은 같은 정보를 보여 줍니다. 처음에는 기본 형식으로 안내문까지 읽고, 익숙해지면 **-s** 로 빠르게 훑는 식으로 함께 씁니다.

## 상황별 빠른 참조: status 출력으로 무엇을 진단하는가

`git status` 를 읽으면 "지금 무엇을 해야 하는가"가 거의 자동으로 결정됩니다. 자주 만나는 출력과 그 의미·다음 행동을 표로 정리합니다. 이 표는 이 책 전체에서 두고두고 씁니다.

| status에 보이는 문구                        | 의미                     | 다음에 할 일                                  |
|---------------------------------------|------------------------|------------------------------------------|
| On branch main                        | 현재 main 브랜치에 있음        | (정상) 작업 계속                               |
| No commits yet                        | 아직 커밋이 하나도 없음          | 첫 파일을 만들고<br>add·commit(3장)              |
| Untracked files:                      | 추적 안 되는 새 파일이 있음       | 이력에 넣을 거면 <code>git add</code> (3장)      |
| nothing to commit, working tree clean | 모든 변경이 커밋됨             | (가장 깨끗한 상태) 안심하고<br>다음 작업                |
| Changes not staged for commit:        | 추적 파일을 고쳤지만<br>add 안 함 | 고친 내용을 <code>git add</code> (3장)         |
| fatal: not a git repository           | 여기는 저장소가 아님            | <code>pwd</code> 로 위치 확인 → 이동 또는<br>init |

특히 `working tree clean` (작업트리가 깨끗함)은 "기록되지 않은 변경이 하나도 없다"는 가장 안전한 상태입니다. 브랜치를 옮기거나(5장) 위험한 명령을 쓰기 전에는, 항상 `git status` 로 이 깨끗한 상태인지 먼저 확인하는 습관이 사고를 예방합니다.

## 추적 파일 목록 확인: `git ls-files`

"지금 이 저장소가 추적하고 있는 파일이 정확히 무엇인지"만 따로 보고 싶을 때가 있습니다(특히 3장의 `.gitignore` 를 다룰 때 유용합니다). `git ls-files` 가 추적 중인 파일만 나열합니다.

```
git ls-files
```

아직 아무것도 `add·commit` 하지 않았다면 이 명령은 **아무것도 출력하지 않습니다**(추적 중인 파일이 없으므로). 이는 2.8에서 다룰 오개념("init만 하면 자동 추적된다")을 명령으로 확인하는 좋은 방법입니다. 추적 안 됨(untracked) 파일은 `ls-files` 에 나타나지 않고, 오직 `git status` 의 `Untracked files` 에만 보입니다.

## 2.6 흔한 문제·해결: 'not a git repository'와 엉뚱한 폴더 init

이 절은 입문자가 자주 막히는 두 상황을 **상황** → **명령** → **문제 발생** → **진단** → **해결** 흐름으로 다룹니다.

### 문제 1: fatal: not a git repository

**상황.** Git 명령을 쳤는데 다음과 같은 오류가 납니다.

```
git status
```

**문제 발생.**

```
$ git status
fatal: not a git repository (or any of the parent directories): .git
```

**진단.** 이 메시지는 **현재 폴더(그리고 상위 폴더들)에 .git 이 없다** = "여기는 저장소가 아니다"라는 뜻입니다. 원인은 보통 둘 중 하나입니다.

1. `git init` 을 아직 안 했습니다.
2. 저장소 폴더가 아닌 **엉뚱한 위치**에서 명령을 쳤습니다.

지금 내가 어느 폴더에 있는지부터 확인합니다.

```
pwd
```

**해결.**

- 의도한 저장소 폴더가 따로 있다면 그곳으로 이동합니다: `cd todo-app`.
- 정말 이 폴더를 저장소로 만들 생각이었다면 여기서 `git init` 을 실행합니다.

이동 후 `git status` 가 `On branch main` 으로 정상 동작하면 해결된 것입니다.

### 문제 2: 엉뚱한 폴더에 init 해 버림

**상황.** 빠르게 작업하다가 실수로 **바탕화면이나 홈 폴더** 같은 넓은 위치에서 `git init` 을 해 버렸습니다.

**문제 발생.** `git status` 를 보니 바탕화면의 수많은 파일이 전부 `Untracked files` 로 잡힙니다.

```
$ git status
On branch main

No commits yet

Untracked files:
  Documents/
  Downloads/
  Pictures/
  ... (수십 개)
```

**진단.** `pwd` 로 현재 위치를 확인합니다. 경로가 `C:/Users/gildong (홈)`이나 바탕화면이면, 저장소를 만들 의도가 아니었던 넓은 폴더에 `.git` 을 만든 것입니다.

**해결.** 잘못 만든 `.git` 폴더만 지우면 그 폴더는 다시 평범한 폴더로 돌아갑니다(작업 파일은 그대로 남습니다).

```
rm -rf .git
```

**경고.** `rm -rf .git` 은 반드시 `pwd` 로 현재 위치가 그 잘못된 폴더가 맞는지 확인한 뒤에만 실행하십시오. `.git` 만 지우는 것이라 작업 파일은 남지만, 다른 경로의 다른 저장소에서 실수로 실행하면 그 저장소의 이력이 사라집니다. 명령을 치기 전 `pwd` 로 위치를 확인하는 습관이 안전장치입니다.

지운 뒤 `git status` 를 다시 치면 `fatal: not a git repository` 가 나오는데, 이는 "이제 다시 평범한 폴더가 됐다"는 정상 신호입니다.

## 2.7 즉시 연습(2종 1세트): 저장소 만들고 상태 진단 채우기

먼저 모범 답안을 보고, 빈 템플릿을 직접 채운 뒤 자가체크로 점검합니다.

### 모범 답안

새 To-Do 프로젝트 폴더를 저장소로 만들고 상태를 진단하는 표준 흐름입니다.

```
mkdir todo-practice
cd todo-practice
git init
git status
echo "<h1>To-Do</h1>" > index.html
git status -s
```

## 실행 결과(요약)

```
$ git init
Initialized empty Git repository in C:/Users/gildong/todo-practice/.git/
$ git status
On branch main

No commits yet

nothing to commit (create/copy files and use "git add" to track)
$ git status -s
?? index.html
```

마지막 `?? index.html` 은 "새로 만든 `index.html` 이 아직 추적 안 됨"이라는 뜻입니다.

## 직접 작성: 저장소 만들고 상태 읽기

아래 빈칸을 채워 실제 터미널에서 실행하고, 각 단계에서 `git status` 가 무엇이라 답하는지 직접 읽어 보십시오.

```
# 1) 작업 폴더 만들고 들어가기
mkdir my-todo
cd my-todo

# 2) 이 폴더를 저장소로 만들기
git _____

# 3) 현재 상태 읽기 — On branch는 무엇이라 나오는가?
git _____

# 4) 첫 파일 만들기
echo "<h1>My Todo</h1>" > index.html

# 5) 짧은 형식으로 상태 보기 — 기호는 무엇인가?
git status _____
```

진단 과제. 다음 짧은 형식 출력을 보고 각 파일의 상태를 한국어로 설명해 보십시오.

```
A app.js
?? style.css
M index.html
```

## 자가체크 체크리스트

- [ ] `git init` 후 `git status` 가 On branch main 을 보여 줍니까?(아니라면 1장의 `init.defaultBranch` 설정을 확인)
- [ ] 새로 만든 파일이 Untracked files 또는 짧은 형식 `??` 로 보입니까?
- [ ] 진단 과제에서 A (스테이징됨)· ?? (추적 안 됨)· M (수정됨, add 전)을 각각 맞게 설명했습니까?
- [ ] fatal: not a git repository 가 나면 가장 먼저 `pwd` 로 위치를 확인하겠다는 점을 이해했습니까?

## 2.7+ 한 걸음 더: status로 작업 흐름 따라 읽기(2종 1세트)

`git status`의 진짜 힘은 "작업이 진행되며 상태가 어떻게 변하는지"를 따라 읽을 때 드러납니다. To-Do 프로젝트에 파일을 하나씩 올려 가며 status가 무엇이냐 답하는지 추적해 봅니다. (스테이징·`git add`는 3장에서 자세히 다루지만, 여기서는 status 출력의 변화를 읽는 연습으로만 미리 봅니다.)

### 모범 답안

```
mkdir todo-trace
cd todo-trace
git init
echo "<h1>To-Do</h1>" > index.html
git status -s
echo "body { font-family: sans-serif; }" > style.css
git status -s
```

### 한 줄씩 읽기.

- 1~3번째 줄 — `todo-trace` 폴더를 만들어 저장소로 만듭니다.
- 4번째 줄 — 첫 파일 `index.html` 을 만듭니다.
- 5번째 줄 — 짧은 형식으로 상태를 봅니다. 새 파일이므로 `??` 로 보일 것입니다.

- 6번째 줄 — 두 번째 파일 `style.css` 를 만듭니다.
- 7번째 줄 — 다시 상태를 봅니다. 이제 추적 안 됨 파일이 둘이 되었습니다.

## 실행 결과

```
$ echo "<h1>To-Do</h1>" > index.html
$ git status -s
?? index.html
$ echo "body { font-family: sans-serif; }" > style.css
$ git status -s
?? index.html
?? style.css
```

파일이 늘 때마다 `??` 목록이 함께 늘어나는 것을 볼 수 있습니다. 이렇게 한 단계마다 `git status` 로 확인하는 습관이, 나중에 복잡한 작업에서도 길을 잃지 않게 해 줍니다.

## 기본 형식과 짧은 형식 함께 보기

같은 상태를 기본 형식으로 보면 다음과 같습니다.

```
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    index.html
    style.css

nothing added to commit but untracked files present (use "git add" to track)
```

기본 형식은 "다음에 무엇을 하라"는 안내(`use "git add ..."`)까지 보여 주므로, 처음에는 기본 형식으로 안내를 읽고, 익숙해지면 `-s` 로 빠르게 훑는 식으로 함께 쓰면 좋습니다.

## 직접 작성: 상태 변화를 추적하며 채우기

아래 빈칸을 채워 실행하고, **각 단계의 `git status` 출력을 직접 적어** 보십시오. 무엇이 추적 안 됨으로 잡히는지 눈으로 확인하는 것이 목표입니다.



```
# 1) 폴더 만들고 저장소로
mkdir todo-fill
cd todo-fill
git -----

# 2) 첫 파일 만들기
echo "<h1>Todo</h1>" > index.html

# 3) 짧은 형식으로 상태 보기 — 출력을 적어 보세요
git status ----
# 내가 본 출력: -----

# 4) 두 번째 파일(스크립트) 만들기
echo "console.log('todo');" > app.js

# 5) 다시 상태 보기 — 목록이 어떻게 늘었나요?
git status ----
# 내가 본 출력: -----
```

**진단 과제.** 다른 폴더로 `cd ..` 한 뒤 `git status` 를 쳤더니 `fatal: not a git repository` 가 나왔습니다. 무엇을 가장 먼저 확인해야 합니까? 한 줄로 답해 보십시오.

## 자가체크 체크리스트

- [ ] 3번 단계에서 `index.html` 이 ?? 로 보였습니까?
- [ ] 5번 단계에서 `index.html` 과 `app.js` 가 모두 ?? 로 보였습니까?
- [ ] `git status` 를 여러 번 쳐도 파일이 바뀌지 않는다는 것을 확인했습니까?(읽기 전용)
- [ ] 진단 과제의 답이 "`pwd` 로 현재 위치(저장소 폴더가 맞는지)를 확인한다"입니까?

## FAQ: 2장에서 자주 나오는 질문

**Q. `.git` 폴더가 안 보입니다. 제대로 만들어진 건가요?** `.git` 은 이름이 점으로 시작해 기본적으로 숨김 처리됩니다. 탐색기의 "숨긴 항목 표시"를 켜거나, Git Bash에서 `ls -a` 로 확인하면 보입니다. `git status` 가 정상 동작하면 저장소는 제대로 만들어진 것입니다.

**Q. 이미 저장소인 폴더에서 `git init` 을 또 하면 어떻게 되나요?** 기존 이력을 지우지 않고 `Reinitialized existing Git repository ...` 라고만 나옵니다. 보통은 다시 할 필요가 없으며, 실수로 해도 큰 문제는 없습니다.

**Q. `git status` 가 너무 자주 칠 명령 같은데 괜찮나요?** 오히려 권장합니다. `git status` 는 읽기 전용이라 아무리 자주 쳐도 안전하고, 헛갈릴 때 가장 먼저 쳐야 할 명령입니다.

**Q. 짧은 형식의 두 칸 기호가 헛갈립니다.** 지금은 대표 패턴만 기억하면 됩니다. ?? 는 추적 안 됨, A 는 스테이징된 새 파일, M (앞 공백+M)은 수정했지만 아직 add 안 함입니다. 왼쪽 칸=스테이징, 오른쪽 칸=작업트리라는 구조는 3장에서 add를 배운 뒤 더 분명해집니다.

## 2.8 오개념 교정: init은 저장이 아니다·status는 안전한 읽기

### 흔한 오개념과 바로잡기

**오개념 1:** "git init 을 하면 파일이 저장(백업)된다." 아닙니다. git init 은 저장소라는 빈 틀만 만들 뿐, 파일을 하나도 기록하지 않습니다. git status 가 No commits yet 이라고 알려 주는 것이 그 증거입니다. 실제 기록은 3장에서 배울 git add 와 git commit 을 해야 남습니다.

**오개념 2:** "폴더 안에 파일이 있으면 init 직후 자동으로 추적된다." 아닙니다. git init 직후 폴더 안 모든 파일은 추적 안 됨(untracked) 상태로 시작합니다. Git은 우리가 git add 로 명시적으로 지정하기 전까지는 어떤 파일도 이력에 넣지 않습니다.

**오개념 3:** "git status 를 자주 치면 파일이 바뀌거나 위험하다." 아닙니다. git status 는 읽기 전용 명령이라 파일이나 이력을 전혀 바꾸지 않습니다. 아무리 자주 실행해도 안전하므로, 헛갈릴 때마다 무조건 git status 부터 치는 것이 좋은 습관입니다.

**오개념 4:** ".git 폴더는 쓸데없으니 지워도 된다." .git 은 저장소의 두뇌로, 모든 커밋과 이력이 그 안에 있습니다. 의도적으로 "저장소를 평범한 폴더로 되돌릴 때"만 지웁니다(2.6 참고). 그 외에는 절대 손대지 않습니다.

## 한눈에 보는 2장 명령 요약

이 장에서 익힌 명령과 상태 신호를 한 표로 모았습니다.

| 명령                         | 하는 일                                   | 자주 쓰는 형태                                                   |
|----------------------------|----------------------------------------|------------------------------------------------------------|
| <code>git init</code>      | 현재 폴더를 저장소로 만들기( <code>.git</code> 생성) | <code>git init</code> 또는 <code>git init &lt;폴더명&gt;</code> |
| <code>git status</code>    | 브랜치·커밋 유무·파일 상태 진단(읽기 전용)              | <code>git status</code>                                    |
| <code>git status -s</code> | 짧은 형식으로 상태 한눈에 보기                      | <code>git status -s</code>                                 |
| <code>git ls-files</code>  | 추적 중인 파일만 나열                           | <code>git ls-files</code>                                  |
| <code>ls .git</code>       | 저장소 두뇌( <code>.git</code> ) 내부 들여다보기   | <code>ls .git</code>                                       |
| <code>rm -rf .git</code>   | 저장소를 평범한 폴더로 되돌리기(이력 삭제)               | (위치 확인 후) <code>rm -rf .git</code>                         |

상태 기호 빠른 참조입니다.

| 기호 | 의미                             |
|----|--------------------------------|
| ?? | 추적 안 됨(untracked) — 새 파일       |
| A  | 스테이징됨(added) — 다음 커밋에 들어갈 새 파일 |
| M  | 수정됨(modified) — 추적 파일을 고침      |
| D  | 삭제됨(deleted) — 추적 파일을 지움       |

무슨 작업을 하든 헛갈리면 **가장 먼저** `git status` 를 칩니다. 읽기 전용이라 안전하고, "지금 무엇을 해야 하는가"를 거의 자동으로 알려 주기 때문입니다.

## 2장 정리

- `git init` 은 폴더 안에 숨은 `.git` 을 만들어 그 폴더를 저장소로 바꿉니다. `.git` 은 모든 이력을 담는 저장소의 두뇌입니다.

- `git init` 은 저장소(Repository)가 아니라 빈 폴더를 만들기입니다. 직후 모든 파일은 추적 안 됨(untracked)으로 시작합니다.
- `git status` 는 현재 브랜치·커밋 유무·파일 상태를 알려 주는, 가장 자주 쓰는 읽기 전용 진단 명령입니다.
- `git status -s` 의 기호 `??` (추적 안 됨)·`A` (스테이징됨)·`M` (수정됨)을 읽을 수 있어야 합니다.
- `fatal: not a git repository` 는 "여기는 저장소가 아니다"라는 신호이며, `pwd` 로 위치를 확인하는 것이 진단의 첫걸음입니다.

이제 우리는 평범한 폴더를 저장소로 바꾸고, 그 안에서 무슨 일이 일어나는지를 `git status` 로 읽을 수 있게 됐습니다. 아직 아무것도 "기록"하지는 않았지만, 기록을 시작하기 위한 모든 준비가 끝났습니다. 추적 안 됨과 추적됨의 경계선을 또렷이 구분하는 눈을 갖춘 것이 이 장의 가장 큰 수확입니다.

다음 장에서는 추적 안 됨 상태의 파일을 `git add` 로 스테이징하고 `git commit` 으로 기록해, 드디어 첫 스냅샷(커밋)을 남깁니다. 작업트리·스테이징·저장소라는 3영역이 어떻게 연결되는지, 그리고 비밀키 같은 파일을 `.gitignore` 로 어떻게 추적에서 제외하는지까지 배우며 1단원(u1)을 마무리합니다.



# 변경 기록하기: add·commit과 3영역, .gitignore

## 학습 목표

- 작업트리·스테이징·저장소 3영역 사이를 변경이 어떻게 이동하는지 설명할 수 있습니다.
- `git add`로 이번 커밋에 담을 변경을 골라 스테이징하고 `git commit -m`으로 커밋할 수 있습니다.
- 좋은 커밋 메시지와 적절한 커밋 단위를 작성하고 `git commit --amend`로 직전 커밋을 손볼 수 있습니다.
- `.gitignore`로 추적 제외 규칙을 작성하고, 이미 추적 중인 파일을 `git rm --cached`로 빼낼 수 있습니다.

앞 장(ch\_02)에서 우리는 `git init`으로 평범한 폴더를 저장소로 바꾸고, `git status`로 "지금 어떤 파일이 추적 안 됨(untracked)·수정됨(modified) 상태인지"를 읽는 법을 익혔습니다. 그런데 거기까지는 아직 아무것도 **기록되지 않았습니다**. 저장소라는 빈 장부만 만들었을 뿐, 그 장부에 한 줄도 적지 않은 상태입니다. 이 장에서는 드디어 변경을 영구 기록으로 남기는 핵심 동작, `git add`와 `git commit`을 배웁니다.

선수 개념 상기: 앞 장의 `git status`를 떠올리십시오. status가 보여 주던 `Untracked files`, `Changes to be committed`, `Changes not staged for commit`이라는 세 묶음은 사실 이 장에서 배울 **3영역**(작업트리·스테이징·저장소)을 그대로 비추는 거울이었습다. status를 읽을 줄 알면, add·commit이 무엇을 어디로 옮기는지가 눈에 보입니다.

이 장의 개념을 커리큘럼에서는 **변경 기록하기 — add·commit과 3영역, .gitignore**라고 부릅니다. 이름 그대로, 변경이 기록으로 굳어지는 전 과정과 "기록하면 안 되는 것"을 걸러 내는 `.gitignore`까지를 한 묶음으로 다룹니다.

이 장은 Windows 11을 기준으로 설명합니다. 명령은 Git Bash 또는 PowerShell 어디서 입력해도 동일하게 동작합니다. 책에서는 프롬프트 기호를 `$`로 통일합니다 (PowerShell이라면 `PS C:\...>` 자리에 같은 명령을 입력하면 됩니다).

## 1. 도입: 변경은 어떻게 '기록'이 되는가

파일을 고쳐 저장(Ctrl+S)하면 디스크의 파일 내용은 분명히 바뀝니다. 하지만 Git의 입장에서 그것은 아직 "기록"이 아닙니다. 단지 작업 공간의 파일이 달라졌을 뿐입니다. Git에서 변경이 영구 이력으로 남으려면 두 번의 명시적인 손길이 필요합니다.

1. `git add` 로 "이번에 기록할 변경은 이것입니다"라고 **고릅니다**(스테이징).
2. `git commit` 으로 고른 변경을 하나의 **스냅샷**으로 묶어 저장소에 박습니다(커밋).

처음 Git을 만나면 "왜 한 번에 저장이 안 되고 두 단계나 거치지?"라는 의문이 듭니다. 다른 도구는 보통 저장 버튼 하나로 끝나기 때문입니다. 그러나 이 두 단계 분리가 Git의 강력함입니다. 한 번에 여러 파일을 고쳤더라도, **그중 일부만 골라** 의미 있는 단위로 끊어 커밋할 수 있습니다. 마치 사진을 찍기 전에 프레임에 무엇을 담을지 고르는 것과 같습니다. `git add` 는 프레임을 잡는 일, `git commit` 은 셔터를 누르는 일입니다.

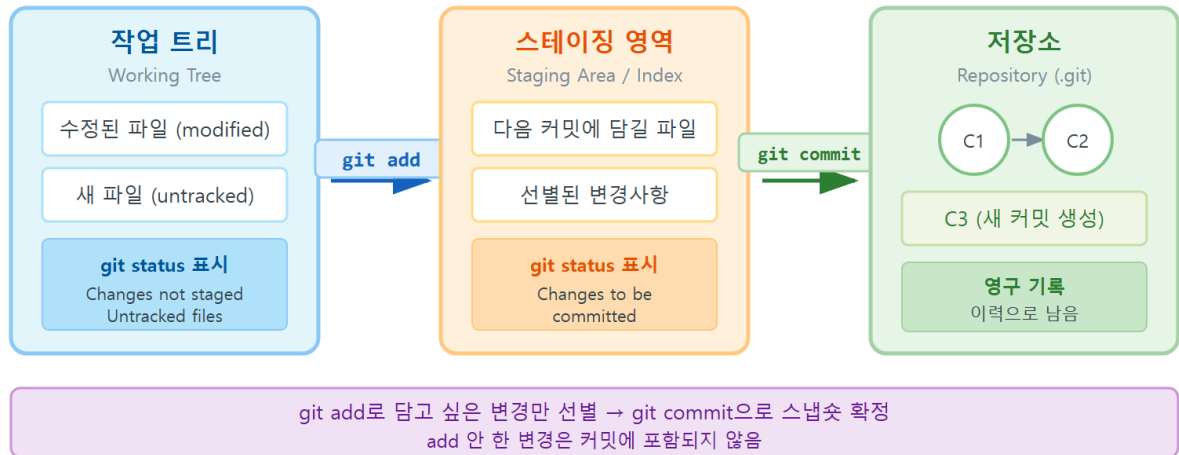
### 이 장에서 만나는 핵심 용어

- 작업트리(working tree): 지금 내가 파일을 편집하는 실제 작업 공간
- 스테이징 영역(staging area / index): 다음 커밋에 담을 변경을 모아 두는 대기실
- 저장소(repository): 커밋들이 영구히 쌓이는 이력 보관소( `.git` 안)
- 커밋(commit): 어느 시점의 변경을 묶어 메시지와 함께 박은 하나의 스냅샷
- `.gitignore`: 추적하지 않을 파일을 Git에게 알려 주는 목록 파일

## 개념: 변경 기록하기

### 2. 동작 원리: 작업트리 → 스테이징 → 저장소 3영역

## Git 3영역 흐름



Git이 변경을 기록하는 과정은 **세 개의 영역**을 거치는 여행으로 이해하면 가장 명확합니다.

| 영역      | 다른 이름                           | 역할                    | 어떻게 들어오나                |
|---------|---------------------------------|-----------------------|-------------------------|
| 작업트리    | working tree, working directory | 지금 파일을 편집하는 공간        | 파일을 만들거나 수정             |
| 스테이징 영역 | staging area, index             | 다음 커밋에 담을 변경을 모으는 대기실 | <code>git add</code>    |
| 저장소     | repository( .git )              | 커밋이 영구히 쌓이는 곳         | <code>git commit</code> |

변경은 항상 **왼쪽에서 오른쪽으로** 흐릅니다. 파일을 고치면 변경이 작업트리에 생기고, `git add`를 하면 그 변경이 스테이징 영역으로 올라가며, `git commit`을 하면 스테이징에 모인 것이 하나의 커밋으로 저장소에 박힙니다.

이 흐름을 `git status`의 출력과 짝지어 보면 머릿속에 확실히 자리잡습니다.

- 작업트리에만 있는 변경 → status에서 빨간색 `Changes not staged for commit` 또는 `Untracked files`
- 스테이징에 올라간 변경 → status에서 초록색 `Changes to be committed`
- 커밋되어 저장소에 들어간 변경 → status에서 더 이상 보이지 않음(`nothing to commit, working tree clean`)



```
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    README.md
    index.html

nothing added to commit but untracked files present (use "git add" to track)
```

- Untracked files 아래 빨간 목록( README.md , index.html )은 작업트리에만 있는, 아직 한 번도 추적되지 않은 파일입니다.
- Git이 친절하게 use "git add <file>..." 라고 다음에 할 일을 안내해 줍니다. status는 늘 "지금 상태 + 다음에 할 일"을 함께 알려 줍니다.
- No commits yet 은 이 저장소에 아직 커밋이 하나도 없다는 뜻입니다(저장소 장부가 비어 있음).

핵심은 "고쳤다고 자동으로 기록되지 않는다" 는 점입니다. 변경이 저장소까지 가려면 반드시 add → commit 두 관문을 통과해야 합니다. 이 사실을 모르면 "분명히 저장했는데 왜 커밋에 안 들어갔지?"라는 가장 흔한 혼란에 빠집니다.

### 3. git add: 이번 커밋에 담을 것 고르기

git add 는 작업트리의 변경을 스테이징 영역으로 올리는 명령입니다. 기본 형태는 다음과 같습니다.

```
$ git add <파일이름>      # 특정 파일 하나만 스테이징
$ git add .                # 현재 폴더 아래 모든 변경을 스테이징
$ git add -p              # 변경의 일부(조각)만 골라 스테이징
```

가장 많이 쓰는 것은 git add . (점)입니다. 현재 폴더와 그 하위의 모든 변경을 한 번에 올립니다. 다만 "모든 변경"을 무심코 올리면 의도하지 않은 파일(로그·비밀키 등)까지 함께 담길 수 있으므로, 무엇이 올라가는지 git status 로 확인하는 습관이 중요합니다.

특정 파일만 골라 올릴 수도 있습니다.

```
$ git add README.md      # README.md의 변경만 스테이징
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   README.md

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    index.html
```

- `git add README.md` 로 `README.md` 만 스테이징하자, `status`가 그것만 초록색 `Changes to be committed` 로 보여 줍니다.
- `index.html` 은 아직 `add` 하지 않아 여전히 `Untracked files` (빨간색)에 남아 있습니다. 이렇게 **파일별로 골라** 담을 수 있다는 점이 `add`의 핵심 가치입니다.
- `new file:` 은 이 파일이 이번 커밋에서 처음 추적된다는 표시입니다(기존 파일을 고쳤다면 `modified:` 로 나옵니다).

### git add -p로 한 파일 안에서 조각만 고르기

한 파일 안에서도 일부 변경만 커밋하고 싶을 때가 있습니다. 예를 들어 한 파일에 "기능 수정"과 "임시 디버그 코드"가 섞여 있다면, 기능 수정만 커밋하고 싶을 것입니다. 이때 `git add -p` (`patch`)를 쓰면 변경 덩어리(`hunk`)마다 `담을지(y)` `말지(n)`를 물어봅니다.

```
$ git add -p index.html
diff --git a/index.html b/index.html
@@ -1,3 +1,5 @@
<h1>My Page</h1>
+<p>소개 문단</p>
+<!-- TODO: 디버그용, 나중에 삭제 -->
(1/1) Stage this hunk [y,n,q,a,d,e,?]?
```

여기서 `y`를 누르면 이 덩어리를 스테이징하고, `n`을 누르면 건너뛵니다. `?`를 누르면 각 키의 의미를 보여 줍니다. 입문 단계에서는 `git add <파일>` 과 `git add .` 만으로 충분하지만, `-p`라는 정밀 도구가 있다는 것은 알아 두면 좋습니다.

## 4. worked example: 샘플 페이지 첫 커밋 만들기

이제 빈 저장소에서 시작해 첫 커밋을 남기는 정석 흐름을 처음부터 끝까지 따라가 보겠습니다. 작은 소개 웹페이지 프로젝트라고 가정합니다.

## 상황

폴더 `my-site` 를 만들고, 그 안에 `index.html` 과 `README.md` 를 만든 직후입니다. 아직 저장소로 만들지도, 커밋하지도 않았습니다.

## 명령과 진단

다음은 명령을 차례로 입력했을 때의 실행 결과입니다.

```
$ git init
Initialized empty Git repository in C:/Users/you/my-site/.git/

$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    README.md
    index.html

nothing added to commit but untracked files present (use "git add" to track)

$ git add .

$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   README.md
    new file:   index.html

$ git commit -m "프로젝트 초기 구조 추가"
[main (root-commit) a1b2c3d] 프로젝트 초기 구조 추가
 2 files changed, 12 insertions(+)
 create mode 100644 README.md
 create mode 100644 index.html
```

- `git init` 으로 저장소를 만든 직후, status는 두 파일을 `Untracked files` 로 봅니다(아직 추적 안 됨).
- `git add .` 로 두 파일을 모두 스테이징하자, status가 `Changes to be committed` 로 올라온 것을 보여 줍니다. 변경이 작업트리에서 스테이징으로 **이동**한 것입니다.
- `git commit -m "..."` 으로 스테이징의 내용을 하나의 커밋으로 박습니다. `-m` 뒤에 큰따옴표로 커밋 메시지를 적습니다.

- 출력의 [main (root-commit) a1b2c3d] 에서 `root-commit` 은 이 저장소의 **첫 커밋**이라는 뜻이고, `a1b2c3d` 는 커밋의 짧은 해시(지문)입니다. `2 files changed, 12 insertions(+)` 는 2개 파일에 12줄이 추가됐다는 요약입니다.

커밋이 끝나면 `status`는 깨끗해집니다. 변경이 모두 저장소로 들어갔기 때문입니다.

```
$ git status
On branch main
nothing to commit, working tree clean
```

이 `working tree clean` 은 "작업트리에 커밋되지 않은 변경이 하나도 없다"는 가장 평온한 상태입니다. 새 작업을 시작하기 좋은 깨끗한 출발점입니다.

### 변경을 `add` 한 뒤 또 고치면?

흔히 겪는 상황 하나를 짚고 넘어갑니다. 파일을 `add` 한 다음 **같은 파일을 또 수정**하면 어떻게 될까요? `status`가 같은 파일을 두 군데에 동시에 보여 줍니다.

```
$ git add README.md
$ # 여기서 README.md를 한 번 더 수정함
$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    modified:   README.md

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
    modified:   README.md
```

- `README.md` 가 `Changes to be committed` (스테이징됨)와 `Changes not staged for commit` (작업 트리에 또 변경 있음) **양쪽에** 나타납니다.
- 이는 모순이 아니라 정확한 상태 보고입니다. `add` 시점의 내용은 스테이징에 올라가 있고, 그 뒤에 더 고친 부분은 아직 작업트리에만 있는 것입니다.
- 마지막으로 고친 내용까지 커밋하려면 `git add README.md` 를 **한 번 더** 실행해 스테이징을 최신 상태로 갱신해야 합니다. `add`는 "지금 이 순간의 작업트리"를 스냅샷으로 떠 가는 것이라 기억하십시오.

## 5. 좋은 커밋 메시지와 커밋 단위, `--amend`로 직전 수정

**커밋 메시지: 미래의 나와 동료들을 위한 기록**

커밋 메시지는 "무엇을 왜 바꿨는가"를 미래에 전하는 편지입니다. `git log` 로 이력을 되짚을 때, 좋은 메시지는 길잡이가 되고 나쁜 메시지는 미로가 됩니다.

| 나쁜 메시지  | 왜 나쁜가           | 좋은 메시지                 |
|---------|-----------------|------------------------|
| fix     | 무엇을 고쳤는지 알 수 없음 | 로그인 버튼 클릭이 안 되던 문제 수정  |
| update  | 무엇이 갱신됐는지 모름    | README에 설치 방법 섹션 추가    |
| □ L O R | 의미 없음           | 소개 페이지에 연락처 정보 추가      |
| 작업함     | 어떤 작업인지 모름      | index.html 헤더 글꼴 크기 조정 |

관례적으로 메시지는 다음을 따르면 좋습니다.

- **요약 줄**(첫 줄)은 50자 안팎으로, "무엇을 했는지" 명령형 또는 명사형으로 간결하게.
- 더 설명할 게 있으면 한 줄 비우고 **본문**에 "왜" 바꿨는지를 적습니다.

본문이 있는 커밋은 `-m` 을 두 번 쓰거나, `-m` 없이 `git commit` 만 실행해 편집기에서 작성합니다.

```
$ git commit -m "README에 설치 방법 추가" -m "처음 받는 사람이 곧바로 실행할 수 있도록 단계별 명
```

- 첫 `-m` 은 요약 줄, 둘째 `-m` 은 본문이 됩니다. 둘 사이에 자동으로 빈 줄이 들어가 형식이 맞춰집니다.
- 요약은 "README에 설치 방법 추가"처럼 결과를 한눈에 보여 주고, 본문은 "왜"(처음 받는 사람을 위해)를 보충합니다.

## 커밋 단위: 하나의 논리 = 하나의 커밋

좋은 커밋은 **하나의 논리적 변경**을 담습니다. "로그인 기능 추가"와 "오타 수정"을 한 커밋에 뭉뚱그리면, 나중에 둘 중 하나만 되돌리기 어렵습니다. 반대로 의미 없이 너무 잘게 쪼개도(한 글자 고칠 때마다 커밋) 이력이 지저분해집니다. "이 변경을 한 문장으로 설명할 수 있는가?"가 좋은 기준입니다. 한 문장에 "그리고"가 자꾸 들어간다면 커밋을 나눌 신호입니다.

## --amend로 직전 커밋 손보기

방금 한 커밋의 메시지에 오타가 있거나, 빠뜨린 파일을 그 커밋에 포함하고 싶을 때 `git commit --amend` 를 씁니다. 새 커밋을 또 만드는 대신 **직전 커밋을 고쳐** 다시 만듭니다.

```
$ git commit -m "README 소개 작성" # 오차가 있는 메시지로 커밋해 버림
$ git commit --amend -m "README 소개 작성" # 직전 커밋 메시지를 바로잡음
[main 7f8e9d0] README 소개 작성
1 file changed, 3 insertions(+)
```

빠뜨린 파일을 직전 커밋에 추가하려면, 먼저 그 파일을 `git add` 한 뒤 `--amend` 를 실행합니다.

```
$ git add forgotten.css
$ git commit --amend --no-edit # 메시지는 그대로 두고 파일만 직전 커밋에 합침
```

`--no-edit` 은 "메시지는 건드리지 말고 그대로 두라"는 옵션입니다.

**주의:** `--amend` 는 직전 커밋을 **새로운 커밋으로 교체**하는 것이라 해시가 바뀝니다. 아직 원격에 올리지(push) 않은 커밋에만 안전하게 쓰십시오. 이미 공유한 커밋을 amend 하면 동료의 이력과 어긋납니다(이 위험은 ch\_14의 황금률에서 자세히 다룹니다).

## 6. .gitignore로 로그·env·빌드 산출물 제외하기

저장소에는 **추적하면 안 되는 파일**이 있습니다. 비밀번호가 든 설정 파일, 자동 생성되는 로그, 빌드 결과물, 용량 큰 임시 파일 등입니다. 이런 파일은 커밋하면 보안 위험이 되거나 이력을 더럽힙니다. `.gitignore` 는 "이런 파일은 무시하라"고 Git에게 알려 주는 목록입니다.

저장소 최상단에 `.gitignore` 라는 이름의 파일을 만들고, 무시할 패턴을 한 줄에 하나씩 적습니다.

```
# 주석은 # 으로 시작합니다

# 로그 파일 전부 무시
*.log

# 환경 변수/비밀 파일 무시
.env
secret.env

# 빌드 산출물 폴더 무시
build/
dist/

# OS가 만드는 잡파일
Thumbs.db
.DS_Store
```

- `*.log` 는 "확장자가 `.log` 인 모든 파일"을 뜻합니다. `*` 는 아무 글자나 의미하는 와일드카드입니다.

- `.env`, `secret.env` 처럼 파일명을 그대로 적으면 그 파일을 무시합니다.
- `build/` 처럼 끝에 슬래시(/)를 붙이면 그 이름의 **폴더 전체**를 무시합니다.
- `Thumbs.db` (Windows)·`.DS_Store` (macOS)는 운영체제가 자동으로 만드는 잡파일이라 보통 무시 목록에 넣습니다.

`.gitignore`에 등록된 파일은 `git status`에서 `Untracked files`에 더 이상 나타나지 않습니다. Git이 아예 못 본 척하기 때문입니다. `.gitignore` 자체는 커밋하는 것이 정석입니다. 팀원 모두가 같은 무시 규칙을 공유해야 하기 때문입니다.

**팁:** 무시 규칙을 처음부터 직접 다 적을 필요는 없습니다. GitHub가 운영하는 `github.com/github/gitignore` 저장소에 언어·환경별 표준 `.gitignore`가 정리돼 있습니다. 새 프로젝트를 만들 때 자기 환경에 맞는 것을 가져와 출발하면 편리합니다.

## 7. 흔한 문제·해결: 비밀키 커밋 → `git rm --cached`로 빼내기

`.gitignore`에는 결정적인 함정이 하나 있습니다. `.gitignore`는 **"아직 추적 안 된" 파일에만 효과가 있습니다**. 이미 한 번 커밋되어 추적 중인 파일은 나중에 `.gitignore`에 적어도 계속 추적됩니다. 이 사실을 모르면 "분명히 무시 목록에 적었는데 왜 계속 따라다니지?"라는 함정에 빠집니다.

### 상황

실수로 비밀번호가 든 `secret.env`를 커밋해 버린 직후 이를 발견했습니다.

### 진단

```
$ git status
On branch main
nothing to commit, working tree clean

$ git ls-files
README.md
index.html
secret.env
```

- `git ls-files`는 **현재 추적 중인 파일 목록**을 보여 줍니다. 여기에 `secret.env`가 들어 있다는 것은, 이미 추적되고 있다는 확정 진단입니다.
- 이 상태에서 `.gitignore`에 `secret.env`를 적어도, 이미 추적 중이므로 `status`에서 사라지지 않습니다. `.gitignore`만으로는 부족하다는 신호입니다.

### 해결

추적에서 빼내려면 `git rm --cached` 를 씁니다. 이 명령은 **작업트리의 파일은 그대로 두고**, Git의 추적(인덱스)에서만 제거합니다.

```
$ git rm --cached secret.env
rm 'secret.env'

$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    deleted:    secret.env

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    secret.env

$ # .gitignore에 secret.env를 추가한 뒤
$ git add .gitignore
$ git commit -m "secret.env를 추적에서 제외하고 gitignore 추가"
[main 9c0d1e2] secret.env를 추적에서 제외하고 gitignore 추가
2 files changed, 1 insertion(+), 0 deletions(-)
delete mode 100644 secret.env
create mode 100644 .gitignore
```

- `git rm --cached secret.env` 직후 `status`를 보면, `deleted: secret.env` (추적 제거가 스테이징됨)와 동시에 `Untracked files` 에 `secret.env` 가 다시 나타납니다. 파일 자체는 디스크에 남아 있고(작업트리), Git의 추적에서만 빠졌다는 뜻입니다.
- `.gitignore` 에 `secret.env` 를 적고 함께 커밋하면, 이후로는 그 파일이 `status`에 아예 나타나지 않습니다.

**중요:** `git rm --cached` 로 추적을 빼도, 이미 커밋된 과거 이력에는 비밀이 그대로 남아 있습니다. 진짜 비밀번호·API 키가 커밋됐다면 반드시 그 키를 **폐기·재발급**해야 합니다. 이력에서 완전히 지우는 것은 입문 범위를 넘는 별도 작업(`git filter-repo` 등)이며, 가장 확실한 안전은 "유출된 비밀은 즉시 무효화한다"는 원칙입니다.

`--cached` 없이 그냥 `git rm secret.env` 를 쓰면 작업트리의 파일까지 **삭제**되므로, 파일은 남기고 추적만 뺄 때는 반드시 `--cached` 를 붙여야 합니다.

## 8. 즉시 연습 (2종 1세트)

### 직접 작성 1 — 첫 커밋 만들기 (빈칸 채우기)

새 폴더 `notes` 에서 `memo.txt` 한 파일을 만든 뒤, 저장소로 만들어 첫 커밋을 남기는 명령을 완성하십시오. 빈칸에 알맞은 명령·옵션을 직접 작성하면 됩니다.



```
$ git init                                # 폴더를 저장소로 만들기
$ _____                             # 모든 변경을 스테이징하는 명령(점 포함)
$ git status                             # 무엇이 Changes to be committed에 올랐는지 확인
$ git commit _____ "메모 파일 첫 작성" # 메시지를 함께 주는 옵션을 빈칸에
$ git status                             # working tree clean 인지 확인
```

## 자가체크 체크리스트

- [ ] 첫 빈칸에 `git add .` 를 넣어 변경을 스테이징했는가
- [ ] 둘째 빈칸에 `-m` 을 넣어 메시지와 함께 커밋했는가
- [ ] 커밋 후 `git status` 가 `nothing to commit, working tree clean` 을 보여 주는가
- [ ] 커밋 메시지가 "무엇을 했는지" 알 수 있게 적혔는가

## 직접 작성 2 — 실수로 커밋한 비밀 파일 빼내기 (빈칸 채우기)

`config.env` (비밀 설정)를 이미 커밋한 상태입니다. 추적에서 빼고 앞으로 무시되게 만드는 과정을 완성하십시오. `config.env` 파일 자체는 디스크에 남겨야 합니다.

```
$ git ls-files                            # config.env가 추적 중인지 진단(목록에 보이는지)
$ git rm _____ config.env           # 작업트리 파일은 남기고 추적만 빼는 옵션을 빈칸에
$ # .gitignore 파일에 한 줄을 추가: _____ (config.env를 무시하는 규칙)
$ git add .gitignore
$ git commit -m "config.env 추적 제외 및 gitignore 추가"
```

## 자가체크 체크리스트

- [ ] 첫 빈칸에 `--cached` 를 넣어 작업트리 파일은 보존했는가
- [ ] `.gitignore` 에 `config.env` (또는 `*.env`) 한 줄을 추가했는가
- [ ] 커밋 후 `git status` 에 `config.env` 가 더 이상 나타나지 않는가
- [ ] (개념 점검) 만약 실제 비밀이었다면 그 비밀을 재발급해야 한다는 점을 떠올렸는가

# 9. 흔한 오류와 오개념 교정

## 오개념 1 — "수정하고 저장만 하면 커밋된다"

파일을 고쳐 저장(Ctrl+S)해도 그것은 작업트리의 변경일 뿐, 커밋이 아닙니다. `git add` 로 스테이징한 뒤 `git commit` 해야 비로소 저장소에 기록됩니다.

```
$ # index.html을 수정하고 저장만 한 상태
$ git commit -m "헤더 수정"
On branch main
Changes not staged for commit:
  modified:   index.html
no changes added to commit (use "git add" and/or "git commit -a")
```

**교정:** add 없이 commit 하면 "no changes added to commit"이라며 아무것도 커밋되지 않습니다. 수정 → git add → git commit 순서를 지키십시오. 또한 add 후 또 고쳤다면 다시 add 해야 그 변경까지 담깁니다.

## 오개념 2 — ".gitignore에 적기만 하면 이미 커밋된 파일도 빠진다"

.gitignore 는 아직 추적 안 된 파일에만 효과가 있습니다. 이미 커밋된 파일은 .gitignore 에 적어도 계속 추적됩니다.

```
$ # data.log를 이미 커밋한 뒤 .gitignore에 *.log 추가
$ git status
On branch main
nothing to commit, working tree clean
$ git ls-files
data.log          # 여전히 추적 중!
```

**교정:** 이미 추적 중인 파일은 git rm --cached <파일> 로 먼저 추적을 빼고, .gitignore 에 규칙을 추가한 뒤 커밋해야 무시됩니다.

## 오개념 3 — "커밋 메시지는 아무렇게나 적어도 된다"

fix, update, 작업함 같은 메시지는 당장은 편하지만, 나중에 git log 로 "언제 무엇을 왜 바꿨는지" 추적할 때 아무 도움이 안 됩니다. 메시지는 미래의 나와 동료들을 위한 기록입니다.

**교정:** "무엇을 했는지" 한 줄로 분명히 적으십시오. 예: update 대신 README에 설치 방법 섹션 추가. 한 문장으로 설명되지 않으면 커밋을 나눌 신호입니다.

## 오개념 4 — "git rm은 추적만 빼는 명령이다"

git rm <파일> 은 추적뿐 아니라 작업트리의 실제 파일까지 삭제합니다. 파일은 남기고 추적만 빼려면 반드시 --cached 를 붙여야 합니다.

```
$ git rm secret.env          # 위험: 디스크의 secret.env도 삭제됨
$ git rm --cached secret.env # 안전: 파일은 남고 추적만 빠짐
```

**교정:** 파일을 보존하면서 추적만 제거하려면 `git rm --cached` 를 쓰십시오. `--cached` 가 작업트리를 지켜 줍니다.

## 10. 단원 미니 프로젝트: 내 첫 Git 저장소 (2종 1세트)

이 단원(u1)에서 배운 세 개념 — `git config` (ch\_01) · `git init` 과 `git status` (ch\_02) · `git add` · `git commit` · `.gitignore` (이 장) — 을 모두 동원해 작은 산출물 하나를 완성합니다. 작은 소개 웹페이지를 만드는 1인 프로젝트를 Git으로 처음부터 기록하는 것이 목표입니다.

**적용 개념:** c01(config) · c02(init·status) · c03(add·commit·gitignore)

**산출물 목표:** `index.html` · `README.md` · `.gitignore` 를 갖춘 저장소 하나. `git log --oneline` 에 의미 있는 커밋 2~3개가 남고, 실수로 커밋한 `secret.env` 를 추적에서 빼낸 복구 이력까지 포함합니다.

### 모범 — 완성된 흐름

다음은 미니 프로젝트를 마일스톤(증분) 3단계로 끝까지 수행한 모범 기록입니다.

```

$ # M1: 환경 준비 → 저장소 만들기 → 상태 확인
$ git config --global user.name "Hong Gildong"
$ git config --global user.email "gildong@example.com"
$ git init
Initialized empty Git repository in C:/Users/you/my-site/.git/
$ git status
On branch main
No commits yet
Untracked files:
  README.md
  index.html

$ # M2: 첫 커밋 → 로그 확인
$ git add .
$ git commit -m "프로젝트 초기 구조 추가"
[main (root-commit) a1b2c3d] 프로젝트 초기 구조 추가
 2 files changed, 12 insertions(+)
$ git commit --allow-empty -m "README 소개 문단 작성" 2>/dev/null # 예시용
$ git log --oneline
a1b2c3d 프로젝트 초기 구조 추가

$ # M3: secret.env 실수 커밋 → 발견 → 추적 제외 후 재커밋
$ git add secret.env
$ git commit -m "임시 설정 추가"
[main b2c3d4e] 임시 설정 추가
$ git ls-files
README.md
index.html
secret.env
$ git rm --cached secret.env
rm 'secret.env'
$ # .gitignore에 'secret.env'와 '*.log' 추가 후
$ git add .gitignore
$ git commit -m "secret.env 추적 제외 및 gitignore 추가"
[main c3d4e5f] secret.env 추적 제외 및 gitignore 추가

```

완성 후 `git log --oneline` 으로 확인한 이력은 다음과 같은 모양이 됩니다.

```

$ git log --oneline
c3d4e5f secret.env 추적 제외 및 gitignore 추가
b2c3d4e 임시 설정 추가
a1b2c3d 프로젝트 초기 구조 추가

```

- M1에서 `git config` 로 저자 정보를 먼저 갖춰야 커밋에 작성자가 올바르게 박힙니다(설정이 없으면 ch\_01에서 본 'Author identity unknown' 경고가 납니다).
- M2에서 의미 있는 메시지로 커밋을 싹고, `git log --oneline` 으로 한 줄씩 압축해 확인합니다(자세한 log 읽기는 다음 장 ch\_04에서 배웁니다).
- M3에서 `secret.env` 를 일부러 커밋했다가 `git rm --cached` 로 빼내는 복구를 직접 경험합니다. 이 "사고 → 진단(`git ls-files`) → 해결(`git rm --cached`)" 흐름이 Git 학습의 핵심

패턴입니다.

## 직접 작성 — 학생용 빈 체크리스트 (2종 1세트)

### 미니 프로젝트 실습 — 단계별로 직접 명령을 채워 수행하기

아래 마일스톤을 따라 자신의 저장소를 직접 만들고, 각 단계에서 실행한 명령을 빈칸에 적으며 진행하십시오. 명령은 책을 덮고 스스로 떠올려 보는 것이 학습에 가장 좋습니다.

```
$ # M1 환경 준비 + 저장소 만들기
$ git config --global user.name "____" # 본인 이름
$ git config --global user.email "____" # 본인 이메일
$ _____ # 폴더를 저장소로 만드는 명령
$ git status # untracked 파일 확인

$ # M2 첫 커밋
$ _____ # 변경 스테이징(점 포함)
$ git commit ____ "프로젝트 초기 구조 추가" # 메시지 옵션 채우기
$ git log --oneline # 커밋이 남았는지 확인

$ # M3 secret.env 사고 복구
$ # (먼저 secret.env를 만들어 실수로 커밋했다고 가정)
$ git ls-files # secret.env가 추적 중인지 진단
$ git rm _____ secret.env # 작업트리는 남기고 추적만 빼는 옵션
$ # .gitignore에 다음 한 줄을 추가: _____
$ git add .gitignore
$ git commit -m "____" # 무엇을 왜 했는지 드러나는 메시지
```

### 자가체크 체크리스트 (스스로 점검)

- [ ] `git status` 가 최종적으로 `working tree clean` 인가
- [ ] `git log --oneline` 에 의미 있는 메시지의 커밋이 2~3개 있는가
- [ ] `secret.env` 가 더 이상 추적되지 않는가(`git ls-files` 에 없는가)
- [ ] `.gitignore` 가 커밋되어 저장소에 포함됐는가
- [ ] 각 커밋 메시지가 "무엇을 했는지" 한 줄로 설명되는가

## 11. 자주 묻는 질문 (FAQ)

**Q1. `git add .` 와 `git add -A` 는 무엇이 다른가요?** A. 최신 Git에서는 둘 다 "추가·수정·삭제를 모두 스테이징"하지만, `.` 은 현재 폴더 기준이고 `-A` 는 저장소 전체 기준이라는 미세한 차이가 있습니다. 입문 단계에서는 저장소 최상단에서 `git add .` 를 쓰면 충분합니다. 무엇이 올라가는지를 `git status` 로 확인하는 습관이 더 중요합니다.

**Q2. 커밋을 했는데 메시지를 잘못 적었습니다. 되돌릴 수 있습니까?** A. 아직 push 하지 않은 직전 커밋이라면 `git commit --amend -m "올바른 메시지"` 로 바로 고칠 수 있습니다. 더 과거의 커밋이나 이미 공유한 커밋은 다른 방법이 필요하며(ch\_08의 되돌리기, ch\_14의 rebase), 공유한 이력은 함부로 고치지 않는 것이 원칙입니다.

**Q3. 빈 폴더는 커밋이 안 되나요?** A. Git은 파일을 추적할 뿐 빈 폴더 자체는 추적하지 않습니다. 폴더 구조를 유지하고 싶다면 관례적으로 그 안에 `.gitkeep` 이라는 빈 파일을 하나 두어 폴더가 사라지지 않게 합니다.

**Q4. .gitignore 에 적었는데도 파일이 status에 보입니다. 왜죠?** A. 그 파일이 이미 추적 중일 가능성이 큼니다. `git ls-files` 로 확인해 목록에 있으면, `git rm --cached <파일>` 로 추적을 빼고 다시 커밋해야 합니다. `.gitignore` 는 아직 추적되지 않은 파일에만 작동한다는 점을 기억하십시오.

## 12. 마무리

이 장에서는 변경이 기록으로 굳어지는 전 과정을 익혔습니다. 작업트리에서 시작한 변경은 `git add` 로 **스테이징**에 모이고, `git commit` 으로 **저장소**에 스냅샷으로 박힙니다. 이 3영역 모델은 Git의 모든 작업을 이해하는 토대이며, `git status` 가 그 세 영역을 비추는 거울이라는 점도 확인했습니다.

또한 좋은 커밋 메시지와 적절한 커밋 단위로 "미래에 읽을 수 있는 이력"을 남기는 법, `--amend` 로 직전 커밋을 손보는 법, `.gitignore` 로 추적하면 안 되는 파일을 걸러 내는 법, 그리고 실수로 커밋한 비밀 파일을 `git rm --cached` 로 빼내는 복구까지 다뤘습니다. 특히 `.gitignore` 는 이미 추적 중인 파일에는 효과가 없다"는 함정은 입문자가 반드시 한 번은 겪는 관문이니 기억해 두십시오.

다음 장(ch\_04)에서는 이렇게 쌓은 커밋 이력을 **읽는** 힘을 기릅니다. `git log` 로 누가 언제 무엇을 했는지 훑고, `git diff` 로 무엇이 어떻게 바뀌었는지 줄 단위로 비교하는 법을 배웁니다. 기록하는 능력(이 장)과 읽는 능력(다음 장)이 만나야, 비로소 충돌 해결과 되돌리기 같은 모든 고급 작업을 자신 있게 진단할 수 있습니다.



# 이력 읽기: git log와 git diff로 진단하기

## 학습 목표

- `git log` 와 옵션(`--oneline` · `--graph` · `-n` · `--author`)으로 이력을 효율적으로 읽을 수 있습니다.
- 커밋 해시(SHA)·HEAD·부모 커밋의 의미를 해석할 수 있습니다.
- `git diff` 로 작업트리·스테이징·커밋 사이의 차이를 비교할 수 있습니다.
- `git show` 로 특정 커밋의 변경 내용을 확인하고, log-diff로 문제를 진단할 수 있습니다.

앞 장(ch\_03)에서 우리는 `git add` 와 `git commit` 으로 변경을 기록하는 법을 익혔습니다. 이제 저장소에는 커밋이 차곡차곡 쌓이기 시작합니다. 그런데 기록만 쌓고 읽지 못하면, 그것은 펼쳐 보지 않는 일기장과 같습니다. 이 장에서는 쌓인 이력을 읽는 두 도구, `git log` 와 `git diff` 를 배웁니다. 이 단원의 개념을 커리큘럼에서는 **이력 읽기 — log와 diff** 라고 부릅니다.

선수 개념 상기: 앞 장의 **커밋(commit)** 을 떠올리십시오. 우리는 변경을 add 해 스테이징에 모으고 commit으로 스냅샷을 박았습니다. 그 스냅샷 하나하나가 이 장에서 `git log` 로 훑고 `git diff` 로 비교할 대상입니다. 또 ch\_03의 **3영역**(작업트리·스테이징·저장소)은 `git diff` 가 "무엇과 무엇을 비교하는지"를 결정하는 좌표가 됩니다.

이력을 정확히 읽는 능력은 단순한 구경이 아닙니다. 앞으로 배울 충돌 해결(ch\_07)·되돌리기(ch\_08) 같은 모든 작업은 "지금 무슨 일이 일어났는가"를 log-diff로 진단하는 데서 출발합니다. 그래서 이 장은 모든 복구의 출발점입니다.

이 장은 Windows 11을 기준으로 설명합니다. 명령은 Git Bash·PowerShell 어디서나 동일합니다.

## 1. 도입: 이력을 읽는 힘이 모든 복구의 출발점



코드를 짜다 보면 이런 순간이 반드시 옵니다. "어제까지 잘 되던 게 왜 안 되지?", "이 줄을 누가, 언제, 왜 바꿨지?", "방금 고친 게 정말 커밋에 들어갔나?" 이 질문들의 답은 모두 저장소의 이력 안에 있습니다. 그 이력을 꺼내 읽는 도구가 `git log` 와 `git diff` 입니다.

- `git log` 는 "어떤 커밋들이 있었는가"를 시간순으로 보여 줍니다. 이력의 **목차**를 읽는 도구입니다.
- `git diff` 는 "두 시점 사이에 무엇이 바뀌었는가"를 줄 단위로 보여 줍니다. 변경의 **내용**을 들여다보는 도구입니다.
- `git show` 는 "특정 커밋 하나가 무엇을 바꿨는가"를 메시지와 함께 펼쳐 줍니다.

이 셋은 모두 **읽기 전용**입니다. 아무리 자주 실행해도 파일이나 이력을 바꾸지 않습니다. 그래서 "지금 상태가 헛갈리면 일단 log-diff-status부터"가 안전하고 좋은 습관입니다.

### 이 장에서 만나는 핵심 용어

- 커밋 해시(commit hash, SHA): 커밋마다 붙는 40자리 고유 지문(보통 앞 7자만 표시)
- HEAD: 지금 내가 보고 있는 커밋(현재 위치)을 가리키는 포인터
- 부모 커밋(parent): 어떤 커밋의 바로 직전 커밋
- diff: 두 시점 사이의 줄 단위 차이(추가는 `+`, 삭제는 `-`)
- 페이저(pager): 긴 출력을 화면 단위로 넘겨 보여 주는 프로그램( `less` )

## 개념: 이력 읽기

### 2. git log 기본과 `--oneline``---graph``---all`

## 커밋 이력 그래프 (git log --graph)



```
$ git log --oneline --graph
```

```
* f9e8d7c (HEAD -> main) 버그 수정
* b4c3d2e 스타일 수정
* a1b2c34 기능 추가
* 0d1e2f3 초기 커밋
```

git log 는 가장 최근 커밋부터 거슬러 올라가며 이력을 보여 줍니다. 아무 옵션 없이 실행하면 커밋마다 해시·작성자·날짜·메시지가 자세히 나옵니다.

```
$ git log
commit c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2 (HEAD -> main)
Author: Hong Gildong <gildong@example.com>
Date: Mon Jun 16 10:42:11 2025 +0900
```

secret.env 추적 제외 및 gitignore 추가

```
commit b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1
Author: Hong Gildong <gildong@example.com>
Date: Mon Jun 16 10:38:02 2025 +0900
```

README 소개 문단 작성

```
commit a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0
Author: Hong Gildong <gildong@example.com>
Date: Mon Jun 16 10:30:55 2025 +0900
```

프로젝트 초기 구조 추가

- 커밋은 **최신이 맨 위**입니다. 위에서 아래로 갈수록 과거입니다.
- 각 커밋은 `commit` 뒤의 40자리 해시, `Author` (작성자), `Date` (작성 시각), 그리고 들여쓰인 커밋 메시지로 이뤄집니다.
- 맨 위 커밋 옆 `(HEAD -> main)` 은 "현재 HEAD가 main 브랜치를 가리키고, main이 이 커밋을 가리킨다"는 뜻입니다(다음 절에서 자세히 설명합니다).

이 기본 형식은 자세하지만 길어서 한눈에 흐름을 보기 어렵습니다. 그래서 실무에서는 다음 옵션을 자주 씁니다.

**--oneline: 한 줄로 압축**

```
$ git log --oneline
c3d4e5f (HEAD -> main) secret.env 추적 제외 및 gitignore 추가
b2c3d4e README 소개 문단 작성
a1b2c3d 프로젝트 초기 구조 추가
```

**--oneline** 은 커밋 하나를 "짧은 해시 + 메시지" 한 줄로 줄여 줍니다. 이력을 빠르게 훑을 때 가장 많이 쓰는 형태입니다.

**--graph --all: 브랜치 갈래를 그림으로**

**--graph** 는 커밋들의 갈래(브랜치)를 ASCII 선으로 그려 줍니다. **--all** 은 현재 브랜치뿐 아니라 모든 브랜치를 함께 보여 줍니다(브랜치는 ch\_05에서 배웁니다). 두 옵션을 합치면 이력 구조가 한눈에 들어옵니다.

```
$ git log --oneline --graph --all
* c3d4e5f (HEAD -> main) secret.env 추적 제외 및 gitignore 추가
* b2c3d4e README 소개 문단 작성
* a1b2c3d 프로젝트 초기 구조 추가
```

지금은 갈래가 없어 한 줄로 곧게 내려오지만, 브랜치를 나누고 병합하면 이 그림이 갈라지고 합쳐지는 모습을 보여 줍니다. `git log --oneline --graph --all` 은 "지금 이 저장소의 전체 지도"를 보는 명령이라 기억해 두면 두고두고 유용합니다.

**좁혀 읽는 옵션: -n, --author**

이력이 길어지면 원하는 부분만 좁혀 읽습니다.

| 옵션                                                     | 의미                 | 예                                                   |
|--------------------------------------------------------|--------------------|-----------------------------------------------------|
| <code>-n &lt;개수&gt;</code> 또는 <code>-&lt;개수&gt;</code> | 최근 N개만             | <code>git log -n 3</code> , <code>git log -3</code> |
| <code>--author=&lt;이름&gt;</code>                       | 특정 작성자의 커밋만        | <code>git log --author="Hong"</code>                |
| <code>--oneline</code>                                 | 한 줄 요약             | <code>git log --oneline</code>                      |
| <code>--stat</code>                                    | 커밋마다 바뀐 파일·줄 수 요약  | <code>git log --stat</code>                         |
| <code>-p</code>                                        | 커밋마다 실제 변경(diff)까지 | <code>git log -p</code>                             |

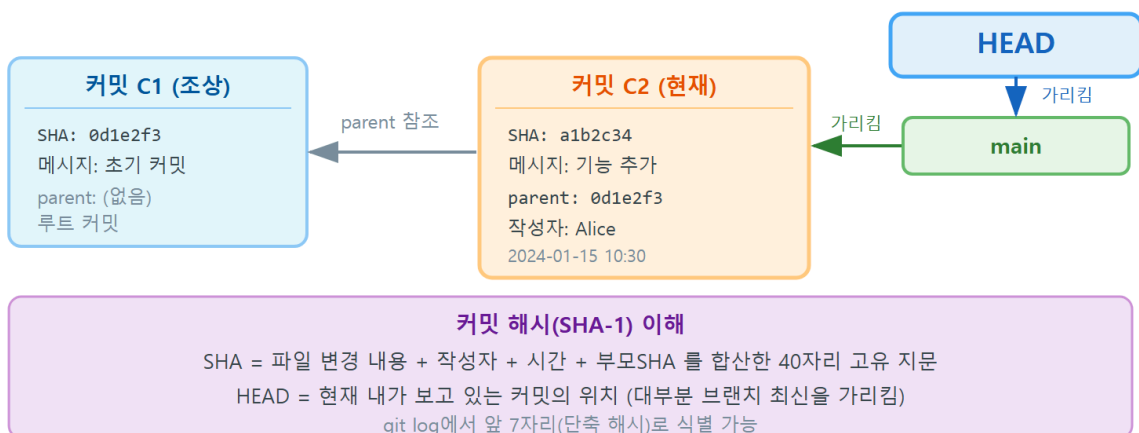
```
$ git log -2 --oneline
c3d4e5f (HEAD -> main) secret.env 추적 제외 및 gitignore 추가
b2c3d4e README 소개 문단 작성

$ git log --author="Hong" --oneline
c3d4e5f (HEAD -> main) secret.env 추적 제외 및 gitignore 추가
b2c3d4e README 소개 문단 작성
a1b2c3d 프로젝트 초기 구조 추가
```

- `git log -2` 는 최근 2개 커밋만 보여 줍니다. 긴 이력에서 최근 것만 빠르게 볼 때 편리합니다.
- `--author="Hong"` 은 작성자 이름에 "Hong"이 포함된 커밋만 거릅니다. 협업에서 "누가 한 작업인지"로 좁혀 읽을 때 씁니다.

### 3. 커밋 해시(SHA)·HEAD·부모 커밋 이해하기

#### 커밋 해시(SHA)·HEAD·부모 커밋 관계



`git log` 를 제대로 읽으려면 세 가지 개념이 필요합니다. 커밋 **해시**, **HEAD**, 그리고 **부모** 커밋입니다.

#### 커밋 해시(SHA): 커밋의 지문

커밋마다 붙는 `a1b2c3d...` 로 시작하는 40자리 문자열을 **커밋 해시** 또는 SHA라고 합니다. 이것은 무작위로 붙인 번호가 아니라, 커밋의 내용(바뀐 파일·메시지·작성자·시각·부모 커밋)을 바탕으로 계산된 **지문**입니다. 내용이 조금이라도 다르면 해시가 완전히 달라집니다. 그래서 같은 변경이라도 작성 시각이나 작성자가 다르면 해시가 다릅니다.

해시는 40자리이지만, 보통 앞 7자( a1b2c3d )만으로도 한 저장소 안에서 커밋을 유일하게 가리키기에 충분합니다. 명령에서 커밋을 지목할 때도 짧은 해시를 그대로 쓸 수 있습니다.

```
$ git show a1b2c3d      # 짧은 해시로 그 커밋을 지목
```

## HEAD: 지금 내가 있는 위치

**HEAD**는 "지금 내가 보고 있는 커밋"을 가리키는 포인터입니다. 보통 HEAD는 현재 브랜치(예: main)를 가리키고, 그 브랜치가 최신 커밋을 가리킵니다. 그래서 `git log`에서 맨 위 커밋에 (HEAD -> main)이 붙는 것입니다. 새 커밋을 만들면 HEAD가 그 새 커밋으로 함께 전진합니다.

HEAD를 기준으로 과거 커밋을 상대적으로 가리킬 수도 있습니다.

| 표기              | 의미            |
|-----------------|---------------|
| HEAD            | 현재 커밋         |
| HEAD~1 또는 HEAD~ | 현재의 부모(한 칸 전) |
| HEAD~2          | 두 칸 전         |
| HEAD~3          | 세 칸 전         |

```
$ git show HEAD~1      # 현재 커밋의 바로 직전(부모) 커밋을 본다
```

## 부모 커밋: 이력을 잇는 사슬

모든 커밋(첫 커밋만 빼고)은 **부모 커밋**을 가집니다. 부모란 그 커밋의 바로 직전 커밋입니다. 커밋들은 이렇게 자식 → 부모 → 부모의 부모로 이어진 **사슬**을 이룹니다. 첫 커밋은 부모가 없어서 `git log`에서 `root-commit`이라 표시됩니다(ch\_03에서 본 그 표시입니다).

```
a1b2c3d (부모 없음, root-commit)
  ↑ 부모
b2c3d4e
  ↑ 부모
c3d4e5f ← HEAD -> main (현재)
```

HEAD는 이 사슬의 어느 지점을 "현재"로 가리키는 손가락이고, 각 커밋은 부모를 통해 과거로 연결됩니다. 이 사슬 구조를 이해하면 HEAD~2 같은 표기와 다음 장의 브랜치(같은 사슬에서 갈라지는 또 다른 손가락)가 자연스럽게 이해됩니다.

## 4. worked example: log로 이력 훑고 show로 한 커밋 들여다보기

### 상황

커밋 세 개가 쌓인 저장소에서, "두 번째 커밋이 정확히 무엇을 바꿨는지" 확인하려 합니다.

### 명령과 결과

먼저 `git log --oneline` 으로 목차를 훑어 대상 커밋의 해시를 찾습니다.

```
$ git log --oneline
c3d4e5f (HEAD -> main) secret.env 추적 제외 및 gitignore 추가
b2c3d4e README 소개 문단 작성
a1b2c3d 프로젝트 초기 구조 추가
```

"README 소개 문단 작성" 커밋( b2c3d4e )을 들여다보고 싶습니다. `git show <해시>` 로 그 커밋 하나의 변경을 펼칩니다.

```
$ git show b2c3d4e
commit b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1
Author: Hong Gildong <gildong@example.com>
Date: Mon Jun 16 10:38:02 2025 +0900

    README 소개 문단 작성

diff --git a/README.md b/README.md
index e69de29..a1c2b3d 100644
--- a/README.md
+++ b/README.md
@@ -1,3 @@
-# My Site
+
+간단한 소개 웹사이트 프로젝트입니다.
```

- `git show` 는 위쪽에 커밋 정보(해시·작성자·날짜·메시지)를, 아래쪽에 그 커밋이 만든 **변경(diff)** 을 함께 보여 줍니다.
- `diff --git a/README.md b/README.md` 는 "README.md의 변경"이라는 머리말입니다. `a/` 는 변경 전, `b/` 는 변경 후를 뜻합니다.
- `@@ -1,3 @@` 은 "변경 전 1번째 줄 부근, 변경 후 1~3번째 줄"이라는 위치 안내(hunk header)입니다.
- 맨 앞에 `+` 가 붙은 줄은 **추가된 줄**입니다(여기서는 빈 줄과 소개 문장 두 줄이 추가됨). 만약 삭제된 줄이 있으면 `-` 로 표시됩니다.

이렇게 `log` 로 목차를 잡고(어떤 커밋이 있나) → `show` 로 본문을 펼치는(그 커밋이 무엇을 바꿨나) 흐름이 이력 탐색의 기본 콤보입니다.

## 실행 결과 정리

이 worked example의 실행 결과를 요약하면, `git log --oneline` 으로 세 커밋의 목차를 확인했고, `git show b2c3d4e` 로 두 번째 커밋이 README에 소개 문장 두 줄을 추가했다는 사실을 줄 단위로 확인했습니다. "누가.언제.무엇을"이 한 화면에 모두 드러납니다.

## 한 걸음 더: `log -p`로 이력과 변경을 한 번에

`git show` 가 커밋 하나를 펼친다면, `git log -p` 는 여러 커밋을 거슬러 올라가며 각 커밋의 변경(diff)까지 차례로 보여 줍니다. "최근 며칠 사이 이 파일이 어떻게 진화했는지"를 따라갈 때 유용합니다.

```
$ git log -p -2 README.md
commit c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2 (HEAD -> main)
Author: Hong Gildong <gildong@example.com>
Date:   Mon Jun 16 10:42:11 2025 +0900

    secret.env 추적 제외 및 gitignore 추가

diff --git a/README.md b/README.md
@@ -1,3 +1,4 @@
     # My Site

    간단한 소개 웹페이지 프로젝트입니다.
    +연락처: gildong@example.com

commit b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1
Author: Hong Gildong <gildong@example.com>
Date:   Mon Jun 16 10:38:02 2025 +0900

    README 소개 문단 작성

diff --git a/README.md b/README.md
@@ -1 +1,3 @@
     # My Site
    +
    +간단한 소개 웹페이지 프로젝트입니다.
```

- `git log -p` 는 커밋 정보 다음에 그 커밋의 diff를 함께 보여 줍니다. `-2` 로 최근 2개만, `README.md` 로 그 파일만 좁혔습니다.
- 위에서 아래로 최신 → 과거 순이라, README가 "제목만 있던 상태 → 소개 문단 추가 → 연락처 추가"로 자라 온 흐름이 그대로 읽힙니다.

- 출력이 길면 페이지로 넘어가니, 다 봤으면 q로 빠져나오면 됩니다(다음 절에서 다룹니다).

이처럼 `log` (목차) · `show` (한 커밋) · `log -p` (여러 커밋 + 변경) · `diff` (아직 커밋 안 한 변경)를 상황에 맞게 골라 쓰면, 저장소의 과거와 현재를 자유롭게 진단할 수 있습니다.

## 5. git diff(작업트리 vs 스테이징)와 --staged

`git show`가 "이미 커밋된" 변경을 보는 도구라면, `git diff`는 주로 "아직 커밋되지 않은" 변경을 보는 도구입니다. 핵심은 **diff가 무엇과 무엇을 비교하는가**입니다. ch\_03의 3영역이 여기서 좌표로 쓰입니다.

| 명령                             | 비교 대상          | 한마디로                       |
|--------------------------------|----------------|----------------------------|
| <code>git diff</code>          | 작업트리 vs 스테이징   | "add 하지 않은 변경"             |
| <code>git diff --staged</code> | 스테이징 vs 마지막 커밋 | "add 했지만 아직 commit 안 한 변경" |
| <code>git diff HEAD</code>     | 작업트리 vs 마지막 커밋 | "마지막 커밋 이후 모든 변경"          |

가장 중요한 함정은 `git diff` (옵션 없음)는 기본적으로 "작업트리 vs 스테이징"만 보여 준다는 점입니다. 즉, 이미 `git add`한 변경은 그냥 `git diff`에는 나오지 않습니다. 이걸 모르면 "분명 고쳤는데 diff에 안 보인다"며 당황하게 됩니다.

```
$ # README.md를 수정한 직후 (아직 add 전)
$ git diff
diff --git a/README.md b/README.md
index a1c2b3d..f4e5d6c 100644
--- a/README.md
+++ b/README.md
@@ -1,3 +1,4 @@
 # My Site
```

```
간단한 소개 웹사이트 프로젝트입니다.
+연락처: gildong@example.com
```

이제 이 변경을 `git add`한 뒤 다시 `git diff`를 실행하면 아무것도 나오지 않습니다. 변경이 스테이징으로 올라가, "작업트리 vs 스테이징" 사이에는 차이가 없어졌기 때문입니다. 이때는 `--staged`로 봐야 합니다.



```
$ git add README.md
$ git diff
$ # (아무 출력도 없음 — 작업트리와 스테이징이 같아짐)

$ git diff --staged
diff --git a/README.md b/README.md
index a1c2b3d..f4e5d6c 100644
--- a/README.md
+++ b/README.md
@@ -1,3 +1,4 @@
# My Site
```

간단한 소개 웹사이트 프로젝트입니다.  
+연락처: gildong@example.com

- add 전에는 변경이 작업트리에만 있으므로 `git diff` 에 보입니다.
- add 후에는 변경이 스테이징으로 옮겨져 "작업트리 vs 스테이징" 차이가 사라지므로 `git diff` 가 빈 출력을 냅니다(이상한 게 아니라 정상입니다).
- `git diff --staged` (또는 같은 뜻의 `--cached`) 는 "스테이징 vs 마지막 커밋"을 비교하므로, add 한 변경을 여기서 볼 수 있습니다.
- 커밋 전에 "지금 커밋하면 무엇이 들어갈까?"를 확인하려면 `git diff --staged` 가 정답입니다.

## 6. 범위 비교: `git diff / main..feature`

`git diff` 는 두 커밋이나 두 브랜치 사이의 차이도 비교할 수 있습니다. 커밋 두 개의 해시를 나란히 주면 그 사이의 변화를 보여 줍니다.

```
$ git diff a1b2c3d c3d4e5f
```

이것은 "a1b2c3d 시점에서 c3d4e5f 시점까지 무엇이 바뀌었는가"를 보여 줍니다. 첫 커밋과 최신 커밋을 비교하면 그동안의 모든 변화를 한 번에 볼 수 있습니다.

브랜치 사이도 마찬가지로입니다. 점 두 개(..) 표기를 자주 씁니다(브랜치는 ch\_05에서 배웁니다).

```
$ git diff main..feature
```

`main..feature` 는 "main 기준으로 feature에 무엇이 더해졌는가"를 비교합니다. 다음 단원의 미니 프로젝트에서 "feature 브랜치가 main과 무엇이 다른지" 확인할 때 바로 이 명령을 씁니다.

```
$ git diff HEAD~1 HEAD      # 직전 커밋과 현재 커밋의 차이(= 마지막 커밋이 바꾼 것)
$ git diff HEAD~2 HEAD      # 두 커밋 전과 현재의 차이
```

- `git diff HEAD~1 HEAD` 는 "부모 커밋과 현재 커밋"의 차이, 사실상 `git show HEAD` 와 같은 변경을 보여 줍니다.
- 범위 비교는 "이번 작업 전체가 무엇을 바꿨나"를 검토할 때 유용합니다. 예를 들어 PR(ch\_13)을 올리기 전 `git diff main..내브랜치` 로 전체 변경을 한 번 훑어보는 습관이 좋습니다.

## diff를 더 읽기 좋게: --stat과 파일 지정

변경이 많을 때는 줄 단위 전체 diff가 너무 길어 한눈에 안 들어옵니다. 이때 `--stat` 을 붙이면 "어떤 파일이 몇 줄 바뀌었는지" 요약만 보여 줍니다.

```
$ git diff --stat HEAD~2 HEAD
README.md      | 4 +++-
index.html     | 6 ++++++
.gitignore     | 5 +++++
3 files changed, 14 insertions(+), 1 deletion(-)
```

- 각 줄은 "파일 이름 | 바뀐 줄 수"를 보여 주고, `+/-` 막대의 길이로 추가·삭제 규모를 한 눈에 가늠하게 합니다.
- 맨 아래 `3 files changed, 14 insertions(+), 1 deletion(-)` 는 전체 요약입니다. "큰 그림 먼저(`--stat`) → 궁금한 파일만 자세히"가 효율적인 읽기 순서입니다.

특정 파일의 변경만 보고 싶으면 명령 끝에 파일 이름을 붙입니다. 여러 영역·범위 표기와 자유롭게 조합됩니다.

```
$ git diff README.md          # README.md의 작업트리 변경만
$ git diff --staged README.md # README.md의 스테이징 변경만
$ git diff HEAD~1 HEAD index.html # 직전 커밋이 index.html에 한 변경만
```

이렇게 "비교 대상(작업트리·스테이징·커밋·범위) × 좁히기(파일·`--stat`)"를 조합하면, 원하는 변경을 정확히 겨냥해 읽을 수 있습니다.

## 7. 흔한 문제·해결: 고쳤는데 diff에 안 보임·log 페이지 멈춤

입문자가 `log·diff`에서 가장 자주 마주치는 두 가지 상황을 정면으로 다룹니다.

### 문제 1 — "분명히 고쳤는데 git diff에 아무것도 안 나와요"

**상황:** 파일을 고쳤는데 `git diff` 가 빈 화면입니다.

**진단:** 십중팔구 그 변경을 이미 `git add` 했기 때문입니다. `git diff` 는 "작업트리 vs 스테이징"만 보므로, 이미 스테이징된 변경은 보이지 않습니다.

**해결:** 어느 영역에 변경이 있는지 `git status` 로 먼저 확인하고, 스테이징된 변경은 `git diff --staged` 로 봅니다.

```
$ git status
On branch main
Changes to be committed:
  modified:   README.md

$ git diff
$ # (빈 출력 — 변경이 스테이징에 있음)
$ git diff --staged
diff --git a/README.md b/README.md
@@ -1,3 +1,4 @@
+연락처: gildong@example.com
```

Changes to be committed (초록)에 파일이 있다는 것은 스테이징된 상태라는 뜻이니, `--staged` 로 봐야 한다는 신호입니다.

## 문제 2 — "git log를 쳤더니 화면이 멈췄어요"

**상황:** `git log` 를 실행했더니 커밋 목록이 뜬 채 멈춘 듯하고, 어떤 키를 눌러도 명령이 안 먹는 것 같습니다.

**진단:** 멈춘 것이 아닙니다. 출력이 길면 Git은 **페이저**(less)라는 프로그램으로 화면 단위로 보여 줍니다. 지금은 그 페이저 화면 안에 들어가 있는 상태입니다.

**해결:** 페이저 안에서는 다음 키를 씁니다.

| 키        | 동작                |
|----------|-------------------|
| q        | 페이저를 빠져나오기(가장 중요) |
| 스페이스 · f | 다음 화면으로           |
| b        | 이전 화면으로           |
| /단어      | 단어 검색             |
| ↑ ↓      | 한 줄씩 위아래          |

q 만 기억하면 됩니다. q 를 누르면 페이지에서 나와 다시 평소의 명령 프롬프트로 돌아옵니다.

```
$ git log
... (커밋 목록이 화면을 채우고 맨 아래 ':' 커서가 깜빡임)
:      ← 여기서 q 를 누르면 빠져나옴
$
```

팁: 페이지가 번거롭다면 짧은 출력을 쓰면 페이지가 안 뜹니다. `git log --oneline -10` 처럼 개수를 제한하거나, 한 화면에 들어오는 출력은 그냥 위로 스크롤해서 볼 수 있습니다. 그래도 화면 아래에 `:` 가 보이면 페이지 안이라는 신호이니 q 를 떠올리십시오.

## 8. 즉시 연습 (2종 1세트)

### 직접 작성 1 — log로 이력 훑고 한 커밋 들여다보기 (빈칸 채우기)

커밋이 여러 개 쌓인 저장소에서, 이력을 한 줄로 훑은 뒤 특정 커밋의 변경을 확인하는 과정을 완성하십시오. 빈칸에 알맞은 명령·옵션을 직접 작성하면 됩니다.

```
$ git log _____ # 커밋을 한 줄씩 압축해 보는 옵션
$ git log _____ # 모든 브랜치를 그래프로 보는 두 옵션(--graph 포함)
$ git ____ b2c3d4e   # b2c3d4e 커밋 하나의 변경을 펼쳐 보는 명령
$ git show _____ # 현재 커밋의 '부모'를 상대 표기로 지목(HEAD 기준)
```

### 자가체크 체크리스트

- [ ] 첫 빈칸에 `--oneline` 을 넣어 한 줄 요약으로 봤는가
- [ ] 둘째 빈칸에 `--graph --all` 을 넣어 전체 지도를 봤는가
- [ ] 셋째 빈칸에 `show` 를 넣어 특정 커밋을 펼쳤는가
- [ ] 넷째 빈칸에 `HEAD~1` 을 넣어 부모 커밋을 지목했는가

### 직접 작성 2 — diff로 어느 영역의 변경인지 진단하기 (빈칸 채우기)

파일을 고치고 일부를 add 한 상황에서, 작업트리와 스테이징의 변경을 각각 확인하는 과정을 완성하십시오.

```
$ # file.txt를 수정한 직후 (아직 add 전)
$ git ____          # add 안 한 변경(작업트리 vs 스테이징)을 보는 명령
$ git add file.txt
$ git diff          # 이제 빈 출력이 나오는 이유를 설명해 보기
$ git diff _____ # add 한 변경(스테이징 vs 마지막 커밋)을 보는 옵션
$ git status        # 변경이 어느 영역에 있는지 교차 확인
```

## 자가체크 체크리스트 (스스로 점검)

- [ ] 첫 빈칸에 `diff` 를 넣어 작업트리 변경을 봤는가
- [ ] `add` 후 `git diff` 가 비는 이유를 "변경이 스테이징으로 옮겨져서"로 설명할 수 있는가
- [ ] 둘째 빈칸에 `--staged` (또는 `--cached`)를 넣어 스테이징 변경을 봤는가
- [ ] `git status` 의 초록/빨강 묶음과 `diff` 결과가 서로 들어맞는지 확인했는가

## 9. 흔한 오류와 오개념 교정

### 오개념 1 — "고친 게 안 보이면 git diff가 고장 난 것이다"

`git diff` (옵션 없음)는 작업트리 vs 스테이징만 보여 줍니다. 이미 `git add` 한 변경은 여기에 나오지 않습니다.

```
$ git add README.md
$ git diff
$ # (빈 출력 — 고장이 아니라 정상)
```

**교정:** `add` 한 변경은 `git diff --staged` 로 봅니다. 어느 영역에 변경이 있는지는 늘 `git status` 로 먼저 확인하는 습관을 들이십시오. 빨강(작업트리)은 `git diff`, 초록(스테이징)은 `git diff --staged` 입니다.

### 오개념 2 — "git log가 멈췄다(프로그램이 죽었다)"

출력이 길면 Git은 페이지(`less`)로 보여 주는데, 이를 멈춤으로 오해하기 쉽습니다. 화면 아래 `:` 가 보이면 페이지 안입니다.

**교정:** `q` 를 누르면 페이지에서 빠져나옵니다. 스페이스로 다음 화면, `b` 로 이전 화면을 봅니다. 죽은 것이 아니라 "읽기 모드"에 들어간 것뿐입니다.

### 오개념 3 — "커밋 해시는 아무렇게나 붙은 번호다"

해시는 무작위 번호가 아니라 커밋 내용(파일·메시지·작성자·시각·부모)으로 계산된 **지문**입니다. 그래서 내용이 같아 보여도 시각·작성자가 다르면 해시가 달라지고, 반대로 해시가 같으면 내용이 같다는 강한 보장이 됩니다.

**교정:** 커밋을 지목할 때는 짧은 해시(앞 7자)면 충분합니다. 해시가 "내용의 지문"이라는 점은 이후 충돌·되돌리기에서 "같은 변경인데 왜 해시가 다른가"를 이해하는 열쇠가 됩니다.

## 오개념 4 — "git diff와 git show는 같은 것이다"

둘 다 변경을 보여 주지만 대상이 다릅니다. `git diff` 는 주로 **아직 커밋되지 않은** 변경(작업트리·스테이징)을, `git show` 는 **이미 커밋된** 특정 커밋 하나의 변경을 봅니다.

**교정:** "지금 작업 중인 변경"은 `git diff`, "과거 커밋이 무엇을 바꿨나"는 `git show <해시>` 로 기억하십시오. 둘을 함께 쓰면 현재와 과거를 모두 진단할 수 있습니다.

## 10. 자주 묻는 질문 (FAQ)

**Q1. `git diff --staged` 와 `git diff --cached` 는 다른가요?** A. 완전히 같습니다. `--cached` 가 오래된 이름이고 `--staged` 가 나중에 추가된 더 직관적인 이름입니다. 어느 것을 써도 "스테이징 vs 마지막 커밋"을 비교합니다. 의미가 분명한 `--staged` 를 권장합니다.

**Q2. 특정 파일의 이력만 보고 싶습니다.** A. `git log <파일이름>` 처럼 파일을 지정하면 그 파일을 건드린 커밋만 보여 줍니다. `git log --oneline README.md` 처럼 옵션과 함께 쓸 수 있고, `git log -p README.md` 로 그 파일의 변경 내용까지 함께 볼 수 있습니다.

**Q3. 해시를 매번 길게 복사해야 하나요?** A. 아닙니다. 한 저장소 안에서 충돌하지 않는 한 앞 7자 정도의 짧은 해시면 충분합니다. 또 `HEAD`, `HEAD~1`, 브랜치 이름, 태그 이름으로도 커밋을 가리킬 수 있어, 해시를 직접 쓰는 일은 생각보다 많지 않습니다.

**Q4. `git log --graph` 가 지금은 그냥 직선인데, 언제 갈라지나요?** A. 브랜치를 만들어 따로 커밋하면 갈래가 생기고, 병합하면 다시 합쳐집니다(`ch_05·ch_06`에서 배웁니다). 그때 `git log --oneline --graph --all` 을 실행하면 갈라지고 합쳐지는 그림이 나타납니다. 지금 직선인 것은 아직 브랜치가 하나뿐이기 때문입니다.

**Q5. 특정 기간의 커밋만 보고 싶습니다.** A. `--since` 와 `--until` 로 기간을 좁힐 수 있습니다. 예를 들어 `git log --since="2 days ago"` 는 최근 이틀 안의 커밋만, `git log --since="2025-06-01" --until="2025-06-16"` 은 그 기간의 커밋만 보여 줍니다. `--author` (작성자)·파일 이름과 함께 조합하면 "지난주에 내가 README에 한 작업"처럼 아주 구체적으로 좁혀 읽을 수 있습니다.

**Q6. 커밋 메시지로 검색할 수 있나요?** A. `git log --grep="검색어"` 를 쓰면 커밋 메시지에 그 단어가 든 커밋만 거릅니다. 예를 들어 `git log --oneline --grep="버그"` 는 메시지에 "버그"가 들어간 커밋만 한 줄씩 보여 줍니다. "그 수정이 어느 커밋이었더라?"를 찾을 때 유용합니다.

## 11. 마무리

이 장에서는 쌓인 커밋 이력을 읽는 두 도구를 익혔습니다. `git log` 는 이력의 목차를 보여 주며, `--oneline` 으로 압축하고 `--graph --all` 로 전체 지도를 그리며 `-n` · `--author` 로 좁혀 읽을 수 있습니다. `git diff` 는 변경의 내용을 줄 단위로 보여 주되, **무엇과 무엇을 비교하는지**가 핵심이라 `git diff` (작업트리 vs 스테이징)와 `git diff --staged` (스테이징 vs 커밋)를 구분해야 합니다. `git show` 로는 과거의 특정 커밋 하나를 펼쳐 볼 수 있었습니다.

또한 커밋 해시(SHA)는 내용의 지문이고, HEAD는 현재 위치를 가리키는 포인터이며, 커밋들은 부모를 통해 사슬로 이어진다는 구조를 확인했습니다. 그리고 "고쳤는데 diff에 안 보임"(이미 add 됨)과 "log가 멈춤"(페이지, q로 탈출) 같은 가장 흔한 두 함정을 진단·해결하는 법을 배웠습니다.

이력을 읽는 힘은 앞으로의 모든 작업의 토대입니다. 다음 장(ch\_05)에서는 `git branch` 와 `git switch` 로 본류(main)를 안전하게 둔 채 따로 실험하는 **분기**를 배웁니다. 이때 이 장에서 익힌 `git log --oneline --graph --all` 이 "지금 어느 브랜치가 어디에 있는지"를 비추는 가장 중요한 진단 도구가 됩니다. 읽는 힘이 있어야, 갈라지고 합쳐지는 협업의 세계를 두려움 없이 누빌 수 있습니다.





# 분기 만들기: git branch와 git switch

## 이 챕터에서 배우는 것

- 브랜치가 "특정 커밋을 가리키는 가벼운 이름표(포인터)"이고 HEAD가 "지금 내가 있는 브랜치"임을 설명할 수 있습니다.
- `git branch` 로 브랜치를 만들고, 목록을 확인하고, `-d` 로 삭제할 수 있습니다.
- `git switch` 로 브랜치를 갈아타고, `git switch -c` 로 만들면서 한 번에 이동할 수 있습니다.
- `git switch` 와 옛 `git checkout` 의 관계, 그리고 `checkout` 이 가졌던 두 역할이 어떻게 나뉘었는지 설명할 수 있습니다.
- 기능 브랜치 워크플로우와 브랜치 이름 짓기 관례를 적용할 수 있습니다.
- 커밋하지 않은 변경 때문에 전환이 거부될 때, 원인을 진단하고 커밋 또는 `git stash` 로 해결할 수 있습니다.

이 챕터의 핵심 개념은 **분기** — **branch**와 **switch** 한 가지입니다. 선수 개념은 바로 앞 챕터(ch\_04)의 **이력 읽기** — **log**와 **diff**입니다. 브랜치는 결국 "커밋을 가리키는 이름표"이기 때문에, ch\_04에서 익힌 `git log --oneline --graph` 로 "지금 어느 브랜치가 어느 커밋을 가리키는지"를 눈으로 확인하는 장면이 끊임없이 등장합니다. 커밋 해시·HEAD·부모 커밋이라는 ch\_04의 어휘가 여기서 곧바로 도구가 됩니다.

## 5.1 도입: main을 지키며 따로 실험하기

지금까지 우리는 하나의 줄기(main) 위에 커밋을 차곡차곡 쌓았습니다. 그런데 실제 개발에서는 이런 상황이 자주 생깁니다. "지금 잘 돌아가는 소개 페이지(main)는 그대로 둔 채, 새 'About' 섹션을 한번 실험해 보고 싶다. 잘되면 합치고, 망하면 통째로 버리고 싶다."

만약 브랜치가 없다면 어떻게 할까요? ch\_01에서 비웃었던 그 방법, 즉 폴더를 통째로 복사해서 `프로젝트_About실험본` 같은 사본을 만드는 길로 되돌아가게 됩니다. 사본은 원본과 따로 놀아 나중에 어느 줄이 진짜인지 헷갈리고, 합치는 일은 손으로 복사·붙여넣기 하는 악몽이 됩니다.

**브랜치(branch)**는 바로 이 문제를 해결합니다. 같은 폴더 안에서, 같은 이력 위에서, main을 안전하게 둔 채로 새로운 갈래를 따로 파서 실험하고, 끝나면 깔끔하게 다시 합칠 수 있게 해 줍니다.

다. 그것도 폴더 복사 없이, 거의 즉시, 아주 가볍게 말입니다.

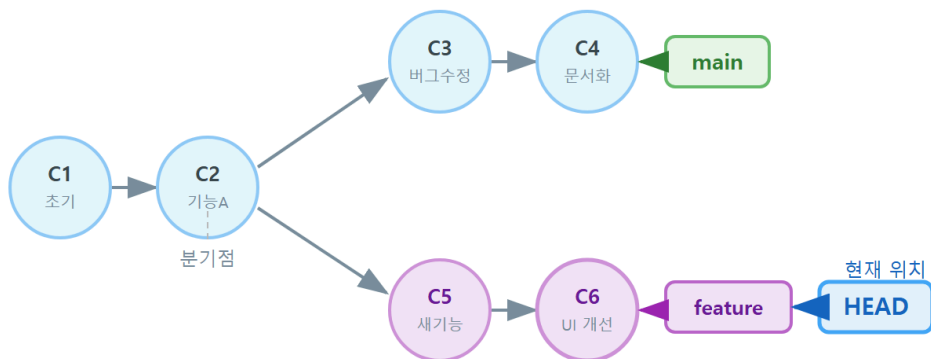
### 한눈에 보는 브랜치

브랜치(branch)는 커밋 이력에서 갈라져 나온 독립된 작업 갈래입니다. 사실 그 정체는 **특정 커밋을 가리키는 가벼운 이름표(포인터)**이고, **HEAD**는 "지금 내가 있는 브랜치"를 가리킵니다. `main` 을 건드리지 않고 새 기능을 따로 개발하려면 `git switch -c <이름>` 으로 브랜치를 만들며 이동하고, `git switch <이름>` 으로 돌아옵니다. 이는 옛 `git checkout` 의 "브랜치 전환" 역할을 떴어 낸 신명령입니다.

브랜치를 자유롭게 쓸 수 있게 되면 작업 습관 자체가 달라집니다. "이걸 건드렸다가 `main` 이 깨지면 어쩌지" 하는 두려움이 사라지고, 부담 없이 시도하고, 실패하면 갈래째 버리는 실험 정신이 생깁니다. 이것이 협업과 안전한 실험의 토대이며, 다음 chapter의 병합(merge), 그리고 종합 프로젝트의 PR 협업으로 곧장 이어집니다.

## 5.2 동작 원리: 브랜치=포인터, HEAD=현재 브랜치

브랜치 = 포인터, HEAD = 현재 브랜치



브랜치는 커밋을 가리키는 이름표(포인터) — 새 커밋을 만들면 포인터가 자동으로 앞으로 이동  
HEAD = 내가 지금 보고 있는 브랜치 (`git switch`로 변경)

브랜치를 처음 배울 때 가장 흔한 오해는 "브랜치는 코드의 복사본"이라는 생각입니다. 사실은 정반대입니다. 브랜치는 무겁게 복사된 무엇이 아니라, **딱 하나의 커밋을 가리키는 41바이트짜리 이름표**입니다. Git 내부에서 `main` 이라는 브랜치는 `.git/refs/heads/main` 파일에 커밋 해시 한 줄이 적혀 있는 것이 전부입니다.

그림으로 이해해 봅시다. 커밋은 부모에서 자식으로 이어진 사슬입니다. 브랜치는 그 사슬의 한 지점을 손가락으로 가리키는 포인터입니다.

```
(A) --- (B) --- (C)    <- main 이 가리키는 곳
                  ^
                HEAD -> main
```

여기서 두 가지 포인터가 등장합니다.

- **브랜치 포인터:** `main` 은 커밋 `C` 를 가리킵니다. 새 커밋을 만들면 `main` 은 자동으로 새 커밋으로 한 칸 전진합니다.
- **HEAD:** "지금 내가 있는 브랜치"를 가리키는 특별한 포인터입니다. 보통 HEAD는 어떤 브랜치 이름을 가리키고(예: `HEAD -> main`), 그 브랜치가 다시 커밋을 가리킵니다.

이제 `feature` 라는 새 브랜치를 `C` 에서 만들면 이렇게 됩니다.

```
(A) --- (B) --- (C)    <- main, feature (둘 다 C를 가리킴)
                  ^
                HEAD -> feature (switch 후)
```

브랜치를 만든 직후에는 `main` 과 `feature` 가 **같은 커밋 C**를 가리킵니다. 파일은 한 글자도 복사되지 않았습니다. 달라진 것은 "이름표가 하나 늘었다"는 것뿐입니다. 그래서 브랜치 생성이 그토록 빠르고 가벼운 것입니다.

이 상태에서 `feature` 로 전환(`HEAD -> feature`)한 뒤 커밋 `D` 를 만들면, **HEAD가 가리키는 브랜치(feature)만** 전진합니다.

```
(A) --- (B) --- (C)          <- main
                  \
                  (D)        <- feature
                  ^
                HEAD -> feature
```

`main` 은 여전히 `C` 에 머물러 있고, `feature` 만 `D` 로 나아갔습니다. 바로 이것이 "main을 안전하게 둔 채 따로 실험한다"의 정확한 의미입니다. 핵심을 정리하면 다음과 같습니다.

| 용어              | 정체                       | 비유                          |
|-----------------|--------------------------|-----------------------------|
| 커밋(commit)      | 변경 스냅샷(고정·불변)            | 사진 한 장                      |
| 브랜치<br>(branch) | 특정 커밋을 가리키는 포인터<br>(움직임) | 사진에 붙인 포스트잇 이름표             |
| HEAD            | 현재 브랜치를 가리키는 포인터         | "지금 보고 있는 이름표"를 가리키는<br>화살표 |

이 세 가지 관계를 손에 익히면, 앞으로 배울 병합·되돌리기·rebase가 전부 "포인터를 어디로 옮기느냐"의 이야기라는 것이 보입니다.

## 5.3 git branch: 생성·목록·삭제(-d)

`git branch` 명령은 이름과 달리 여러 일을 합니다. 인자를 어떻게 주느냐에 따라 목록을 보여 주기도 하고, 만들기도 하고, 지우기도 합니다.

| 명령                                                 | 하는 일                        |
|----------------------------------------------------|-----------------------------|
| <code>git branch</code>                            | 로컬 브랜치 목록 출력(현재 브랜치는 *로 표시) |
| <code>git branch &lt;이름&gt;</code>                 | <이름> 브랜치를 생성(이동은 하지 않음)     |
| <code>git branch -d &lt;이름&gt;</code>              | 병합이 끝난 브랜치를 안전하게 삭제         |
| <code>git branch -D &lt;이름&gt;</code>              | 병합 여부와 무관하게 강제 삭제(주의)       |
| <code>git branch -m &lt;옛이름&gt; &lt;새이름&gt;</code> | 브랜치 이름 변경                   |

여기서 가장 헷갈리는 점은 `git branch <이름>` 은 브랜치를 만들지만 하고 그쪽으로 이동시키지는 않는다는 것입니다. 만든 뒤에도 HEAD는 여전히 원래 브랜치에 남아 있습니다. 그래서 실무에서는 "만들며 이동"을 한 번에 하는 `git switch -c` (5.4절)를 훨씬 자주 씁니다.

삭제할 때 `-d` (소문자)와 `-D` (대문자)의 차이도 중요합니다. `-d` 는 "이 브랜치의 작업이 이미 다른 곳(예: main)에 병합되어 안전하게 지워도 되는가"를 확인하고, 아직 합쳐지지 않았으면 거부

합니다. 이는 실수로 작업을 통째로 잃는 것을 막아 주는 안전장치입니다. 정말로 버릴 작업이라면 `-D`로 강제할 수 있습니다.

```
git branch
git branch feature/about-page
git branch
```

```
* main
* main
  feature/about-page
```

위에서 첫 `git branch`는 `main` 하나만 보여 주고, `feature/about-page`를 만든 뒤 다시 실행하니 두 개가 보입니다. `*`가 여전히 `main`에 있다는 점에 주목하세요. 브랜치를 만들었어도 우리는 아직 `main`에 있습니다.

## 5.4 worked example: switch -c로 기능 브랜치 파고 돌아 오기

이제 가장 표준적인 흐름을 한 번에 따라가 보겠습니다. 상황은 이렇습니다. 소개 웹페이지 저장소(main)에 'About' 섹션을 실험하려 합니다. main을 망가뜨리지 않도록 `feature/about-page` 브랜치에서 작업하고, 끝나면 main으로 돌아와 main이 그대로인지 확인하려 합니다.

### 상황 → 명령

```
git switch -c feature/about-page
git branch
```

```
Switched to a new branch 'feature/about-page'
main
* feature/about-page
```

`git switch -c`는 브랜치를 만들면서 동시에 그쪽으로 이동합니다. 출력의 `Switched to a new branch`가 생성과 이동이 한 번에 일어났음을 알려 주고, `git branch`의 `*`가 이제 `feature/about-page`로 옮겨졌습니다.

이 브랜치에서 About 섹션을 추가하고 커밋합니다.

```
git add about.html
git commit -m "feat: About 소개 섹션 추가"
git log --oneline
```

```
[feature/about-page 7a1c9e2] feat: About 소개 섹션 추가
 1 file changed, 12 insertions(+)
7a1c9e2 (HEAD -> feature/about-page) feat: About 소개 섹션 추가
3f2b6a0 (main) docs: README 소개 작성
9d4e1c8 프로젝트 초기 구조 추가
```

`git log --oneline`의 첫 줄을 보면 `HEAD -> feature/about-page`가 새 커밋 `7a1c9e2`를 가리키고, `main`은 여전히 한 칸 아래 `3f2b6a0`에 머물러 있습니다. 5.2절에서 본 그림 그대로, **HEAD가 가리키는 브랜치만 전진한 것**입니다.

## 진단: main은 정말 그대로일까

이제 `main`으로 돌아와 확인합니다.

```
git switch main
git log --oneline
```

```
Switched to branch 'main'
3f2b6a0 (HEAD -> main) docs: README 소개 작성
9d4e1c8 프로젝트 초기 구조 추가
```

`git switch main`의 결과는 `Switched to branch 'main'` (이번엔 `a new branch`가 아니라 기존 브랜치로의 전환)입니다. 그리고 `main`의 `git log`에는 방금 만든 About 커밋이 **보이지 않습니다**. About 작업은 `feature/about-page` 갈래에만 있고 `main`은 안전하게 그대로입니다. 이것이 우리가 원했던 결과입니다.

## 결과: 두 갈래를 그래프로 확인

ch\_04에서 배운 `--all --graph`로 두 갈래를 한눈에 봅니다.

```
git log --oneline --all --graph
```

```
* 7a1c9e2 (feature/about-page) feat: About 소개 섹션 추가
* 3f2b6a0 (HEAD -> main) docs: README 소개 작성
* 9d4e1c8 프로젝트 초기 구조 추가
```

- `git switch -c feature/about-page`: 브랜치 **생성 + 이동**을 한 번에 합니다. 출력 `Switched to a new branch`가 두 일이 동시에 일어났음을 확인해 줍니다.
- `git commit` 후 `HEAD -> feature/about-page`만 새 커밋으로 전진하고 `main`은 제자리입니다. 포인터가 따로 논다는 5.2절 원리를 실제 출력으로 확인한 셈입니다.
- `git switch main`으로 돌아오니 작업 폴더의 파일 내용이 `main` 시점으로 바뀌고, `About` 커밋은 로그에서 사라집니다(브랜치에만 존재).
- `--all --graph`는 모든 브랜치를 그래프로 그려, 어디서 갈라졌고 `HEAD`가 어디인지를 시각적으로 진단하게 해 줍니다.

**직전 브랜치로 빠르게:** `git switch -`

`main`과 `feature`를 자주 오갈 때는 `git switch -`가 편리합니다. 셸의 `cd -`처럼 **직전에 있던 브랜치로** 한 번에 돌아갑니다. 브랜치 이름을 매번 타이핑하지 않아도 됩니다.

## 5.5 git switch와 옛 checkout -b의 대응(두 역할 분리)

기존 자료나 검색 결과를 보면 브랜치 전환에 `git checkout`을 쓰는 경우가 많습니다. 둘의 관계를 정확히 알아 두어야 혼동하지 않습니다.

`git checkout`은 역사적으로 **두 가지 전혀 다른 일**을 한 명령에 묶여놓고 있었습니다.

1. 브랜치를 전환한다(`git checkout main`).
2. 파일을 특정 커밋 상태로 되돌린다(`git checkout -- index.html`).

이 두 역할이 한 명령에 섞여 있어 입문자에게 큰 혼란을 줬습니다. 그래서 Git 2.23부터 역할을 둘로 쪼갰습니다.

| 옛 명령(checkout)                           | 새 명령                                   | 역할                |
|------------------------------------------|----------------------------------------|-------------------|
| <code>git checkout &lt;브랜치&gt;</code>    | <code>git switch &lt;브랜치&gt;</code>    | 브랜치 전환            |
| <code>git checkout -b &lt;브랜치&gt;</code> | <code>git switch -c &lt;브랜치&gt;</code> | 브랜치 생성 + 전환       |
| <code>git checkout -- &lt;파일&gt;</code>  | <code>git restore &lt;파일&gt;</code>    | 파일을 마지막 커밋 상태로 복원 |

즉 `git switch`는 "브랜치 전환" 전담, `git restore`는 "파일 복원" 전담입니다(파일 복원은 `ch_08`에서 자세히 다룹니다). 의미가 명확하게 갈렸기 때문에 이 책은 전환에 `switch`를 사용합니다.

## checkout이 틀린 것은 아닙니다

`git checkout` 은 지금도 완전히 유효하며 사라지지 않았습니다. 회사 코드나 오래된 튜토리얼에서 `git checkout -b feature` 를 봐도 당황할 필요 없습니다. `git switch -c feature` 와 **똑같은 일**을 합니다. 다만 새로 배우는 입장에서는 역할이 분명한 `switch / restore` 를 쓰는 편이 헛갈리지 않습니다.

## 5.6 기능 브랜치 워크플로우와 이름 짓기 관례

브랜치를 "왜, 언제" 파는지에 대한 표준 작업 방식이 **기능 브랜치 워크플로우(feature branch workflow)**입니다. 핵심 규칙은 간단합니다.

main에는 직접 작업하지 않는다. 모든 새 기능·수정은 main에서 갈라낸 별도 브랜치에서 만들고, 완성되면 main으로 병합한다.

이렇게 하면 main은 언제나 "동작하는 안정 버전"으로 유지되고, 진행 중인 실험은 각자의 갈래에 격리됩니다. 한 바퀴를 단계로 적으면 다음과 같습니다.

1. main 에서 최신 상태를 확인한다(`git switch main`, 이후 단위에서는 `git pull`).
2. `git switch -c feature/<할일>` 로 기능 브랜치를 판다.
3. 그 브랜치에서 작업하고 의미 단위로 커밋한다.
4. 완성되면 `main` 으로 돌아와 병합한다(ch\_06).
5. 역할을 다한 브랜치를 `git branch -d` 로 정리한다.

브랜치 이름은 팀이 한눈에 의도를 알 수 있도록 관례를 따릅니다. 널리 쓰이는 접두어는 다음과 같습니다.

| 접두어      | 용도         | 예                     |
|----------|------------|-----------------------|
| feature/ | 새 기능       | feature/about-page    |
| fix/     | 버그 수정      | fix/login-typo        |
| hotfix/  | 운영 중 긴급 수정 | hotfix/crash-on-start |
| docs/    | 문서 작업      | docs/readme-update    |



이름은 소문자와 하이픈(-)으로 짧고 구체적으로 짓습니다. `feature/test` 나 `feature/new` 처럼 모호한 이름은 일주일만 지나도 무슨 작업이었는지 알 수 없게 됩니다. 종합 프로젝트(ch\_15~18)의 팀 협업 시뮬레이션에서도 이 이름 관례를 그대로 사용합니다.

## 5.7 흔한 문제·해결: 미커밋 변경으로 전환 거부 → 커밋/stash

브랜치를 처음 다룰 때 거의 모두가 한 번은 마주치는 상황입니다. **문제**는 이렇게 나타납니다. `feature/about-page` 에서 `about.html` 을 고치다가, 아직 커밋하지 않은 채로 `main` 으로 전환하려 했더니 Git이 거부합니다.

### 문제 발생

```
git switch main
```

```
error: Your local changes to the following files would be overwritten by checkout:
  about.html
Please commit your changes or stash them before you switch branches.
Aborting
```

### 진단

Git은 왜 막았을까요? `about.html` 은 두 브랜치에서 내용이 다른데, 작업트리에 **커밋되지 않은 변경**이 떠 있습니다. 이대로 전환하면 그 변경을 어디에 두어야 할지 Git이 알 수 없고, 잘못하면 변경이 덮여 사라질 수 있습니다. 그래서 데이터를 잃지 않도록 전환 자체를 멈춘 것입니다. 먼저 `git status` 로 무엇이 떠 있는지 확인합니다.

```
git status -s
```

```
M about.html
```

**M** 은 수정됐지만(modified) 아직 스테이징·커밋되지 않았다는 뜻입니다(ch\_02의 상태 기호). 이 변경을 어떻게든 "안전한 곳"에 뒤야 전환할 수 있습니다. 길은 두 가지입니다.

### 해결 1: 커밋한다(변경이 일단락된 경우)

작업이 한 단위로 마무리됐다면 그냥 커밋해서 `feature` 갈래에 박아 두면 됩니다.

```
git add about.html
git commit -m "feat: About 섹션 문구 다듬기"
git switch main
```

```
[feature/about-page 8b2d0f4] feat: About 섹션 문구 다듬기
1 file changed, 3 insertions(+), 1 deletion(-)
Switched to branch 'main'
```

커밋하니 작업트리가 깨끗해졌고, 전환이 정상적으로 됩니다.

## 해결 2: 잠시 치워 둔다(아직 커밋하기 애매한 경우)

아직 커밋할 만큼 정리되지 않았다면, `git stash`로 변경을 임시 선반에 치워 작업트리를 깨끗하게 만든 뒤 전환합니다(stash는 ch\_09에서 본격적으로 배웁니다).

```
git stash
git switch main
```

```
Saved working directory and index state WIP on feature/about-page: 8b2d0f4 feat: About 섹션 문구
Switched to branch 'main'
```

나중에 다시 `feature/about-page`로 돌아와 `git stash pop`으로 치워 둔 변경을 꺼내 이어 가면 됩니다.

### 전환이 항상 거부되는 것은 아닙니다

커밋하지 않은 변경이 있어도, 그 파일이 두 브랜치에서 동일하다면 Git은 변경을 그대로 들고 전환시켜 줍니다(거부하지 않습니다). 거부는 "전환하면 변경이 덮여 사라질" 때만 일어납니다. 그러니 거부 메시지는 에러가 아니라 데이터를 지키려는 안전장치로 받아들이고, 침착하게 `status` → 커밋 또는 `stash`로 처리하면 됩니다.

## 5.8 즉시 연습(2종 1세트): 브랜치 생성·전환·비교 채우기

배운 명령을 직접 손에 익힐 차례입니다. 먼저 **모범 풀이**를 본 뒤, 빈 템플릿을 직접 채워 보세요.

## 모범 풀이

main에서 `feature/contact` 브랜치를 만들어 전환하고, 변경을 커밋한 뒤 main으로 돌아와 갈래가 나뉘었는지 확인하는 한 바퀴입니다.

```
git switch -c feature/contact
git add contact.html
git commit -m "feat: 연락처 섹션 추가"
git switch main
git log --oneline --all --graph
```

```
Switched to a new branch 'feature/contact'
[feature/contact 1f9a3c7] feat: 연락처 섹션 추가
 1 file changed, 8 insertions(+)
Switched to branch 'main'
* 1f9a3c7 (feature/contact) feat: 연락처 섹션 추가
* 3f2b6a0 (HEAD -> main) docs: README 소개 작성
* 9d4e1c8 프로젝트 초기 구조 추가
```

HEAD -> main 이 분기 지점에 머물고, `feature/contact` 만 한 칸 위로 전진했습니다. 실행 결과의 그래프가 두 갈래로 갈린 것을 직접 확인하세요.

## 학생 작성형 빈 템플릿

### 직접 작성: 기능 브랜치 한 바퀴

다음 빈칸( \_\_\_\_\_ )을 채워 명령을 완성하고, 본인 저장소에서 실제로 실행해 출력을 확인하세요. 트랙은 코드 실행 검증입니다 — 명령이 실제로 동작해야 정답입니다.

```
# 1) feature/menu 브랜치를 만들면서 이동한다
git _____ -c _____

# 2) 변경 파일을 스테이징하고 커밋한다
git add menu.html
git _____ -m "feat: 메뉴 섹션 추가"

# 3) main으로 돌아간다
git _____ main

# 4) 두 갈래를 그래프로 비교한다
git log --oneline --all --_____
```

**확인 질문:** 4)의 그래프에서 `main` 과 `feature/menu` 중 어느 쪽이 더 위(최신)에 있어야 정상일까요? 그 이유를 5.2절의 포인터 그림으로 설명해 보세요.

## 자가체크 체크리스트

- [ ] 1)에서 Switched to a new branch 'feature/menu' 가 출력되었는가
- [ ] 3) 직후 git log --oneline 에 메뉴 커밋이 **보이지 않는가**(main은 그대로)
- [ ] git branch 의 \* 가 의도한 브랜치에 있는가
- [ ] --all --graph 에서 두 갈래가 한 지점에서 갈라져 보이는가

## 직접 작성: 전환 거부 상황 재현·해결

일부러 "미커밋 변경으로 전환 거부"를 만들고 스스로 해결해 보세요.

```
# 1) feature/menu에서 menu.html을 고친다(커밋하지 않는다)
#   (편집기로 한 줄 수정)

# 2) main으로 전환을 시도한다 -> 거부 메시지를 직접 본다
git _____ main

# 3) 무엇이 떠 있는지 진단한다
git _____ -s

# 4) 둘 중 하나로 해결한다
#   (가) 커밋: git add menu.html && git commit -m "...
#   (나) 치우기: git _____ (변경을 임시 선반에 치움)

# 5) 다시 main으로 전환한다
git switch main
```

## 자가체크 체크리스트

- [ ] 2)에서 Please commit your changes or stash them 메시지를 직접 확인했는가
- [ ] 3)의 git status -s 에서 해당 파일이 M으로 보였는가
- [ ] 해결 후 git status 가 working tree clean 이 되었는가
- [ ] 거부가 "에러"가 아니라 "안전장치"임을 설명할 수 있는가

# 5.9 오개념 교정: 폴더가 복사되지 않는다·branch는 이동 아님

## 흔한 오개념과 바로잡기

브랜치는 입문자가 가장 자주 오해하는 주제입니다. 아래 세 가지를 분명히 짚고 넘어가세요.

- **오개념 1: "브랜치를 만들면 폴더나 파일이 복사된다."** 아닙니다. 브랜치는 커밋을 가리키는 41바이트 이름표일 뿐, 파일은 한 글자도 복사되지 않습니다. 같은 폴더에서 Git이 보여 주는 파일 내용만 현재 브랜치에 맞춰 바뀝니다. 그래서 브랜치 생성이 순식간에 끝납니다.
- **오개념 2: "git branch feature 를 하면 그 브랜치로 이동한다."** 아닙니다. `git branch <이름>` 은 **만들기만** 하고 HEAD는 원래 브랜치에 그대로 있습니다(`git branch` 출력의 \* 로 확인). 만들며 이동하려면 `git switch -c <이름>` 을 씁니다.
- **오개념 3: "git switch 가 커밋을 만들거나 변경을 저장한다."** 아닙니다. `switch` 는 HEAD가 가리키는 브랜치를 바꿀 뿐, 커밋을 만들지 않습니다. 오히려 커밋하지 않은 변경이 덮일 위험이 있으면 전환을 **거부**해 데이터를 지킵니다(5.7절).
- **오개념 4: "checkout 은 구식이라 쓰면 안 된다."** 아닙니다. `checkout` 도 지금까지 완전히 유효합니다. `switch / restore` 는 역할을 명확히 나눈 신명령일 뿐이며, 둘은 같은 일을 합니다. 다만 의미가 분명한 `switch` 를 권장합니다.

헛갈릴 때의 행동 강령은 ch\_02와 똑같습니다. **무조건 git status 와 git branch 부터** 봅니다.

"지금 어느 브랜치인가(`git branch` 의 \*), 작업트리에 뜬 변경은 무엇인가(`git status`)"를 확인하면 거의 모든 혼란이 풀립니다.

한 가지 더 짚어 둘 것은 **"브랜치가 가리키는 커밋"과 "작업트리에 보이는 파일"의 관계**입니다. 전환을 하면 두 가지가 동시에 일어납니다. (1) HEAD가 가리키는 브랜치가 바뀌고, (2) 작업트리의 파일 내용이 그 브랜치가 가리키는 커밋의 스냅샷으로 갱신됩니다. 그래서 `feature` 에서 보이던 `about.html` 이 `main` 으로 전환하면 사라진 것처럼 보이는데, 이는 삭제가 아니라 "main 시점에는 그 파일이 아직 없었을 뿐"입니다. 다시 `feature` 로 전환하면 그대로 돌아옵니다. 파일이 "사라졌다"고 당황하지 말고, 지금 HEAD가 어느 커밋을 가리키는지를 떠올리면 됩니다.

## 자주 묻는 질문

**Q. 브랜치를 너무 많이 만들면 저장 공간이 부담되지 않나요?** A. 전혀 그렇지 않습니다. 브랜치는 커밋을 가리키는 41바이트짜리 이름표라, 수백 개를 만들어도 용량은 사실상 0에 가깝습니다. 부담 없이 만들고 끝나면 `git branch -d` 로 정리하면 됩니다.

**Q. git switch -c 로 만든 브랜치는 어느 커밋에서 시작하나요?** A. 명령을 실행한 시점의 HEAD(즉 현재 브랜치가 가리키는 커밋)에서 갈라집니다. `main`에 있을 때 `git switch -c feature` 를 하면 `main`의 현재 커밋이 분기 지점이 됩니다. 특정 커밋이나 다른 브랜치에서 시작하려면 `git switch -c feature <시작점>` 처럼 시작점을 덧붙입니다.

**Q. 브랜치 이름을 잘못 지었습니다. 바꿀 수 있습니까?** A. `git branch -m <옛이름> <새이름>` 으로 이름을 바꿉니다. 현재 있는 브랜치의 이름을 바꿀 때는 `git branch -m <새이름>` 처럼 옛 이름을 생략해도 됩니다.

**Q. `git switch` 와 `git checkout` 중 무엇을 외워야 하나요?** A. 새로 배운다면 `git switch` (브랜치 전환)와 `git restore` (파일 복원)를 권장합니다. 역할이 분명해 헷갈리지 않습니다. 다만 현업·예전 자료에서 `git checkout` 을 자주 보게 되므로, "checkout = switch + restore를 합친 옛 명령"이라는 대응만 알아 두면 충분합니다.

## 5.10 단원 미니 프로젝트: 기능 브랜치로 실험하기(2종 1세트)

이 단원(u2: 이력을 읽고 갈래를 나누기)에서 배운 `git log`·`git diff` (**ch\_04**)와 `git branch`·`git switch` (**ch\_05**)만으로 완성하는 작은 산출물입니다. 목표는 한 줄로 이렇습니다.

샘플 저장소에서 log·diff로 이력을 읽고, `feature/about-page` 브랜치를 만들어 변경한 뒤, main과 비교(diff)해 두 갈래의 차이를 직접 설명한다.

**산출물(deliverable):** `feature/about-page` 브랜치를 가진 저장소 — main은 그대로 두고 브랜치에서만 소개(About) 페이지를 추가했으며, `git log --graph` 와 `git diff main..feature/about-page` 로 두 갈래의 차이를 말로 설명할 수 있는 상태.

### 마일스톤(증분으로 쌓기)

| 마일스톤 | 할 일                                                                                                         | 쓰는 개념                       |
|------|-------------------------------------------------------------------------------------------------------------|-----------------------------|
| M1   | <code>git log --oneline --graph</code> 로 현재 이력을 읽고, <code>git switch -c feature/about-page</code> 로 분기      | c04(log),<br>c05(switch -c) |
| M2   | About 섹션 추가 커밋 → <code>git switch main</code> 으로 돌아와 main 이 그대로임을 확인                                        | c05(switch),<br>c04(log)    |
| M3   | <code>git diff main..feature/about-page</code> 로 두 갈래 차이 비교,<br><code>git log --all --graph</code> 로 분기 시각화 | c04(diff),<br>c05(branch)   |

## 모범 흐름

```
# M1: 현재 이력을 읽고 분기
git log --oneline --graph
git switch -c feature/about-page

# M2: 브랜치에서 작업 커밋 후 main으로 복귀
git add about.html
git commit -m "feat: About 소개 섹션 추가"
git switch main

# M3: 두 갈래 비교
git diff main..feature/about-page
git log --oneline --all --graph
```

```
Switched to a new branch 'feature/about-page'
[feature/about-page 7a1c9e2] feat: About 소개 섹션 추가
1 file changed, 12 insertions(+)
Switched to branch 'main'
diff --git a/about.html b/about.html
new file mode 100644
index 0000000..3c1e0ab
--- /dev/null
+++ b/about.html
@@ -0,0 +1,12 @@
+<section id="about">
+  <h2>About</h2>
+  ...
* 7a1c9e2 (feature/about-page) feat: About 소개 섹션 추가
* 3f2b6a0 (HEAD -> main) docs: README 소개 작성
* 9d4e1c8 프로젝트 초기 구조 추가
```

실행 결과에서 `git diff main..feature/about-page` 는 main에는 없고 feature에만 있는 변경 (About 파일 추가)을 보여 주고, 그래프는 두 갈래가 갈린 모습을 그립니다.

### 흔한 문제·진단·해결(미니 프로젝트용)

- 고쳤는데 `git diff` 에 안 나옴 → 진단: 이미 add 되어 스테이징에 있음 → 해결: `git diff --staged` 로 확인(ch\_04).
- 커밋 안 한 변경이 있어 `switch` 가 거부됨 → 진단: `git status` 로 M 확인 → 해결: 커밋하거나 `git stash` 후 전환(5.7절).
- `git log` 가 화면에 멈춘 듯 보임 → 진단: less 페이지 화면임 → 해결: q로 빠져나오기(ch\_04).

## 학생 작성형 빈 스타터

### 직접 작성: 내 손으로 갈래 나누기

아래 스타터의 빈칸을 채우고, 본인 저장소에서 M1 → M2 → M3을 순서대로 실행하세요.

```
# M1) 이력을 읽고 분기한다
git log --oneline --_____
git switch _____ feature/about-page

# M2) 작업·커밋 후 main으로 복귀
git add about.html
git commit -m "feat: _____"
git switch _____

# M3) 두 갈래를 비교한다
git diff _____..feature/about-page
git log --oneline --all --_____
```

**제출 시 함께 적기:** M3의 `diff` 결과를 보고, "main과 feature는 무엇이 어떻게 다른가"를 두세 문장으로 설명하세요. 5.2절의 포인터 그림을 근거로 들면 좋습니다.

### 자가체크 체크리스트(스스로 점검)

- [ ] main에는 About 변경이 **없는가**(`git switch main` 후 `git log` 로 확인)
- [ ] feature에만 변경이 있는가(`git diff main..feature/about-page` 에 변경이 보임)
- [ ] `git log --all --graph` 에서 두 갈래가 한 지점에서 갈라져 보이는가
- [ ] `diff` 결과로 두 갈래의 차이를 말로 설명할 수 있는가
- [ ] 작업 종료 시 `git status` 가 `working tree clean` 인가



이렇게 갈래를 안전하게 나누는 법을 익혔습니다. 다음 챕터(ch\_06)에서는 따로 만든 이 갈래를 본류(main)로 **합치는** `git merge` 를 배웁니다. 분기(branch)가 "나누기"였다면, 병합(merge)은 그 반대인 "합치기"입니다.



# 갈래 합치기: git merge(fast-forward vs 3-way)

## 이 챕터에서 배우는 것

- fast-forward 병합과 3-way 병합의 차이를 그림으로 설명할 수 있습니다.
- `git merge` 로 기능 브랜치를 main에 통합할 수 있습니다(합치는 방향을 정확히 잡습니다).
- 공통 조상(merge base)과 두 부모를 가진 병합 커밋이 무엇인지 설명할 수 있습니다.
- `--no-ff` 로 병합 커밋을 명시적으로 남겨 기능 단위를 이력에 드러낼 수 있습니다.
- 병합 후 `git branch -d` 로 브랜치를 정리하고, 진행 중인 병합을 `git merge --abort` 로 되돌릴 수 있습니다.
- "병합 방향 혼동"과 "사라진 것처럼 보이는 변경"을 진단하고 바로잡을 수 있습니다.

이 챕터의 핵심 개념은 **병합 — merge** 한 가지입니다. 선수 개념은 바로 앞 챕터(ch\_05)의 **분기 — branch**와 **switch**입니다. ch\_05에서 "브랜치는 커밋을 가리키는 포인터, HEAD는 현재 브랜치"라는 그림을 익혔는데, 병합은 결국 **그 포인터들을 어떻게 합치고 어디로 옮기느냐**의 이야기입니다. ch\_04의 `git log --graph` 로 병합 결과를 확인하는 장면도 계속 나옵니다.

## 6.1 도입: 따로 만든 기능을 본류로 합치기

ch\_05에서 우리는 `feature/about-page` 갈래를 파서 main을 안전하게 둔 채 About 섹션을 실험했습니다. 그런데 실험이 성공했다면 그다음은 무엇일까요? 당연히 **그 작업을 main에 합쳐** 정식 기능으로 만들어야 합니다. 갈래를 영원히 따로 둘 거라면 애초에 나눈 의미가 없습니다.

손으로 파일을 복사·붙여넣기 해서 합치는 것은 ch\_01에서 비웃던 옛 방식입니다. Git은 `git merge` 한 명령으로 두 갈래의 작업을 자동으로 합쳐 줍니다. 그것도 단순히 파일을 덮어쓰는 것이 아니라, "어디서 갈라졌고 양쪽이 각각 무엇을 바꿨는지"를 따져 가며 똑똑하게 통합합니다.

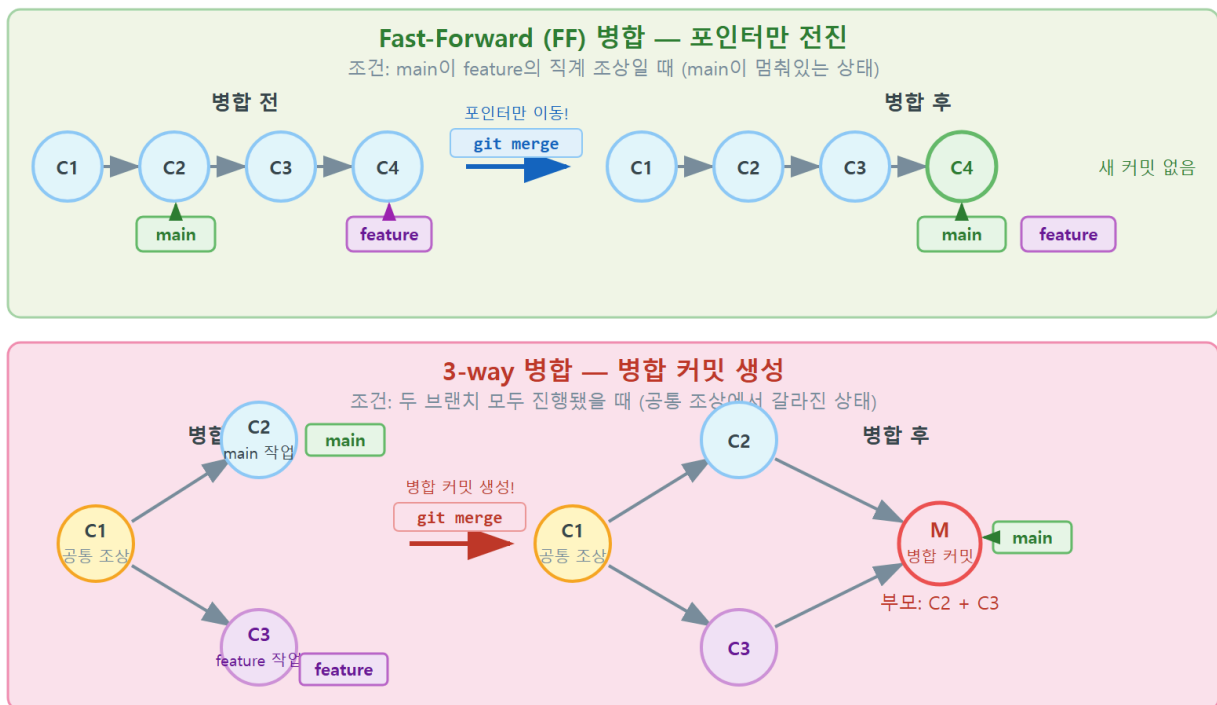
이때 Git은 상황에 따라 **두 가지 방식** 중 하나로 병합합니다. main이 갈라진 뒤 한 발짝도 움직이지 않았다면 포인터만 속 옮기는 **fast-forward**가, main도 그사이 따로 전진했다면 두 갈래를 합쳐 새 커밋을 만드는 **3-way 병합**이 일어납니다. 이 두 방식을 그림으로 구분하는 것이 이 챕터의 첫 목표입니다.

## 한눈에 보는 병합

병합(merge)은 다른 브랜치의 작업을 현재 브랜치로 합치는 것입니다. 갈라진 뒤 합쳐 받을 브랜치(main)가 그대로면 포인터만 앞으로 옮기는 **fast-forward 병합**이, 양쪽이 모두 진행됐으면 공통 조상(merge base)과 두 갈래를 합쳐 **두 부모를 가진 병합 커밋(merge commit)**을 만드는 **3-way 병합**이 일어납니다. 합치려면 **받을 브랜치로 먼저 git switch 한 뒤 git merge <합칠 브랜치>**를 실행합니다.

## 6.2 동작 원리: fast-forward vs 3-way merge

### Fast-Forward vs 3-way Merge 비교



두 병합 방식의 차이는 "main이 갈라진 뒤에도 움직였는가"라는 단 하나의 질문으로 갈립니다.

### 경우 1: fast-forward (main이 제자리)

feature를 만든 뒤 main에서는 아무 커밋도 하지 않았다고 합시다. 그러면 이력은 사실상 한 줄입니다.

```
(A) --- (B) --- (C) --- (D)
                   ^       ^
                 main    feature
```

여기서 main으로 가서 `git merge feature` 를 하면, main을 그냥 D로 옮기기만 하면 됩니다. main에서 갈라진 뒤 main이 추가한 것이 없으니, 합칠 것도 없이 **포인터만 앞으로(fast-forward) 전진**시키면 끝입니다.

```
(A) --- (B) --- (C) --- (D)
                   ^
                 main, feature (둘 다 D)
```

새 커밋은 생기지 않습니다. 이력이 한 줄로 깔끔하게 유지됩니다. Git은 이 경우 **Fast-forward** 라고 알려 줍니다.

## 경우 2: 3-way merge (양쪽이 모두 전진)

이번에는 `feature` 를 만든 뒤 **main에서도** 다른 작업(예: பு터 수정)을 커밋했다고 합시다. 이력이 진짜로 두 갈래로 벌어집니다.

```
      (E)                <- main (main이 따로 전진)
      /
(A)-(B)-(C)
      \
      (D)                <- feature
```

이제는 포인터만 옮길 수가 없습니다. main에는 E가, feature에는 D가 각각 있어서, **양쪽의 변경을 모두 살려** 합쳐야 합니다. Git은 세 지점을 봅니다 — 공통 조상 C, main의 끝 E, feature의 끝 D. 이 셋을 비교해(그래서 "3-way") 합친 결과를 담은 **새 병합 커밋 M**을 만듭니다.

```
      (E)----- (M)      <- main
      /           /
(A)-(B)-(C)      /
      \           \
      (D)---      <- feature
```

M은 부모가 둘(E와 D)인 특별한 커밋입니다. Git은 이 경우 **Merge made by the 'ort' strategy.** 라고 알려 줍니다('ort'는 현재 Git의 기본 병합 전략 이름입니다).

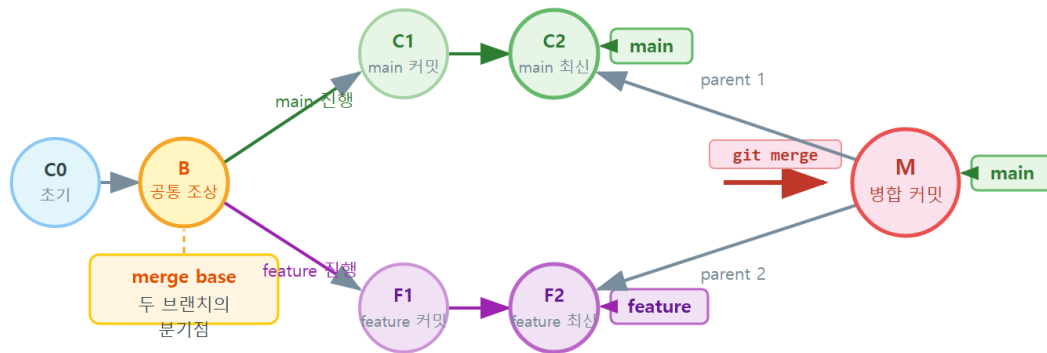
|         | fast-forward            | 3-way merge                       |
|---------|-------------------------|-----------------------------------|
| 조건      | 받을 브랜치(main)가 갈라진 뒤 제자리 | 양쪽이 모두 전진(이력이 갈라짐)                |
| 새 커밋    | 만들지 않음(포인터만 전진)         | 두 부모를 가진 병합 커밋 생성                 |
| 이력 모양   | 한 줄(직선)                 | 갈라졌다가 합쳐지는 다이아몬드                  |
| Git 메시지 | Fast-forward            | Merge made by the 'ort' strategy. |

"fast-forward(빨리 감기)"라는 이름은 비유로 이해하면 쉽습니다. 영상에서 빨리 감기를 하면 새로운 장면을 만들지 않고 **재생 위치만 앞으로 점프**하지요. fast-forward 병합도 똑같습니다. 새 커밋(새 장면)을 만들지 않고, main이라는 재생 위치를 feature가 있는 곳까지 그냥 앞으로 점프 시킵니다. 반대로 3-way 병합은 양쪽에 각자 다른 장면이 있어 점프만으로는 안 되고, 둘을 엮은 새 장면(병합 커밋)을 만들어야 합니다.

이 차이를 미리 알아 두면 실무에서 유용합니다. 어떤 병합이 일어날지 예측하려면 "내가 받을 브랜치(main)가 갈라진 뒤에도 커밋을 추가했는가?"만 자문하면 됩니다. 추가하지 않았으면 fast-forward, 추가했으면 3-way입니다. 병합을 실행하기 전에 `git log --oneline --all --graph`로 갈래 모양을 미리 확인하는 습관을 들이면, 결과 메시지가 Fast-forward 일지 Merge made by ... 일지 미리 가늠할 수 있습니다.

## 6.3 공통 조상(merge base)과 두 부모를 가진 병합 커밋

## 공통 조상(merge base)과 병합 커밋



병합 커밋 M은 두 개의 부모(C2, F2)를 가지는 유일한 커밋  
Git이 세 지점(merge base B, C2, F2)을 비교해 최종 결과를 만드는 것이 3-way 병합

3-way 병합의 핵심에는 **공통 조상(merge base)**이라는 개념이 있습니다. 6.2절 경우 2의 그림에서 main(E)과 feature(D)가 마지막으로 함께 있었던 지점은 c입니다. 이 c가 두 갈래의 **공통 조상**입니다.

왜 공통 조상이 필요할까요? 양쪽의 최종 상태(E와 D)만 보서는, 어떤 줄이 "새로 추가된 것"이고 어떤 줄이 "원래 있던 것"인지 알 수 없습니다. 공통 조상 c를 기준으로 삼아야 비로소 이렇게 따질 수 있습니다.

- c → E로 가며 main이 **무엇을 바꿨는가**(예: 푸터 추가).
- c → D로 가며 feature가 **무엇을 바꿨는가**(예: About 섹션 추가).

두 변경이 서로 다른 파일·다른 줄을 건드렸다면, Git은 양쪽 변경을 모두 적용한 결과를 자동으로 만들어 병합 커밋에 담습니다. (만약 두 변경이 **같은 줄**을 다르게 건드렸다면 Git이 스스로 정할 수 없어 충돌이 나는데, 그것이 다음 챕터 ch\_07의 주제입니다.)

병합 커밋이 **두 부모**를 갖는다는 점도 중요합니다. 보통 커밋은 부모가 하나(직전 커밋)지만, 병합 커밋 M은 부모가 둘(E와 D)입니다. 이 두 부모 덕분에 나중에 `git log --graph`로 "이 두 갈래가 여기서 합쳐졌다"는 역사를 그대로 추적할 수 있습니다.

```
git log --oneline --graph
```

```
* a1b2c3d (HEAD -> main) Merge branch 'feature/about-page'
|\
| * 7a1c9e2 (feature/about-page) feat: About 소개 섹션 추가
* | 5e6f7a8 docs: 푸터 저작권 문구 수정
|/
* 3f2b6a0 docs: README 소개 작성
* 9d4e1c8 프로젝트 초기 구조 추가
```

위 실행 결과에서 맨 위 Merge branch ... 커밋이 병합 커밋이고, 그 아래 `|\` 와 `|/` 표시가 "여기서 두 갈래( 5e6f7a8 와 7a1c9e2 )가 갈라졌다가 합쳐졌다"는 것을 그림으로 보여 줍니다. 공통 조상은 3f2b6a0 입니다.

### 공통 조상을 직접 확인하기

`git merge-base main feature/about-page` 를 실행하면 두 브랜치의 공통 조상 커밋 해시를 바로 알려 줍니다. "어디서 갈라졌는지"가 헛갈릴 때 진단 도구로 유용합니다.

## 6.4 worked example: main으로 switch 후 merge feature

이제 표준 통합 흐름을 처음부터 끝까지 따라가 보겠습니다. **상황:** feature/about-page 에서 About 섹션 작업을 마쳤습니다. 이를 main에 합쳐 정식 기능으로 만들려 합니다.

### 상황 → 명령(방향을 정확히)

병합에서 가장 중요한 것은 방향입니다. "feature를 main에 합친다"이므로, 받을 쪽인 **main으로 먼저 전환**한 다음 `git merge feature` 를 실행합니다. 순서를 거꾸로 하면 엉뚱한 방향으로 합쳐 집니다.

```
git switch main
git merge feature/about-page
```

main이 갈라진 뒤 제자리였다면 fast-forward가 일어납니다.



```
Switched to branch 'main'
Updating 3f2b6a0..7a1c9e2
Fast-forward
 about.html | 12 ++++++++
 1 file changed, 12 insertions(+)
```

`Fast-forward` 라는 단어가 보이면, 새 커밋 없이 main 포인터가 feature 자리로 전진했다는 뜻입니다.

## 다른 경우: main도 전진했을 때(3-way)

만약 그사이 main에서도 푸터를 고쳐 커밋했다면, 같은 `git merge` 가 이번엔 3-way 병합을 합니다.

```
git switch main
git merge feature/about-page
```

```
Switched to branch 'main'
Merge made by the 'ort' strategy.
 about.html | 12 ++++++++
 1 file changed, 12 insertions(+)
```

`Merge made by the 'ort' strategy.` 는 두 부모를 가진 병합 커밋이 새로 생겼다는 뜻입니다(6.3절).

## 진단: 정말 합쳐졌는지 확인

병합이 끝났으면 그래프로 결과를 검증하는 습관을 들이세요.

```
git log --oneline --graph
```

```
* a1b2c3d (HEAD -> main) Merge branch 'feature/about-page'
|\
| * 7a1c9e2 (feature/about-page) feat: About 소개 섹션 추가
* | 5e6f7a8 docs: 푸터 저작권 문구 수정
|/
* 3f2b6a0 docs: README 소개 작성
```

`main( HEAD -> )`에 About 작업과 푸터 작업이 **둘 다** 들어왔고, 두 갈래가 합쳐진 모양이 그래프로 확인됩니다.

- `git switch main`: 합쳐 **받을** 브랜치로 먼저 이동합니다. 병합은 "현재 브랜치(main)로 다른 브랜치(feature)를 끌어온다"이므로 방향이 핵심입니다.
- `git merge feature/about-page`: 현재 브랜치(main)에 feature의 작업을 합칩니다. main이 제자리였으면 `Fast-forward`, 둘 다 전진했으면 `Merge made by the 'ort' strategy.` 가 출력됩니다.
- `git log --oneline --graph`: 병합 결과를 시각적으로 검증합니다. 3-way였다면 병합 커밋과 `| \ / |` 모양이 보입니다.

## 병합 전에 무엇이 들어올지 미리 보기

큰 변경을 합치기 전에는 "main에 무엇이 새로 들어오는지"를 먼저 확인하는 것이 안전합니다. ch\_04에서 배운 범위 비교(`main..feature`)가 바로 이때 쓰입니다.

```
git switch main
git log --oneline main..feature/about-page
```

```
7a1c9e2 feat: About 소개 섹션 추가
6b5a4c3 feat: About 섹션 스타일 정리
```

`main..feature/about-page` 는 "feature에는 있는데 main에는 아직 없는 커밋"만 보여 줍니다. 실행 결과의 두 커밋이 이번 병합으로 main에 합류할 것들입니다. 합칠 양과 내용을 미리 파악하고 나서 `git merge` 를 실행하면, 예상치 못한 변경에 놀랄 일이 줄어듭니다. 변경 내용까지 줄 단위로 보고 싶다면 `git diff main..feature/about-page` 를 함께 쓰면 됩니다.

## 6.5 git merge --no-ff와 이력 가독성

fast-forward는 이력을 한 줄로 깔끔하게 유지하는 장점이 있지만, 단점도 있습니다. **"어떤 커밋들이 한 기능이었는지"가 이력에서 사라진다는** 점입니다. fast-forward로 합치면 병합 커밋이 없어서, 나중에 보면 About 작업 커밋이 그냥 main 줄에 섞여 버립니다.

이를 막으려면 `--no-ff` (no fast-forward) 옵션을 씁니다. fast-forward가 가능한 상황에서도 **일부러 병합 커밋을 하나 만들어**, "여기서 feature 하나가 통째로 합쳐졌다"는 경계를 이력에 남깁니다.

```
git switch main
git merge --no-ff feature/about-page -m "Merge: About 섹션 기능 통합"
git log --oneline --graph
```

```
Switched to branch 'main'
Merge made by the 'ort' strategy.
  about.html | 12 ++++++++
  1 file changed, 12 insertions(+)
*    c4d5e6f (HEAD -> main) Merge: About 섹션 기능 통합
|\
| * 7a1c9e2 (feature/about-page) feat: About 소개 섹션 추가
|/
* 3f2b6a0 docs: README 소개 작성
```

실행 결과를 보면, main이 제자리였는데도( 7a1c9e2 하나만 갈라져 있었음에도) 병합 커밋 c4d5e6f 가 생겼습니다. 덕분에 "About 기능이 여기서 한 덩어리로 합류했다"는 사실이 이력에 명확히 드러납니다.

|         | 기본(fast-forward) | --no-ff            |
|---------|------------------|--------------------|
| 병합 커밋   | 없음               | 항상 생성              |
| 이력 모양   | 한 줄(직선)          | 기능 단위로 갈라졌다 합쳐짐    |
| 장점      | 단순.깔끔            | 기능 경계가 보임, 되돌리기 쉬움 |
| 자주 쓰는 곳 | 혼자 작업.사소한 변경     | 팀 협업.PR 머지         |

같은 작업을 fast-forward로 합쳤을 때와 --no-ff 로 합쳤을 때 이력 그래프가 어떻게 달라지는지 나란히 보면 차이가 분명합니다.

```
# (가) 기본 fast-forward 로 합친 경우 — 한 줄로 섞임
* 7a1c9e2 (HEAD -> main) feat: About 소개 섹션 추가
* 3f2b6a0 docs: README 소개 작성

# (나) --no-ff 로 합친 경우 — 기능 경계가 보임
*    c4d5e6f (HEAD -> main) Merge: About 섹션 기능 통합
|\
| * 7a1c9e2 feat: About 소개 섹션 추가
|/
* 3f2b6a0 docs: README 소개 작성
```

(가)는 About 커밋이 main 줄에 그냥 녹아들어 "이게 한 기능이었다"는 정보가 없습니다. 반면 (나)는 병합 커밋 c4d5e6f 가 경계를 만들어, 나중에 "이 기능 전체를 되돌리고 싶다"면 그 병합

커밋 하나만 다루면 됩니다(ch\_08의 `revert -m`). 이력을 "사람이 읽는 문서"로 본다면 (나)가 더 친절한 셈입니다.

### GitHub의 PR 머지와 `--no-ff`

종합 프로젝트(ch\_13~18)에서 다룰 GitHub의 "Create a merge commit" 방식이 사실상 `--no-ff` 병합입니다. 팀에서는 어떤 PR이 언제 합쳐졌는지를 이력에 남기려고 병합 커밋을 선호하는 경우가 많습니다.

## 6.6 병합 후 정리: `git branch -d, merge --abort`

### 끝난 브랜치 정리: `git branch -d`

병합이 끝나면 그 기능 브랜치는 역할을 다했습니다. 더 뒤도 헛갈리기만 하므로 정리합니다. 이때 ch\_05에서 본 `git branch -d` (소문자)를 씁니다.

```
git branch -d feature/about-page
```

```
Deleted branch feature/about-page (was 7a1c9e2).
```

`-d` 는 "이 브랜치가 이미 어딘가에 병합되었는가"를 확인하고 안전할 때만 지웁니다. 만약 아직 합치지 않은 브랜치를 `-d` 로 지우려 하면 다음처럼 거부해 작업 유실을 막아 줍니다.

```
error: The branch 'feature/about-page' is not fully merged.
If you are sure you want to delete it, run 'git branch -D feature/about-page'.
```

이 메시지가 뜨면 "어? 아직 안 합쳤구나" 하고 먼저 병합을 끝낸 뒤 지우면 됩니다. 정말 버릴 작업이라면 `-D` 로 강제할 수 있지만, 이는 작업을 영영 잃을 수 있으니 신중해야 합니다.

### 진행 중인 병합 되돌리기: `git merge --abort`

3-way 병합 중 같은 줄이 충돌하면(ch\_07) Git이 병합을 멈추고 충돌 상태로 들어갑니다. 이때 "그냥 처음부터 다시 하고 싶다"면 `git merge --abort` 로 **병합을 시작하기 직전 상태로 안전하게 되돌릴 수** 있습니다.

```
git merge --abort
git status
```

```
On branch main
nothing to commit, working tree clean
```

`--abort` 는 병합을 깨끗이 취소하고 작업트리를 병합 전으로 복원합니다. 충돌이 너무 복잡해 막혔을 때의 든든한 안전망입니다(충돌 해결의 본격적인 내용은 ch\_07).

## 어떤 브랜치가 이미 병합됐는지 확인하기

브랜치가 여러 개일 때 "무엇을 지워도 안전한지"를 일일이 따지기는 번거롭습니다. Git은 이를 한눈에 보여 주는 옵션을 제공합니다.

```
git branch --merged
git branch --no-merged
```

```
* main
  feature/about-page
  feature/contact
```

`git branch --merged` 는 **현재 브랜치(main)에 이미 합쳐진** 브랜치만 보여 줍니다. 위 출력의 `feature/about-page` · `feature/contact` 는 main에 병합이 끝났으므로 `git branch -d` 로 안전하게 지워도 됩니다. 반대로 `git branch --no-merged` 는 아직 합치지 않은 브랜치를 보여 주는데, 이 목록에 있는 브랜치를 `-d` 로 지우려 하면 Git이 거부합니다(작업 유실 방지).

이 두 명령은 정리 작업의 좋은 길잡이입니다. "**--merged 에 보이면 지워도 안전, --no-merged 에 보이면 먼저 합칠지 판단**"이라는 규칙으로 브랜치 목록을 깔끔하게 유지하세요. 종합 프로젝트(ch\_15~18)처럼 브랜치가 많아지는 상황에서 특히 유용합니다.

### 병합 후 정리는 "의식"으로 굳히세요

기능을 합쳤으면 곧바로 (1) `git log --graph` 로 병합 확인 → (2) `git branch --merged` 로 안전 확인 → (3) `git branch -d` 로 정리, 이 세 단계를 한 묶음으로 실행하는 습관을 들이면 브랜치 목록이 어지러워지지 않습니다.

## 6.7 흔한 문제·해결: 병합 방향 혼동·사라진 변경 진단

병합에서 입문자가 가장 자주 겪는 두 가지 문제를 진단부터 해결까지 다룹니다.

## 문제 1: 병합 방향을 거꾸로 잡았다

**문제 발생:** "feature를 main에 합치려" 했는데, 실수로 feature 브랜치에 머문 채 `git merge main`을 실행했습니다.

```
git branch
git merge main
```

```
* feature/about-page
  main
Already up to date.
```

**진단:** `git branch`의 \*가 `feature/about-page`에 있습니다. 즉 현재 브랜치가 feature인데 `git merge main`은 "feature에 main을 합쳐라"라는 뜻이 됩니다. 이 경우 흔히 `Already up to date.` (합칠 것이 없음)가 뜨거나, 의도와 반대로 main이 feature에 들어옵니다. **병합은 항상 "현재 브랜치 ← 인자 브랜치" 방향임을 기억하세요.**

**해결:** 받을 브랜치(main)로 먼저 전환한 뒤 다시 합칩니다.

```
git switch main
git merge feature/about-page
```

```
Switched to branch 'main'
Updating 3f2b6a0..7a1c9e2
Fast-forward
 about.html | 12 ++++++++
 1 file changed, 12 insertions(+)
```

이제 의도대로 main이 feature의 작업을 받았습니다.

## 문제 2: 합쳤는데 한쪽 변경이 사라진 것 같다

**문제 발생:** 병합 후 페이지를 열어 보니 feature에서 분명히 만든 About 섹션이 안 보입니다.

**진단:** 침착하게 `ch_04`의 도구로 추적합니다. 먼저 `git log`로 그 커밋이 정말 main에 들어왔는지 확인합니다.

```
git log --oneline --all --graph
```

```
* 5e6f7a8 (HEAD -> main) docs: 푸터 저작권 문구 수정
| * 7a1c9e2 (feature/about-page) feat: About 소개 섹션 추가
|/
* 3f2b6a0 docs: README 소개 작성
```

여기서 7a1c9e2 (About 커밋)가 여전히 feature 갈래에만 있고 main(HEAD ->)에는 합쳐지지 않은 것이 보입니다. 즉 변경이 "사라진" 것이 아니라 **애초에 병합이 안 된** 것입니다. (병합이 정말 됐다면 6.4절처럼 main 줄에 About 커밋이 보여야 합니다.)

**해결:** main에서 병합을 제대로 실행합니다.

```
git switch main
git merge feature/about-page
```

**진짜로 변경이 사라졌다면?**

병합 자체는 양쪽 변경을 **둘 다 살리는** 작업입니다. 그러므로 한쪽 변경이 정말로 없어졌다면 그것은 "병합"의 결과가 아니라 (1) 충돌 해결 중 한쪽을 통째로 버렸거나, (2) `reset /force push` 같은 사고일 가능성이 큼니다. 이런 경우의 복구는 ch\_08(되돌리기·reflog)에서 배웁니다. 핵심은 **추측하지 말고 `git log --all --graph` 로 먼저 진단하는** 것입니다.

## 6.8 즉시 연습(2종 1세트): ff-3-way 재현 채우기

fast-forward와 3-way를 **직접 재현**해 차이를 눈으로 확인하는 연습입니다. 모범 풀이를 본 뒤 빈 템플릿을 채우세요.

### 모범 풀이: fast-forward 재현

main을 건드리지 않고 feature만 전진시킨 뒤 병합하면 fast-forward가 일어납니다.

```
git switch -c feature/ff-demo
git commit --allow-empty -m "feat: ff 데모 커밋"
git switch main
git merge feature/ff-demo
```

```
Switched to a new branch 'feature/ff-demo'
[feature/ff-demo 2c3d4e5] feat: ff 데모 커밋
Switched to branch 'main'
Updating 3f2b6a0..2c3d4e5
Fast-forward
```

Fast-forward 가 떴고 새 병합 커밋은 생기지 않았습니다. 실행 결과로 직접 확인하세요.

## 학생 작성형 빈 템플릿 1

### 직접 작성: 3-way 병합 재현하기

이번엔 양쪽을 모두 전진시켜 3-way 병합과 병합 커밋을 만들어 보세요. 빈칸(\_\_\_\_\_)을 채우고 실제로 실행해 출력을 확인합니다.

```
# 1) feature/3way-demo를 만들며 이동하고, 커밋한다
git switch _____ feature/3way-demo
git commit --allow-empty -m "feat: feature 쪽 작업"

# 2) main으로 돌아가 main에서도 따로 커밋한다(여기서 이력이 갈라짐)
git _____ main
git commit --allow-empty -m "docs: main 쪽 작업"

# 3) main에 feature를 병합한다 -> 이번엔 3-way!
git merge _____

# 4) 그래프로 병합 커밋(두 부모)을 확인한다
git log --oneline --_____
```

확인 질문: 3)에서 출력 메시지가 Fast-forward 가 아니라 Merge made by the 'ort' strategy. 로 바뀐 이유를, 6.2절의 "양쪽이 모두 전진" 조건으로 설명하세요.

### 자가체크 체크리스트

- [ ] 3)에서 Merge made by the 'ort' strategy. 가 출력되었는가
- [ ] 4)의 그래프에 병합 커밋(Merge branch ...)과 `\ /` 모양이 보이는가
- [ ] 그 병합 커밋이 부모를 둘 가진다고 설명할 수 있는가(공통 조상 기준)

## 학생 작성형 빈 템플릿 2

### 직접 작성: 통합 후 정리까지

병합을 끝낸 뒤 브랜치 정리까지 한 바퀴를 완성하세요.



```
# 1) main으로 전환한다
git switch _____

# 2) --no-ff로 기능 브랜치를 병합해 병합 커밋을 남긴다
git merge _____ feature/3way-demo -m "Merge: 3way 데모 통합"

# 3) 역할을 다한 브랜치를 안전하게 삭제한다
git branch _____ feature/3way-demo

# 4) 최종 이력을 확인한다
git log --oneline --graph
```

**확인 질문:** 3)에서 만약 `not fully merged` 거부 메시지가 떴다면 무엇을 먼저 해야 할까요?

### 자가체크 체크리스트(스스로 점검)

- [ ] 2)에서 병합 커밋이 생겼는가( `--no-ff` 효과)
- [ ] 3)에서 `Deleted branch feature/3way-demo (was ...)` 가 출력되었는가
- [ ] 4)의 그래프에서 기능이 한 덩어리로 합류한 경계가 보이는가
- [ ] 작업 종료 시 `git status` 가 `working tree clean` 인가

## 6.9 오개념 교정: 합치는 방향·ff와 이력·병합은 둘 다 살림

### 흔한 오개념과 바로잡기

병합에서 입문자가 자주 빠지는 함정 네 가지입니다.

- **오개념 1: "어느 브랜치에서 merge 하든 결과는 같다."** 아닙니다. 병합은 **"현재 브랜치 ← 인자 브랜치"** 방향입니다. feature를 main에 합치려면 반드시 `git switch main` 후 `git merge feature` 를 해야 합니다. 방향을 거꾸로 하면 main이 feature로 들어가거나 `Already up to date.` 가 떠 헛갈립니다(6.7절).
- **오개념 2: "fast-forward는 뭔가 잘못된 병합이다."** 아닙니다. fast-forward는 main이 갈라진 뒤 제자리였을 때 일어나는 **정상적이고 가장 깔끔한 병합**입니다. 다만 기능 경계가 이력에 안 남으니, 그것을 남기고 싶으면 `--no-ff` 를 씁니다.
- **오개념 3: "병합하면 한쪽 변경만 남고 다른 쪽은 덮인다."** 아닙니다. 병합은 양쪽 변경을 **둘 다 살리는** 작업입니다. 한쪽이 사라졌다면 그건 병합이 아니라 충돌 해결 실수나 `reset/force push` 사고입니다. `git log --all --graph` 로 먼저 진단하세요.

- **오개념 4: "병합 커밋은 쓸데없으니 항상 fast-forward가 낫다."** 상황에 따라 다릅니다. 혼자 하는 사소한 변경엔 fast-forward가 깔끔하지만, 팀 협업에서는 "이 기능이 언제 합쳐졌는지"를 병합 커밋으로 남기는 편이 추적·되돌리기에 유리합니다.

병합도 헛갈릴 때의 행동은 한결같습니다. `git branch` 로 지금 어느 브랜치인지, `git log --all --graph` 로 갈래가 어떻게 생겼는지부터 확인하면 방향 실수와 "사라진 변경" 착각의 대부분이 풀립니다.

## 이 chapter 명령 한눈에 정리

| 명령                                       | 하는 일                | 기억할 점             |
|------------------------------------------|---------------------|-------------------|
| <code>git switch main</code>             | 받을 브랜치로 이동          | 병합 전 항상 먼저 받을 쪽으로 |
| <code>git merge feature</code>           | 현재 브랜치에 feature를 합침 | 방향: 현재 ← 인자       |
| <code>git merge --no-ff feature</code>   | 병합 커밋을 항상 남김        | 기능 경계를 이력에 남길 때   |
| <code>git merge --abort</code>           | 진행 중 병합 취소          | 충돌로 막혔을 때 안전망     |
| <code>git branch -d feature</code>       | 병합 끝난 브랜치 삭제        | 안 합쳤으면 거부(안전)     |
| <code>git merge-base main feature</code> | 공통 조상 해시 확인         | "어디서 갈라졌나" 진단     |

## 자주 묻는 질문

**Q. fast-forward를 항상 피하고 병합 커밋을 남기고 싶습니다.** A. `git config --global merge.ff false` 로 설정하면 매번 `--no-ff` 를 붙이지 않아도 항상 병합 커밋이 생깁니다. 팀 정책에 따라 선택하면 됩니다(혼자 작업이면 굳이 끌 필요는 없습니다).

**Q. git merge 를 했더니 편집기 화면이 떠서 당황했습니다.** A. 3-way 병합이나 `--no-ff` 병합은 병합 커밋 메시지를 입력받기 위해 편집기를 엽니다. 기본 메시지(Merge branch ...)를 그대로 쓸 거라면 저장 후 닫으면 됩니다(Vim이면 `:wq`, ch\_01에서 설정한 편집기에 따라 다름). 메시지를 미리 정하려면 `-m "..."` 을 붙입니다.

**Q. 병합한 뒤 방금 그 병합을 취소하고 싶습니다.** A. 충돌로 진행 중인 병합은 `git merge --abort` 로 되돌립니다. 그러나 이미 **완료된**(커밋된) 병합을 되돌리는 것은 다른 이야기로, ch\_08의 `git reset` (아직 push 안 했을 때)이나 `git revert` (이미 공유했을 때)를 씁니다. 공유 여부에 따라 수단이 다르다는 점을 기억하세요.

**Q. fast-forward와 3-way 중 무엇이 "더 좋은" 건가요?** A. 우열이 아니라 상황입니다. 혼자 하는 사소한 변경은 fast-forward가 깔끔하고, 여럿이 기능 단위로 협업할 때는 병합 커밋(`--no-ff` 또는 3-way)이 "어느 기능이 언제 합쳐졌는지"를 남겨 추적·되돌리기에 유리합니다.

이렇게 갈래를 합치는 법을 익혔습니다. 정리하면, 병합의 본질은 "두 포인터가 가리키는 작업을 하나로 모으는 것"이며, main이 제자리였으면 포인터만 전진하는 fast-forward가, 양쪽이 모두 전진했으면 공통 조상을 기준으로 두 변경을 엮은 병합 커밋을 만드는 3-way 병합이 일어납니다. 그리고 방향(현재 ← 인자)과 검증(`git log --graph`)과 정리(`git branch -d`)라는 세 가지 습관이 병합을 안전하게 만들어 줍니다. 이 흐름은 ch\_05의 분기와 짝을 이뤄, 종합 프로젝트의 팀 협업 워크플로우(기능 브랜치 → 병합)의 뼈대가 됩니다.

그런데 지금까지의 병합은 모두 두 갈래가 **서로 다른 파일·다른 줄**을 건드린, 자동으로 깔끔하게 합쳐지는 경우였습니다. 만약 두 갈래가 **같은 파일의 같은 줄**을 다르게 고쳤다면 Git은 어느 쪽이 옳은지 스스로 정할 수 없어 합치다 멈춥니다. 그것이 바로 다음 챕터(ch\_07)의 주제인 **충돌 (merge conflict) 해결**입니다. 충돌은 실패가 아니라 협업의 정상적인 한 과정이며, 병합의 동작 원리를 제대로 이해한 지금이 충돌을 배우기에 가장 좋은 때입니다.



## 7장 충돌 해결: merge conflict 진단부터 마무리까지

이 장은 단원 3(u3) '갈래를 합치고 충돌을 풀기'의 두 번째 챕터입니다. 이 장에서 배우는 개념은 **충돌 해결(merge conflict resolution, c07)** 한 가지입니다. 앞 장에서 배운 병합(merge, c06)은 두 갈래를 합치는 행복한 경우였습니다. 하지만 실제 협업에서는 두 사람이 같은 곳을 서로 다르게 고치는 일이 흔하고, 그때 Git은 스스로 결정하지 못해 멈춥니다. 이 멈춤이 바로 병합 충돌(merge conflict)이며, 이 장은 그 충돌을 **두려움 없이 진단하고 해결하는** 한 가지 능력에 온전히 집중합니다.

### 이 장을 마치면 할 수 있는 것

- 충돌이 왜·언제 생기는지 설명하고, 충돌 표시(<<<<<<< ===== >>>>>>>)를 정확히 해석할 수 있습니다.
- `git status`로 충돌 파일(Unmerged paths)을 식별하고 병합 진행 상태를 진단할 수 있습니다.
- 충돌 부분을 원하는 최종본으로 편집한 뒤 `git add`·`git commit`으로 해결을 마무리할 수 있습니다.
- `git merge --abort`로 충돌 전 상태로 안전하게 되돌릴 수 있습니다.
- 작은 커밋·작은 통합으로 충돌을 애초에 줄이는 습관을 설명할 수 있습니다.

### 7.1 도입: 충돌은 실패가 아니라 정상 과정

처음 충돌(conflict)을 만난 입문자는 대개 화면 가득한 <<<<<<< 기호와 빨간 에러 메시지에 당황합니다. "내가 뭘 잘못했지?", "이력이 망가진 건가?" 하는 불안이 먼저 듭니다. 그래서 이 장의 첫 문장은 이렇게 시작합니다. **충돌은 실패가 아닙니다.** 충돌은 두 사람이 같은 부분을 동시에 다르게 고쳤다는, 협업에서 지극히 자연스러운 상황을 Git이 정직하게 알려 주는 것입니다.

생각해 봅시다. 두 동료가 같은 문서의 제목 줄을 각자 다르게 고쳤습니다. 한 명은 "함께 만드는 할 일 관리"로, 다른 한 명은 "우리 팀 협업 할 일 보드"로 바꿨습니다. 이 둘을 합칠 때, **어느 쪽**

을 택해야 옳은지 Git은 알 수 없습니다. 그것은 사람의 의도에 달린 문제이기 때문입니다. 그래서 Git은 자기 마음대로 한쪽을 버리지 않고, 양쪽 내용을 나란히 보여 주며 "여기는 당신이 정해 주세요"라고 멈춰 섭니다. 충돌은 Git이 신중하다는 증거이지 고장이 아닙니다.

## 왜 이 개념이 지금 필요한가

앞 장(c06)에서 우리는 `git merge` 로 갈래를 합쳤습니다. 그때 다룬 두 방식 중 **3-way 병합** — 공통 조상에서 갈라진 두 갈래를 합쳐 병합 커밋을 만드는 경우 — 이 충돌의 무대입니다. 양쪽이 서로 다른 곳을 고쳤으면 Git이 자동으로 합치지만, **같은 곳을 다르게 고쳤으면** 자동 병합이 불가능해 충돌이 납니다. 즉 충돌은 병합의 한 갈래일 뿐이며, 병합을 배운 지금이 충돌을 배우기에 가장 알맞은 자리입니다. 협업(원격·PR, u5·u6)으로 나아가기 전에 반드시 손에 익혀야 할 마지막 1인 기술이기도 합니다.

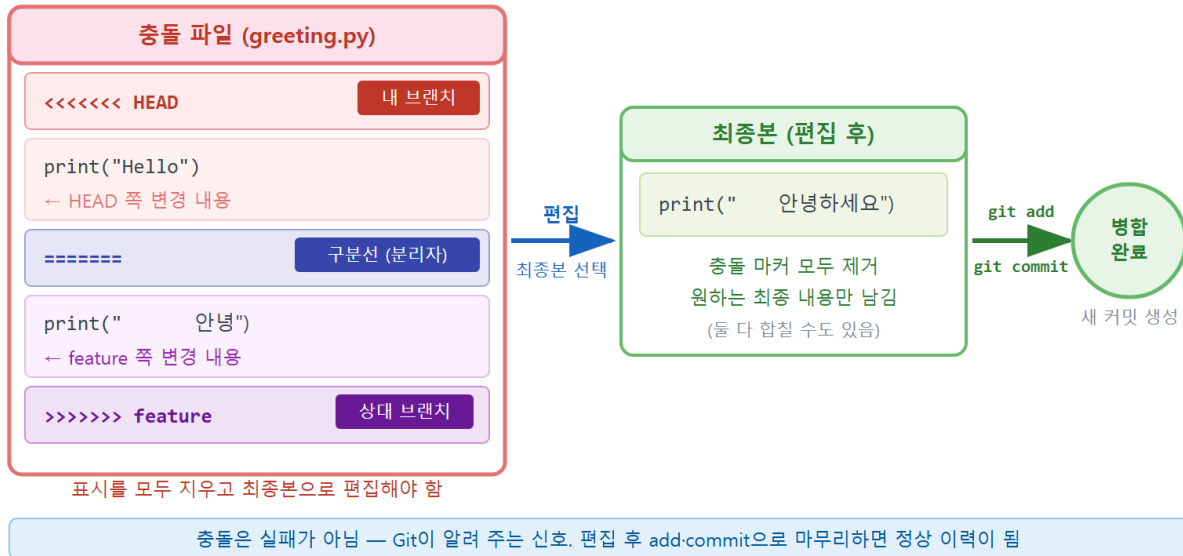
**선수 개념 한 줄 상기(c06).** 병합은 합쳐 받을 브랜치(main)로 먼저 `switch` 한 뒤 `git merge <기능브랜치>` 를 실행하는 것이라고 배웠습니다. 충돌도 바로 이 `git merge` 도중에 일어납니다. 그리고 충돌을 진단하는 첫 도구는 늘 그렇듯 `git status` (c02)이며, 양쪽이 무엇을 어떻게 바꿨는지는 `git diff`·`git log` (c04)로 읽습니다. 새로운 마법이 아니라, **이미 배운 진단 명령들을 충돌 상황에 적용**하는 것이 이 장의 전부입니다.

이 장이 일관되게 따르는 흐름은 다음 다섯 박자입니다. 모든 worked example과 연습이 이 박자를 따릅니다.

1. **상황** — 두 갈래가 같은 곳을 다르게 고쳤다.
2. **명령** — `git merge` 를 실행한다.
3. **문제 발생** — 충돌이 나고 병합이 멈춘다.
4. **진단** — `git status` 로 충돌 파일을, 파일을 열어 충돌 표시를 읽는다.
5. **해결·검증** — 최종본으로 편집 → `git add` → `git commit` → `git log` 실행으로 확인한다.

## 7.2 동작 원리: 충돌 표시 <<<<<<< =====>>>>>>> 읽기

## 충돌 표시(conflict marker)와 해결 흐름



## 정의

이 장의 개념인 **충돌 해결** — **merge conflict**를 한 문장으로 정의하면 이렇습니다. **병합 충돌 (merge conflict)**이란, 두 브랜치가 같은 파일의 같은 부분(보통 같은 줄)을 서로 다르게 바꿔, Git이 어느 쪽을 택할지 스스로 정하지 못하고 병합을 멈춘 상태를 말합니다. 이때 Git은 충돌이 난 파일 안에 **충돌 표시(conflict marker)** 라는 특수한 기호로 양쪽 내용을 함께 적어 넣고, 사람이 최종 형태를 정해 주기를 기다립니다.

## 충돌이 생기는 조건과 안 생기는 경우

핵심은 "같은 곳"입니다. 같은 파일을 고쳤더라도 **서로 다른 줄**을 고쳤으면 Git이 알아서 둘 다 반영해 충돌이 나지 않습니다. 충돌은 두 갈래가 **공통 조상(merge base)**으로부터 **같은 줄을 다른 내용으로** 바꿨을 때만 일어납니다.

| 상황                          | 충돌이 나는가                     | 이유                  |
|-----------------------------|-----------------------------|---------------------|
| 서로 다른 파일을 수정                | 안 남                         | 겹치는 부분이 없어 자동 병합    |
| 같은 파일의 서로 다른 줄을 수정          | 보통 안 남                      | Git이 양쪽 변경을 모두 반영   |
| 같은 파일의 <b>같은 줄</b> 을 다르게 수정 | <b>남</b>                    | 어느 쪽을 택할지 사람만 결정 가능 |
| 한쪽은 줄을 삭제, 다른 쪽은 그 줄을 수정    | <b>남</b>                    | 삭제와 수정이 상충          |
| 한쪽은 파일을 삭제, 다른 쪽은 그 파일을 수정  | <b>남</b><br>(modify/delete) | 파일을 남길지 지울지 상충      |

## 충돌 표시의 구조

충돌이 난 파일을 열면 이런 모양을 보게 됩니다. 이 다섯 줄의 구조를 정확히 읽는 것이 충돌 해결의 절반입니다.

```
<<<<<<< HEAD
# 우리 팀 To-Do 앱 — 함께 만드는 할 일 관리
=====
# Team To-Do — 우리 팀 협업 할 일 보드
>>>>>>> feature/title-b
```

각 줄의 의미는 다음과 같습니다.

- <<<<<<< HEAD — 여기서부터가 **현재 내 브랜치(HEAD, 보통 main)**의 내용이라는 시작 표시입니다. HEAD는 지금 내가 합쳐 받고 있는 쪽, 즉 "우리(ours)"입니다.
- 그 아래 줄 # 우리 팀 To-Do 앱 — 함께 만드는 할 일 관리 — HEAD 쪽이 가진 실제 내용입니다.
- ===== — 위쪽(우리)과 아래쪽(상대)을 **가르는 경계선**입니다. 이 줄을 기준으로 위는 내 브랜치, 아래는 들어오는 브랜치입니다.
- # Team To-Do — 우리 팀 협업 할 일 보드 — 합쳐 들어오는 브랜치(theirs)의 내용입니다.
- >>>>>>> feature/title-b — 여기까지가 **들어오는 브랜치(feature/title-b)**의 내용이라는 끝 표시입니다. 어느 브랜치를 합치는 중인지 이름이 함께 적힙니다.



그림으로 외우면 쉽습니다. 위( <<<<<<< HEAD ~ ===== )는 나, 아래( ===== ~ >>>>>>> )는 상대입니다. 해결이란 이 일곱 표시 줄(시작·경계·끝)을 모두 지우고, 위·아래 두 내용 중 하나를 고르거나 둘을 적절히 합쳐 내가 원하는 최종 한 형태만 남기는 작업입니다.

### 공통 조상까지 보고 싶다면 diff3 스타일

기본 충돌 표시는 양쪽(나·상대)만 보여 줍니다. `git config --global merge.conflictStyle zdiff3` 을 한 번 설정해 두면, 경계 위에 ||||| 구역이 추가되어 **공통 조상(원래 어떤 내용이었는지)** 까지 보여 줍니다. 양쪽이 무엇을 어떻게 바꿨는지 비교하기 쉬워, 복잡한 충돌에서 특히 유용합니다. 입문 단계에서는 기본 스타일로 충분하지만, 이런 선택지가 있다는 것만 알아 두면 좋습니다.

## 7.3 git status로 Unmerged paths 진단하기

충돌이 나면 가장 먼저 할 일은 편집이 아니라 **진단**입니다. 헛갈리면 무조건 `git status` 부터 — 이 습관은 충돌 상황에서 더욱 빛을 발합니다. 병합 중 `git status` 는 단순히 파일 상태만 알려 주는 것이 아니라, **지금 병합이 진행 중이며 무엇을 하면 되는지**까지 친절히 안내합니다.

충돌 직후의 전형적인 `git status` 출력을 봅시다.

```
On branch main
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
    both modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
```

이 출력을 한 줄씩 해부합니다.

- `You have unmerged paths.` — **병합이 끝나지 않고 멈춰 있다**는 핵심 신호입니다. 지금 저 장소는 '병합 진행 중(merging)' 상태입니다.
- `(fix conflicts and run "git commit")` — 해야 할 일을 알려 줍니다. 충돌을 해결하고 커밋하라는 뜻입니다.
- `(use "git merge --abort" to abort the merge)` — 마음을 바꿔 처음으로 돌아가고 싶으면 `--abort` 를 쓰라는 **안전망 안내**입니다.

- Unmerged paths: 아래의 both modified: README.md — **양쪽이 모두 고친(both modified) 충돌 파일**이 README.md임을 정확히 짚어 줍니다. 충돌 파일이 여러 개면 여기 모두 나열됩니다.

짧은 형식 `git status -s` 로 보면 충돌 파일은 UU 기호로 나타납니다. U 는 unmerged(병합 안 됨)를 뜻하며, 양쪽이 모두 고친 충돌은 UU README.md 로 표시됩니다.

```
UU README.md
```

또 충돌 부분만 빠르게 훑고 싶으면 `git diff` 가 양쪽 변경이 겹친 자리를 충돌 표시와 함께 보여 줍니다. 정리하면, 충돌 진단에는 `git status` (어느 파일이 충돌인가) → **파일 열기(충돌 표시 읽기)** → `git diff` (겹친 내용 비교) 세 도구면 충분합니다. 모두 앞에서 이미 배운 안전한 읽기 명령들입니다.

### 진단 없이 손대지 않습니다

충돌을 처음 만나면 급한 마음에 파일부터 마구 고치기 쉽습니다. 하지만 어느 파일이 충돌인지, 양쪽이 무엇을 바꿨는지 모르고 손대면 한쪽 의도를 통째로 잃기 쉽습니다. `git status` 로 **충돌 파일을 확인하고, 표시를 읽어 양쪽 의도를 파악한 뒤** 편집하는 순서를 지켜세요. 진단은 1초면 되고, 잘못된 해결을 되돌리는 것은 훨씬 오래 걸립니다.

## 7.4 worked example: 같은 줄 충돌을 편집·add·commit 으로 해결

이제 충돌을 처음부터 끝까지 한 번 완주합니다. 앞서 약속한 다섯 박자(상황 → 명령 → 문제 발생 → 진단 → 해결·검증)를 그대로 따라갑니다. 실습 환경은 Windows 11의 Git Bash 또는 PowerShell이며, 명령은 동일합니다.

### 1단계 상황: 두 갈래가 같은 제목 줄을 다르게 고친다

작은 To-Do 웹앱 저장소가 있고, README.md 의 첫 줄(제목)을 두 사람이 각자 다르게 바꾸려 합니다. 혼자서 두 역할을 연기하기 위해, 공통 지점에서 두 개의 브랜치를 만들어 같은 줄을 다르게 수정합니다. 출발 시점의 README.md 는 이렇습니다.

```
# 우리 팀 To-Do 앱
```

```
할 일을 관리하는 작은 웹앱입니다.
```

먼저 공통 조상에서 두 브랜치를 갈라 각각 제목을 다르게 고치고 커밋합니다.

```
# A 역할: 공통 지점에서 feature/title-a를 만들어 제목 수정
git switch main
git switch -c feature/title-a
# README.md 첫 줄을 '# 우리 팀 To-Do 앱 — 함께 만드는 할 일 관리'로 편집한 뒤
git add README.md
git commit -m "docs: 제목에 부제 '함께 만드는 할 일 관리' 추가"

# B 역할: 다시 공통 지점(main)에서 feature/title-b를 만들어 제목을 다르게 수정
git switch main
git switch -c feature/title-b
# README.md 첫 줄을 '# Team To-Do — 우리 팀 협업 할 일 보드'로 편집한 뒤
git add README.md
git commit -m "docs: 제목을 'Team To-Do 협업 할 일 보드'로 변경"
```

두 브랜치는 같은 첫 줄을 서로 다른 내용으로 바꿨습니다. 이것이 충돌의 씨앗입니다.

## 2단계 명령: main으로 돌아와 두 브랜치를 차례로 병합

이제 main으로 돌아와 둘을 차례로 합칩니다. 첫 병합은 깔끔하게, 두 번째에서 충돌이 납니다.

```
git switch main
git merge feature/title-a
```

실행 결과는 다음과 같습니다.

```
Updating a1b2c3d..e4f5a6b
Fast-forward
 README.md | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
```

main이 갈라진 뒤 한 발도 움직이지 않았으므로, 첫 병합은 **fast-forward**로 깔끔하게 끝났습니다(c06). 이제 main의 제목은 title-a의 내용입니다. 이어서 title-b를 합칩니다.

```
git merge feature/title-b
```

## 3단계 문제 발생: 충돌로 병합이 멈춘다

이번에는 다음과 같은 메시지가 나오며 병합이 멈춥니다.

```
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
```

CONFLICT (content): Merge conflict in README.md — README.md의 내용이 충돌했다는 뜻입니다. main은 이미 title-a의 제목으로 바뀌어 있는데, 들어오는 title-b가 같은 줄을 또 다르게 바꾸려 하니 Git이 어느 쪽을 택할지 결정하지 못한 것입니다. Automatic merge failed 는 자동 병합 실패를 알리고, 무엇을 해야 하는지(fix conflicts and then commit)까지 알려 줍니다.

## 4단계 진단: status로 파일을, 파일을 열어 표시를 읽는다

당황하지 말고 `git status` 부터 봅니다.

```
On branch main
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
    both modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
```

충돌 파일은 README.md 하나(both modified)입니다. 파일을 열어 보면 Git이 양쪽 내용을 충돌 표시와 함께 넣어 두었습니다.

```
<<<<<<< HEAD
# 우리 팀 To-Do 앱 — 함께 만드는 할 일 관리
=====
# Team To-Do — 우리 팀 협업 할 일 보드
>>>>>>> feature/title-b

할 일을 관리하는 작은 웹앱입니다.
```

### 충돌 표시 줄별 해설

- <<<<<<< HEAD — 여기부터가 현재 main(우리)의 내용입니다. main에는 이미 title-a 병합 결과가 들어 있으므로, HEAD 쪽 제목은 '함께 만드는 할 일 관리'입니다.
- # 우리 팀 To-Do 앱 — 함께 만드는 할 일 관리 — main(HEAD)이 가진 제목 줄입니다.
- ===== — 경계선. 위는 우리(main), 아래는 상대(feature/title-b).

- # Team To-Do — 우리 팀 협업 할 일 보드 — 들어오는 브랜치 feature/title-b의 제목 줄입니다.
- >>>>>> feature/title-b — 여기까지가 상대 브랜치 내용이라는 끝 표시입니다.
- 그 아래 할 일을 관리하는 작은 웹앱입니다. 는 양쪽이 건드리지 않은 줄이라 충돌 표시 밖에 그대로 있습니다 — 충돌은 **상충한 부분에만** 표시가 붙습니다.

## 5단계 해결: 최종본으로 편집 → add → commit

이제 사람이 의도를 정합니다. 두 제목의 좋은 점을 합쳐 'Team To-Do — 함께 만드는 우리 팀 할 일 보드'로 정하기로 합니다. **충돌 표시(<<<<<<, =====, >>>>>>)**를 한 줄도 남김없이 지우고, 원하는 최종 한 줄만 남기도록 파일을 편집합니다. 해결 후 README.md는 이렇게 됩니다.

```
# Team To-Do — 함께 만드는 우리 팀 할 일 보드

할 일을 관리하는 작은 웹앱입니다.
```

편집을 저장한 뒤, 이 파일을 다 고쳤다고 Git에 알립니다. **충돌 해결을 표시하는 방법은 git add** 입니다.

```
git add README.md
git status
```

```
On branch main
All conflicts fixed but you are still merging.
  (use "git commit" to conclude merge)

Changes to be committed:
  modified:   README.md
```

All conflicts fixed but you are still merging. — 충돌은 모두 해결됐고, 이제 커밋만 하면 병합이 마무리된다는 안내입니다. 커밋합니다.

```
git commit
```

git commit 을 인자 없이 실행하면 편집기가 열리며 Merge branch 'feature/title-b' 같은 **병합 커밋 메시지가 미리 채워져** 있습니다. 그대로 저장해도 되고, 편집기 없이 바로 마무리하려면 다음처럼 할 수 있습니다.

```
git commit --no-edit
```

```
[main 7a8b9c0] Merge branch 'feature/title-b'
```

## 6단계 검증: log와 파일로 확인한다

병합이 끝났으니 결과를 확인합니다. 충돌 해결 병합은 두 부모를 가진 병합 커밋을 남깁니다.

```
git log --oneline --graph -n 4
```

```
* 7a8b9c0 (HEAD -> main) Merge branch 'feature/title-b'
|\
| * 9d8c7b6 (feature/title-b) docs: 제목을 'Team To-Do 협업 할 일 보드'로 변경
* | e4f5a6b (feature/title-a) docs: 제목에 부제 '함께 만드는 할 일 관리' 추가
|/
* a1b2c3d 프로젝트 초기 구조 추가
```

그래프가 a1b2c3d 에서 두 갈래로 갈라졌다가 7a8b9c0 에서 다시 합쳐지는 것이 보입니다. 마지막으로 파일에 충돌 표시가 남지 않았는지, 의도한 최종본인지 직접 눈으로 확인합니다.

```
git show HEAD:README.md
```

```
# Team To-Do — 함께 만드는 우리 팀 할 일 보드

할 일을 관리하는 작은 웹앱입니다.
```

충돌 표시가 모두 사라지고 원하는 한 줄만 남았습니다. **상황 → 명령 → 문제 발생 → 진단 → 해결 → 검증**의 한 바퀴를 완주했습니다. 다 끝난 기능 브랜치는 `git branch -d feature/title-a` `feature/title-b` 로 정리하면 이력이 깔끔해집니다(c06).

### 충돌 해결의 4동작만 기억하면 됩니다

복잡해 보이지만 해결의 핵심은 단 네 동작입니다. (1) **상황 확인**: `git status` 로 충돌 파일을 본 다. (2) **편집**: 파일을 열어 충돌 표시를 지우고 최종본을 만든다. (3) **해결 표시**: `git add <파일>`. (4) **마무리**: `git commit`. 이 네 동작은 어떤 충돌에서도 똑같습니다. 충돌 파일이 여러 개면 (2)~(3)을 파일마다 반복한 뒤 한 번 커밋하면 됩니다.

## 7.5 git merge --abort로 충돌 전으로 안전 복귀

충돌을 해결하다 보면 "이거 너무 복잡한데, 차라리 처음부터 다시 하고 싶다"거나 "지금 병합할 때가 아니었다"는 생각이 들 때가 있습니다. 다행히 Git은 **병합을 통째로 취소하고 충돌 전 상태로 돌아가는** 안전한 탈출구를 제공합니다.

```
git merge --abort
```

이 명령은 진행 중이던 병합을 취소하고, **병합을 시작하기 직전의 깨끗한 상태로 작업트리와 인덱스를 되돌립니다**. 충돌 표시가 끼어든 파일도 원래대로 복구됩니다. 짧은 시나리오로 확인해 봅시다.

```
git merge feature/title-b
```

```
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
```

충돌이 났지만, 지금은 해결할 마음이 없습니다. 깔끔히 취소합니다.

```
git merge --abort
git status
```

```
On branch main
nothing to commit, working tree clean
```

`working tree clean` — 충돌도 충돌 표시도 사라지고, 병합을 시도하기 전의 평온한 상태로 완전히 돌아왔습니다. 나중에 준비가 되면 다시 `git merge feature/title-b` 로 시도하면 됩니다.

**--abort**는 '진행 중인 병합'을 취소할 때만

`git merge --abort` 는 **아직 커밋하지 않은, 진행 중인 병합**을 되돌리는 명령입니다. 이미 충돌을 해결하고 `git commit` 까지 마쳐 병합 커밋이 만들어졌다면, 그것은 더 이상 '진행 중인 병합'이 아니므로 `--abort` 대상이 아닙니다. 그 경우는 8장에서 배울 `git reset` (아직 공유 안 한 경우) 이나 `git revert` (공유한 경우)로 되돌립니다. 즉, **충돌 해결 도중이라면 --abort, 병합을 이미 커밋했다면 다음 장의 되돌리기입니다**.

## 7.6 충돌을 줄이는 습관(작은 커밋·작은 통합)

충돌을 잘 푸는 것만큼 중요한 것이 **충돌을 애초에 덜 만드는 습관**입니다. 충돌은 두 갈래가 같은 곳을 오래 따로 고칠수록 늘어나므로, 다음 습관이 효과적입니다.

- **작은 커밋·작은 변경.** 한 커밋이 다루는 범위가 작을수록 다른 사람의 변경과 겹칠 확률이 줄고, 충돌이 나도 해결할 범위가 작습니다.
- **자른 통합(integrate often).** 기능 브랜치를 며칠씩 묵히지 말고, main의 변경을 자주 받아와 통합하면 충돌이 작게 자주 일어나 다루기 쉽습니다. 큰 충돌 한 번보다 작은 충돌 여러 번이 훨씬 쉽습니다.
- **파일·함수 분리.** 서로 다른 사람이 자주 건드리는 코드는 파일이나 함수를 나눠 두면 같은 줄을 동시에 고칠 일이 줄어듭니다.
- **소통.** "나 지금 README 제목 손볼게"라는 한마디가, 같은 줄을 동시에 고치는 충돌을 미리 막습니다. 협업은 도구만이 아니라 사람의 약속이기도 합니다.

이 습관들은 종합 프로젝트(팀 협업 시뮬레이션)와 이후 실무에서 그대로 적용됩니다. 충돌이 두렵지 않은 사람은 충돌을 잘 푸는 사람이자, 충돌을 작게 유지하는 사람입니다.

## 7.7 흔한 문제·해결: 표시 잔존 커밋·어느 쪽 선택 막힘

충돌 해결에서 입문자가 가장 자주 겪는 두 가지 사고를 '문제 → 진단 → 해결' 형식으로 정면으로 다룹니다.

### 문제 1: 충돌 표시를 깜빡 남긴 채 커밋해 코드가 깨졌다

가장 흔한 사고입니다. 충돌을 해결한다고 했는데 ===== 나 >>>>>>> 같은 표시를 한 줄 빠뜨리고 git add·git commit 을 해 버린 것입니다. 코드 파일이라면 문법 오류로 실행이 깨지고, 문서라면 이상한 기호가 그대로 남습니다.

**진단.** 무엇이 잘못됐는지 git diff 나 직접 검색으로 충돌 표시가 남았는지 확인합니다. 방금 한 커밋이라면 git show HEAD 로 내용을 봅니다.

```
git show HEAD:src/app.js
```



```
function addTask(title) {
<<<<<<< HEAD
  return { id: Date.now(), title, done: false };
=====
  return { id: crypto.randomUUID(), title, done: false };
>>>>>>> feature/uuid-id
}
```

<<<<<<<, =====, >>>>>>> 가 그대로 코드에 남아 있어 JavaScript 문법이 깨졌습니다.

**해결.** 파일을 다시 열어 충돌 표시를 모두 지우고 최종본만 남긴 뒤, 다시 `git add · git commit` 합니다.

```
# src/app.js에서 충돌 표시를 모두 지우고 원하는 한 형태만 남긴 뒤
git add src/app.js
git commit -m "fix: 충돌 표시 잔존 제거 — UUID 방식으로 통일"
```

**예방.** 커밋 전에 충돌 표시가 남았는지 검색하는 습관을 들이면 이 사고를 거의 막을 수 있습니다. `git diff --check` 는 충돌 표시 같은 문제를 알려 줍니다.

```
git diff --check
```

```
src/app.js:2: leftover conflict marker
```

## 문제 2: 어느 쪽을 골라야 할지 몰라 막혔다

충돌 표시를 읽었는데 위(HEAD)와 아래(상대) 중 무엇이 맞는지 판단이 서지 않아 손이 멈추는 경우입니다.

**진단.** 양쪽이 각각 무엇을, 왜 바꿨는지 맥락을 확인합니다. 각 브랜치의 마지막 커밋 메시지를 보면 의도가 드러납니다.

```
git log --oneline -1 feature/title-a
git log --oneline -1 feature/title-b
```

**해결의 선택지.** 막혔을 때 떠올릴 수 있는 길은 네 가지입니다.

1. **위(우리)를 택한다** — 표시를 지우고 HEAD 쪽 내용만 남깁니다.
2. **아래(상대)를 택한다** — 표시를 지우고 들어오는 브랜치 내용만 남깁니다.
3. **둘을 합친다** — 양쪽의 좋은 점을 모아 새 최종본을 직접 만듭니다(7.4의 방식).

4. **일단 멈추고 되돌린다** — 판단이 도저히 안 서면 `git merge --abort` 로 충돌 전으로 돌아가, 동료와 상의하거나 맥락을 더 파악한 뒤 다시 시도합니다.

대부분의 협업 충돌은 3번(둘을 합치기)이 정답일 때가 많습니다. 한쪽을 통째로 버리면 그 사람의 의도가 사라지기 때문입니다. **막히는 것은 잘못이 아닙니다.** 막히면 4번으로 안전하게 후퇴할 수 있다는 사실이 충돌을 두렵지 않게 만듭니다.

## 7.8 즉시 연습(2종 1세트): 충돌 최종본 직접 작성

### 즉시 연습 A — 충돌 표시 읽고 최종본 직접 작성 (2종 1세트 · 1/2)

아래는 To-Do 앱의 `index.html` 제목 영역에서 난 충돌입니다. A 브랜치(HEAD)는 한국어 제목으로, B 브랜치는 영문 부제를 추가하는 방향으로 같은 줄을 고쳤습니다. 두 의도를 **모두 살리는** 최종본을 직접 작성해 보세요.

```
<<<<<<< HEAD
    <h1>우리 팀 할 일 보드</h1>
=====
    <h1>Team To-Do Board</h1>
>>>>>>> feature/en-title
```

### 모범 흐름(읽고 참고)

1. `git status` 로 충돌 파일이 `index.html` 임을 확인한다.
2. 위(우리)는 '우리 팀 할 일 보드', 아래(상대)는 'Team To-Do Board'임을 읽는다.
3. 두 의도를 합쳐 한 줄로 만들고, 충돌 표시 세 줄을 모두 지운다.
4. `git add index.html` → `git commit` 으로 마무리한다.

**빈 템플릿(직접 작성)** — 충돌 표시를 모두 지우고, 두 제목을 합친 최종 한 줄을 적어 보세요.

```
<h1>_____</h1>
```

이어서, 해결을 마무리하는 명령을 순서대로 적어 보세요.

```
[해결 표시] git _____ index.html
[마무리]     git _____
[검증]      git show HEAD:index.html → 충돌 표시가 _____ (남아 있음 / 모두 사라짐)
```

자가체크:

- [ ] 충돌 표시 세 줄(<<<<<<, =====, >>>>>>)을 빠짐없이 지웠는가.
- [ ] 한쪽을 통째로 버리지 않고 두 의도를 최종본에 반영했는가(또는 의식적으로 한쪽을 택한 이유가 있는가).
- [ ] 해결 표시는 `git add`, 마무리는 `git commit` 임을 정확히 적었는가.
- [ ] 검증 단계에서 충돌 표시가 모두 사라졌음을 확인했는가.

### 즉시 연습 B — 막혔을 때의 판단과 안전 복귀 (2종 1세트 · 2/2)

다음 상황에서 어떤 명령을 왜 쓸지 판단해 보세요. 충돌이 너무 복잡해 지금은 해결할 자신이 없고, 동료와 상의한 뒤 다시 하고 싶습니다.

상황: `git merge feature/big-refactor` 도중 5개 파일에서 충돌이 났고, 무엇이 옳은지 판단이 서지 않습니다.

빈 템플릿(직접 작성) — 빈칸을 채우세요.

```
[진단] 먼저 git _____ 로 충돌 파일이 몇 개이고 어떤 파일인지 확인한다.
[결정] 지금은 해결하지 않고 충돌 전으로 돌아가기로 한다.
[명령] git merge _____ (병합을 통째로 취소)
[확인] git status → '_____ clean' 이 나오면 안전하게 복귀 완료
[다음] 동료와 상의해 의도를 정한 뒤 다시 git merge feature/big-refactor
```

모범 답안(스스로 채운 뒤 비교하세요): [진단] `git status`, [명령] `git merge --abort`, [확인] `working tree clean`. 핵심은 **막혔을 때 억지로 해결하려 들지 않고 안전하게 후퇴하는 선택지를 안다는 것**입니다.

자가체크:

- [ ] 충돌을 만나면 편집보다 `git status` 진단을 먼저 두었는가.
- [ ] `git merge --abort` 가 '진행 중인(커밋 전) 병합'을 되돌리는 명령임을 이해했는가.
- [ ] 복귀 후 `working tree clean` 으로 충돌 전 상태임을 확인했는가.

## 7.9 오개념 교정: 충돌은 정상·표시 제거 필수·add로 해결 표시

### 오개념 1 — "충돌이 났다는 건 내가 뭔가 잘못했다는 뜻이다"

(틀림) 충돌을 에러·실패·고장으로 여겨 당황하거나 작업을 포기한다. (교정) 충돌은 **두 사람이 같은 곳을 다르게 고쳤다는 정상적인 협업 상황**입니다. Git이 함부로 한쪽을 버리지 않고 사람에게

결정을 넘기는 신중함의 표현입니다. 침착하게 `git status` 부터 읽으면 무엇을 해야 할지 Git이 친절히 안내합니다.

## 오개념 2 — "충돌 표시는 대충 뒤도 되겠지"

(틀림) <<<<<<, =====, >>>>>> 표시를 일부 남긴 채 `add · commit` 한다. (교정) 충돌 표시는 반드시 **한 줄도 남김없이 모두 제거**해야 합니다. 표시를 남기면 코드는 문법 오류로 깨지고 문서는 이상한 기호가 남습니다. 커밋 전 `git diff --check` 로 표시 잔존을 점검하는 습관을 들이세요.

## 오개념 3 — "파일만 고치면 충돌이 해결된 것이다"

(틀림) 충돌 파일을 최종본으로 편집만 하고 `git add` 를 하지 않은 채 다음 단계로 넘어가려 한다. (교정) Git은 `git add <파일>` 을 해야 그 파일을 '해결됨'으로 인식합니다. 고치기만 하고 `add` 를 안 하면 `git commit` 이 거부되거나 병합이 마무리되지 않습니다. 순서는 **편집 → `git add` → `git commit`** 입니다.

## 오개념 4 — "충돌 해결은 둘 중 하나를 무조건 골라 버리는 것이다"

(틀림) 항상 위(우리) 또는 아래(상대) 한쪽을 통째로 택하고 다른 쪽을 버린다. (교정) 한쪽을 택하는 것도 한 방법이지만, 협업에서는 **양쪽 의도를 모두 살려 합치는 것**이 정답일 때가 많습니다. 한쪽을 통째로 버리면 동료의 작업이 사라집니다. 표시를 지우고 원하는 최종 형태를 **직접** 작성할 수 있다는 것을 기억하세요.

# 7.10 단원 미니 프로젝트: 두 기능 병합·충돌 해결(2종 1세트)

이제 단원 3(병합·충돌)에서 배운 것을 하나로 모아, **충돌을 직접 일으키고 직접 해결**하는 작은 프로젝트를 완성합니다. 이 단원의 개념 c06(merge)·c07(충돌 해결)만으로 완결됩니다.

## 프로젝트 목표

같은 파일을 서로 다르게 고친 두 브랜치를 만들어, 하나는 fast-forward로, 다른 하나는 충돌이 나는 3-way 병합으로 통합합니다. 충돌을 **상태 진단 → 표시 해석 → 최종본 편집 → `add·commit` → 검증**의 흐름으로 직접 해결해, 충돌 표시가 모두 사라지고 페이지가 정상 동작하는 저장소를 만듭니다.

## 마일스톤 (중분)

- **M1.** feature/title-a 와 feature/title-b 가 README.md 의 같은 제목 줄을 서로 다르게 수정·커밋한다(두 브랜치 모두 공통 지점 main에서 분기).
- **M2.** main에 feature/title-a 를 병합한다(main이 안 움직였으므로 fast-forward). 이어 feature/title-b 를 병합하면 충돌이 발생한다.
- **M3.** git status 로 충돌 파일을 확인하고, 충돌 표시를 제거해 최종본을 작성한 뒤 git add → git commit 으로 병합을 완료하고, git log --graph 와 파일 내용으로 검증한다.

## 모범 — 완성 흐름 (읽고 참고)

```
# M1: 공통 지점에서 두 브랜치를 갈라 같은 줄을 다르게 수정
git switch main
git switch -c feature/title-a
# README.md 첫 줄을 A안으로 편집
git add README.md
git commit -m "docs: 제목 A안으로 수정"
git switch main
git switch -c feature/title-b
# README.md 첫 줄을 B안으로 편집
git add README.md
git commit -m "docs: 제목 B안으로 수정"

# M2: 차례로 병합 — 첫 번째는 fast-forward, 두 번째는 충돌
git switch main
git merge feature/title-a
git merge feature/title-b      # CONFLICT 발생

# M3: 진단 → 해결 → 검증
git status                    # both modified: README.md 확인
# README.md에서 충돌 표시 제거하고 최종본 작성
git add README.md
git commit --no-edit          # 병합 커밋으로 마무리
git log --oneline --graph -n 4 # 두 부모를 가진 병합 커밋 확인
```

병합 커밋이 만들어진 직후의 이력 검증 출력은 다음과 같은 모양입니다.

```
*   c0ffee1 (HEAD -> main) Merge branch 'feature/title-b'
|\
| * b2b2b2b (feature/title-b) docs: 제목 B안으로 수정
* | a1a1a1a (feature/title-a) docs: 제목 A안으로 수정
|/
* 0000abc 프로젝트 초기 구조 추가
```

## 학생 작성형 — 빈 템플릿 (직접 작성)

## 미니 프로젝트 작성형 — 충돌 최종본과 명령 직접 채우기

아래 충돌 블록을 보고, 빈칸을 직접 채워 프로젝트를 완성하세요. 두 제목 의도를 살린 최종본을 만들고, 해결 명령을 적습니다.

충돌 블록(주어진 양쪽 버전):

```
<<<<<<< HEAD
# 우리 팀 To-Do 앱 (A안: 한국어 중심)
=====
# Team To-Do App (B안: 영문 중심)
>>>>>> feature/title-b
```

작성할 최종본(충돌 표시 모두 제거):

```
# _____
```

해결·검증 명령(빈칸 채우기):

```
[진단] git _____ # 충돌 파일 both modified 확인
[해결] (편집기로 충돌 표시를 모두 지우고 최종본 작성)
[표시] git _____ README.md # 해결됨으로 표시
[마무리] git _____ --no-edit # 병합 커밋 생성
[검증1] git log --oneline --_____ -n 4 # 병합 커밋이 두 부모를 갖는지
[검증2] git show HEAD:README.md # 충돌 표시가 모두 사라졌는지
```

### 자가체크(완성 기준)

- [ ] 충돌 표시(<<<, ==, >>>)가 최종본에서 모두 사라졌는가.
- [ ] 병합 커밋이 두 부모를 갖는가(`git log --graph`에서 갈라졌다 합쳐지는 모양 확인).
- [ ] 첫 병합은 fast-forward, 두 번째 병합은 충돌이었음을 설명할 수 있는가.
- [ ] (선택) 막혔을 때 `git merge --abort`로 되돌릴 수 있음을 떠올렸는가.
- [ ] README의 최종 제목이 의도한 한 형태로 정상 표시되는가.

## 7.11 마무리: 다음 장으로

이 장에서 우리는 협업에서 가장 자주 마주치는 사건인 **병합 충돌**을 정면으로 다뤘습니다. 핵심은 네 가지였습니다. 첫째, 충돌은 실패가 아니라 두 사람이 같은 곳을 다르게 고쳤다는 **정상적인 상황**입니다. 둘째, 충돌이 나면 `git status`로 충돌 파일을 진단하고 충돌 표시(<<<<<< 나 /

===== 경계 / >>>>>> 상대)를 읽습니다. 셋째, 표시를 모두 지우고 최종본을 만든 뒤 `git add` 로 해결을 표시하고 `git commit` 으로 마무리합니다. 넷째, 막히면 `git merge --abort` 로 안전하게 후퇴할 수 있습니다.

그런데 한 가지 더 깊은 질문이 남습니다. 충돌을 잘못 해결해 이미 커밋해 버렸다면? 아예 잘못된 커밋을 만들었거나, 실수로 작업을 날렸다면 어떻게 되돌릴까요? 다음 장(8장)에서는 바로 이 '되돌리기'의 세계로 들어갑니다. 작업트리를 되돌리는 `git restore`, 커밋 이력을 되감는 `git reset` (`--soft`·`--mixed`·`--hard`), 공유한 이력을 안전하게 취소하는 `git revert`, 그리고 잃은 것처럼 보이는 커밋을 되살리는 최후의 안전망 `git reflog` 까지 — 실수해도 되돌릴 수 있다는 자신감을 손에 넣게 됩니다.





## 8장 되돌리기: restore·reset·revert 구분과 reflog 안전망

이 장은 단원 4(u4) '실수를 되돌리고 작업을 잠시 미루기'의 첫 챕터입니다. 이 장에서 배우는 개념은 **되돌리기 — restore·reset·revert 구분(git restore, reset, revert, c08)** 한 가지(세 명령 + 안전망 reflog)입니다. 앞 장에서 충돌을 해결하다 잘못 커밋하는 일이 생길 수 있다고 했습니다. 이 장은 바로 그 '잘못'을 되돌리는 방법을 다룹니다. Git을 배우며 가장 큰 자신감을 주는 단원 — **실수해도 거의 다 되돌릴 수 있다** — 의 출발점입니다.

### 이 장을 마치면 할 수 있는 것

- `restore · reset · revert` 가 각각 **무엇을** 되돌리는지 구분해 설명할 수 있습니다.
- `git reset` 의 `--soft · --mixed · --hard` 가 3영역(저장소·스테이징·작업트리)에 미치는 영향을 표로 설명할 수 있습니다.
- 이미 공유한(push한) 커밋은 `reset` 대신 `revert` 로 되돌려야 하는 이유를 설명할 수 있습니다.
- `git reflog` 로 `reset --hard` 등으로 잃은 것처럼 보이는 커밋을 복구할 수 있습니다.

### 8.1 도입: 실수는 반드시 일어난다 — 복구라는 자신감

코드를 다루는 일에서 실수는 예외가 아니라 일상입니다. 엉뚱한 파일을 스테이징하고, 커밋 단위를 잘못 나누고, 방금 한 수정을 다 망치고, 심지어 멀쩡한 커밋을 날려 버리기도 합니다. 초보일수록 이런 실수 앞에서 얼어붙습니다. "이거 영영 못 되돌리는 거 아니야?" 하는 불안 때문입니다.

이 장의 메시지는 단호합니다. **Git에서는 한 번이라도 커밋된 것은 거의 다 되돌릴 수 있습니다.** 작업트리의 변경은 `git restore` 로, 커밋 이력은 `git reset` 이나 `git revert` 로, 그리고 '날아간 것처럼 보이는' 커밋조차 `git reflog` 라는 안전망으로 되살릴 수 있습니다. 되돌리기를 제대로

익히면, 실수가 두려워 머뭇거리는 대신 **과감하게 실험하고 안전하게 복구하는** 개발자가 됩니다.

### 왜 세 명령으로 나뉘어 있는가

되돌리기 명령이 세 개나 되는 이유는, **되돌리는 대상이 서로 다르기** 때문입니다. `restore` 는 아직 커밋 안 한 **파일 내용**을, `reset` 은 **브랜치가 가리키는 커밋(이력)**을, `revert` 는 **이미 만들어진 과거 커밋의 변경**을 다룹니다. 하나의 만능 명령으로 뭉뚱그리면 위험합니다. 어떤 상황에 어떤 명령을 쓰는지 구분하는 판단력 자체가 이 장의 핵심 학습 목표입니다. 명령을 외우기보다 **"지금 나는 무엇을 되돌리려 하는가?"**를 먼저 묻는 습관을 들이세요.

**선수 개념 한 줄 상기(c07·c03·c04)**. 앞 장에서 충돌 병합을 잘못 커밋할 수 있다고 했고, 그것을 되돌리는 길이 이 장입니다. 또 되돌리기를 이해하려면 c03의 **3영역**(작업트리 → 스테이징 → 저장소)과 c04의 **이력 읽기**(`git log` · 커밋 해시·HEAD)가 바탕이 됩니다. `reset` 이 3영역에 어떤 영향을 주는지, 어떤 커밋으로 되감는지는 모두 이 두 개념 위에서 이해됩니다. 헛갈리면 진단의 출발점은 언제나 `git status` 와 `git log` 입니다.

## 8.2 git restore: 작업트리·스테이징 되돌리기(--staged)

가장 가볍고 안전한 되돌리기부터 시작합니다. `git restore` 는 **아직 커밋하지 않은 변경**을 되돌립니다. 두 가지 쓰임이 있으며, 둘을 구분하는 것이 전부입니다.

### 두 가지 쓰임

```
git restore <파일>           # (1) 작업트리의 수정을 마지막 커밋 상태로 되돌림
git restore --staged <파일>  # (2) 스테이징(add)을 취소 — 작업트리 변경은 그대로 둠
```

- `git restore <파일>` — 파일을 고쳤는데 그 수정을 통째로 버리고 마지막 커밋(HEAD) 상태로 되돌립니다. 작업트리의 변경이 **사라지므로**, 되살릴 수 없는 변경에는 주의합니다.
- `git restore --staged <파일>` — `git add` 로 스테이징한 것을 **언스테이지(unstage)** 합니다. 즉 스테이징 영역에서만 빼내고, 작업트리의 수정 내용 자체는 그대로 보존합니다. "add를 잘못했다" 싶을 때 쓰는 안전한 취소입니다.

이 둘의 차이가 핵심입니다. `--staged` 는 **스테이징만** 되돌려 작업 내용을 보존하지만, 옵션 없는 `git restore` 는 **작업트리 변경 자체를 버립니다**. 둘을 헛갈리면 애써 한 작업을 날릴 수 있습니다.

## 빠른 시연

```
# 파일을 고치고 실수로 add 했다고 하자
git add app.js
git status
```

```
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    modified:   app.js
```

`git status` 가 친절하게 `git restore --staged <file>` 로 언스테이지하라고 알려 줍니다(이것도 진단의 일부입니다). `add`만 취소합니다.

```
git restore --staged app.js
git status
```

```
On branch main
Changes not staged for commit:
  (use "git restore <file>..." to discard changes in working directory)
    modified:   app.js
```

스테이징은 풀렸지만 `app.js` 의 수정 내용은 작업트리에 그대로 남았습니다(`Changes not staged`). 만약 이 수정 자체도 버리고 싶다면, 그때 `git restore app.js` 를 씁니다 — 이번에는 작업트리 변경이 사라집니다.

### 옛 git checkout과의 관계

`git restore` 는 Git 2.23에서 도입된 비교적 새 명령으로, 과거 `git checkout` 이 한꺼번에 맡던 두 역할 중 **파일 복원** 역할을 떼어낸 것입니다(브랜치 전환 역할은 `git switch` 가 가져갔습니다, c05). 옛 자료에서 `git checkout -- <파일>` 이나 `git reset HEAD <파일>` 로 소개되는 동작이, 지금은 각각 `git restore <파일>` 과 `git restore --staged <파일>` 로 더 명확해졌습니다. 둘 다 여전히 동작하지만, 의미가 또렷한 `restore` 를 권합니다.

## 8.3 동작 원리: reset --soft/--mixed/--hard와 3영역

## git reset 모드별 3영역 영향



## 정의

`git reset` 은 현재 브랜치가 가리키는 위치(HEAD)를 다른 커밋으로 옮겨 이력을 되감는 명령입니다. `restore` 가 파일 내용을 다루는 것과 달리, `reset` 은 브랜치 포인터(이력) 자체를 움직입니다. 이때 세 영역(저장소·스테이징·작업트리)을 어디까지 함께 되돌릴지를 `--soft` · `--mixed` · `--hard` 옵션이 결정합니다.

## 세 옵션의 차이 — 3영역 영향 비교

c03에서 배운 3영역을 떠올리세요. `reset` 은 항상 브랜치가 가리키는 커밋(HEAD)을 옮기지만, 스테이징과 작업트리를 함께 되돌릴지는 옵션마다 다릅니다.

| 옵션                        | 저장소<br>(HEAD 이동) | 스테이징(인덱스)   | 작업트리(파일)    | 한마디                       |
|---------------------------|------------------|-------------|-------------|---------------------------|
| <code>--soft</code>       | 옮김               | 그대로 둠       | 그대로 둠       | 커밋만 풀어 변경을 스테이징에 되살림      |
| <code>--mixed</code> (기본) | 옮김               | HEAD에 맞춰 해제 | 그대로 둠       | 커밋·스테이징을 풀어 변경을 작업트리에 되살림 |
| <code>--hard</code>       | 옮김               | HEAD에 맞춰 해제 | HEAD에 맞춰 폐기 | 커밋·스테이징·작업트리 모두 되돌림(위험)   |

핵심을 비유로 정리합니다. 세 옵션은 "얼마나 깊이 되돌리느냐"의 차이입니다.

- `--soft` — 커밋만 살짝 풀어, 그 커밋이 담았던 변경을 스테이징에 그대로 올려 둡니다. 바로 다시 커밋할 수 있는 상태입니다. "방금 커밋을 다시 만들고 싶다"에 알맞습니다.
- `--mixed` (옵션을 안 쓰면 이것) — 커밋과 스테이징을 풀어, 변경을 작업트리(수정 상태)로 내려놓습니다. "커밋과 add를 둘 다 취소하고 다시 고르고 싶다"에 알맞습니다.
- `--hard` — 커밋·스테이징·작업트리를 모두 대상 커밋 상태로 되돌립니다. 커밋하지 않은 작업트리 변경이 영구히 사라지므로 가장 위험합니다.

## 어떤 커밋으로 되감을지 — HEAD~1

되감을 대상은 커밋으로 지정합니다. 자주 쓰는 표기가 `HEAD~1` 입니다.

- `HEAD` — 현재 커밋(가장 최신).
- `HEAD~1` — 현재의 바로 직전(부모) 커밋. `HEAD~2` 는 두 단계 전입니다.
- `git reset --soft HEAD~1` — "직전 커밋 하나를 풀어, 그 변경을 스테이징에 되살려라"는 가장 흔한 사용입니다.

`--hard`는 미커밋 변경을 영구히 지웁니다

`git reset --hard` 는 가장 강력하지만 가장 위험합니다. 아직 커밋하지 않은 작업트리·스테이징 변경을 되돌릴 수 없게 지웁니다. 한 시간 동안 고친 코드를 커밋도 stash도 하지 않은 채 `--hard` 를 실행하면, 그 작업은 사라지고 reflog로도 되살리기 어렵습니다(reflog는 '커밋된' 지점만 복구합니다). 더하여, 이미 push해 공유한 브랜치에는 절대 쓰지 않습니다(8.6에서 다룹니다). `--hard` 를 쓰기 전에는 "지울 미커밋 변경이 없는가?"를 `git status` 로 반드시 확인하세요.

## 8.4 worked example: 잘못 add·잘못 커밋을 restore·reset으로 복구

restore와 reset을 실제 복구 흐름으로 묶어 봅니다. 다섯 박자(상황 → 명령 → 문제 발생 → 진단 → 해결·검증)를 따릅니다.

### 1단계 상황: 두 가지 실수가 겹쳤다

To-Do 앱을 작업하다 두 실수를 했습니다. (가) 디버그용 임시 파일 `debug.log` 를 실수로 `git add` 했고, (나) "기능 추가"와 "오타 수정"을 한 커밋에 뭉쳐 잘못 커밋했습니다. 아직 push 전(공유 전)이라 마음 편히 정리할 수 있습니다.

## 2단계~3단계 명령과 문제 발생: 잘못 add, 잘못 커밋

```
# (가) 임시 파일을 실수로 스테이징
git add app.js debug.log
git commit -m "기능 추가와 오타 수정" # (나) 두 가지를 한 커밋에 뭉침
```

커밋하고 나니 두 가지가 마음에 걸립니다. `debug.log` 는 커밋에 들어가면 안 됐고, 한 커밋에 서로 다른 두 작업을 섞은 것도 나중에 이력을 어지럽힙니다.

## 4단계 진단: log와 show로 무엇이 잘못됐는지 확인

```
git log --oneline -n 2
```

```
3c4d5e6 (HEAD -> main) 기능 추가와 오타 수정
0a1b2c3 프로젝트 초기 구조 추가
```

```
git show --stat HEAD
```

```
commit 3c4d5e6 (HEAD -> main)
  기능 추가와 오타 수정

 app.js      | 12 ++++++++
 debug.log   |  3 +++
 2 files changed, 15 insertions(+)
```

진단 결과가 분명합니다. 방금 커밋( 3c4d5e6 )에 `debug.log` 가 딸려 들어갔고, 의미가 다른 변경이 한 커밋에 섞여 있습니다.

## 5단계 해결: reset --soft로 커밋을 풀고, restore로 임시 파일을 빼고, 다시 커밋

아직 push 전이므로 직전 커밋을 풀어 다시 만드는 것이 가장 깔끔합니다. `--soft` 로 커밋만 풀어 변경을 스테이징에 되살립니다.

```
git reset --soft HEAD~1
git status
```

```
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   app.js
        new file:   debug.log
```

## 무슨 일이 일어났나

- `git reset --soft HEAD~1` — 직전 커밋(3c4d5e6)을 풀어 HEAD를 그 부모(0a1b2c3)로 옮겼습니다. `--soft`라서 커밋이 담았던 변경(app.js, debug.log)은 **스테이징에 그대로 남아 있습니다**(Changes to be committed). 작업 내용은 하나도 잃지 않았습니다.
- 이제 스테이징을 다시 고를 수 있습니다. debug.log는 애초에 들어가면 안 됐으니 스테이징에서 빼냅니다.

debug.log만 언스테이지하고, 추적도 하지 않도록 정리한 뒤 다시 커밋합니다.

```
git restore --staged debug.log      # 스테이징에서만 제거(파일은 남음)
git commit -m "feat: 할 일 완료 토글 기능 추가"
git status
```

```
On branch main
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        debug.log

nothing added to commit but untracked files present
```

debug.log는 다시 추적 안 됨(untracked) 상태가 되었고(필요하면 .gitignore에 넣으면 됩니다, c03), app.js 변경만 의미 있는 메시지로 깔끔하게 커밋했습니다.

## 6단계 검증

```
git log --oneline -n 2
```

```
9f8e7d6 (HEAD -> main) feat: 할 일 완료 토글 기능 추가
0a1b2c3 프로젝트 초기 구조 추가
```

```
git show --stat HEAD
```

```
commit 9f8e7d6 (HEAD -> main)
  feat: 할 일 완료 토글 기능 추가

app.js | 12 ++++++++
1 file changed, 12 insertions(+)
```

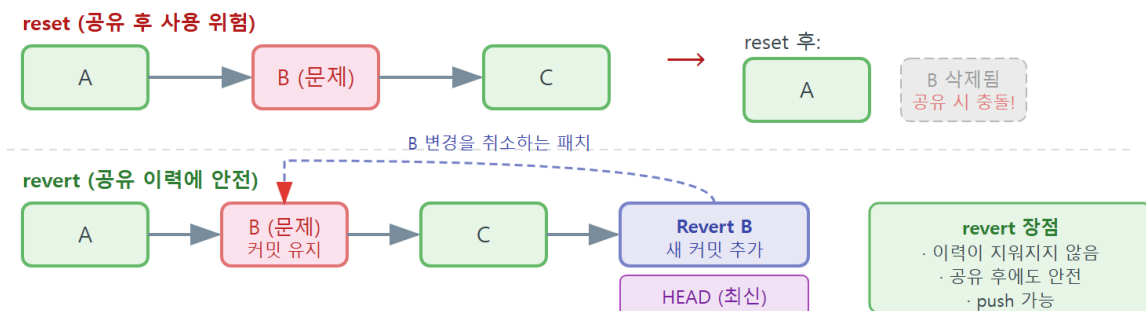
debug.log 가 빠지고 app.js 만 담긴 깨끗한 커밋이 됐습니다. `restore` 는 파일·스테이징을, `reset --soft` 는 커밋(이력)을 되돌린다는 역할 분담이 한 흐름에 모두 드러났습니다.

push 전이라서 `reset`이 안전했습니다

이 예제가 `reset`을 마음 편히 쓸 수 있었던 결정적 이유는 **잘못된 커밋을 아직 push하지 않았다 (공유 전)**는 점입니다. 내 컴퓨터 안에서만 존재하는 이력을 정리하는 것이라 누구의 이력도 어긋나지 않습니다. 만약 이미 push해 동료가 받아 간 커밋이었다면, `reset` 대신 `revert`를 써야 합니다 — 바로 다음 절의 주제입니다.

## 8.5 git revert: 공유 이력을 안전하게 되돌리는 새 커밋

### git revert — 공유 이력을 안전하게 되돌리기



`git revert <커밋해시>` — 새 되돌림 커밋이 이력에 추가됨

### 정의

`git revert <커밋>` 은 지정한 과거 커밋의 변경을 **취소하는 '새 커밋'을 만들어** 그 효과를 되돌립니다. `reset`이 이력을 **지워서** 되감는 것과 달리, `revert`는 이력을 **지우지 않고**, 되돌리는 내용을 담은 커밋을 **앞에 하나 더 쌓습니다**. 그래서 이미 공유한(push한) 이력에도 안전합니다.

### 동작 원리 — 되돌림 커밋을 새로 쌓는다



예를 들어 커밋 B가 어떤 줄을 추가했다고 합시다. `git revert B`를 실행하면, Git은 그 줄을 **다시 제거하는** 새 커밋 B'를 만듭니다. 이력은 다음처럼 변합니다.

```
revert 전:  A — B — C          (B가 잘못된 변경)
revert 후:  A — B — C — B'      (B'가 B의 변경을 되돌림, 이력은 그대로 보존)
```

B는 이력에 그대로 남고, B의 효과만 B'로 상쇄됩니다. 과거를 **지우는** 게 아니라, "이 변경을 취소합니다"라는 새 기록을 **더하는** 방식입니다. 그래서 동료가 이미 받아 간 B를 건드리지 않아 이력이 어긋나지 않습니다.

## 빠른 시연

이미 push해 공유한 커밋 B(잘못된 변경)를 되돌립니다.

```
git log --oneline -n 3
```

```
c3c3c3c (HEAD -> main, origin/main) 잘못된 가격 계산 로직 추가
b2b2b2b 할 일 정렬 기능 추가
a1a1a1a 프로젝트 초기 구조 추가
```

`origin/main`이 함께 가리키는 것을 보니 이 커밋들은 **이미 원격에 공유**되었습니다. 가장 최근의 잘못된 커밋을 revert합니다.

```
git revert HEAD
```

`git revert`는 되돌림 커밋의 메시지를 담은 편집기를 엽니다. 기본 메시지는 `Revert "잘못된 가격 계산 로직 추가"`처럼 채워져 있습니다. 그대로 저장하면 됩니다.

```
[main d4d4d4d] Revert "잘못된 가격 계산 로직 추가"
1 file changed, 1 insertion(+), 8 deletions(-)
```

```
git log --oneline -n 3
```

```
d4d4d4d (HEAD -> main) Revert "잘못된 가격 계산 로직 추가"
c3c3c3c (origin/main) 잘못된 가격 계산 로직 추가
a... ...
```

잘못된 커밋 `c3c3c3c`는 이력에 남아 있고, 그 효과를 되돌리는 `d4d4d4d`가 새로 쌓였습니다. 이제 이 되돌림 커밋을 `git push`하면, 동료의 이력과 충돌 없이 안전하게 잘못을 취소할 수 있습니다.

니다.

### revert도 충돌이 날 수 있습니다

되돌리려는 커밋 이후에 같은 부분이 또 바뀌었다면, revert 도중에도 병합 충돌과 똑같은 충돌이 날 수 있습니다. 그때는 7장에서 배운 방식 그대로 — `git status` 로 진단하고, 충돌 표시를 정리해 최종본을 만들고, `git add` 후 `git revert --continue` 로 마무리합니다. 포기하려면 `git revert --abort` 로 되돌립니다. 한 번 배운 충돌 해결이 여기서 그대로 재사용됩니다.

## 8.6 reset vs revert: 비공유 정리 vs 공유 취소

이제 가장 중요한 판단 기준을 정리합니다. **reset**과 **revert** 중 무엇을 쓸지는 "그 커밋을 이미 공유(push)했는가?"로 갈립니다.

| 구분                 | git reset                           | git revert                   |
|--------------------|-------------------------------------|------------------------------|
| 하는 일               | 브랜치를 과거 커밋으로 <b>되감음</b><br>(이력을 지움) | 되돌리는 <b>새 커밋</b> 을 쌓음(이력 보존) |
| 이력                 | 커밋이 사라짐                             | 커밋이 그대로 남고 취소 커밋이 추가됨        |
| 공유 전(내 로컬만)        | <b>적합</b> — 깔끔히 정리                  | 가능하나 과함                      |
| 공유 후(push해 동료가 받음) | <b>위험</b> — 이력이 어긋나 force push 사고   | <b>적합</b> — 안전하게 취소          |
| 한마디                | 아직 안 보낸 초안을 고쳐 쓰기                   | 이미 보낸 편지에 정정 편지를 더 보내기       |

핵심 원리는 이렇습니다. 이미 **push한 커밋을 reset으로 지우면**, 내 이력과 원격(동료) 이력이 어긋납니다. 그 어긋남을 억지로 밀어 넣으려면 force push를 하게 되고, 그 순간 동료가 그 위에 쌓은 커밋이 사라지는 사고로 번집니다. 그래서 **공유한 이력은 reset으로 지우지 않고 revert로 되돌린다는** 것이 협업의 황금률입니다(rebase의 황금률, 14장과 같은 원리).

판단을 한 문장으로: "내 컴퓨터 안에만 있는 커밋이면 **reset**, 이미 세상에 내보낸 커밋이면 **revert**." `git log` 에서 그 커밋에 `origin/...` 이 함께 붙어 있으면 공유된 것이니 revert를 택합니다.

## 공유 이력을 reset으로 지우면 생기는 일

이미 push한 커밋을 `git reset --hard HEAD~1` 로 지우고 force push하면, 원격의 그 커밋이 사라집니다. 그런데 동료도 그 커밋을 이미 받아 그 위에서 작업하고 있었다면, 동료의 이력과 어긋나 다음에 그가 push-pull할 때 혼란과 충돌이 생기고, 최악의 경우 누군가의 커밋이 유실됩니다. "공유한 것은 지우지 말고 되돌린다(revert)" — 이 한 줄이 팀의 이력을 지킵니다.

## 8.7 worked example: reset --hard 사고를 git reflog로 복구

가장 극적인 복구를 다룹니다. 실수로 `git reset --hard` 를 해 커밋이 사라진 것처럼 보일 때, `git reflog` 라는 안전망으로 되살리는 흐름입니다.

### 1단계 상황: 커밋 두 개를 날린 줄 알았다

며칠 작업해 의미 있는 커밋 두 개를 쌓았습니다. 그런데 옛 상태를 잠깐 보려고 `git reset --hard` 를 잘못 입력해, **방금까지 있던 커밋 두 개가 사라진 것처럼** 보입니다.

### 2단계~3단계 명령과 문제 발생

```
git log --oneline -n 3
```

```
e5e5e5e (HEAD -> main) feat: 마감일 표시 기능
d4d4d4d feat: 우선순위 정렬 기능
a1a1a1a 프로젝트 초기 구조 추가
```

```
git reset --hard HEAD~2    # 실수: 두 커밋을 되감아 버림
```

실행 결과는 다음과 같습니다.

```
HEAD is now at a1a1a1a 프로젝트 초기 구조 추가
```

```
git log --oneline -n 3
```

```
a1a1a1a (HEAD -> main) 프로젝트 초기 구조 추가
```

feat: 마감일 표시 기능 과 feat: 우선순위 정렬 기능 이 git log 에서 사라졌습니다. 초보라면 여기서 가슴이 철렁합니다. 하지만 커밋은 사라지지 않았습니 — 단지 어떤 브랜치도 가리키지 않게 되었을 뿐입니다.

## 4단계 진단: git reflog로 사라진 커밋의 흔적을 찾는다

git log 는 현재 브랜치가 가리키는 이력만 보여 주지만, git reflog 는 HEAD가 거쳐 간 모든 발자취를 보여 줍니다. reset으로 옮기기 전의 위치가 여기 기록돼 있습니다.

```
git reflog
```

```
a1a1a1a (HEAD -> main) HEAD@{0}: reset: moving to HEAD~2
e5e5e5e HEAD@{1}: commit: feat: 마감일 표시 기능
d4d4d4d HEAD@{2}: commit: feat: 우선순위 정렬 기능
a1a1a1a HEAD@{3}: commit (initial): 프로젝트 초기 구조 추가
```

### reflog 줄별 해설

- HEAD@{0}: reset: moving to HEAD~2 — 가장 최근 동작이 바로 그 reset --hard 였음을 보여 줍니다. 지금 HEAD는 a1a1a1a 에 있습니다.
- HEAD@{1}: commit: feat: 마감일 표시 기능 — reset 직전 HEAD가 가리키던 커밋 e5e5e5e 입니다. 사라진 줄 알았던 그 커밋이 여기 멀쩡히 있습니다.
- HEAD@{2}: commit: feat: 우선순위 정렬 기능 — 그 이전 커밋 d4d4d4d 입니다.
- 즉 복구의 열쇠는 e5e5e5e (또는 HEAD@{1} ) — reset 직전의 위치입니다. 이 해시만 알면 그 지점으로 돌아갈 수 있습니다.

## 5단계 해결: 직전 위치로 되돌린다

reflog에서 찾은 해시(또는 HEAD@{1} )로 브랜치를 다시 옮깁니다.

```
git reset --hard e5e5e5e
```

```
HEAD is now at e5e5e5e feat: 마감일 표시 기능
```

(git reset --hard "HEAD@{1}" 처럼 reflog 참조를 그대로 써도 됩니다. Windows의 PowerShell·Git Bash에서는 중괄호가 든 참조를 따옴표로 감싸면 안전합니다.)

## 6단계 검증

```
git log --oneline -n 3
```

```
e5e5e5e (HEAD -> main) feat: 마감일 표시 기능
d4d4d4d feat: 우선순위 정렬 기능
a1a1a1a 프로젝트 초기 구조 추가
```

사라진 줄 알았던 두 커밋이 모두 돌아왔습니다. **reflog**는 "Git의 블랙박스"입니다 — HEAD가 움직인 기록을 한동안 보관하므로, `reset`·잘못된 머지·브랜치 삭제 같은 사고에서 직전 지점을 찾아 복구하는 최후의 안전망이 됩니다.

### reflog가 복구하는 것은 '커밋'뿐입니다

reflog는 강력하지만 만능은 아닙니다. reflog가 되살리는 것은 **한 번이라도 커밋된 지점**입니다. 그래서 `reset --hard`로 사라진 커밋은 되살릴 수 있지만, **커밋도 stash도 하지 않은 작업트리 변경**은 reflog에 기록이 없어 복구하지 못할 수 있습니다. 교훈은 분명합니다. 위험한 작업(특히 `--hard`) 전에는 일단 `git commit`이나 `git stash` (9장)로 작업을 '커밋된 지점'으로 만들어 두세요. 그러면 reflog가 늘 당신을 구해 줍니다.

## 8.8 흔한 문제·해결: 미커밋 변경 소실·공유 이력 어긋남

되돌리기에서 가장 뼈아픈 두 사고를 '문제 → 진단 → 해결' 형식으로 다룹니다.

### 문제 1: `reset --hard`로 커밋 안 한 작업을 날렸다

한 시간 작업한 변경을 커밋도 stash도 하지 않은 채 `git reset --hard`를 실행해 작업이 사라졌습니다.

**진단.** `git reflog`로 복구 가능 여부를 확인합니다. 핵심 질문은 "그 작업이 한 번이라도 커밋된 적이 있는가?"입니다.

- 커밋된 지점이 있으면 → reflog에서 그 해시를 찾아 `git reset --hard <해시>` 또는 `git checkout <해시>`로 복구합니다(8.7).
- 한 번도 커밋·stash되지 않은 미커밋 변경이라면 → reflog에 기록이 없어 복구가 어렵습니다.

**해결과 교훈.** 커밋된 지점은 복구하되, 미커밋 변경은 되살리기 어렵다는 것을 받아들입니다. 그리고 **예방**이 진짜 해결입니다. `--hard` 나 위험한 명령 전에는 반드시 `git status` 로 잃을 변경이 있는지 확인하고, 있으면 먼저 `git commit` 이나 `git stash` 로 안전한 지점을 만든 뒤 진행합니다.

## 문제 2: 공유한 커밋을 reset으로 지웠다가 이력이 어긋났다

이미 push한 커밋을 `git reset` 으로 지우고 다시 push하려다 거부되거나, force로 밀어 동료 이력과 어긋났습니다.

**진단.** `git status` 와 `git log` 로 내 브랜치와 원격(origin/main)이 얼마나 벌어졌는지 확인합니다. push가 non-fast-forward 로 거부되면(11장), 그것은 "원격이 내가 지운 커밋을 여전히 갖고 있다"는 신호입니다.

**해결.** 억지로 force push하지 않습니다. 이미 공유한 이력을 reset으로 지운 것이 잘못이므로, **원래 상태로 되돌린 뒤 reset이 아니라 revert로 다시 처리**합니다. reflog로 reset 직전 위치(공유와 일치하던 지점)를 찾아 복구하고, 정말 되돌리고 싶은 커밋은 `git revert` 로 되돌림 커밋을 만들어 push합니다.

**예방.** 8.6의 황금률을 지킵니다 — **공유한 이력은 reset/force로 지우지 않고 revert로 되돌린다.** `git log` 에 `origin/...` 이 붙은 커밋을 되돌릴 때는 반사적으로 revert를 떠올리세요.

## 8.9 즉시 연습(2종 1세트): 상황별 복구 명령 고르기

### 즉시 연습 A — 상황에 맞는 복구 명령 직접 고르기 (2종 1세트 · 1/2)

다음 다섯 상황 각각에 가장 알맞은 명령을 보기에서 골라 빈칸에 직접 작성하세요. 핵심은 "무엇을 되돌리려 하는가"와 "공유했는가"를 먼저 묻는 것입니다.

보기: `git restore <파일>` / `git restore --staged <파일>` / `git reset --soft HEAD~1` / `git revert <커밋>` / `git reflog`

- 1) 파일을 git add 했는데, 작업 내용은 남기고 스테이징만 취소하고 싶다.  
→ \_\_\_\_\_
- 2) 방금 수정한 내용을 통째로 버리고 마지막 커밋 상태로 되돌리고 싶다.  
→ \_\_\_\_\_
- 3) 아직 push 안 한 직전 커밋을 풀어, 변경을 스테이징에 되살려 다시 커밋하고 싶다.  
→ \_\_\_\_\_
- 4) 이미 push해 동료가 받아 간 잘못된 커밋을 안전하게 되돌리고 싶다.  
→ \_\_\_\_\_
- 5) 실수로 reset --hard 해 커밋이 사라진 것 같다. 직전 위치를 찾고 싶다.  
→ \_\_\_\_\_

모범 답안(스스로 채운 뒤 비교하세요): 1) `git restore --staged <파일>` 2) `git restore <파일>`  
 3) `git reset --soft HEAD~1` 4) `git revert <커밋>` 5) `git reflog` (로 직전 해시를 찾아 `git reset --hard <해시>` 로 복구).

자가체크:

- [ ] 1,2번에서 `--staged` (스테이징만)와 옵션 없는 `restore`(작업트리까지)의 차이를 구분했는가.
- [ ] 4번에서 **공유한 커밋이라 reset이 아니라 revert**를 골랐는가.
- [ ] 5번에서 `reflog`가 '커밋된 지점'을 찾는 안전망임을 이해했는가.

### 즉시 연습 B — reset 3옵션의 3영역 영향 표 직접 채우기 (2종 1세트 · 2/2)

`git reset` 의 세 옵션이 3영역에 미치는 영향을 직접 채워 표를 완성하세요. 각 칸에 '되돌림(폐기/해제)' 또는 '그대로 둠'을 적습니다.

| 옵션                        | 저장소(HEAD) | 스테이징  | 작업트리  |
|---------------------------|-----------|-------|-------|
| <code>--soft</code>       | 옳김        | _____ | _____ |
| <code>--mixed</code> (기본) | 옳김        | _____ | _____ |
| <code>--hard</code>       | 옳김        | _____ | _____ |

이어서, 각 옵션을 한 문장으로 설명해 보세요.

- `[--soft]` 변경을 \_\_\_\_\_ 에 되살려 바로 다시 커밋할 수 있다.  
`[--mixed]` 변경을 \_\_\_\_\_ 에 되살려 다시 add부터 고를 수 있다.  
`[--hard]` 커밋·스테이징·작업트리를 모두 되돌린다. 미커밋 변경이 \_\_\_\_\_ 으로 위험하다.

모범 답안: `--soft` (스테이징: 그대로 둠 / 작업트리: 그대로 둠), `--mixed` (스테이징: 해제 / 작업트리: 그대로 둠), `--hard` (스테이징: 해제 / 작업트리: 폐기). 설명: `--soft` 는 **스테이징**에, `--mixed` 는 **작업트리**에 되살리며, `--hard` 는 미커밋 변경이 **영구히 사라지므로** 위험합니다.

자가체크:

- [ ] 세 옵션 모두 저장소(HEAD)는 옮긴다는 공통점을 적었는가.
- [ ] `--soft` 는 스테이징에, `--mixed` 는 작업트리에 변경을 되살린다는 차이를 구분했는가.
- [ ] `--hard` 만 작업트리를 폐기해 위험하다는 점을 명시했는가.

## 8.10 오개념 교정: hard 위험·공유엔 revert·restore≠reset

### 오개념 1 — "reset --hard는 그냥 되돌리기 버튼이다"

(틀림) `--hard` 를 가벼운 되돌리기로 여겨 아무 때나 누른다. (교정) `git reset --hard` 는 **커밋하지 않은 작업트리·스테이징 변경을 영구히 지웁니다**. `reflog`로도 '커밋된 지점'만 복구되므로, 미커밋 변경은 못 살릴 수 있습니다. `--hard` 전에는 `git status` 로 잃을 변경이 없는지 확인하고, 있으면 먼저 `commit`이나 `stash`로 안전한 지점을 만드세요.

### 오개념 2 — "이미 push한 커밋도 reset으로 지우면 된다"

(틀림) 공유한 잘못된 커밋을 `reset`으로 지우고 `force push`한다. (교정) 공유한 이력을 `reset`으로 지우면 내 이력과 동료 이력이 어긋나 사고가 납니다. **공유한(push한) 커밋은 reset이 아니라 `git revert`로 되돌림 커밋을 만들어** 안전하게 취소합니다. 판단 기준은 단순합니다 — `git log`에 `origin/...`이 붙어 있으면 `revert`입니다.

### 오개념 3 — "restore와 reset은 비슷한 명령이다"

(틀림) 둘을 같은 '되돌리기'로 뭉뚱그려 아무거나 쓴다. (교정) `restore` 는 **파일 내용**(작업트리·스테이징)을 되돌리고, `reset` 은 **브랜치가 가리키는 커밋(이력)**을 옮깁니다. 대상이 완전히 다릅니다. "파일을 되돌리나, 이력을 되돌리나?"를 먼저 물으면 둘은 헛갈리지 않습니다.

### 오개념 4 — "reflog만 있으면 무엇이든 되살릴 수 있다"

(틀림) `reflog`를 만능 복구 수단으로 믿고 위험한 명령을 함부로 쓴다. (교정) `reflog`는 **한 번이라도 커밋된 지점**을 되살리는 안전망입니다. 커밋·stash되지 않은 변경은 `reflog`에 흔적이 없어 복구하지 못할 수 있습니다. `reflog`는 '커밋한 것'을 지켜 줄 뿐이므로, 중요한 작업은 자주 커밋하는 습관이 진짜 안전망입니다.



## 8.11 마무리: 다음 장으로

이 장에서 우리는 Git이 주는 가장 든든한 능력 — **되돌리기** — 를 배웠습니다. 핵심은 "무엇을 되돌리는가"로 명령이 갈린다는 것이었습니다. 첫째, 아직 커밋 안 한 **파일·스테이징**은 `git restore` (와 `--staged`)로 되돌립니다. 둘째, **커밋 이력**은 `git reset`으로 되감되, `--soft` (스테이징에), `--mixed` (작업트리에), `--hard` (모두 폐기, 위험)의 3영역 영향을 구분합니다. 셋째, **이미 공유한** 잘못된 커밋은 `reset`이 아니라 `git revert`로 되돌림 커밋을 만들어 안전하게 취소합니다. 넷째, 사고가 나도 `git reflog`가 커밋된 지점을 찾아 주는 최후의 안전망이 됩니다.

다음 장(9장)에서는 또 하나의 든든한 도구 `git stash`를 배웁니다. 커밋하기엔 이른 변경을 잠시 선반에 치워 작업트리를 깨끗이 만들고, 급한 핫픽스로 브랜치를 옮겼다가 돌아와 다시 꺼내는 흐름입니다. 그리고 단원 4의 끝에서는 이 장의 `restore·reset·revert·reflog`와 다음 장의 `stash`를 모두 동원해, 네 가지 사고를 직접 만들고 알맞게 복구하는 '**잘못된 커밋 복구 훈련소**' 미니 프로젝트로 이어집니다. 실수를 두려워하지 않는 개발자의 길에 한 발 더 들어선 셈입니다.



# 임시 저장: git stash로 하던 일 미뤄 두기

## 학습 목표

- `git stash` 로 미완성 변경을 안전하게 치우고 작업트리를 깨끗하게 만들 수 있습니다.
- `git stash list` · `pop` · `apply` · `drop` 으로 치워 둔 변경을 관리할 수 있습니다.
- `git stash` 를 활용해 급한 브랜치 전환(핫픽스)을 처리할 수 있습니다.
- `git stash pop` 시 생길 수 있는 충돌을 진단하고 해결할 수 있습니다.

앞 장(8장)에서 우리는 `restore` · `reset` · `revert` 로 이미 만든 변경이나 커밋을 되돌리는 법을 배웠습니다. 그런데 현실에서는 "되돌리고 싶지는 않은데, 지금 당장은 치워 두고 싶은" 변경이 자주 생깁니다. 한창 기능을 만들다 말았는데 커밋하기에는 너무 어설프고, 그렇다고 버리기는 아까운 변경입니다. 이럴 때 쓰는 도구가 이 장의 주인공 `git stash` 입니다. `stash`는 8장의 복구 도구들과 함께 이 단원(실수를 되돌리고 작업을 미루기)을 마무리하는 마지막 조각입니다.

이 장의 선수 개념 상기: 5장의 **분기(branch-switch)** 를 그대로 씁니다. `stash`가 가장 빛나는 순간이 바로 "커밋 안 한 변경이 있는 채로 다른 브랜치로 옮겨야 할 때"이기 때문입니다. 5장에서 "미커밋 변경이 있으면 `switch` 가 거부될 수 있다"고 배운 것을 떠올리면, `stash`가 왜 필요한지 곧바로 이해됩니다.

## 1. 도입: 커밋하기 애매한 변경, 잠시 치워 두기

To-Do 웹앱의 `app.js` 에 "할 일 정렬 기능"을 절반쯤 만들었다고 합시다. 함수 이름만 잡아 두고 본문은 비어 있는, 그야말로 작업 중인 상태입니다. 바로 이때 팀에서 급한 연락이 옵니다. "방금 배포한 메인 화면이 깨졌습니다. 지금 당장 고쳐서 올려야 합니다."

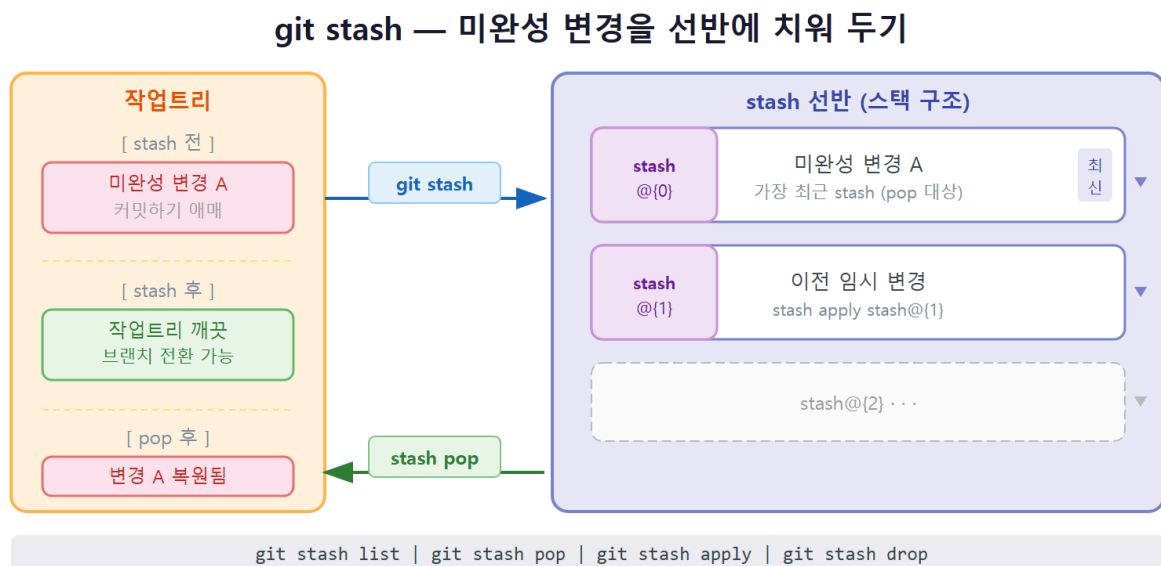
이 상황에서 여러분의 손에는 두 가지 곤란이 있습니다.

- **지금 변경을 커밋하기엔 너무 어설프습니다.** 빈 함수를 "정렬 기능 추가"라고 커밋하면 이력이 거짓말을 하게 됩니다.
- **그렇다고 변경을 버릴 수도 없습니다.** 어렵게 잡아 둔 작업이니까요. 그리고 핫픽스를 하려면 다른 브랜치로 옮겨야 하는데, 미완성 변경이 발목을 잡습니다.

`git stash` 는 이 딜레마를 정확히 풀어 줍니다. 하던 변경을 **선반(stash)**에 잠시 올려 두고 작업 트리를 마지막 커밋 상태로 깨끗하게 되돌립니다. 그러면 홀가분하게 브랜치를 옮겨 핫픽스를 하고, 돌아와서 선반에 둔 변경을 다시 내려 이어 가면 됩니다. "저장도 아니고 삭제도 아닌, 잠깐 미뤄 두기"가 `stash`의 본질입니다.

## 개념: 임시 저장 — `stash (git stash)`

### 2. 동작 원리: 스테시 선반(스택) 구조



#### 정의

**임시 저장(stash)**이란, 아직 커밋하기 이른 작업트리:스테이징의 변경을 잠시 **선반(스택, stack)**에 치워 두고 작업 공간을 마지막 커밋 상태로 깨끗하게 만드는 기능입니다. `git stash` (정식 이름은 `git stash push`)는 변경을 선반에 올리고, 나중에 `git stash pop`으로 다시 내립니다.

핵심 용어를 정리합니다.

- **스테시(stash):** 치워 둔 변경 한 덩어리. 영어 단어 그대로 "감춰 둔 물건"이라는 뜻입니다.
- **선반(스택):** 스테시가 쌓이는 곳. 가장 최근에 올린 것이 맨 위(`stash@{0}`)에 오는 **스택** 구조입니다.

#### 동작 원리: 올리고(push) 내리는(pop) 스택

선반은 책을 쌓아 두는 것과 같습니다. 나중에 올린 책이 맨 위에 오고, 꺼낼 때도 맨 위부터 꺼냅니다. stash도 똑같이 동작합니다.

1. **올리기**(`git stash`): 지금 작업트리·스테이징의 변경을 한 덩어리로 묶어 선반 맨 위에 올립니다. 동시에 작업트리는 마지막 커밋(HEAD) 상태로 **깨끗하게** 돌아갑니다.
2. **쌓이기**: 또 stash를 하면 새 덩어리가 맨 위(`stash@{0}`)에 오고, 기존 것은 한 칸 밀려 `stash@{1}` 이 됩니다.
3. **내리기**(`git stash pop`): 맨 위 스테시를 꺼내 지금 브랜치의 작업트리에 다시 적용하고, 선반에서 그 항목을 **제거**합니다.

이 구조를 표로 보면 다음과 같습니다.

| 시점                               | 선반(스택) 상태                                                   | 작업트리                              |
|----------------------------------|-------------------------------------------------------------|-----------------------------------|
| 변경 작업 중                          | (비어 있음)                                                     | 변경 있음(dirty)                      |
| <code>git stash</code> 직후        | <code>stash@{0}</code> 한 덩어리                                | 깨끗함(clean)                        |
| 또 다른 변경 후 <code>git stash</code> | <code>stash@{0}</code> (새 것), <code>stash@{1}</code> (이전 것) | 깨끗함                               |
| <code>git stash pop</code>       | <code>stash@{0}</code> (이전 것 한 칸 내려옴)                       | <code>stash@{0}</code> 이던 변경이 복원됨 |

여기서 중요한 점은 stash가 **이력(커밋)에 남지 않는 임시 보관**이라는 것입니다. 커밋은 `git log`에 영구히 남지만, 스테시는 선반 위의 임시 메모일 뿐입니다. 그래서 잠깐 미뤘다가 곧 돌아올 변경에만 쓰고, 오래 보관할 작업은 커밋이나 브랜치로 남기는 것이 안전합니다.

**stash가 치우는 것과 치우지 않는 것**: `git stash`는 기본적으로 Git이 **추적 중인(tracked)** 파일의 변경만 치웁니다. 즉 한 번이라도 커밋·add 된 적 있는 파일의 수정만 선반에 올립니다. 아직 한 번도 add 하지 않은 **새 파일(untracked)**은 그대로 작업트리에 남습니다. 새 파일까지 함께 치우려면 `-u` 옵션이 필요합니다(뒤에서 자세히 다룹니다).

### 3. git stash로 치우고 작업트리 정리하기

가장 기본적인 사용법부터 봅니다. 변경이 있는 상태에서 그냥 `git stash`를 실행하면 됩니다.

```
git status      # 먼저 지금 상태를 확인(습관)
git stash       # 추적 중인 변경을 선반에 올리고 작업트리를 깨끗이
git status      # 작업트리가 깨끗해졌는지 확인
```

실행 결과는 다음과 같습니다. `app.js` 를 수정해 둔 상태에서 `stash`를 한 흐름입니다.

```
$ git status
On branch feature/sort
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
       modified:   app.js

no changes added to commit (use "git add" and/or "git commit -a")

$ git stash
Saved working directory and index state WIP on feature/sort: 7c1d9a2 정렬 함수 골격 추가

$ git status
On branch feature/sort
nothing to commit, working tree clean
```

- 첫 `git status` 는 `app.js` 가 `modified` (수정됨)로 잡혀 있음을 보여 줍니다. 아직 커밋도 `add` 도 안 한 작업 중 변경입니다.
- `git stash` 는 `Saved working directory and index state ...` 라는 메시지로 변경을 선반에 올렸음을 알립니다. `WIP on feature/sort` 는 "feature/sort 브랜치에서 작업 중(Work In Progress)이던 것"이라는 자동 설명이고, 뒤의 `7c1d9a2 ...` 는 그 시점의 마지막 커밋입니다.
- 마지막 `git status` 가 `working tree clean` (작업트리 깨끗함)이라고 합니다. 변경이 사라진 것이 아니라 **선반으로 옮겨졌을 뿐**입니다.

만약 변경이 하나도 없는 상태에서 `stash`를 하면 Git은 치울 것이 없다고 알려 줍니다.

```
$ git stash
No local changes to save
```

설명을 붙이고 싶으면 `-m` 옵션으로 메시지를 남길 수 있습니다. 스테시가 여러 개 쌓일 때 무엇이 무엇인지 구분하기 좋습니다.

```
git stash -m "정렬 기능 작업 중간 저장"
```

#### 4. worked example: 작업 중 핫픽스 전환 → 돌아와 pop

이제 도입에서 본 상황을 끝까지 풀어 봅니다. **상황**은 이렇습니다. `feature/sort` 브랜치에서 정렬 기능을 만들다 말았는데, `main` 의 화면이 깨졌다는 긴급 신고가 들어왔습니다. 핫픽스를 한 뒤 다시 정렬 작업으로 돌아와야 합니다.

## 명령

```
# (1) 지금 feature/sort에서 정렬 작업 중 — 미커밋 변경이 있음
git status

# (2) 미완성 변경을 선반에 치워 작업트리를 깨끗이
git stash -m "정렬 기능 WIP"

# (3) 핫픽스를 위해 main으로 전환(이제 미커밋 변경이 없어 깨끗이 전환됨)
git switch main

# (4) 핫픽스 브랜치를 파서 버그 수정
git switch -c hotfix/header
# ... style.css의 깨진 헤더를 한 줄 고침 ...
git add style.css
git commit -m "헤더 레이아웃 깨짐 수정"

# (5) 핫픽스를 main에 통합(6장에서 배운 merge)
git switch main
git merge hotfix/header

# (6) 원래 하던 정렬 작업으로 복귀
git switch feature/sort

# (7) 선반에 치워 둔 변경을 다시 내려 이어 작업
git stash pop
```

## 진단과 결과

핵심 단계의 출력을 따라가 봅니다. (2)에서 stash를 하고, (3)에서 `switch` 가 거부되지 않고 깨끗하게 넘어가는 것이 이 흐름의 묘미입니다.

```
$ git stash -m "정렬 기능 WIP"
Saved working directory and index state On feature/sort: 정렬 기능 WIP

$ git switch main
Switched to branch 'main'

$ git switch feature/sort
Switched to branch 'feature/sort'

$ git stash pop
On branch feature/sort
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
    modified:   app.js

no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (b3f81c4e2a9d5f7c8e1a2b3c4d5e6f7a8b9c0d1e)
```

- `git switch main` 이 아무 거부 없이 `Switched to branch 'main'` 으로 넘어갑니다. (2)에서 stash로 작업트리를 깨끗이 만들어 둔 덕분입니다. 만약 stash를 안 했다면 5장에서 본 "미커밋 변경 때문에 전환 거부" 메시지를 만났을 것입니다.
- 핫픽스를 마치고 `git switch feature/sort` 로 돌아온 뒤 `git stash pop` 을 하면, `app.js` 의 수정이 다시 `modified` 상태로 **복원**됩니다. 작업이 끝난 그 지점에서 그대로 이어 갈 수 있습니다.
- 마지막 줄 `Dropped refs/stash@{0} (...)` 이 중요합니다. `pop` 은 변경을 복원하면서 그 스테시를 선반에서 **제거**했다는 뜻입니다. 즉 한 번 `pop` 하면 선반에서 사라집니다.

이 흐름의 핵심은 "치우기 → 전환 → 고치기 → 돌아오기 → 내리기"라는 순서입니다. stash 덕분에 미완성 변경을 잃지도, 어설픔게 커밋하지도 않고 급한 일을 처리할 수 있습니다.

## 5. stash list·pop·apply·drop과 -u 옵션

선반을 관리하는 명령들을 정리합니다. stash 하나만 쓸 때는 단순하지만, 여러 개를 쌓아 두면 목록을 보고 골라 꺼낼 줄 알아야 합니다.

| 명령                                       | 하는 일                                                                   |
|------------------------------------------|------------------------------------------------------------------------|
| <code>git stash list</code>              | 선반에 쌓인 스테시 목록 보기( <code>stash@{0}</code> , <code>stash@{1}</code> ...) |
| <code>git stash show -p stash@{0}</code> | 특정 스테시의 변경 내용(패치) 미리 보기                                                |
| <code>git stash pop</code>               | 맨 위 스테시를 꺼내 적용하고 선반에서 <b>제거</b>                                        |
| <code>git stash apply</code>             | 스테시를 적용하되 선반에 <b>그대로 남김</b>                                            |
| <code>git stash drop stash@{1}</code>    | 특정 스테시를 적용하지 않고 선반에서 제거                                                |
| <code>git stash clear</code>             | 모든 스테시를 선반에서 비우기(주의: 복구 어려움)                                           |
| <code>git stash -u</code>                | 추적 안 된 새 파일(untracked)까지 함께 치우기                                        |

### 목록을 보고 골라 꺼내기

여러 개를 쌓아 두면 `git stash list` 로 확인하고, 원하는 것을 지정해 적용할 수 있습니다.



```
git stash list           # 쌓인 스테시 확인
git stash show -p stash@{1} # 두 번째 스테시 내용 미리 보기
git stash apply stash@{1} # 두 번째를 적용(선반엔 남김)
git stash drop stash@{1}  # 다 썼으면 제거
```

```
$ git stash list
stash@{0}: On feature/sort: 정렬 기능 WIP
stash@{1}: WIP on main: a1b2c3d README 소개 문구 작성 중

$ git stash apply stash@{1}
On branch feature/sort
Changes not staged for commit:
  modified:   README.md
```

- `git stash list` 는 최신 것이 `stash@{0}`, 그다음이 `stash@{1}` 로 나옵니다. 번호가 작을수록 최근입니다.
- `git stash apply stash@{1}` 은 두 번째 스테시를 적용하지만, 출력에 `Dropped ...` 가 없습니다. `apply` 는 선반에 그대로 남기기 때문입니다. 같은 변경을 여러 브랜치에 적용해 보고 싶을 때 유용합니다.
- 다 쓴 스테시는 `git stash drop stash@{1}` 로 직접 제거합니다. `apply` 는 자동으로 지우지 않으므로, 선반이 지저분해지지 않게 정리하는 습관이 좋습니다.

## pop과 apply의 차이

둘은 "적용한다"는 점은 같지만 **선반에서 지우느냐**가 다릅니다.

- `git stash pop` = 적용 + 제거. 한 번 쓰고 끝낼 변경에 씁니다(가장 흔함).
- `git stash apply` = 적용 + 보존. 같은 변경을 다른 곳에도 적용하거나, 적용이 잘 되는지 먼저 시험해 보고 싶을 때 씁니다.

## -u: 추적 안 된 새 파일까지 치우기

기본 `git stash` 는 새로 만든 파일(untracked)을 치우지 않는다고 했습니다. 새 파일까지 함께 미뤄 두려면 `-u` (또는 `--include-untracked`)를 씁니다.

```
git stash -u -m "새 모듈 파일 포함 임시 저장"
```

```
$ git status -s
M app.js
?? sort.js

$ git stash -u -m "새 모듈 파일 포함 임시 저장"
Saved working directory and index state On feature/sort: 새 모듈 파일 포함 임시 저장

$ git status -s
$
```

- 첫 `git status -s` 에서 `app.js` 는 `M` (추적 중 수정), `sort.js` 는 `??` (추적 안 된 새 파일)로 나옵니다.
- `-u` 없이 `git stash` 만 했다면 `sort.js` 는 작업트리에 그대로 남았을 것입니다. `-u` 를 붙였으므로 새 파일까지 선반에 올라가, 마지막 `git status -s` 출력이 **완전히 비어**(깨끗함) 있습니다.

## 6. 흔한 문제와 해결

### 문제 1 — stash pop 충돌

stash를 치워 둔 사이 같은 파일이 다른 작업으로 바뀌면, `git stash pop` 때 **충돌(conflict)** 이 날 수 있습니다. 7장에서 본 병합 충돌과 똑같은 원리입니다.

**상황:** `feature/sort` 에서 `app.js` 를 고치다 stash 했는데, 그 사이 다른 커밋이 같은 줄을 건드렸습니다. 돌아와 `git stash pop` 을 하니 충돌이 납니다.

```
$ git stash pop
Auto-merging app.js
CONFLICT (content): Merge conflict in app.js
On branch feature/sort
Unmerged paths:
  (use "git restore --staged <file>..." to unstage)
  (use "git add <file>..." to mark resolution)
    both modified:   app.js

The stash entry is kept in case you need it again.
```

**진단:** 충돌이 나면 7장에서 배운 `<<<<<<<`, `=====`, `>>>>>>>` 표시가 `app.js` 에 들어갑니다. `git status` 로 `Unmerged paths` (병합 안 된 경로)를 확인합니다. 그리고 마지막 줄 `The stash entry is kept in case you need it again.` (스태시 항목은 혹시 몰라 남겨 둡니다)에 주목합니다. **충돌이 난 pop은 스테시를 자동으로 지우지 않습니다.**

**해결:** 충돌 표시를 보고 최종본으로 편집한 뒤, `git add` 로 해결을 표시합니다. 그다음, `pop`이 남겨 둔 스테시를 직접 정리합니다.

```
# ... app.js의 충돌 표시를 지우고 최종본으로 편집 ...
git add app.js           # 해결 표시(7장과 동일)
git stash drop           # pop이 충돌로 남겨 둔 스테시를 직접 제거
git stash list           # 선반이 비었는지 확인
```

**왜 직접 drop 해야 하나:** 정상 `pop` 은 적용에 성공하면 스테시를 자동으로 지웁니다. 하지만 충돌이 나면 "혹시 다시 시도할지 모른다"며 스테시를 남깁니다. 충돌을 잘 해결했다면 그 스테시는 더 필요 없으므로 `git stash drop` 으로 정리해 선반을 깨끗이 유지하십시오. 정리하지 않으면 다음에 `git stash list` 에서 이미 푼 변경이 계속 남아 헷갈립니다.

## 문제 2 — untracked 파일이 stash에 안 들어감

**상황:** 새로 만든 `sort.js` 와 수정한 `app.js` 가 있는데, `git stash` 후 브랜치를 옮겼더니 `sort.js` 가 새 브랜치에도 그대로 보입니다.

**진단:** 기본 `git stash` 는 추적 중인 파일의 변경만 치우고, 한 번도 `add` 하지 않은 새 파일은 작업트리에 남깁니다. `git status -s` 에서 그 파일이 `??` 로 표시되는지 확인합니다.

**해결:** 새 파일까지 치우려면 `-u` 를 씁니다. 이미 stash를 한 뒤라면, 새 파일을 한 번 `git add` 로 추적 대상으로 만든 뒤 다시 stash 하거나, 처음부터 `git stash -u` 로 다시 시도합니다.

```
git stash -u              # 처음부터 새 파일까지 함께 치우기
```

## 7. 즉시 연습 (2종 1세트)

### 직접 작성 1 — 핫픽스 전환 흐름 (빈칸 채우기)

`feature/cart` 브랜치에서 장바구니 기능을 만들다 말았는데, `main` 에 긴급 버그 신고가 들어왔습니다. 미완성 변경을 잃지 않고 핫픽스를 처리한 뒤 돌아와 작업을 이어 가는 명령을 완성하십시오.

```

git status                                # 미커밋 변경 확인(습관)
git _____ -m "장바구니 WIP"         # 변경을 선반에 치워 작업트리 정리
git switch main                          # 이제 깨끗하므로 전환됨
git switch -c hotfix/login
# ... 버그 수정 ...
git add .
git commit -m "로그인 버튼 동작 수정"
git switch main
git merge hotfix/login
git switch feature/cart                  # 원래 작업으로 복귀
git stash _____                     # 치워 둔 변경을 내려 이어 작업(적용+제거)

```

## 자가체크 체크리스트

- [ ] 첫 빈칸에 `stash` 를 넣어 미완성 변경을 선반에 올렸는가
- [ ] 둘째 빈칸에 `pop` 을 넣어 변경을 복원하면서 선반에서 제거했는가
- [ ] `git stash` 직후 `git status` 가 `working tree clean` 이 되어 `switch` 가 거부되지 않았는가
- [ ] 핫픽스를 마치고 돌아와 `pop` 한 뒤 작업이 끊긴 지점에서 이어졌는가

## 직접 작성 2 — 여러 스테시 관리와 `pop` 충돌 (직접 작성)

다음 상황을 명령으로 직접 작성하십시오. 빈칸과 빈 줄을 채워 완성하면 됩니다.

1. 선반에 쌓인 스테시 목록을 본다.
2. 두 번째 스테시(`stash@{1}`)의 내용을 미리 본다.
3. 두 번째 스테시를 적용하되 선반에 남긴다.
4. `pop` 도중 충돌이 났다고 가정하고, 충돌을 해결한 뒤 해결을 표시하고 남은 스테시를 직접 제거한다.

```

# 1. 목록 보기
git stash _____

# 2. 두 번째 스테시 내용 미리 보기
git stash show -p _____

# 3. 두 번째 스테시 적용(선반에 남김)
git stash _____ stash@{1}

# 4-1. (충돌 해결 후) 해결 표시
git add _____

# 4-2. pop이 충돌로 남겨 둔 스테시 직접 제거
git stash _____

```

## 자가체크 체크리스트

- [ ] 1번에 `list` 를 넣어 목록을 확인했는가
- [ ] 2번에 `stash@{1}` 을 지정해 특정 스테시를 미리 봤는가
- [ ] 3번에 `apply` 를 넣어 선반에 남기는 방식으로 적용했는가(`pop` 이 아님)
- [ ] 4-1에 충돌 파일 이름을 넣고, 4-2에 `drop` 을 넣어 남은 스테시를 정리했는가

## 8. 흔한 오류와 오개념 교정

### 오개념 1 — "stash는 커밋이니까 오래 뒤도 안전하다"

stash는 커밋이 **아닙니다**. `git log` 에 남지 않고 선반 위에만 임시로 올라가는 보관입니다. 그래서 중요한 작업을 오래 stash에만 두면 잊고 잃기 쉽습니다. 특히 `git stash clear` 를 무심코 실행하면 선반의 모든 스테시가 한 번에 사라집니다.

**교정:** stash는 "곧 돌아올 짧은 미루기"에만 씁니다. 며칠 이상 보관할 작업이라면 차라리 어설피더라도 **임시 커밋**(나중에 `--amend` 나 `reset --soft` 로 정리)이나 **별도 브랜치**로 남기는 편이 안전합니다.

### 오개념 2 — "pop과 apply는 똑같다"

둘 다 변경을 적용하지만 **선반에서 지우느냐**가 다릅니다.

```
$ git stash pop      # 적용 + 선반에서 제거
... Dropped refs/stash@{0} (...)

$ git stash apply    # 적용만, 선반에 그대로 남음
... (Dropped 메시지 없음)
```

**교정:** 한 번 쓰고 끝이면 `pop`, 같은 변경을 다른 브랜치에도 적용하거나 시험만 해 보려면 `apply` 를 쓰고 다 쓴 뒤 `git stash drop` 으로 정리합니다. 또한 충돌이 난 `pop` 은 스테시를 자동으로 지우지 않으니, 해결 후 직접 `drop` 해야 합니다.

### 오개념 3 — "git stash면 새로 만든 파일도 당연히 치워진다"

기본 `git stash` 는 **추적 중인** 파일의 변경만 치웁니다. 한 번도 `add` 하지 않은 **새 파일 (untracked)** 은 작업트리에 그대로 남습니다.

**교정:** 새 파일까지 함께 미뤄 두려면 `git stash -u (--include-untracked)` 를 씁니다. stash 후 브랜치를 옮겼는데 새 파일이 따라온다면, `-u` 를 빼먹은 것이 원인일 가능성이 큼니다. 무조건 `git status -s` 로 `??` 표시를 먼저 확인하는 습관이 진단의 출발입니다.

## 9. 단원 미니 프로젝트: 잘못된 커밋 복구 훈련소 (2종 1세트)

이 단원(u4)에서 우리는 8장의 `restore`·`reset`·`revert`·`reflog` 와 9장의 `stash` 를 배웠습니다. 이제 이들을 한자리에 모아, **상황마다 알맞은 복구 도구를 고르는 판단력**을 기르는 미니 프로젝트를 진행합니다. "실수는 반드시 일어난다. 그러나 침착하게 진단하면 복구할 수 있다"는 자신감이 이 단원의 목표입니다.

**적용 개념:** c08(`restore`·`reset`·`revert`·`reflog`) + c09(`stash`). 새 개념은 없고, 단원에서 배운 복구 도구를 통합 적용합니다.

**미션:** 일부러 네 가지 사고를 만들고, 각 상황에 맞는 복구 명령을 골라 처리한 뒤, "어떤 상황에 어떤 명령을 왜 썼는지"를 표로 정리합니다.

- **사고 A — 잘못 스테이징:** 엉뚱한 파일을 `git add` 했다 → 스테이징에서 빼야 한다.
- **사고 B — 잘못 나눈 커밋(아직 `push` 전):** 직전 커밋을 취소해 변경을 되살리고 다시 커밋하고 싶다.
- **사고 C — 공유한 커밋 취소(이미 `push`):** 동료가 받은 잘못된 커밋을 안전하게 되돌려야 한다.
- **사고 D — 작업 중 급한 전환:** 미완성 변경이 있는데 핫픽스로 다른 브랜치에 가야 한다.

### 마일스톤(증분)

- **M1 — 잘못 스테이징·잘못 수정 복구:** 엉뚱하게 `add` 한 파일을 `git restore --staged` 로 스테이징에서 빼고, 잘못 고친 파일을 `git restore` 로 마지막 커밋 상태로 되돌립니다.
- **M2 — 아직 `push` 안 한 직전 커밋 되살리기:** `git reset --soft HEAD~1` 로 직전 커밋을 취소하되 변경은 스테이징에 되살려, 다시 의미 있게 커밋합니다. `reset`의 `--soft`·`--mixed`·`--hard`가 3영역(작업트리·스테이징·저장소)에 미치는 영향을 표로 정리합니다.
- **M3 — 공유한 커밋 안전하게 취소:** 이미 `push` 한 잘못된 커밋을 `git revert <commit>` 로 되돌리는 새 커밋을 만들어 처리합니다. 왜 여기서는 `reset` 이 아니라 `revert` 를 써야 하는지 한 줄로 적습니다.
- **M4 — 급한 전환과 `reflog` 안전망:** 작업 중 `git stash` 로 치우고 `main` 으로 가 핫픽스한 뒤 돌아와 `git stash pop` 으로 복귀합니다. 마지막으로 `git reset --hard` 로 일부러 사고를 낸 뒤 `git reflog` 로 잃은 것처럼 보이던 커밋을 되찾습니다.

## 모범 산출물(요약)

완성된 훈련소는 다음을 갖습니다.

- 네 사고를 각각 올바른 명령(`restore` / `reset` / `revert` / `stash`)으로 처리한 **실습 기록**.
- 상황 → 사용 명령 → 이유를 정리한 **판단 표**(아래 빈 템플릿의 채운 버전).
- `git reset --hard` 사고를 `git reflog` 로 복구한 흔적(복구한 커밋 해시 기록).

상황별 정답 매핑은 다음과 같습니다.

| 사고            | 공유 여부        | 알맞은 명령                                                    | 이유                       |
|---------------|--------------|-----------------------------------------------------------|--------------------------|
| A 잘못 스테이징     | 미공유          | <code>git restore --staged &lt;file&gt;</code>            | 커밋 이력은 그대로 두고 스테이징만 해제   |
| B 잘못 나눈 직전 커밋 | 미공유 (push 전) | <code>git reset --soft HEAD~1</code>                      | 변경을 스테이징에 되살려 다시 커밋      |
| C 공유한 잘못된 커밋  | 공유(push 후)   | <code>git revert &lt;commit&gt;</code>                    | 되돌리는 새 커밋이라 동료 이력과 안 어긋남 |
| D 작업 중 급한 전환  | 미공유          | <code>git stash</code> → 핫픽스 → <code>git stash pop</code> | 미완성 변경을 잃지 않고 잠시 미룸      |

## 학생 작성형 빈 템플릿

### 직접 작성 — 복구 판단 표 채우기

아래 표의 빈칸을 직접 채우고, 각 사고를 실제로 재현해 명령으로 복구한 뒤 결과를 확인하십시오.

| 사고                       | 공유 여부 | 고려해야 할 명령                                                 | 왜 그 명령인가 |
|--------------------------|-------|-----------------------------------------------------------|----------|
| A 엉뚱한 파일을 add 함          | 미공유   | <code>git _____ --staged &lt;file&gt;</code>              | (직접 작성)  |
| B 직전 커밋을 합치고 싶음 (push 전) | 미공유   | <code>git reset _____ HEAD~1</code>                       | (직접 작성)  |
| C 이미 push 한 커밋을 취소       | 공유    | <code>git _____ &lt;commit&gt;</code>                     | (직접 작성)  |
| D 작업 중 급한 핫픽스 전환         | 미공유   | <code>git _____</code> → 핫픽스 → <code>git stash pop</code> | (직접 작성)  |

추가 과제: `git reset --hard` 로 커밋 하나를 일부러 날린 뒤, `git _____` 로 잃은 커밋의 해시를 찾아 `git reset --hard <해시>` 로 복구하십시오.

### 자가체크 체크리스트

- [ ] A에 `restore`, B에 `--soft`, C에 `revert`, D에 `stash` 를 넣었는가
- [ ] 공유한 이력(C)에는 `reset` 이 아니라 `revert` 를 골랐는가(황금률)
- [ ] D에서 `stash` 후 `git status` 가 깨끗해져 전환이 막히지 않았는가
- [ ] 추가 과제의 빈칸에 `reflog` 를 넣어 잃은 커밋의 해시를 되찾았는가
- [ ] 네 사고를 모두 복구한 뒤 `git status` 와 `git log` 로 결과를 검증했는가

## 10. 자주 묻는 질문 (FAQ)

### Q1. stash 한 변경은 다른 브랜치에서도 꺼낼 수 있나요?

네. 스테시는 특정 브랜치에 묶이지 않습니다. `feature/sort` 에서 `stash` 한 변경을 `main` 으로 가서 `git stash pop` 하면, 그 변경이 `main` 의 작업트리에 적용됩니다. "브랜치를 잘못 파서 작업했다"는 실수를 바로잡을 때 유용합니다. 다만 그 사이 같은 파일이 다르게 바뀌어 있으면 충돌이 날 수 있으니, `git status` 로 먼저 상태를 확인하는 습관이 좋습니다.

### Q2. git stash 메시지의 WIP on ... 은 무슨 뜻인가요?

WIP 는 "Work In Progress(작업 중)"의 약자입니다. `-m` 으로 직접 메시지를 주지 않으면 Git이 "어느 브랜치에서 어떤 커밋 위에 작업 중이었는지"를 자동으로 `WIP on <브랜치>: <커밋>` 형태



로 적어 줍니다. 스테시가 여러 개 쌓이면 구분이 어려우니, 중요한 것은 `git stash -m "설명"` 으로 직접 메시지를 다는 편이 좋습니다.

### Q3. 실수로 `git stash drop` 이나 `clear` 로 스테시를 지웠는데 되살릴 수 있나요?

쉽지는 않지만 방법이 있습니다. 스테시도 내부적으로는 커밋 객체라, `git fsck --no-reflogs` 같은 방법으로 잃어버린 스테시 커밋을 찾아낼 수 있는 경우가 있습니다. 그러나 이는 복잡하고 항상 성공하지도 않습니다. 그래서 애초에 **중요한 작업은 stash가 아니라 커밋·브랜치로 보존**하는 것이 정답입니다. stash는 "곧 다시 꺼낼 짧은 미루기"에만 쓰십시오.

### Q4. stash에 올린 변경의 내용을 적용하기 전에 미리 볼 수 있나요?

네. `git stash show stash@{0}` 은 어떤 파일이 몇 줄 바뀌었는지 요약(diffstat)을 보여 주고, `git stash show -p stash@{0}` 은 실제 변경 내용(패치)을 보여 줍니다. 여러 스테시 중 무엇을 꺼낼지 고를 때 먼저 내용을 확인하면 실수를 줄일 수 있습니다.

## 11. 마무리

이 장에서 우리는 "되돌리지는 않되 잠시 치워 두는" 도구 `git stash` 를 익혔습니다. 핵심을 다시 정리합니다.

- `git stash` 는 미완성 변경을 **선반(스택)** 에 올리고 작업트리를 깨끗이 만듭니다. 커밋이 아니라 임시 보관입니다.
- 가장 빛나는 순간은 **작업 중 급한 브랜치 전환**입니다. 치우기 → 전환 → 고치기 → 돌아오기 → `pop` 의 흐름을 몸에 익히십시오.
- `pop` 은 적용 후 제거, `apply` 는 적용 후 보존입니다. **충돌이 난 `pop` 은 스테시를 자동으로 지우지 않으니** 해결 후 직접 `drop` 합니다.
- 새 파일(untracked)까지 치우려면 `-u` 가 필요합니다.

다음 단원(u5)부터는 드디어 혼자 쓰던 Git을 **GitHub와 잇는 원격(remote)** 의 세계로 넘어갑니다. 10장에서 `git clone` 과 `git remote` 로 로컬 저장소를 원격과 연결하는 법을 배웁니다.



# 원격 저장소: git remote와 git clone으로 GitHub 잇기

## 학습 목표

- 원격 저장소(remote)와 로컬 저장소의 관계를 설명할 수 있습니다.
- `git clone` 으로 원격 저장소를 이력째 복제할 수 있습니다.
- `git remote add origin <URL>` 로 로컬을 원격과 연결할 수 있습니다.
- `git remote -v` 로 연결된 원격을 확인하고 `origin` 의 의미를 설명할 수 있습니다.

지난 단원(u4)까지 우리는 **내 컴퓨터 안에서** Git을 다뤘습니다. 저장소를 만들고(init), 변경을 기록하고(add-commit), 이력을 읽고(log-diff), 갈래를 나누고 합치고(branch-merge), 실수를 되돌리고 미뤘습니다(restore-reset-revert-stash). 이제부터는 그 저장소를 **세상과 공유**할 차례입니다. 이 단원(u5)은 내 로컬 저장소를 GitHub의 **원격 저장소(remote)** 와 연결하는 데서 시작합니다. 그 첫 명령이 이 장의 주인공인 `git clone` 과 `git remote` 입니다.

이 장의 선수 개념 상기: 3장의 **변경 기록하기(add-commit과 3영역)** 를 그대로 씁니다. 원격은 결국 "내가 커밋한 이력을 올려 두는 공유 장소"이므로, 커밋이 무엇인지 알아야 원격에 무엇을 보내는지 이해됩니다. 이 장에서는 "연결"까지만 다루고, 실제로 올리고 받는 `push`·`pull` 은 다음 장(11장)에서 배웁니다.

## 1. 도입: 혼자 쓰던 Git을 공유의 세계로

지금까지 만든 To-Do 웹앱 저장소는 오직 내 컴퓨터 한 곳에만 있습니다. 여기에는 두 가지 한계가 있습니다.

- **백업이 없습니다.** 노트북이 고장 나거나 폴더를 실수로 지우면 모든 이력이 사라집니다.
- **협업이 불가능합니다.** 동료에게 코드를 보여 주거나 함께 고치려면, 모두가 접근할 수 있는 **공동의 중심점**이 필요합니다.

**GitHub**가 바로 그 중심점입니다. 내 저장소를 GitHub 서버에 올려 두면, 그것이 백업이자 협업의 허브가 됩니다. 이렇게 서버에 둔 저장소 복사본을 **원격 저장소(remote repository)** 라고 부릅니다.

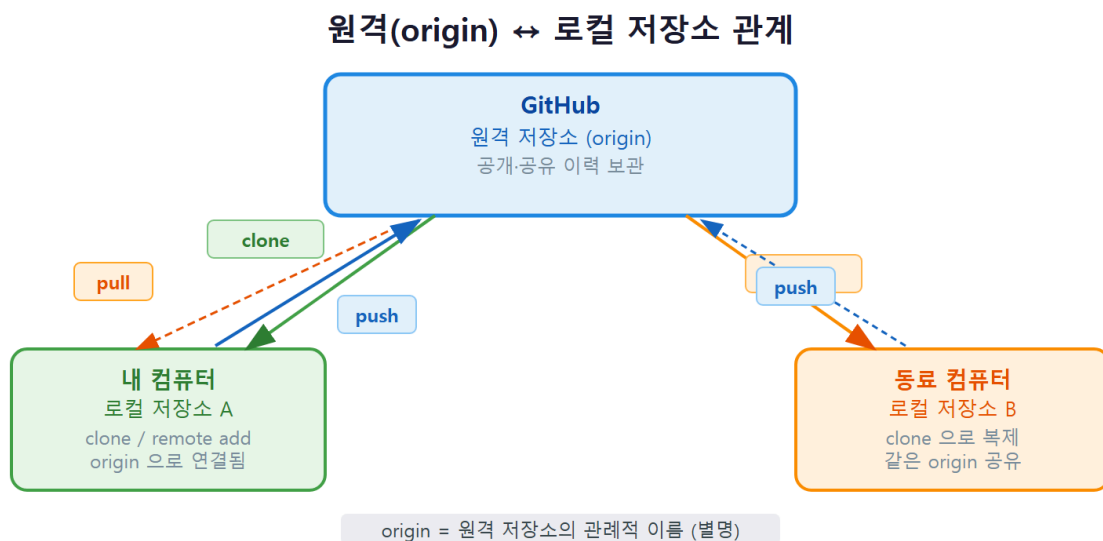
원격과 잇는 길은 두 갈래입니다.

- 남이 만든 저장소를 통째로 받아오기: `git clone`. 빈손에서 시작해 기존 프로젝트를 받습니다.
- 내가 이미 만든 로컬 저장소를 원격과 연결하기: `git remote add`. 지금까지 만든 To-Do 저장소가 여기에 해당합니다.

이 장에서 두 길을 모두 익힙니다. 1장에서 "Git과 GitHub는 다르다 — Git은 내 컴퓨터의 도구, GitHub는 저장소를 올려 두는 웹 서비스"라고 배운 그 GitHub를, 이제 실제로 내 저장소와 연결합니다.

## 개념: 원격 저장소와 복제 — remote-clone

### 2. 동작 원리: 원격(origin) ↔ 로컬 저장소



#### 정의

**원격 저장소(remote)** 는 GitHub 같은 서버에 올려 둔 저장소 복사본으로, 여러 사람이 같은 프로젝트를 공유하는 중심점입니다. **복제(clone)** 는 원격 저장소 전체(이력 포함)를 내 컴퓨터로

가져와 곧바로 작업할 수 있게 만드는 일이고, 이미 만든 로컬 저장소는 **원격 연결**(`git remote add`) 로 원격과 짝지어 둡니다.

핵심 용어를 정리합니다.

- **로컬 저장소(local)**: 내 컴퓨터 안의 `.git` 을 가진 저장소. 지금까지 다뤄 온 그것입니다.
- **원격 저장소(remote)**: GitHub 서버에 둔 공유 복사본.
- **origin**: 기본 원격 저장소에 **흔히 붙이는 이름**(관례). 특별한 예약어가 아니라, `clone` 할 때 Git이 자동으로 붙여 주는 별명일 뿐입니다.

### 동작 원리: 같은 이력을 가진 여러 복사본

Git은 **분산** 버전관리 시스템입니다(1장). 이 말의 핵심이 여기서 드러납니다. 원격과 로컬은 "서버에는 코드, 내 컴퓨터에는 화면"처럼 역할이 나뉜 것이 **아닙니다**. 양쪽 모두 **완전한 이력을 가진 동등한 저장소**이고, 그중 하나(GitHub의 것)를 "모두가 모이는 약속 장소"로 삼은 것뿐입니다.

이 구조를 단계로 보면 다음과 같습니다.

1. **원격이 중심에 있습니다**. GitHub에 저장소 하나(origin)가 있습니다.
2. **여러 로컬이 그 하나를 공유합니다**. 동료 A의 컴퓨터, 동료 B의 컴퓨터가 각자 같은 origin 을 복제(clone)해 똑같은 이력을 갖습니다.
3. **로컬은 origin이라는 이름으로 원격을 가리킵니다**. 매번 긴 URL을 입력하지 않도록, 로컬은 원격 주소에 `origin` 이라는 짧은 별명을 붙여 둡니다.

| 구분     | 어디에 있나    | 무엇을 갖나                    | 별명                       |
|--------|-----------|---------------------------|--------------------------|
| 로컬 저장소 | 내 컴퓨터     | <code>.git</code> + 전체 이력 | (내 작업 공간)                |
| 원격 저장소 | GitHub 서버 | 공유용 전체 이력                 | <code>origin</code> (관례) |

여기서 꼭 기억할 점은 **연결을 맺는다고 자동으로 동기화되지 않는다**는 것입니다. `git remote add` 는 "이 원격을 origin이라 부르겠다"는 **주소록 등록**일 뿐입니다. 실제로 올리고 받는 일은 다음 장의 `push`·`pull` 이 합니다. 이 장은 "전화번호를 저장하는 단계"까지입니다.

**왜 하필 origin인가**: `origin` 은 영어로 "출처·근원"이라는 뜻입니다. "내 저장소가 비롯된 곳"이라는 의미로, `git clone` 이 자동으로 붙여 주는 기본 별명입니다. 원하면 `upstream`, `backup` 같은 다른 이름의 원격을 추가로 둘 수도 있습니다. 즉 `origin`은 **법칙이 아니라 널리 쓰이는 관례**입니다.

### 3. git clone으로 이력재 복제하기

남이 만든(또는 GitHub에서 먼저 만든) 저장소를 받아오는 명령이 `git clone`입니다. URL 하나만 주면 됩니다.

```
git clone https://github.com/octocat/Hello-World.git
```

이 한 줄이 하는 일은 단순한 다운로드가 아닙니다. 네 가지를 한꺼번에 처리합니다.

1. Hello-World 라는 폴더를 새로 만듭니다.
2. 그 안에 `.git` 을 두고 **전체 커밋 이력**을 복사합니다(파일의 최신본만이 아니라 과거 전부).
3. 원격 주소를 `origin` 이라는 이름으로 자동 등록합니다.
4. 기본 브랜치(보통 `main`)를 작업트리에 펼쳐 둡니다.

실행하면 다음과 같이 진행 상황이 출력됩니다.

```
$ git clone https://github.com/octocat/Hello-World.git
Cloning into 'Hello-World'...
remote: Enumerating objects: 13, done.
remote: Counting objects: 100% (13/13), done.
remote: Total 13 (delta 2), reused 0 (delta 0), pack-reused 11
Receiving objects: 100% (13/13), done.
Resolving deltas: 100% (2/2), done.
```

복제가 끝나면 곧바로 그 폴더로 들어가 이력과 브랜치를 확인할 수 있습니다. clone은 원격의 브랜치 정보까지 함께 가져옵니다.

```
cd Hello-World
git log --oneline -3      # 받은 이력 확인
git branch -a            # 로컬·원격 브랜치 모두 보기
git remote -v            # 자동 등록된 origin 확인
```

`git branch -a` 의 실행 결과는 다음과 같습니다.

```
$ git branch -a
* main
remotes/origin/HEAD -> origin/main
remotes/origin/main
```

- `* main` 은 지금 내가 있는 로컬 브랜치입니다. clone이 기본 브랜치를 자동으로 펼쳐 둔 것입니다.

- `remotes/origin/main` 은 **원격 추적 브랜치(remote-tracking branch)** 입니다. "origin의 main이 마지막으로 본 시점에 어디였는지"를 기억하는 표시로, 다음 장에서 push·pull의 기준점이 됩니다.
- `remotes/origin/HEAD -> origin/main` 은 "origin의 기본 브랜치가 main"이라는 정보입니다. 즉 clone은 단순한 파일 묶음이 아니라 **브랜치 구조와 기본 브랜치 정보까지** 받아 온다는 뜻입니다.

```
$ git log --oneline -3
7fd1a60 Merge pull request #6 from Spaceghost/patch-1
762941318 New line at end of file.
553c207 first commit

$ git remote -v
origin https://github.com/octocat/Hello-World.git (fetch)
origin https://github.com/octocat/Hello-World.git (push)
```

- `git log --oneline` 에 과거 커밋이 줄줄이 나옵니다. 단순히 파일만 받은 것이 아니라 **누가 언제 무엇을 했는지의 전체 이력**을 함께 받았다는 증거입니다.
- `git remote -v` 가 `origin` 을 보여 줍니다. `clone` 이 원격 주소를 자동으로 `origin` 이라는 이름에 등록해 준 것입니다. 우리가 따로 `remote add` 를 하지 않아도 됩니다.
- 같은 URL이 `(fetch)` 와 `(push)` 두 줄로 나옵니다. 받을 때(fetch)와 올릴 때(push) 쓰는 주소를 각각 보여 주는 것으로, 보통은 같은 주소입니다.

폴더 이름을 다르게 두고 싶으면 URL 뒤에 원하는 이름을 붙입니다. 같은 원격을 두 번 복제해 "두 기여자"를 흉내 낼 때 유용합니다.

```
git clone https://github.com/myname/todo-app.git dev-alice
git clone https://github.com/myname/todo-app.git dev-bob
```

## 두 클론으로 협업 흉내 내기 — 종합 프로젝트의 준비 운동

위처럼 같은 원격을 두 폴더로 복제해 두면, 혼자서도 **두 기여자의 협업**을 결정론적으로 재현할 수 있습니다. 이 책의 종합 프로젝트(팀 협업 시뮬레이션)가 바로 이 방식을 씁니다. `dev-alice` 폴더에서는 앨리스가, `dev-bob` 폴더에서는 밥이 작업하는 것처럼 번갈아 연기하면, 동시 수정 충돌이나 동기화 사고를 안전하게 만들어 보고 풀 수 있습니다.

두 클론의 구조를 표로 보면 다음과 같습니다.

| 폴더         | 연기하는 기여자 | origin이 가리키는 곳 | 갖는 것              |
|------------|----------|----------------|-------------------|
| dev-alice/ | 앨리스      | 같은 GitHub 저장소  | 전체 이력 + origin 연결 |
| dev-bob/   | 밥        | 같은 GitHub 저장소  | 전체 이력 + origin 연결 |

두 폴더는 **각자 독립된 완전한 저장소**이고, 둘 다 같은 origin을 중심으로 삼습니다. 그래서 한쪽이 변경을 push 하면 다른 쪽은 pull로 받아 동기화하는, 실제 팀 협업과 똑같은 흐름을 1인 2역으로 체험할 수 있습니다. 이 장에서는 "두 클론을 만들 수 있다"까지가 목표이고, 실제로 주고 받는 push·pull은 다음 장에서 배웁니다.

**팁 — 지금 어느 폴더에 있는지 늘 확인:** 두 클론을 오갈 때 가장 흔한 실수는 "지금 앨리스 폴더인지 밥 폴더인지" 헷갈리는 것입니다. 터미널 프롬프트의 현재 경로를 확인하거나, 작업 전 `git log --oneline -1`로 마지막 커밋을 보고 위치를 가늠하는 습관을 들이십시오.

#### 4. worked example: GitHub 빈 저장소 만들고 remote add

이번에는 반대 상황입니다. 지금까지 **내 컴퓨터에서 이미 만든** To-Do 저장소를 GitHub와 연결하는 흐름입니다. `clone`은 빈손에서 받아오는 것이고, 이쪽은 이미 있는 로컬을 원격과 잇는 것입니다.

**상황:** 로컬에 커밋 이력이 쌓인 `todo-app` 저장소가 있습니다. 이것을 GitHub에 올리고 싶습니다.

##### 1단계 — GitHub에서 빈 저장소 만들기

먼저 GitHub 웹에서 빈 저장소를 만듭니다. 브라우저에서 다음을 합니다.

1. GitHub 오른쪽 위 + → **New repository** 클릭.
2. Repository name에 `todo-app` 입력.
3. **중요:** Add a README file, .gitignore, license 는 **모두 체크하지 않습니다**. 이미 로컬에 파일이 있으므로, 원격을 완전히 비워 두어야 충돌 없이 깔끔하게 올릴 수 있습니다.
4. **Create repository** 클릭.

그러면 GitHub가 "빈 저장소를 푸시하는 안내"와 함께 저장소 URL을 보여 줍니다(예:

`https://github.com/myname/todo-app.git`).

##### 2단계 — 로컬을 원격과 연결하기

이제 로컬 저장소 폴더에서 그 URL을 `origin`으로 등록합니다.



```
# (1) 로컬 저장소 폴더에서 시작 — 이미 커밋 이력이 있는 상태
git log --oneline

# (2) GitHub에서 받은 URL을 origin이라는 이름으로 등록
git remote add origin https://github.com/myname/todo-app.git

# (3) 연결이 잘 됐는지 확인
git remote -v
```

## 진단과 결과

실행 결과는 다음과 같습니다.

```
$ git log --oneline
a3c19f8 gitignore로 로그.env 제외
5e2b7d1 README 소개 작성
9f4a0c2 프로젝트 초기 구조 추가

$ git remote add origin https://github.com/myname/todo-app.git

$ git remote -v
origin https://github.com/myname/todo-app.git (fetch)
origin https://github.com/myname/todo-app.git (push)
```

- `git remote add origin <URL>` 은 화면에 아무 출력도 내지 않습니다. 조용히 성공한 것입니다. Git에서 "메시지가 없다"는 것은 보통 "정상 완료"라는 신호입니다.
- 바로 이어 `git remote -v` 로 확인하면 `origin` 이 등록됐음을 볼 수 있습니다. 이제부터 긴 URL 대신 `origin` 이라는 짧은 이름으로 이 원격을 가리킬 수 있습니다.
- 여기까지가 **연결**입니다. 아직 코드는 GitHub에 올라가지 않았습니다. GitHub 웹페이지를 새로고침해도 저장소는 비어 있습니다. 실제로 올리는 `git push -u origin main` 은 다음 장에서 배웁니다.

**원격을 비워 두라고 한 이유:** 1단계에서 README나 .gitignore를 체크하면 GitHub가 그 파일로 커밋 하나를 미리 만들어 둡니다. 그러면 원격에는 그 커밋이, 로컬에는 내 커밋들이 따로 있어 이력이 갈라집니다. 이 상태에서 push 하면 "원격이 앞서 있다"며 거부되어(11장에서 다룰 non-fast-forward), 입문 단계에서 당황하기 쉽습니다. 그러니 **이미 로컬이 있는 경우엔 원격을 완전히 비워** 만드는 것이 정석입니다.

## 5. git remote -v와 HTTPS-SSH 인증 개요

원격을 관리하는 명령들을 정리합니다.

| 명령                                                 | 하는 일                         |
|----------------------------------------------------|------------------------------|
| <code>git remote -v</code>                         | 등록된 원격과 그 URL 보기(fetch/push) |
| <code>git remote add &lt;이름&gt; &lt;URL&gt;</code> | 새 원격을 이름 붙여 등록               |
| <code>git remote rename origin upstream</code>     | 원격 이름 바꾸기                    |
| <code>git remote remove origin</code>              | 원격 연결 제거                     |
| <code>git remote set-url origin &lt;URL&gt;</code> | 등록된 원격의 URL 교체(오타 수정 등)      |

## HTTPS와 SSH — 두 가지 주소와 인증

GitHub 저장소 URL에는 두 형식이 있습니다. 둘은 **인증 방식**이 다릅니다.

| 형식    | 주소 예                                                | 인증 방법                        |
|-------|-----------------------------------------------------|------------------------------|
| HTTPS | <code>https://github.com/myname/todo-app.git</code> | 사용자명 + <b>개인 액세스 토큰(PAT)</b> |
| SSH   | <code>git@github.com:myname/todo-app.git</code>     | 미리 등록한 <b>SSH 키</b>          |

- **HTTPS**는 시작하기 쉽습니다. Windows 11에서 Git for Windows를 설치하면 함께 깔리는 **Git Credential Manager**가 처음 push 할 때 브라우저 로그인 창을 띄워 인증을 처리하고, 이후에는 자격 증명을 안전하게 저장해 다시 묻지 않습니다. 단, 비밀번호 대신 **개인 액세스 토큰(Personal Access Token)**을 쓴다는 점만 기억하면 됩니다(GitHub는 2021년 이후 HTTPS에 계정 비밀번호를 받지 않습니다).
- **SSH**는 한 번 키를 등록해 두면 매번 인증 없이 매끄럽게 동작합니다. `ssh-keygen`으로 키 쌍을 만들고 공개키를 GitHub 설정에 등록하는 사전 준비가 필요합니다.

Windows 11에서 HTTPS로 처음 push 할 때 일어나는 일을 미리 알아 두면 당황하지 않습니다. Git Credential Manager가 작은 로그인 창(또는 브라우저)을 띄워 GitHub 계정 인증을 한 번 요구합니다. 여기서 인증을 마치면, 토큰이 Windows 자격 증명 저장소에 안전하게 보관되어 **그다음부터는 묻지 않습니다**. 만약 옛 방식대로 비밀번호를 직접 입력하려 하면 `Support for password authentication was removed.` (비밀번호 인증 지원이 제거되었습니다)라는 거부 메시지를 만나는데, 이는 GitHub가 2021년부터 HTTPS에 계정 비밀번호 대신 토큰만 받기 때문입니다. 비밀번호가 아니라 **개인 액세스 토큰**을 쓰거나 Credential Manager의 브라우저 로그인을 따르면 됩니다.

입문 단계에서는 **HTTPS + Credential Manager** 조합이 가장 무난합니다. 이 책의 예제도 HTTPS 주소를 기준으로 합니다.

**팁 — 주소 형식만 봐도 인증이 보입니다:** URL이 `https://` 로 시작하면 토큰 인증, `git@` 로 시작하면 SSH 키 인증입니다. `push`가 인증 문제로 막힐 때, 먼저 `git remote -v` 로 어떤 형식의 주소가 등록돼 있는지 확인하는 것이 진단의 시작입니다.

### 같은 저장소를 두 형식으로 오가기

이미 HTTPS로 등록한 원격지를 나중에 SSH로 바꾸고 싶다면, 6절에서 배울 `git remote set-url` 로 주소만 갈아 끼우면 됩니다. 저장소 내용은 그대로 두고 **연결 주소만** 바뀌는 것이라 안전합니다.

```
git remote set-url origin git@github.com:myname/todo-app.git # HTTPS → SSH로 교체
git remote -v # 형식이 바뀌었는지 확인
```

실행 결과는 다음과 같이 `git@` 로 시작하는 SSH 주소로 바뀝니다.

```
$ git remote -v
origin git@github.com:myname/todo-app.git (fetch)
origin git@github.com:myname/todo-app.git (push)
```

- 같은 저장소라도 `https://github.com/myname/todo-app.git` (HTTPS)와 `git@github.com:myname/todo-app.git` (SSH)는 **가리키는 대상이 같고 인증 방식만 다릅니다.**
- 처음에는 HTTPS로 시작하고, SSH 키를 등록한 뒤 `set-url` 로 갈아타는 흐름이 흔합니다. 둘 중 무엇을 쓰든 `push·pull` 명령 자체는 똑같습니다.

## 6. 흔한 문제와 해결: 원격 주소 오타 → `remote set-url`

원격을 등록할 때 가장 흔한 사고는 **URL 오타**입니다. 저장소 이름을 잘못 적거나, 사용자명을 틀리거나, `.git` 을 빠뜨리는 식입니다. 연결할 때는 조용히 성공하지만(아무 검증도 하지 않으므로), 나중에 `push·pull` 할 때 비로소 실패합니다.

**상황:** `todo-app` 을 `todo-ap` 으로 잘못 적어 origin에 등록했습니다. 다음 장에서 `push`를 시도하니 저장소를 찾을 수 없다는 오류가 납니다.

**진단:** 무엇보다 `git remote -v` 로 **지금 등록된 주소를 눈으로 확인**합니다. 오타가 바로 보입니다.

```
$ git remote -v
origin https://github.com/myname/todo-ap.git (fetch)
origin https://github.com/myname/todo-ap.git (push)
```

todo-ap 으로 잘못 등록돼 있습니다. 이때 흔히 `remote add` 를 다시 시도하지만, 같은 이름은 두 번 등록할 수 없어 오류가 납니다.

```
$ git remote add origin https://github.com/myname/todo-app.git
error: remote origin already exists.
```

**해결:** 새로 추가하는 것이 아니라, 기존 origin의 **URL을 교체**해야 합니다. `git remote set-url` 을 씁니다.

```
git remote set-url origin https://github.com/myname/todo-app.git
git remote -v          # 교정됐는지 다시 확인
```

```
$ git remote set-url origin https://github.com/myname/todo-app.git

$ git remote -v
origin https://github.com/myname/todo-app.git (fetch)
origin https://github.com/myname/todo-app.git (push)
```

- `error: remote origin already exists.` 는 "origin이라는 이름은 이미 있다"는 뜻입니다. 같은 이름으로 또 `add` 할 수 없습니다.
- 해결책은 **교체**입니다. `git remote set-url origin <정확한 URL>` 이 기존 origin의 주소만 갈아 끼웁니다.
- 교정 후 반드시 `git remote -v` 로 다시 확인하는 것이 핵심 습관입니다. "고쳤다고 믿지 말고 눈으로 확인"이 진단의 기본입니다.

**연결은 검증되지 않습니다:** `git remote add` 나 `set-url` 은 입력한 주소가 실제로 존재하는지 그 자리에서 확인하지 않습니다. 그저 주소록에 적어 둘 뿐입니다. 그래서 오타가 있어도 **즉시 오류가 나지 않고**, push-pull 같은 실제 통신 때 비로소 드러납니다. 등록 직후 `git remote -v` 로 한번 눈으로 확인하는 습관이 이런 늦은 사고를 막아 줍니다.

## 7. 즉시 연습 (2종 1세트)

### 직접 작성 1 — 내 로컬을 GitHub와 연결하기 (빈칸 채우기)

이미 커밋 이력이 있는 로컬 `todo-app` 저장소를, GitHub에 만든 빈 저장소와 연결하는 명령을 완성하십시오. GitHub URL은 `https://github.com/myname/todo-app.git` 이라고 가정합니다.

```
git log --oneline           # 로컬 이력 확인(연결할 것이 있는지)
git remote _____ origin https://github.com/myname/todo-app.git # origin이라는 이름으로 원격
git remote _____      # 연결이 잘 됐는지 확인
```

### 자가체크 체크리스트

- [ ] 첫 빈칸에 `add` 를 넣어 새 원격을 등록했는가
- [ ] 둘째 빈칸에 `-v` 를 넣어 `fetch/push` 주소를 확인했는가
- [ ] GitHub 저장소를 만들 때 `README.gitignore`를 체크하지 않아 원격을 비워 뒀는가
- [ ] `git remote add` 가 아무 메시지 없이(조용히) 성공했고, 아직 코드는 올라가지 않았음을 이해했는가

### 직접 작성 2 — 복제와 오타 교정 (직접 작성)

다음 두 가지를 명령으로 직접 작성하십시오.

1. 공개 저장소 `https://github.com/myname/todo-app.git` 를 `dev-bob` 이라는 폴더 이름으로 복제한다.
2. origin 주소를 `todo-ap` (오타)로 잘못 등록한 상황을, 정확한 주소 `https://github.com/myname/todo-app.git` 로 교체해 고친다(새로 `add` 하지 않고).

```
# 1. 폴더 이름을 지정해 복제
git _____ https://github.com/myname/todo-app.git _____

# 2-1. 현재 잘못 등록된 주소 확인
git remote ____

# 2-2. URL만 정확한 주소로 교체
git remote _____ origin https://github.com/myname/todo-app.git
```

### 자가체크 체크리스트

- [ ] 1번에 `clone` 을 넣고 URL 뒤에 폴더 이름 `dev-bob` 을 지정했는가
- [ ] 2-1에 `-v` 를 넣어 등록된 주소를 눈으로 확인했는가
- [ ] 2-2에 `set-url` 을 넣어 `add` 가 아니라 **교체**로 고쳤는가
- [ ] 교정 후 다시 `git remote -v` 로 주소가 바르게 바뀌었는지 확인했는가

## 8. 흔한 오류와 오개념 교정

### 오개념 1 — "clone은 그냥 파일 다운로드다"

`git clone` 은 단순 다운로드가 **아닙니다**. 파일의 최신본만 받는 것이 아니라, **전체 커밋 이력**과 **원격 연결(origin)** 까지 함께 가져옵니다.

```
$ git clone https://github.com/myname/todo-app.git
$ cd todo-app
$ git log --oneline      # 과거 커밋이 모두 들어 있음 → 단순 다운로드가 아님
$ git remote -v         # origin이 자동 등록돼 있음
```

**교정:** clone 직후 곧바로 `git log` 로 이력을 보고, `git branch` 로 브랜치를 만들고, `git commit` 으로 작업할 수 있습니다. 받은 폴더는 ZIP 다운로드와 달리 **그 자체로 완전한 저장소**입니다. 이것이 분산 버전관리의 핵심입니다.

### 오개념 2 — "origin은 Git이 정한 특별한 예약어다"

`origin` 은 예약어가 **아니라** 관례입니다. `git clone` 이 첫 원격에 자동으로 붙여 주는 기본 별명일 뿐, 다른 이름도 얼마든지 가능합니다.

```
git remote add backup https://github.com/myname/todo-app.git    # backup이라는 원격도 가능
git remote add upstream https://github.com/team/todo-app.git    # 오픈소스에선 upstream 관례도 있음
```

**교정:** origin이라는 글자에 특별한 힘이 있는 것이 아닙니다. 다만 거의 모든 사람이 기본 원격을 origin이라 부르므로, **관례를 따르는 것이 협업에 유리**합니다. 굳이 다른 이름을 쓰면 동료가 헷갈릴 수 있습니다.

### 오개념 3 — "GitHub에 저장소를 만들면 내 코드가 자동으로 올라간다"

GitHub에서 저장소를 만들고 `git remote add` 로 연결해도, 내 로컬 커밋은 **자동으로 올라가지 않습니다**. `remote add` 는 "전화번호 등록"일 뿐, 실제 전송은 별개입니다.

**교정:** 연결과 전송은 다른 단계입니다.

- `git remote add origin <URL>` = 원격 주소를 origin이라는 이름으로 **등록(연결)**.
- `git push -u origin main` = 로컬 커밋을 실제로 원격에 **올리기(전송, 다음 장)**.

`remote add` 후 GitHub 페이지를 새로고침해도 여전히 비어 있는 것은 정상입니다. 올리려면 직접 `push` 해야 합니다.

## 9. 자주 묻는 질문 (FAQ)

---

### Q1. `git clone` 과 GitHub의 "Download ZIP"은 무엇이 다른가요?

ZIP 다운로드는 파일의 **최신본만** 압축해 받습니다. `.git` 이 없으므로 이력도, 브랜치도, 원격 연결도 없는 "그냥 파일 묶음"입니다. 반면 `git clone` 은 `.git` 과 **전체 이력·브랜치·origin 연결**까지 통째로 받아, 받은 즉시 커밋·로그·푸시가 가능한 **완전한 저장소**가 됩니다. 협업이나 기여를 하려면 반드시 clone을 써야 합니다.

### Q2. 원격을 여러 개 둘 수 있나요?

네. `git remote add <이름> <URL>` 로 이름만 다르게 하면 원격을 여러 개 등록할 수 있습니다. 오픈소스에 기여할 때는 흔히 내 복사본을 `origin`, 원본 저장소를 `upstream` 이라는 이름으로 두 개 둡니다. `git remote -v` 를 하면 등록된 모든 원격이 나열됩니다.

### Q3. 빈 저장소를 clone 하면 어떻게 되나요?

GitHub에서 막 만든 빈 저장소를 clone 하면 Git이 `warning: You appear to have cloned an empty repository.` (빈 저장소를 복제한 것 같습니다)라고 알려 줍니다. 오류가 아니라 정상입니다. 받을 커밋이 아직 없을 뿐이고, origin 연결은 정상적으로 맺어집니다. 여기에 파일을 만들어 커밋하고 push 하면 됩니다.

### Q4. `git remote add` 로 등록한 직후 오류가 안 났는데, 정말 연결된 게 맞나요?

`git remote add` 는 주소가 실제로 존재하는지 그 자리에서 **확인하지 않습니다**. 그래서 오타가 있어도 즉시 오류가 나지 않습니다. 연결이 진짜 유효한지는 실제 통신(다음 장의 push·pull) 때 드러납니다. 등록 직후 `git remote -v` 로 **주소를 눈으로 확인**하는 것이, 늦게 터지는 사고를 막는 가장 확실한 습관입니다.

### Q5. clone 한 폴더를 다른 곳으로 옮기면 원격 연결이 끊기나요?

아닙니다. 원격 연결 정보는 폴더 안 `.git/config` 에 저장되므로, 폴더를 통째로 옮기거나 이름을 바꿔도 origin 연결은 그대로 유지됩니다. `git remote -v` 로 확인해 보면 여전히 같은 origin을 가리킵니다.

## 10. 마무리

---

이 장에서 우리는 혼자 쓰던 Git을 GitHub와 잇는 첫 단계, **원격 저장소 연결**을 익혔습니다. 핵심을 정리합니다.

- **원격 저장소(remote)** 는 GitHub 서버에 둔 공유 복사본이고, 로컬과 원격은 **둘 다 완전한 이력을 가진 동등한 저장소**입니다(분산 버전관리).
- `git clone <URL>` 은 단순 다운로드가 아니라 **이력째 복제**하고 `origin` 을 자동 등록합니다. 폴더 이름을 지정해 두 클론으로 "두 기여자"를 흉내 낼 수도 있습니다.
- 이미 만든 로컬은 `git remote add origin <URL>` 로 연결하고, `git remote -v` 로 확인합니다. `origin` 은 예약어가 아니라 **관례**입니다.
- 연결은 검증되지 않으니 등록 직후 `git remote -v` 로 확인하고, 오타는 `git remote set-url` 로 교체합니다.
- 연결과 전송은 다릅니다. `remote add` 는 **자동 업로드가 아닙니다**.

지금까지의 흐름을 두 갈래로 한 번 더 정리하면 머릿속에 길이 남습니다.

- **빈손에서 시작**(남의/원격 저장소를 받기): `git clone <URL>` → 이력·브랜치·origin이 한꺼번에 따라옴 → 바로 작업 가능.
- **이미 만든 로컬을 잇기**: GitHub에 빈 저장소 생성 → `git remote add origin <URL>` → `git remote -v` 로 확인 → (다음 장에서) `git push` 로 전송.

두 길 모두 끝에는 `origin` 이라는 하나의 중심점이 있습니다. 이 이름 하나로 긴 URL을 대신하고, 여러 로컬이 같은 `origin`을 공유해 협업이 시작됩니다.

다음 장(11장)에서는 드디어 연결한 원격에 실제로 커밋을 **올리고(push)** 원격의 변경을 **받아오는(fetch·pull)** 동기화를 배웁니다. 이 장에서 등록한 `origin` 이 그 모든 동기화의 목적지가 됩니다. 또한 원격이 나보다 앞서 있어 push가 거부되는 상황을, 8장의 되돌리기·이 장의 원격 지식과 엮어 침착하게 풀어 보게 됩니다.





# 주고받기: git push·pull·fetch로 동기화하기

## 이 챕터에서 배우는 것

- `git push` 로 로컬 커밋을 원격 저장소에 올릴 수 있습니다.
- `git fetch` (받기만)와 `git pull` (받아서 합치기)의 차이를 설명할 수 있습니다.
- `git push -u` 로 업스트림(upstream)을 한 번 맺어 이후 동기화를 간단히 할 수 있습니다.
- 원격이 앞서서 push가 거부될 때(non-fast-forward) `git pull` 후 push로 해결할 수 있습니다.

## 도입: 올리고 받고 합치는 일상 동기화

앞 챕터(ch\_10)에서 우리는 로컬 저장소를 GitHub의 원격 저장소(origin)와 연결했습니다. `git remote add origin <URL>` 로 주소를 등록하거나 `git clone` 으로 원격을 통째로 복제하는 일까지 끝냈습니다. 그런데 연결만으로는 아무 일도 일어나지 않습니다. **연결은 전화번호를 저장한 것일 뿐, 아직 전화를 걸지는 않은 상태입니다.**

이제부터가 협업의 핵심입니다. 내가 로컬에서 쌓은 커밋을 원격으로 **올리고**(push), 동료가 원격에 올린 변경을 내 컴퓨터로 **받고**(fetch), 받은 것을 내 작업에 **합치는**(merge) 일이 매일 반복됩니다. 이 세 가지가 Git 협업의 일상 동기화입니다.

핵심 비유: 원격 저장소는 **공유 책장**입니다. `push` 는 내가 새로 쓴 원고를 공유 책장에 꽂는 것, `fetch` 는 책장에 어떤 새 책이 들어왔는지 목록만 확인해 내 책상으로 가져오는 것, `pull` 은 그 새 책을 가져와 내 원고에 곧장 반영(합치기)하는 것입니다. `fetch`는 "구경", `pull`은 "구경 + 반영"인 셈입니다.

이 한 가지 차이를 분명히 구분하는 것이 이 챕터의 가장 중요한 목표입니다. 많은 입문자가 `pull` 을 단순히 "다운로드"로 오해해, 합쳐지는 과정에서 충돌이 났을 때 당황합니다. 우리는

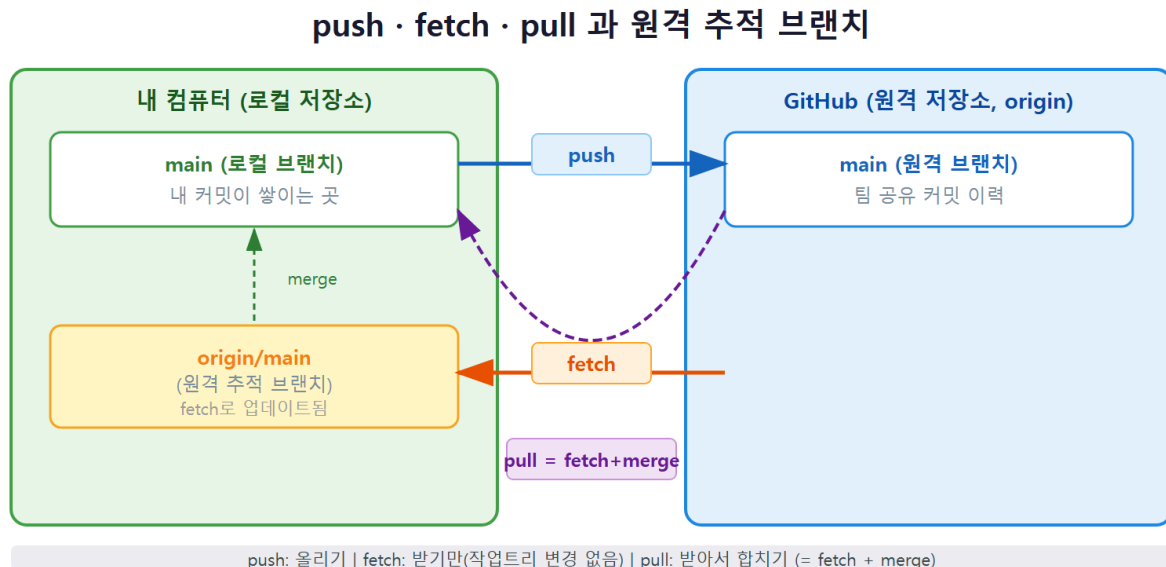
push·fetch·pull이 각각 무엇을 하고 어떤 흔한 문제가 생기는지, 그리고 그 문제를 어떻게 진단하고 해결하는지까지 차근차근 익힙니다.

## 한 문장 정의

**원격과 주고받기 — push·pull·fetch**란, `push`로 로컬 커밋을 원격(origin)에 올리고, `fetch`로 원격의 변경을 받아만 오며, `pull`(= `fetch` + `merge`)로 받아서 현재 브랜치에 합치는 일련의 동기화 작업입니다. `push -u`로 업스트림을 한 번 맺어 두면 이후에는 `git push`·`git pull`만으로 동기화됩니다.

선수 개념 상기: 이 챕터는 **원격 저장소·복제(c10)**와 **병합(c06)**을 선수로 합니다. origin이 무엇이고 로컬과 어떻게 연결되는지(ch\_10), 그리고 두 갈래를 합치는 `merge`가 어떻게 동작하는지(ch\_06)를 떠올리면 `pull`이 "받기 + `merge`"라는 사실이 자연스럽게 이해됩니다. 또 커밋·브랜치(ch\_03, ch\_05)는 모든 동기화의 단위가 됩니다.

## 동작 원리: push·fetch·pull과 origin/main 추적 브랜치



원격과 주고받기를 이해하는 열쇠는 **원격 추적 브랜치(remote-tracking branch)**라는 개념입니다. 이름이 길지만 뜻은 단순합니다. `origin/main`은 "내가 마지막으로 확인했을 때 원격의 `main`이 가리키던 커밋"을 기록해 두는, 로컬에 있는 **읽기 전용 북마크**입니다.

내 컴퓨터에는 사실 두 종류의 `main`이 있습니다.

| 브랜치         | 뜻                             | 누가 옮기나                      |
|-------------|-------------------------------|-----------------------------|
| main        | 내가 작업하는 로컬 브랜치                | 내가 커밋할 때                    |
| origin/main | 원격 main의 마지막 확인 위치(원격 추적 브랜치) | fetch · pull · push 가 통신할 때 |

세 명령은 이 두 브랜치 사이에서 다음과 같이 동작합니다.

1. `git push` — 로컬 `main`의 새 커밋을 원격에 올리고, 성공하면 로컬의 `origin/main`도 그만큼 앞으로 옮깁니다. "내 책상의 원고를 공유 책장에 꽂고, 책장 목록도 최신으로 갱신"입니다.
2. `git fetch` — 원격에 어떤 새 커밋이 있는지 받아와 `origin/main`만 갱신합니다. **로컬 `main`은 건드리지 않습니다.** 그래서 `fetch`는 언제 해도 안전합니다. "책장에 새로 들어온 책을 내 책상 한쪽에 가져다 두기만 함, 아직 내 원고엔 손대지 않음."
3. `git pull` — `fetch`를 한 뒤, 받아 온 `origin/main`을 로컬 `main`에 **merge(합치기)** 합니다. 즉 `pull = fetch + merge`입니다. "가져온 새 책을 곧장 내 원고에 반영."

이 구조에서 중요한 결론 두 가지가 나옵니다.

- `fetch`는 **받아만 오므로 절대 내 작업을 망치지 않습니다.** 불안하면 항상 `fetch`부터 해서 `git log origin/main`으로 원격 상황을 살펴볼 수 있습니다.
- `pull`은 **합치기를 포함하므로 충돌이 날 수 있습니다.** `pull`이 곧 `merge`라는 사실(ch\_06)을 알면, `pull` 중 충돌이 나도 "아, `merge` 충돌이구나"라며 ch\_07에서 배운 방식으로 해결하면 됩니다.

## git push와 -u origin (upstream)

`push`의 가장 기본 형태는 "어디로(원격) 무엇을(브랜치) 올릴지"를 적는 것입니다.

```
git push origin main
```

이는 "origin이라는 원격에 내 main 브랜치를 올려라"라는 뜻입니다. 그런데 매번 `origin main`을 적는 것은 번거롭습니다. 그래서 처음 한 번 **업스트림(upstream)**을 맺어 둡니다.

```
git push -u origin main
```

`-u` (또는 `--set-upstream`)는 "앞으로 로컬 main의 짝은 origin/main이다"라고 **추적 관계를 기억** 시킵니다. 이렇게 한 번 맺어 두면, 이후에는 브랜치 이름을 생략해도 됩니다.

```
git push
git pull
```

업스트림이 설정된 브랜치에서는 `git push` 만 쳐도 Git이 "이 브랜치의 짝은 origin/main이지"라고 알아서 그곳으로 올립니다. `git pull` 도 마찬가지로 어디서 받아 합칠지 자동으로 압니다. 업스트림이 맺어졌는지는 다음으로 확인합니다.

```
git status
git branch -vv
```

`git branch -vv` 는 각 로컬 브랜치 옆에 짝(예: `[origin/main]`)을 대괄호로 보여 줍니다.

## worked example: 첫 push와 이후 간단 동기화

상황부터 정리합니다. ch\_10에서 만든 샘플 저장소( `index.html` · `README.md` )가 로컬에 있고, GitHub에 빈 저장소를 만들어 `git remote add origin` 까지 끝냈다고 가정합니다. 이제 첫 push를 합니다.

### 상황 1: 첫 push로 원격에 올리기

```
git status
git remote -v
git push -u origin main
```

명령을 차례로 실행하면 **실행 결과**는 다음과 같이 나옵니다.

```
$ git status
On branch main
nothing to commit, working tree clean

$ git remote -v
origin  https://github.com/my-id/intro-site.git (fetch)
origin  https://github.com/my-id/intro-site.git (push)

$ git push -u origin main
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Writing objects: 100% (6/6), 612 bytes | 612.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0)
To https://github.com/my-id/intro-site.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

- `git status` 로 먼저 **올릴 것이 커밋된 상태인지** 진단합니다. `working tree clean` 이면 모든 변경이 커밋돼 `push`할 준비가 됐다는 뜻입니다(`push`는 커밋만 올립니다).
- `git remote -v` 로 `origin` 주소가 올바른지 진단합니다. `fetch·push` 두 줄이 같은 주소면 정상입니다.
- `git push -u origin main`: 처음이라 원격에 `main`이 없으므로 `* [new branch] main -> main` 이 출력됩니다.
- 마지막 줄 `set up to track 'origin/main'` 이 바로 `-u` 의 효과입니다. 이제 업스트림이 맺어졌습니다.

## 상황 2: 이후 간단 동기화

업스트림을 맺었으니, 다음 변경부터는 짧게 끝납니다. README에 한 줄을 추가해 커밋한 뒤 올려 봅니다.

```
git add README.md
git commit -m "소개 문구 한 줄 추가"
git push
```

```
$ git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Writing objects: 100% (3/3), 318 bytes | 318.00 KiB/s, done.
To https://github.com/my-id/intro-site.git
 a1b2c3d..e4f5g6h  main -> main
```

- 이번에는 `origin main` 을 적지 않고 `git push` 만 했습니다. 업스트림 덕분에 Git이 알아서 `origin/main`으로 올립니다.

- 출력의 `a1b2c3d..e4f5g6h main -> main` 은 "원격 main을 a1b2c3d에서 e4f5g6h로 전진시켰다"는 뜻입니다. 첫 push의 `[new branch]` 와 달리, 두 점(..)은 **기존 브랜치를 앞으로 전진(fast-forward)**시켰음을 나타냅니다.
- 이 두 단계(상황 1:2)가 1인 작업의 일상 동기화 전부입니다. 커밋하고 `git push`, 이게 끝입니다.

### 상황 3: 기능 브랜치를 처음 push하기

협업에서는 main에 곧장 올리지 않고, ch\_05에서 배운 **기능 브랜치**를 만들어 거기서 작업한 뒤 push합니다. 새 브랜치는 원격에 아직 짝이 없으므로, 첫 push에서 업스트림을 함께 맺어 줍니다.

```
git switch -c feature/contact
git add index.html
git commit -m "연락처 섹션 추가"
git push -u origin feature/contact
```

이때의 **실행 결과**는 다음과 같습니다.

```
$ git push -u origin feature/contact
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Writing objects: 100% (3/3), 402 bytes | 402.00 KiB/s, done.
remote:
remote: Create a pull request for 'feature/contact' on GitHub by visiting:
remote:   https://github.com/my-id/intro-site/pull/new/feature/contact
remote:
To https://github.com/my-id/intro-site.git
 * [new branch]      feature/contact -> feature/contact
branch 'feature/contact' set up to track 'origin/feature/contact'.
```

- `git switch -c feature/contact` 로 기능 브랜치를 만들어 그 안에서 작업·커밋합니다 (ch\_05). main은 그대로 둡니다.
- 새 브랜치도 첫 push에서 `-u` 로 업스트림을 맺어 줍니다. 이후 그 브랜치에서는 `git push` 만으로 올라갑니다.
- GitHub가 `Create a pull request ...` 링크를 친절히 안내합니다. 이 PR 흐름은 ch\_13에서 본격적으로 다룹니다. 지금은 "기능 브랜치도 push로 올린다"만 익히면 충분합니다.

### 상황 4: 연결 상태를 점검하는 `git remote show`

동기화가 꼬일 때 전체 상태를 한눈에 보는 진단 명령이 `git remote show origin` 입니다.

```
git remote show origin
```

이 명령의 **실행 결과**에는 원격 주소, 각 로컬 브랜치가 어느 원격 브랜치를 추적하는지, push-pull 설정이 함께 표시됩니다.

```
$ git remote show origin
* remote origin
  Fetch URL: https://github.com/my-id/intro-site.git
  Push URL: https://github.com/my-id/intro-site.git
  HEAD branch: main
  Remote branches:
    main          tracked
    feature/contact tracked
  Local branch configured for 'git pull':
    main merges with remote main
  Local ref configured for 'git push':
    main pushes to main (up to date)
```

- Fetch URL · Push URL 로 주소가 올바른지 진단합니다(둘이 같아야 정상).
- Remote branches 에 원격에 어떤 브랜치가 있는지, tracked 로 내가 추적 중인지가 보입니다.
- Local branch configured for 'git pull' · Local ref configured for 'git push' 로 업스트림 쪽이 제대로 맺혔는지 확인합니다. (up to date) 면 로컬과 원격이 같은 상태라는 뜻입니다.
- 동기화가 이상할 때 이 한 명령으로 "주소·추적·앞뒤 상태"를 모두 진단할 수 있어 유용합니다.

## git fetch vs git pull(=fetch+merge), --rebase 개요

협업이 시작되면 "받기"가 중요해집니다. 동료의 원격에 올린 변경을 내 쪽으로 가져와야 하기 때문입니다. 여기서 fetch 와 pull 의 차이가 결정적입니다.

```
git fetch origin
git log --oneline origin/main
```

git fetch 는 원격의 새 커밋을 받아 origin/main 만 갱신하고, 내 로컬 main은 그대로 둡니다. 그래서 위처럼 git log origin/main 으로 "원격에 무엇이 새로 들어왔는지" 안전하게 먼저 살펴본 뒤, 합칠지 말지 내가 결정할 수 있습니다.



```
$ git fetch origin
remote: Enumerating objects: 4, done.
Unpacking objects: 100% (3/3), 264 bytes, done.
From https://github.com/my-id/intro-site
   e4f5g6h..h7i8j9k  main       -> origin/main

$ git status
On branch main
Your branch is behind 'origin/main' by 1 commit, and can be fast-forwarded.
(use "git pull" to update your local branch)
```

fetch 후 git status 가 "behind 'origin/main' by 1 commit"이라고 알려 줍니다. 원격이 내 로컬보다 1커밋 앞섰다는 진단입니다. 위 **실행 결과**의 e4f5g6h..h7i8j9k main -> origin/main 은 "origin/main을 h7i8j9k까지 갱신했다"는 뜻이며, 이때 로컬 main은 여전히 e4f5g6h에 그대로 머물러 있습니다(fetch는 작업을 안 건드림). 이제 합칩니다.

```
git pull
```

git pull 은 사실상 git fetch + git merge origin/main 을 한 번에 한 것입니다. 만약 처음부터 git pull 만 했다면 fetch와 merge가 한꺼번에 일어났을 뿐, 내부 동작은 같습니다.

pull 에는 합치는 방식을 바꾸는 --rebase 옵션도 있습니다.

```
git pull --rebase
```

- 기본 git pull 은 받아 온 변경을 **merge**로 합칩니다(필요하면 병합 커밋이 생김).
- git pull --rebase 는 merge 대신 **rebase**로, 내 로컬 커밋을 원격 끝 위로 다시 쌓아 **직선 이력**을 만듭니다. rebase 자체는 ch\_14에서 자세히 다루므로, 여기서는 "pull에 두 가지 합치기 방식이 있다"는 개요만 기억하면 충분합니다.

## worked example: 양쪽이 다른 파일을 고쳐 병합 커밋이 생기는 pull

pull 이 merge라는 사실을 눈으로 확인해 봅시다. 동료가 원격의 index.html 을 고쳐 올렸고, 그 사이 나는 로컬에서 README.md 를 고쳐 커밋했다고 합시다. 서로 다른 파일을 고쳤으므로 충돌은 없지만, 양쪽이 모두 진행됐기 때문에 pull은 3-way merge(ch\_06)로 **병합 커밋**을 만듭니다.

```
git add README.md
git commit -m "소개 문단 다듬기"
git pull
```

이때의 **실행 결과**는 다음과 같습니다.

```
$ git pull
remote: Enumerating objects: 4, done.
Unpacking objects: 100% (3/3), done.
From https://github.com/my-id/intro-site
   e4f5g6h..h7i8j9k  main      -> origin/main
Merge made by the 'ort' strategy.
   index.html | 2 +-
   1 file changed, 1 insertion(+), 1 deletion(-)
```

- Merge made by the 'ort' strategy. 가 핵심입니다. pull이 곧 merge라서, 양쪽 변경을 합치는 **병합 커밋**이 자동으로 만들어졌습니다(서로 다른 파일이라 충돌 없이 자동 병합).
- 합친 뒤 `git log --oneline --graph` 로 보면, 두 갈래가 하나로 모이는 병합 커밋이 보입니다. ch\_06에서 배운 3-way merge가 원격 동기화에서도 똑같이 일어난 것입니다.
- 만약 같은 파일의 같은 줄을 양쪽이 고쳤다면 여기서 **CONFLICT** 가 났을 것이고, 그때는 ch\_07 방식(편집 → add → commit)으로 마무리합니다. **pull = fetch + merge**라는 한 줄을 기억하면 이 모든 동작이 일관되게 이해됩니다.

## 헛갈릴 땐 fetch부터

협업 중 "지금 원격이 어떤 상태지?"가 궁금하면, 곧장 `git pull` 하지 말고 `git fetch` 후 `git log --oneline origin/main` 으로 **먼저 살펴보는 습관**이 안전합니다. fetch는 내 작업을 절대 건드리지 않기 때문에, 상황을 파악한 뒤 합칠지 판단할 여유를 줍니다.

## 흔한 문제·해결: non-fast-forward 거부 → pull 후 push

협업에서 가장 자주 만나는 문제가 **push 거부(non-fast-forward rejected)**입니다. 입문자를 당황시키지만, 원리를 알면 해결은 정해진 한 가지 흐름입니다.

### 상황: 원격이 나보다 앞선 채로 push를 시도

내가 로컬에서 커밋하는 동안, 동료(또는 GitHub 웹에서 내가 직접) 원격 main에 다른 커밋을 먼저 올렸다고 합시다. 이제 내가 push하면 다음과 같이 거부됩니다.

```
git push
```

```
$ git push
To https://github.com/my-id/intro-site.git
! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'https://github.com/my-id/intro-site.git'
hint: Updates were rejected because the remote contains work that you do
hint: not have locally. This is usually caused by another repository pushing
hint: to the same ref. You may want to first integrate the remote changes
hint: (e.g., 'git pull ...') before pushing again.
```

## 진단: 왜 거부됐나

핵심 줄은 `! [rejected] main -> main (fetch first)` 와 `the remote contains work that you do not have locally` 입니다. 원격에는 내가 아직 안 받은 커밋이 있어서, 내가 그대로 push하면 그 커밋을 덮어 잃게 됩니다. Git은 이런 사고를 막기 위해 push를 거부합니다. 즉 이 거부는 오류가 아니라 "먼저 받아서 합치라"는 안전 신호입니다.

## 해결: pull로 받아 합친 뒤 다시 push

```
git pull
git push
```

```
$ git pull
Updating e4f5g6h..h7i8j9k
Fast-forward
 README.md | 1 +
 1 file changed, 1 insertion(+)

$ git push
Enumerating objects: 5, done.
...
h7i8j9k..m9n0p1q  main -> main
```

- `git pull` 로 원격의 앞선 커밋을 받아 내 로컬 main에 합칩니다. 위 예처럼 내 로컬에 충돌이 없으면 `Fast-forward` 로 깔끔히 합쳐집니다.
- 합친 뒤에는 내 로컬이 원격을 포함한 최신 상태이므로, `git push` 가 정상적으로 통과합니다.
- 만약 pull 과정에서 같은 부분을 서로 다르게 고쳤다면 `merge` 충돌이 납니다. 그때는 `ch_07`에서 배운 대로 충돌 표시를 편집하고 `git add` → `git commit` 으로 해결한 뒤 push 하면 됩니다.

## 절대 --force로 밀지 말 것

push가 거부됐다고 `git push --force` 로 밀어붙이면, 원격에 있던 동료의 커밋을 그대로 덮어 없애 버립니다. 입문 단계에서 force push는 사고의 지름길입니다. 거부는 "받아서 합치라"는 신호이니, 답은 언제나 `git pull` 후 `git push` 입니다. (force push의 위험과 예외적 사용은 ch\_14에서 황금률과 함께 다룹니다.)

## pull 방식의 두 갈래: merge로 합칠까, rebase로 정렬할까

거부를 해결할 때 `git pull` 은 기본적으로 merge로 합치지만, 직선 이력을 원하면 `git pull --rebase` 로 해결할 수도 있습니다. 두 방식의 차이를 표로 비교합니다.

| 해결 방식     | 명령                                                     | 결과 이력                    | 병합 커밋   |
|-----------|--------------------------------------------------------|--------------------------|---------|
| merge(기본) | <code>git pull</code> 후 <code>git push</code>          | 두 갈래가 합쳐진 갈래 이력          | 생길 수 있음 |
| rebase    | <code>git pull --rebase</code> 후 <code>git push</code> | 내 커밋이 원격 끝 위로 재정렬된 직선 이력 | 없음      |

입문 단계에서는 기본 `git pull (merge)`로 충분합니다. 병합 커밋이 조금 늘어나도 안전하고 직관적이기 때문입니다. 직선 이력을 선호하는 팀은 rebase 방식을 쓰지만, rebase는 이력 해시가 바뀌는 동작이라 **공유한 이력에는 함부로 쓰면 안 된다는 황금률(ch\_14)**을 먼저 배운 뒤 선택하는 것이 안전합니다. 지금은 "거부되면 받아서 합치고 다시 올린다"는 기본기를 확실히 다지는 데 집중합니다.

## 거부를 미리 예방하는 한 줄 습관

push 거부는 "올리려는 사이에 원격이 먼저 바뀌어서" 생깁니다. 그래서 **작업을 시작하기 전과 올리기 직전에 `git pull` 한 번**을 습관으로 두면, 원격과 보조를 맞춰 거부를 크게 줄일 수 있습니다. 협업에서 "pull 먼저, push 나중"은 가장 값싼 사고 예방책입니다.

## 즉시 연습(2종 1세트): push·pull 동기화 채우기

### 직접 작성 1: 첫 push와 거부 해결 (빈칸 명령)

아래는 "첫 push → 원격 선행으로 거부 → 해결"의 흐름입니다. 빈칸(\_\_\_\_)에 알맞은 명령·옵션을 채우십시오.

```
# (1) 처음 올리며 업스트림까지 맺기
git push ____ origin main

# (2) 이후 동료가 원격을 먼저 갱신 → 내가 push하니 거부됨
git push
# ! [rejected] main -> main (fetch first)

# (3) 거부를 해결하는 두 명령
git ____
git ____
```

**자가체크 체크리스트** — 스스로 점검하십시오.

- ☐ (1)의 빈칸에 업스트림을 맺는 옵션 `-u` 를 넣었는가
- ☐ (3)의 첫 빈칸에 받아서 합치는 명령 `pull` 을 넣었는가
- ☐ (3)의 둘째 빈칸에 다시 올리는 명령 `push` 를 넣었는가
- ☐ 거부됐을 때 `--force` 를 쓰지 않는 이유를 설명할 수 있는가
- ☐ `git push -u origin main` 이후엔 왜 `git push` 만으로 되는지 설명할 수 있는가

## 직접 작성 2: fetch와 pull 구분하기 (백지 작성)

처음부터 직접 명령을 적어 봅니다. 다음 요구사항을 만족하는 명령 순서를 작성하십시오.

1. 원격의 새 커밋을 **받아만 오고** 로컬 `main`은 건드리지 않도록 합니다.
2. 받아 온 원격 상태(`origin/main`)의 커밋 목록을 한 줄 형식으로 살펴봅니다.
3. 살펴본 뒤 문제가 없으면, 받아 온 변경을 로컬 `main`에 **합칩니다**.

또한 다음 표의 빈칸을 채워, `fetch`와 `pull`의 차이를 스스로 정리하십시오.

| 명령                     | 원격에서 받아오나 | 로컬 <code>main</code> 에 합치나 |
|------------------------|-----------|----------------------------|
| <code>git fetch</code> | 예         | (빈칸 A)                     |
| <code>git pull</code>  | 예         | (빈칸 B)                     |

**자가체크 체크리스트**

- ☐ 1번에 `git fetch`(또는 `git fetch origin`)를 적었는가
- ☐ 2번에 `git log --oneline origin/main` 을 적었는가
- ☐ 3번에 `git pull`(또는 `git merge origin/main`)을 적었는가
- ☐ 빈칸 A에 "아니오", 빈칸 B에 "예"를 넣었는가
- ☐ "fetch는 안전하고 pull은 충돌이 날 수 있다"를 그 이유와 함께 설명할 수 있는가

## 오개념 교정: pull은 합치기·거부는 신호·fetch는 안전

이 챕터에서 입문자가 가장 자주 빠지는 오개념들을 흔한 오류 사례와 함께 바로잡습니다.

### 오개념 1: `git pull`은 그냥 다운로드다

가장 흔한 오해입니다. `pull`은 단순 다운로드가 아니라 **다운로드(fetch) + 합치기(merge)**입니다. 그래서 내가 같은 파일의 같은 부분을 고쳐 둔 상태에서 `pull` 하면 **merge 충돌**이 날 수 있습니다.

```
$ git pull
...
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
```

이때 당황할 필요 없습니다. `pull`이 `merge`라는 사실을 알면, ch\_07에서 배운 대로 `<<<<<<<` 표시를 편집해 최종본을 만들고 `git add` → `git commit`으로 마무리하면 됩니다. "받기만 하고 싶다"면 `pull` 대신 `git fetch`를 쓰면 합치기 없이 받아만 옵니다.

### 오개념 2: `push` 거부는 내가 뭔가 잘못된 오류다

! [rejected] ... (fetch first)를 보고 "내가 실수했나" 하며 `force`로 밀어 버리는 것이 가장 위험한 흔한 오류입니다. 거부는 **잘못이 아니라 "원격에 내가 아직 안 받은 변경이 있으니 먼저 받아 합치라"**는 안전 신호입니다.

```
$ git push --force      # 절대 이렇게 해결하지 말 것
+ h7i8j9k...m9n0p1q main -> main (forced update) # 동료 커밋이 사라짐!
```

올바른 해결은 언제나 `git pull` 후 `git push`입니다. `force`는 원격의 남의 커밋을 덮어 없애므로, 협업에서는 사고로 이어집니다.

### 오개념 3: `fetch`는 위험하니 합부로 하면 안 된다

정반대입니다. `fetch`는 **가장 안전한 명령**입니다. 원격의 변경을 `origin/main`으로 받아만 오고 내 로컬 작업트리·로컬 `main`은 전혀 건드리지 않기 때문입니다. 그래서 "지금 원격이 어떤 상태인지" 궁금할 때는 마음껏 `fetch`해도 됩니다.

```
git fetch origin          # 받아만 올 — 내 작업은 안전
git log --oneline origin/main # 원격 상황을 살펴본 뒤
git pull                  # 괜찮으면 그때 합치기
```

정작 합치기(충돌 가능성)는 `pull / merge` 단계에서 일어납니다. `fetch`와 `merge`를 분리해 생각하면 동기화 과정 전체가 또렷해집니다.

## 세 명령 한눈에 정리

지금까지 배운 `push`·`fetch`·`pull`을 한 표로 정리합니다. 협업 중 "지금 무엇을 해야 하지?"가 헛갈릴 때 이 표를 떠올리면 됩니다.

| 명령                     | 방향      | 무엇을 하나                    | 내 작업트리를 바꾸나 | 충돌 가능성        |
|------------------------|---------|---------------------------|-------------|---------------|
| <code>git push</code>  | 로컬 → 원격 | 내 커밋을 원격에 올림              | 아니오         | 없음(거부될 수는 있음) |
| <code>git fetch</code> | 원격 → 로컬 | 원격 변경을 받아 origin/main만 갱신 | 아니오         | 없음            |
| <code>git pull</code>  | 원격 → 로컬 | fetch 후 로컬 main에 merge    | 예           | 있음            |

여기에 일상 협업의 권장 순서를 덧붙입니다.

1. 작업을 시작하기 전 `git pull` 로 원격 최신을 먼저 받아 합칩니다(출발선을 맞춤).
2. 기능 브랜치에서 작업하고 의미 단위로 커밋합니다(ch\_03-ch\_05).
3. 올리기 전 `git pull` 로 그사이 들어온 변경을 합칩니다(거부 예방).
4. `git push` 로 올립니다. 거부되면 다시 `git pull` 후 `git push` 로 해결합니다.

이 순서를 몸에 익히면 non-fast-forward 거부를 사전에 줄이고, 만나더라도 당황하지 않게 됩니다.

## 자주 묻는 질문(FAQ)

**Q. git push 만 쳤더니 "no upstream branch"라고 나옵니다.** A. 아직 업스트림을 안 맺은 새 브랜치입니다. 안내대로 `git push -u origin <브랜치이름>` 을 한 번 실행하면 업스트림이 맺어지고, 이후에는 `git push` 만으로 됩니다.

**Q. fetch 를 했는데 파일이 안 바뀝니다. 정상인가요?** A. 정상입니다. fetch는 `origin/main` 만 갱신하고 로컬 작업 파일은 건드리지 않습니다. 받아 온 내용을 실제 파일에 반영하려면 `git pull` (또는 `git merge origin/main`)을 추가로 해야 합니다.

**Q. pull 과 pull --rebase 중 무엇을 써야 하나요?** A. 입문 단계에서는 기본 `git pull` (merge 방식)로 충분합니다. 직선 이력을 선호하는 팀은 `--rebase` 를 쓰기도 하는데, rebase는 ch\_14에서 황금률과 함께 자세히 배운 뒤 선택해도 늦지 않습니다.

**Q. push할 때 매번 아이디·비밀번호를 묻습니다.** A. HTTPS 주소를 쓰면 인증이 필요합니다 (ch\_10). GitHub는 비밀번호 대신 개인 액세스 토큰(PAT)이나 자격 증명 관리자, 또는 SSH 키를 쓰도록 안내합니다. 한 번 설정해 두면 이후 자동으로 인증됩니다.

**Q. origin/main 과 main 이 자꾸 헛갈립니다.** A. `main` 은 내가 직접 커밋하며 옮기는 로컬 작업 브랜치이고, `origin/main` 은 통신할 때만 갱신되는 원격의 마지막 확인 위치입니다. `git status` 가 "ahead/behind"로 알려 주는 것이 바로 이 둘의 차이입니다. ahead면 내가 안 올린 커밋이 있다는 뜻(push 필요), behind면 원격에 안 받은 커밋이 있다는 뜻(pull 필요)입니다.

**Q. push가 거부됐는데 git pull 을 하니 충돌이 났습니다. 잘못된 건가요?** A. 아닙니다. pull은 merge라서, 같은 부분을 양쪽이 고쳤다면 충돌이 나는 것이 정상입니다(ch\_07). 충돌 표시를 편집해 최종본을 만들고 `git add` → `git commit` 으로 마무리한 뒤 `git push` 하면 됩니다. 충돌은 실패가 아니라 "둘 중 무엇을 남길지 사람이 정하라"는 정상 과정입니다.

## 이 chapter 정리

- 원격과 주고받기는 `push` (올리기)·`fetch` (받기만)·`pull` (받아서 합치기)의 세 동작으로 이뤄집니다.
- `origin/main` 은 "원격 main의 마지막 확인 위치"를 기록하는 로컬의 읽기 전용 북마크(원격 추적 브랜치)입니다.
- `git push -u origin main` 으로 업스트림을 한 번 맺으면, 이후에는 `git push` ·`git pull` 만으로 동기화됩니다.
- `fetch` 는 안전하게 받아만 오고, `pull` 은 `fetch + merge` 라서 충돌이 날 수 있습니다. 충돌은 ch\_07 방식으로 해결합니다.
- push 거부(non-fast-forward)는 잘못이 아니라 "먼저 받아 합치라"는 신호이며, 해결은 언제나 `git pull` 후 `git push` 입니다. `--force` 는 쓰지 않습니다.



- `git remote show origin` 은 주소·추적·앞뒤 상태를 한 명령으로 진단해 주므로, 동기화가 꼬일 때 가장 먼저 떠올릴 만한 도구입니다.
- 다음 챕터(ch\_12)에서는 안정된 지점에 `git tag` 로 버전을 표시하고 GitHub 릴리스를 만드는 법을 배웁니다. push로 커밋을 올리는 일과 달리, 태그는 따로 올려야 한다는 점이 핵심 연결 고리입니다.



# 태그와 릴리스: git tag로 버전 표시하기

## 이 챕터에서 배우는 것

- lightweight 태그와 annotated 태그의 차이를 설명할 수 있습니다.
- `git tag -a` 로 버전 태그를 만들고 `git show` 로 확인할 수 있습니다.
- 태그를 `git push origin <tag>` 또는 `--tags` 로 원격에 올릴 수 있습니다.
- GitHub에서 태그를 기반으로 릴리스를 만드는 흐름을 설명할 수 있습니다(개요).

## 도입: '이 지점이 배포 버전'을 표시하기

앞 챕터(ch\_11)에서 우리는 로컬 커밋을 원격에 올리고(push) 받아 합치는(pull) 동기화를 익혔습니다. 이제 저장소에는 커밋이 차곡차곡 쌓입니다. 그런데 수십 개의 커밋 중에서 **"바로 이 커밋이 사용자에게 내보낸 정식 버전 1.0"**이라는 표시는 어떻게 남길까요?

커밋 해시(예: `a1b2c3d`)는 정확하지만 사람이 외우거나 부르기 어렵습니다. "지난주 화요일에 올린 그 커밋"이라고 말하는 대신, **v1.0.0**이라는 **이름표**를 그 커밋에 붙여 두면 누구나 한눈에 알아봅니다. 이것이 **태그(tag)**입니다.

핵심 비유: 태그는 책에 끼우는 **고정된 책갈피**입니다. 일반 책갈피(브랜치)는 읽던 자리에 따라 계속 옮겨지지만, "여기가 제3장 시작"이라고 라벨을 붙여 영구히 끼워 둔 책갈피(태그)는 그 자리에 그대로 머물러 있습니다. 책을 더 읽어 나가도(새 커밋을 쌓아도) **v1.0.0** 책갈피는 처음 끼운 그 커밋에 고정됩니다.

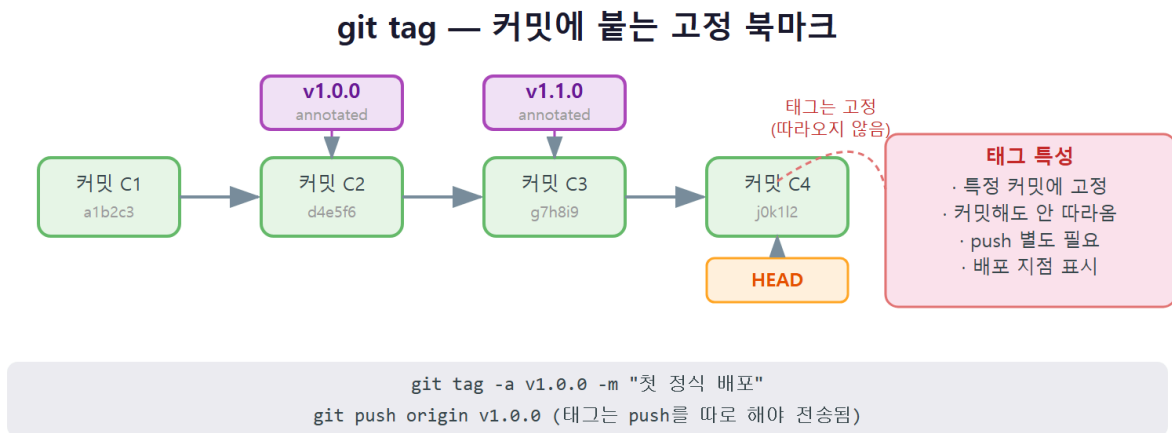
태그는 배포·릴리스의 기준점이 됩니다. 버그 신고가 들어오면 "어느 버전에서요?"라고 묻고, **v1.0.0** 태그가 가리키는 커밋으로 돌아가 그 시점의 코드를 정확히 재현할 수 있습니다. 또 GitHub에서는 이 태그를 토대로 **릴리스(release)**를 만들어, 변경 사항을 정리한 노트와 함께 배포 지점을 공개합니다.

## 한 문장 정의

**태그와 릴리스** — **tag**란, 특정 커밋에 `v1.0.0` 같은 이름을 붙이는 고정 북마크(태그)를 만들고, 보통 `-a`로 작성자·날짜·메시지가 담긴 주석 태그(annotated tag)를 달아 `git push origin <tag>`로 원격에 올린 뒤, GitHub에서 그 태그를 기반으로 배포 지점을 공개(릴리스)하는 작업입니다.

선수 개념 상기: 이 챕터는 **원격과 주고받기** — **push·pull·fetch(c11)**를 선수로 합니다. 태그는 커밋을 가리키는 또 하나의 이름표이므로 커밋·해시(ch\_04)를 떠올리면 자연스럽게, 태그를 원격에 올릴 때 `push(ch_11)`를 그대로 활용합니다. 다만 태그는 **기본적으로 push에 자동으로 달려가지 않는**다는 점이 이 챕터의 핵심 주의 사항입니다.

## 동작 원리: 커밋에 붙는 고정 북마크 태그



태그를 브랜치와 비교하면 동작이 또렷해집니다. 둘 다 "커밋을 가리키는 이름"이지만 움직이는 방식이 정반대입니다.

| 구분       | 브랜치(main 등)        | 태그(v1.0.0 등)    |
|----------|--------------------|-----------------|
| 무엇을 가리키나 | 커밋                 | 커밋              |
| 새 커밋을 하면 | 자동으로 <b>따라 옮겨감</b> | 그 자리에 <b>고정</b> |
| 주된 용도    | 작업이 진행되는 흐름        | 특정 시점(버전) 표시    |

핵심은 이것입니다. 내가 `v1.0.0` 태그를 어떤 커밋에 붙인 뒤 새 커밋을 여러 개 더 쌓아도, `main` 브랜치는 최신 커밋으로 계속 전진하지만 `v1.0.0` 태그는 **처음 붙인 그 커밋에 그대로 머물러 있습니다**. 그래서 언제든지 `v1.0.0` 이라는 이름만으로 "정식 1.0 버전이던 그 시점"을 정확히 가리킬 수 있습니다.

동작을 단계로 정리하면 다음과 같습니다.

- 어떤 커밋이 "내보낼 만한 안정된 지점"이 되면, 그 커밋에 `git tag` 로 이름표를 붙입니다.
- 이후 개발이 계속돼 새 커밋이 쌓여도 태그는 고정된 채 남습니다.
- 태그를 `git push origin <tag>` 로 원격에 올리면, GitHub에서 그 태그가 보입니다.
- GitHub에서 그 태그를 골라 **릴리스**를 만들면, 변경 노트·다운로드 묶음과 함께 배포 지점이 공개됩니다.

## lightweight vs annotated(-a, -m) 태그

태그에는 두 종류가 있습니다.

- **lightweight 태그(가벼운 태그)** — 커밋을 가리키는 **이름만** 있는 단순 북마크입니다. 작성자·날짜·메시지 같은 정보가 없습니다. 잠깐 표시해 두는 임시 용도에 적합합니다.

```
git tag v0.9-test
```

- **annotated 태그(주석 태그)** — `-a` 로 만들며, **작성자·날짜·메시지**가 함께 저장되는 정식 태그입니다. 누가 언제 왜 이 버전을 냈는지가 기록되므로 **릴리스에는 annotated 태그를 씁니다.**

```
git tag -a v1.0.0 -m "첫 정식 릴리스"
```

`-a` 는 annotated(주석)를, `-m` 은 그 태그에 담을 메시지를 뜻합니다. 두 종류의 차이를 표로 정리합니다.

| 구분     | lightweight               | annotated( -a )                         |
|--------|---------------------------|-----------------------------------------|
| 만드는 법  | <code>git tag v0.9</code> | <code>git tag -a v1.0.0 -m "..."</code> |
| 작성자·날짜 | 없음                        | 있음                                      |
| 메시지    | 없음                        | 있음                                      |
| 권장 용도  | 임시·개인 표시                  | 정식 버전·릴리스                               |

## 릴리스에는 annotated를 쓰자

배포 버전을 표시할 때는 거의 항상 `-a` (annotated)를 씁니다. "누가 언제 이 버전을 냈는가"가 이력에 남아야 추적이 가능하기 때문입니다. lightweight 태그는 "잠깐 여기 표시해 두자" 정도

의 가벼운 용도로만 쓰는 것이 좋습니다.

## worked example: v1.0.0 태그 달고 show로 확인·push

상황을 정리합니다. ch\_11에서 push까지 끝낸 샘플 저장소가 있고, 현재 main이 "내보낼 만한 안정된 지점"에 도달했다고 가정합니다. 이제 이 지점에 v1.0.0 태그를 달고 확인한 뒤 원격에 올립니다.

### 상황 1: annotated 태그 만들고 목록·내용 확인

```
git tag -a v1.0.0 -m "첫 정식 릴리스: 소개 페이지 완성"
git tag
git show v1.0.0
```

이때의 실행 결과는 다음과 같습니다.

```
$ git tag
v1.0.0

$ git show v1.0.0
tag v1.0.0
Tagger: Hong Gildong <hong@example.com>
Date: Sat Jun 21 10:20:30 2026 +0900

첫 정식 릴리스: 소개 페이지 완성

commit e4f5g6h7i8j9k0l1m2n3o4p5q6r7s8t9 (HEAD -> main, tag: v1.0.0)
Author: Hong Gildong <hong@example.com>
Date: Sat Jun 21 10:15:00 2026 +0900

소개 문구 한 줄 추가
```

- `git tag -a v1.0.0 -m "..."`: 현재 커밋(HEAD)에 주석 태그를 겁니다. 커밋을 따로 지정하지 않으면 **현재 HEAD가 가리키는 커밋**에 붙습니다.
- `git tag` (옵션 없이): 저장소의 모든 태그를 **목록**으로 보여 줍니다. 방금 만든 v1.0.0 이 보입니다.
- `git show v1.0.0`: 태그의 **작성자·날짜·메시지**(Tagger·Date·메시지)와 그 태그가 가리키는 **커밋 내용**을 함께 보여 줍니다. annotated 태그라서 Tagger 정보가 있는 점에 주목합니다.
- 출력의 `(HEAD -> main, tag: v1.0.0)` 은 지금 이 커밋에 main 브랜치와 v1.0.0 태그가 함께 놓여 있다는 뜻입니다.

## 상황 2: 태그를 원격에 올리기

여기서 입문자가 가장 자주 놓치는 부분입니다. 태그를 만들고 `git push` 만 하면, **태그는 원격에 올라가지 않습니다**. 태그는 따로 올려야 합니다.

```
git push
git push origin v1.0.0
```

```
$ git push
Everything up-to-date

$ git push origin v1.0.0
Enumerating objects: 1, done.
Counting objects: 100% (1/1), done.
Writing objects: 100% (1/1), 180 bytes | 180.00 KiB/s, done.
Total 1 (delta 0), reused 0 (delta 0)
To https://github.com/my-id/intro-site.git
 * [new tag]          v1.0.0 -> v1.0.0
```

- 첫 `git push` 는 `Everything up-to-date` 라고 나옵니다. 커밋은 이미 다 올라가 있지만 **태그는 자동으로 따라가지 않았기** 때문에, 이것만으로는 원격에 태그가 없습니다.
- `git push origin v1.0.0`: 이름을 꼭 집어 그 태그 하나를 원격에 올립니다. `* [new tag] v1.0.0 -> v1.0.0` 이 성공 표시입니다.
- 여러 태그를 한꺼번에 올리려면 `git push origin --tags` 를 씁니다. 다만 `--tags` 는 lightweight 태그까지 모두 올리므로, 보통은 올릴 태그를 꼭 집어 올리는 편이 깔끔합니다.

## 상황 3: 과거 커밋에 태그 달고 잘못된 태그 지우기

태그는 꼭 현재 커밋에만 다는 것은 아닙니다. "사실 두 커밋 전이 진짜 안정 지점이었다" 싶으면, 그 커밋의 해시를 지정해 과거에 태그를 달 수 있습니다.

```
git log --oneline
git tag -a v0.9.0 e4f5g6h -m "베타 지점"
git show v0.9.0
```

이때의 **실행 결과**는 다음과 같습니다.

```
$ git log --oneline
m9n0p1q (HEAD -> main, tag: v1.0.0) 첫 정식 릴리스 지점
e4f5g6h 소개 페이지 초안
a1b2c3d 프로젝트 초기 구조

$ git tag -a v0.9.0 e4f5g6h -m "베타 지점"

$ git show v0.9.0
tag v0.9.0
Tagger: Hong Gildong <hong@example.com>
...
commit e4f5g6h... (tag: v0.9.0)
    소개 페이지 초안
```

이름을 잘못 지었다면 로컬 태그를 지우고 다시 만들 수 있습니다.

```
git tag -d v0.9.0
```

이때의 실행 결과는 다음과 같습니다.

```
$ git tag -d v0.9.0
Deleted tag 'v0.9.0' (was a7b8c9d)
```

- `git tag -a v0.9.0 e4f5g6h -m "..."`: 커밋 해시 `e4f5g6h`를 지정해 과거 커밋에 태그를 붙입니다. 해시를 생략하면 현재 HEAD에 붙습니다.
- `git tag -d v0.9.0`: 로컬 태그를 지웁니다(-d는 delete). 아직 원격에 안 올린 태그라면 이렇게 자유롭게 지우고 다시 만들 수 있습니다.
- 이미 원격에 올린 태그를 지우려면 `git push origin --delete v0.9.0`이 필요하지만, 남이 받았을 수 있어 신중해야 합니다. 입문 단계에서는 올리기 전에 이름을 정확히 짓는 습관이 더 중요합니다.

## 유의적 버전(SemVer)과 GitHub Releases 개요

태그 이름을 아무렇게나 지어도 동작하지만, 널리 쓰이는 약속이 있습니다. **유의적 버전 (SemVer, Semantic Versioning)**은 버전을 MAJOR.MINOR.PATCH 세 자리로 짓는 규칙입니다.



| 자리 | 이름    | 언제 올리나       | 예              |
|----|-------|--------------|----------------|
| 첫째 | MAJOR | 호환되지 않는 큰 변경 | 1.0.0 -> 2.0.0 |
| 둘째 | MINOR | 호환되는 기능 추가   | 1.0.0 -> 1.1.0 |
| 셋째 | PATCH | 호환되는 버그 수정   | 1.0.0 -> 1.0.1 |

예를 들어 v1.0.0 에서 버그를 고치면 v1.0.1, 새 기능을 더하면 v1.1.0, 기존 사용법을 깨는 큰 변경이면 v2.0.0 이 됩니다. 앞에 붙이는 v 는 관례이며 필수는 아니지만 널리 쓰입니다. 한 가지 규칙만 더 기억하면 좋습니다. **윗자리를 올리면 아랫자리는 0으로 되돌립니다.** 그래서 v1.4.2 에서 기능을 추가하면 v1.5.2 가 아니라 v1.5.0 이 되고, 큰 변경이면 v2.0.0 이 됩니다. 이렇게 자리마다 뜻이 분명하면, 버전 번호만 보고도 "이 업데이트가 안전한 수정인지, 새 기능인지, 주의가 필요한 큰 변경인지"를 가늠할 수 있습니다.

**GitHub Releases**는 이 태그를 한 단계 더 포장한 **배포 페이지**입니다. 흐름은 다음과 같습니다 (개요).

1. 안정 지점에 annotated 태그(v1.0.0)를 만들어 원격에 올립니다(앞의 worked example).
2. GitHub 저장소의 Releases 메뉴에서 "Draft a new release"를 누릅니다.
3. 올려 둔 태그(v1.0.0)를 고르고, **릴리스 노트**(이번 버전의 변경 사항)를 적습니다.
4. 게시하면, 사용자는 그 페이지에서 변경 내역과 소스 묶음(zip)을 받아 볼 수 있습니다.

## worked example: 태그로 그 시점 코드 살펴보기

태그의 진짜 쓸모는 "그 버전 시점의 코드를 정확히 재현"하는 데 있습니다. 버그 신고가 "v1.0.0 에서 안 돼요"라고 들어오면, 그 태그 지점으로 잠깐 돌아가 확인할 수 있습니다.

```
git tag --list "v1.*"
git checkout v1.0.0
```

이때의 **실행 결과**는 다음과 같습니다.

```
$ git tag --list "v1.*"
```

```
v1.0.0
```

```
v1.1.0
```

```
$ git checkout v1.0.0
```

```
Note: switching to 'v1.0.0'.
```

You are in 'detached HEAD' state. You can look around, make experimental changes and commit them, and you can discard any commits you make in this state without impacting any branches by switching back to a branch.

HEAD is now at e4f5g6h 소개 페이지 초안

확인이 끝나면 원래 작업하던 브랜치로 돌아옵니다.

```
git switch -
```

- `git tag --list "v1.*"`: 패턴으로 태그를 거릅니다. 태그가 많아지면 `v1.`로 시작하는 것만 추려 보기 편합니다.
- `git checkout v1.0.0`: 그 태그가 가리키는 커밋으로 작업트리를 잠깐 되돌립니다. 이때 `detached HEAD` (분리된 HEAD) 상태라는 안내가 나옵니다. 태그는 브랜치가 아니라 **고정된 한 지점**이라, 그 위에서 작업하려면 브랜치가 아닌 커밋에 직접 올라선 상태가 되기 때문입니다.
- `git switch -`: 직전에 있던 브랜치(예: `main`)로 다시 돌아옵니다. 살펴보기만 했다면 이렇게 안전하게 복귀하면 됩니다.

## detached HEAD에서 당황하지 않기

태그를 `checkout` 하면 나오는 "detached HEAD" 안내는 오류가 아니라 **"지금 브랜치가 아니라 특정 커밋 위에 서 있다"**는 알림입니다. 그냥 둘러보기만 했다면 `git switch -` (또는 `git switch main`)로 브랜치에 돌아오면 됩니다. 그 지점에서 수정을 이어 가고 싶다면 `git switch -c hotfix/v1.0.x` 처럼 **새 브랜치를 만들어** 작업합니다.

## 태그와 릴리스는 층이 다르다

태그는 **Git**의 기능이고, 릴리스는 **GitHub**의 기능입니다. 태그는 "커밋에 붙은 이름표"라는 저장소 안의 표시이고, 릴리스는 그 태그를 골라 변경 노트·다운로드를 붙여 **공개하는 한 겹**입니다. 그래서 태그 없이 릴리스를 만들 수 없고(릴리스는 태그를 가리킴), 태그만 있고 릴리스는 안 만든 상태도 가능합니다.

## 좋은 릴리스 노트 쓰기

릴리스의 핵심은 태그 자체가 아니라 **"이번 버전에서 무엇이 달라졌는가"**를 사람이 읽을 수 있게 정리한 노트입니다. 입문 단계에서도 다음 세 묶음만 지키면 충분히 쓸모 있는 노트가 됩니다.

| 묶음           | 무엇을 적나    | 예                   |
|--------------|-----------|---------------------|
| 추가됨(Added)   | 새로 생긴 기능  | 연락처 섹션 추가           |
| 변경됨(Changed) | 기존 동작의 변화 | 소개 문구 다듬음           |
| 수정됨(Fixed)   | 고친 버그     | 모바일에서 줄바꿈 깨지던 문제 수정 |

예시 릴리스 노트는 다음과 같은 모습입니다.

```
## v1.1.0 (2026-06-21)

### Added
- 연락처 섹션과 이메일 링크를 추가했습니다.

### Changed
- 소개 문구를 한 문단으로 다듬었습니다.

### Fixed
- 좁은 화면에서 제목이 두 줄로 깨지던 문제를 고쳤습니다.
```

이렇게 변경을 묶어 적으면, 사용자는 한눈에 "이 버전을 올려야 할 이유"를 알 수 있고, 나중에 우리도 "v1.1.0이 무엇이었지?"를 빠르게 떠올릴 수 있습니다. 커밋 메시지(ch\_03)를 평소에 잘 써 두면 릴리스 노트를 쓸 때 그대로 재료가 됩니다.

## 흔한 문제·해결: 태그가 원격에 안 보임 → push --tags

태그에서 가장 자주 겪는 문제입니다. 원리를 알면 해결은 명령 하나입니다.

### 상황: 분명히 태그를 만들었는데 GitHub에 안 보임

```
git tag -a v1.0.0 -m "첫 정식 릴리스"
git push
```

이렇게 하고 GitHub의 태그·릴리스 화면을 봐도 `v1.0.0` 이 보이지 않습니다.

## 진단: 태그는 push에 자동으로 딸려 가지 않는다

`git push` 는 **브랜치의 커밋만** 올립니다. 태그는 그 대상에 포함되지 않으므로, 커밋은 올라갔어도 태그는 여전히 로컬에만 있습니다. 로컬에는 태그가 있는지 다음으로 확인합니다.

```
git tag
```

```
$ git tag
v1.0.0
```

로컬에는 `v1.0.0` 이 있는데 원격에 없다면, 진단은 명확합니다. **태그를 아직 안 올린 것**입니다.

## 해결: 태그를 명시적으로 push

```
git push origin v1.0.0
```

특정 태그 하나만 올립니다. 만약 만들어 둔 태그가 여러 개라 한꺼번에 올리고 싶다면 다음을 씁니다.

```
git push origin --tags
```

```
$ git push origin --tags
Total 1 (delta 0), reused 0 (delta 0)
To https://github.com/my-id/intro-site.git
* [new tag]          v1.0.0 -> v1.0.0
```

- `git push origin v1.0.0`: 태그 하나를 콕 집어 올립니다. 어떤 태그가 올라갔는지 분명해 권장됩니다.
- `git push origin --tags`: 로컬의 모든 태그를 한꺼번에 올립니다. 빠르지만 임시·테스트용 lightweight 태그까지 함께 올라갈 수 있으니 주의합니다.
- 올린 뒤 GitHub의 Releases-Tags 화면을 새로고침하면 `v1.0.0` 이 보입니다. 이제 이 태그로 릴리스를 만들 수 있습니다.

## 즉시 연습(2종 1세트): 태그·릴리스 채우기

## 직접 작성 1: v1.0.0 태그 만들고 올리기 (빈칸 명령)

아래는 "주석 태그 생성 → 확인 → 원격에 올리기"의 흐름입니다. 빈칸(\_\_\_\_)에 알맞은 명령·옵션을 채워주세요.

```
# (1) 현재 커밋에 작성자·메시지가 담긴 정식 태그 만들기
git tag ____ v1.0.0 ____ "첫 정식 릴리스"

# (2) 태그 목록과 내용 확인
git tag
git ____ v1.0.0

# (3) 이 태그 하나를 원격에 올리기
git push origin ____
```

**자가체크 체크리스트** — 스스로 점검하십시오.

- ☐ (1)의 첫 빈칸에 annotated를 뜻하는 `-a` 를 넣었는가
- ☐ (1)의 둘째 빈칸에 메시지를 뜻하는 `-m` 을 넣었는가
- ☐ (2)의 빈칸에 태그 내용을 보는 `show` 를 넣었는가
- ☐ (3)의 빈칸에 올릴 태그 이름 `v1.0.0` 을 넣었는가
- ☐ `git push` 만 해서는 왜 태그가 원격에 안 가는지 설명할 수 있는가

## 직접 작성 2: SemVer로 다음 버전 정하기 (백지 작성)

`v1.0.0` 을 낸 뒤 아래 상황이 차례로 생겼습니다. 각 상황에서 **다음 태그 이름**을 SemVer 규칙으로 직접 정해 적으십시오.

- 소개 페이지의 오타를 고쳤다(버그 수정, 호환됨). 다음 버전은? \_\_\_\_
- 그 뒤 '연락처' 섹션을 새로 추가했다(기능 추가, 호환됨). 다음 버전은? \_\_\_\_
- 그 뒤 페이지 구조를 완전히 바꿔 기존 링크가 깨졌다(호환 안 됨). 다음 버전은? \_\_\_\_

또한 각 버전을 실제로 만들어 올리는 명령도 한 줄씩 적어 보십시오(예: `git tag -a <버전> -m "<메시지>"` → `git push origin <버전>`).

**자가체크 체크리스트**

- ☐ 1번에 PATCH를 올린 `v1.0.1` 을 적었는가
- ☐ 2번에 MINOR를 올리고 PATCH를 0으로 되돌린 `v1.1.0` 을 적었는가
- ☐ 3번에 MAJOR를 올린 `v2.0.0` 을 적었는가
- ☐ 각 태그를 `-a` 로 만들고 `git push origin <태그>` 로 올리는 명령을 적었는가
- ☐ MAJOR·MINOR·PATCH를 각각 언제 올리는지 설명할 수 있는가

## 오개념 교정: 태그 자동 전송 아님·고정 북마크·릴리스 층 구분

### 오개념 1: 태그는 push하면 자동으로 따라간다

가장 흔한 오류입니다. `git push` 는 브랜치의 커밋만 올리며 태그는 기본적으로 전송하지 않습니다. 그래서 태그를 만들고 push만 하면 GitHub에 태그가 안 보입니다.

```
$ git push
Everything up-to-date          # 커밋은 올라갔지만 태그는 그대로 로컬에만
```

해결책: 태그는 따로 올립니다. 하나면 `git push origin v1.0.0`, 모두면 `git push origin --tags` 입니다.

### 오개념 2: 태그는 새 커밋을 하면 같이 따라 움직인다

태그를 브랜치처럼 생각해서 생기는 오해입니다. 브랜치(main)는 커밋할 때마다 최신 커밋으로 따라 옮겨가지만, **태그는 처음 붙인 커밋에 고정됩니다**. 새 커밋을 아무리 쌓아도 `v1.0.0` 은 그 자리에 그대로 있습니다.

```
$ git log --oneline
m9n0p1q (HEAD -> main) 다음 작업 커밋      <- main은 여기로 전진
e4f5g6h (tag: v1.0.0) 첫 정식 릴리스 지점  <- v1.0.0은 여기 고정
```

그래서 태그는 "버전 시점"을 영구히 표시하는 데 알맞습니다. 그 시점을 옮기고 싶다면 새 태그(`v1.0.1` 등)를 새 커밋에 붙입니다(이미 올린 태그를 옮겨 다시 올리는 것은 혼란을 주므로 피합니다).

### 오개념 3: 태그를 만들면 곧 릴리스가 된다

태그와 릴리스를 같은 것으로 여기는 오해입니다. 태그는 **Git의 표시**이고, 릴리스는 **GitHub에서 그 태그를 골라 변경 노트·다운로드를 붙여 공개하는 별도 단계**입니다. 태그를 올렸다고 자동으로 릴리스 페이지가 생기지는 않습니다.

올바른 이해: 태그를 원격에 올린 뒤, GitHub의 Releases에서 그 태그를 골라 노트를 적고 게시해야 비로소 릴리스가 공개됩니다. 즉 "태그 → (선택) 릴리스"의 층이 다릅니다.

## 단원 미니 프로젝트: GitHub에 올리고 v1.0.0 릴리스(2종 1세트)

이 단원(u5: 원격·동기화·태그)의 마무리로, 앞의 **원격 저장소(c10)·push·pull·fetch(c11)·태그와 릴리스(c12)**를 모두 사용해 작은 산출물을 완성합니다. 목표는 "혼자 만든 샘플 저장소를 GitHub에 올리고, 원격 선행 상황을 pull로 해결한 뒤, 안정 지점에 **v1.0.0** 태그·릴리스를 남기기"입니다.

### 모범 — 마일스톤으로 나누어 완성하기

**마일스톤 1:** GitHub 빈 저장소와 로컬을 연결하고 첫 push로 올립니다(c10-c11 적용).

```
git remote add origin https://github.com/my-id/intro-site.git
git remote -v
git push -u origin main
```

```
$ git push -u origin main
* [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

**마일스톤 2:** 원격이 앞선 상황(GitHub 웹에서 README를 직접 한 줄 수정)을 일부러 만들어, push 거부를 pull로 해결합니다(c11 적용).

```
git push
git pull
git push
```

```
$ git push
! [rejected]      main -> main (fetch first)
error: failed to push some refs to '...'
```

```
$ git pull
Updating e4f5g6h..h7i8j9k
Fast-forward
 README.md | 1 +
```

```
$ git push
h7i8j9k..m9n0p1q  main -> main
```

**마일스톤 3:** 안정된 지점에 annotated 태그를 달고 원격에 올린 뒤, GitHub에서 릴리스를 만듭니다(c12 적용).

```
git tag -a v1.0.0 -m "첫 정식 릴리스: 소개 페이지 공개"
git push origin v1.0.0
git tag
```

```
$ git push origin v1.0.0
* [new tag]          v1.0.0 -> v1.0.0

$ git tag
v1.0.0
```

마지막으로 GitHub의 Releases 메뉴에서 "Draft a new release"를 눌러 `v1.0.0` 태그를 고르고, 릴리스 노트를 적어 게시하면 배포 지점이 공개됩니다.

- 마일스톤 1은 ch\_10의 `remote add`와 ch\_11의 `push -u`를 묶어, 로컬을 원격과 잇고 업스 트림까지 맺습니다.
- 마일스톤 2는 ch\_11의 핵심 문제 해결(non-fast-forward 거부 → pull 후 push)을 그대로 적용합니다. force를 쓰지 않고 안전하게 해결한 점이 중요합니다.
- 마일스톤 3은 이 챕터의 annotated 태그·태그 push·릴리스 흐름을 적용합니다. 태그를 따 로 push해야 GitHub에 보인다는 점을 확인합니다.
- 세 마일스톤을 합치면 "혼자 만든 저장소를 세상에 공개하고 첫 버전을 고정"하는 한 바퀴 가 완성됩니다.

## 학생 작성형 빈 템플릿

### 직접 작성 3: 나만의 릴리스 한 바퀴 (빈 템플릿)

위 모범을 참고해, 내 저장소로 직접 한 바퀴를 돕니다. 빈칸(\_\_\_\_)을 채우고 실제로 실행하십시오.

```
# 마일스톤 1: 원격 연결 + 첫 push(업스트림 포함)
git remote add origin <내 저장소 URL>
git push ____ origin main          # 빈칸: 업스트림 옵션

# 마일스톤 2: 원격 선행으로 거부되면 해결
git push
# ![rejected] ... (fetch first)
git ____                          # 빈칸: 받아서 합치기
git push

# 마일스톤 3: v1.0.0 태그 달고 올리기
git tag ____ v1.0.0 -m "첫 정식 릴리스" # 빈칸: 주석 태그 옵션
git push origin ____                 # 빈칸: 올릴 태그 이름
```



## 자가체크 체크리스트 — 스스로 점검하십시오.

- [ ] `git push -u origin main` 후 `git branch -vv` 에 `[origin/main]` 이 보이는가(업스트림 확인)
- [ ] non-fast-forward 거부를 `--force` 없이 `git pull` 후 push로 해결했는가
- [ ] `v1.0.0` 태그를 `-a` (annotated)로 만들었는가
- [ ] `git push origin v1.0.0` 까지 해서 GitHub의 Tags 화면에 태그가 보이는가
- [ ] GitHub Releases에서 그 태그로 릴리스를 게시했는가
- [ ] 로컬 main과 origin/main이 같은 커밋을 가리키는가(`git status` 가 up to date인가)

## 자주 묻는 질문(FAQ)

**Q. lightweight와 annotated 중 무엇을 써야 하나요?** A. 정식 버전·릴리스에는 `-a` (annotated)를 쓰십시오. 작성자·날짜·메시지가 남아 추적이 됩니다. lightweight는 잠깐 표시해 두는 임시 용도로만 권합니다.

**Q. 태그 이름을 잘못 지었습니다. 어떻게 고치나요?** A. 아직 원격에 안 올렸다면 `git tag -d v1.0.0` 으로 로컬 태그를 지우고 다시 만들면 됩니다. 이미 원격에 올렸다면 다른 사람이 받았을 수 있으니, 옮기기보다 올바른 이름의 새 태그를 만드는 편이 혼란이 적습니다.

**Q. 과거 커밋에 태그를 달 수 있나요?** A. 가능합니다. `git tag -a v0.5.0 <커밋해시> -m "..."` 처럼 커밋 해시를 지정하면 그 과거 커밋에 태그가 붙습니다. 커밋을 지정하지 않으면 현재 HEAD에 붙습니다.

**Q. `git push origin --tags` 와 `git push origin v1.0.0` 의 차이는?** A. 앞은 로컬의 모든 태그를, 뒤는 지정한 태그 하나만 올립니다. 임시 태그까지 한꺼번에 올라가는 것을 피하려면, 보통 올릴 태그를 꼭 집어 올리는 편이 깔끔합니다.

**Q. 태그 이름에 꼭 `v` 를 붙여야 하나요?** A. 필수는 아닙니다. `v1.0.0` 처럼 `v` 를 붙이는 것은 널리 쓰이는 관례일 뿐, `1.0.0` 이라고 지어도 동작합니다. 다만 팀 안에서 한 가지 방식으로 통일하는 편이 혼란이 적습니다.

**Q. 릴리스를 꼭 만들어야 태그가 의미가 있나요?** A. 아닙니다. 태그만으로도 "그 시점 코드 재현"·"버전 기준점" 같은 쓸모가 충분합니다. GitHub Releases는 거기에 변경 노트·다운로드를 붙여 공개적으로 배포하고 싶을 때 더하는 한 겹입니다. 내부용 표시라면 태그까지만으로도 됩니다.

**Q. 커밋을 더 한 뒤에 보니 `v1.0.0`이 옛 커밋을 가리킵니다. 옮겨야 하나요?** A. 정상입니다. 태그는 처음 붙인 커밋에 고정되는 것이 본래 동작입니다. 새 안정 지점은 옛 태그를 옮기지 말고

v1.0.1·v1.1.0 같은 **새 태그**를 새 커밋에 붙이십시오. 이미 올린 태그를 옮기면 그 태그를 받은 사람과 어긋나 혼란을 줍니다.

## 이 챕터 정리

- 태그는 특정 커밋에 v1.0.0 같은 이름을 붙이는 고정 북마크로, 새 커밋을 쌓아도 그 자리에 머물러 있습니다(브랜치와 반대).
- lightweight 태그는 이름만, annotated 태그(-a -m)는 작성자·날짜·메시지까지 담으며, 릴리스에는 annotated를 씁니다.
- git tag 로 목록을, git show <tag> 로 내용을 확인하고, 태그는 git push origin <tag> (또는 --tags)로 **따로** 올려야 원격에 보입니다. 과거 커밋에는 해시를 지정해 태그를 달 수 있고, 잘못 지은 로컬 태그는 git tag -d 로 지운 뒤 다시 만들면 됩니다.
- 유의적 버전(SemVer)은 MAJOR.MINOR.PATCH 로, 큰 변경·기능 추가·버그 수정에 따라 자리를 올립니다.
- GitHub Releases는 태그를 골라 변경 노트·다운로드를 붙여 공개하는 한 겹으로, 태그(Git)와 릴리스(GitHub)는 층이 다릅니다.
- 태그로 git checkout 하면 detached HEAD 상태가 되는데, 이는 오류가 아니라 "특정 커밋 위에 서 있다"는 알림이며 git switch - 로 돌아오면 됩니다.
- 다음 단계(ch\_13~)에서는 PR·코드 리뷰·이슈로 여럿이 함께 변경을 제안·검토하는 GitHub 협업으로 나아갑니다. 협업으로 안정된 main에 태그·릴리스를 남기면, 이번 챕터의 버전 표시가 팀 단위 배포로 확장됩니다.



# GitHub 협업: PR·코드 리뷰·이슈와 gh CLI

## 이 챕터에서 배우는 것

- 이슈(Issue)로 할 일·버그를 기록하고 담당자·라벨로 추적할 수 있습니다.
- 기능 브랜치를 push하고 풀 리퀘스트(PR)로 변경을 제안할 수 있습니다.
- 코드 리뷰의 라인 코멘트·Approve·Request changes를 구분하고, 반려 후 재작업 흐름을 설명할 수 있습니다.
- PR 머지 방식(merge commit·squash·rebase)의 차이를 개요 수준에서 설명할 수 있습니다.
- gh CLI(gh issue create · gh pr create · gh pr merge 등)로 터미널에서 협업할 수 있습니다.
- PR에 충돌이 생겨 머지가 막힐 때 로컬에서 병합·해결·push로 풀 수 있습니다.

이번 챕터에서 가르치는 개념은 **GitHub 협업 — PR·코드 리뷰·이슈(GitHub Pull Request, code review, issues)** 한 가지입니다. 선수 개념은 두 가지입니다. 하나는 ch\_11의 **push·pull·fetch**입니다. PR은 "내 브랜치를 push한 뒤, 그 변경을 합쳐 달라"고 제안하는 일이라, push가 전제됩니다. 다른 하나는 ch\_07의 **충돌 해결(merge conflict)**입니다. 여러 사람이 같은 파일을 건드리면 PR 단계에서 충돌이 드러나고, 그때 ch\_07에서 익힌 진단·편집·add·commit 흐름을 그대로 씁니다. 두 선수 개념이 흐릿하다면 잠깐 되짚고 오시길 권합니다.

## 13.1 도입: 변경을 제안하고 함께 검토하는 관문, PR

지금까지는 혼자 작업했습니다. 브랜치를 파고(ch\_05), 커밋하고(ch\_03), main에 직접 merge하고(ch\_06), 원격에 push했습니다(ch\_11). 그런데 동료가 셋, 다섯, 열 명이 되는 순간 한 가지 질문이 생깁니다. "누군가 main을 망가뜨리는 변경을 몰래 밀어 넣으면 어떻게 막지?"

혼자일 때는 내 판단만 믿으면 됐지만, 팀에서는 변경이 main에 들어가기 전에 **다른 사람이 한번 봐 주는 관문**이 필요합니다. "이 변경을 main에 합쳐도 될까요?"라고 묻고, 동료가 코드를 읽고 의견을 달고, 괜찮다고 승인하면 그제서야 합치는 절차 말입니다. 이 관문이 바로 **풀 리퀘스트(Pull Request, PR)**입니다.

PR은 git이 아니라 **GitHub가 제공하는 협업 기능**이라는 점이 중요합니다. git의 세계는 push까지입니다. 즉 "내 커밋을 원격 브랜치에 올리는 것"까지가 git이고, "그 브랜치를 main

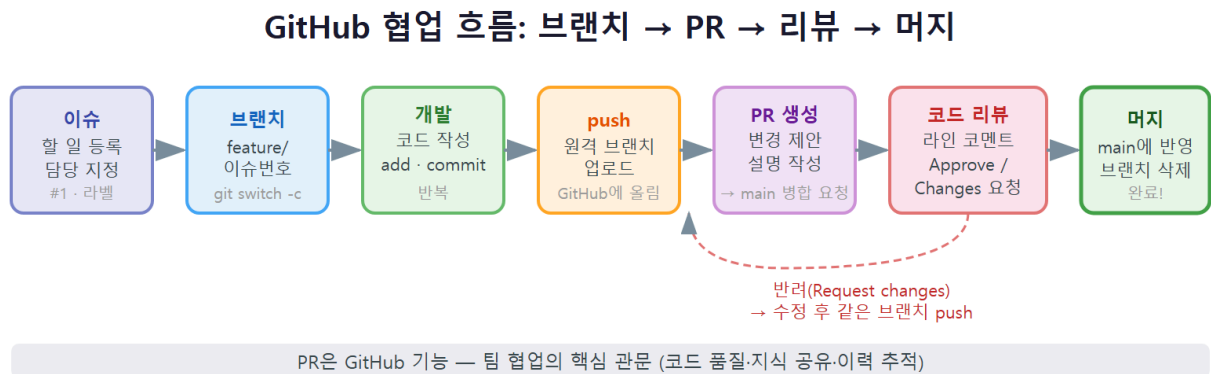
에 합쳐 달라고 제안하고 동료가 리뷰하는 것"은 GitHub라는 웹 서비스가 그 위에 얹은 협업 층입니다.

여기에 **이슈(Issue)**가 짝을 이룹니다. 이슈는 "할 일·버그·논의"를 적어 두는 메모판입니다. "로그인 버튼이 안 눌립니다", "연락처 섹션을 추가합니다" 같은 항목을 이슈로 만들어 담당자를 정하고 라벨을 붙입니다. 보통 **이슈로 할 일을 정의 → 기능 브랜치에서 구현 → PR로 제안 → 리뷰 → 머지 → 이슈 종료**의 한 바퀴를 돕니다.

### 한눈에 보는 GitHub 협업 — PR·코드 리뷰·이슈

풀 리퀘스트(Pull Request, PR)는 "내 브랜치의 변경을 main에 합쳐 달라"고 제안하고, 동료가 코드 리뷰(code review)로 검토·승인(Approve)·반려(Request changes)한 뒤 머지(merge)하는 GitHub의 협업 중심 기능입니다. 이슈(Issue)는 할 일·버그·논의를 기록·할당하는 공간이고, gh CLI로 터미널에서도 같은 작업을 할 수 있습니다. PR은 코드 품질을 팀이 함께 책임지는 표준 관문입니다.

## 13.2 동작 원리: 브랜치 push → PR → 리뷰 → 머지 흐름



PR 협업은 다음 다섯 단계로 흐름입니다. 각 단계가 무슨 일을 하는지 먼저 머릿속에 그림으로 담아 두면, 뒤의 명령들이 한결 쉬워집니다.

| 단계                | 무슨 일이 일어나는가                            | 어디서 하는가                               |
|-------------------|----------------------------------------|---------------------------------------|
| 1. 이슈 등록          | 할 일·버그를 적고 담당자·라벨 지정                   | GitHub(또는 <code>gh issue</code> )     |
| 2. 기능 브랜치<br>push | 내 브랜치를 원격에 올림                          | 로컬 <code>git push</code>              |
| 3. PR 생성          | "이 브랜치를 <code>main</code> 에 합쳐 달라" 제안  | GitHub(또는 <code>gh pr create</code> ) |
| 4. 코드 리뷰          | 동료가 라인 코멘트·승인·반려                       | GitHub(또는 <code>gh pr review</code> ) |
| 5. 머지·정리          | 승인된 PR을 <code>main</code> 에 합치고 브랜치 삭제 | GitHub(또는 <code>gh pr merge</code> )  |

여기서 한 가지 흔한 오해를 미리 풀고 갑니다. PR은 **브랜치와 브랜치 사이의 비교**입니다. "어느 브랜치(base, 보통 `main`)에 어느 브랜치(head, 내 기능 브랜치)를 합칠지"를 GitHub가 비교해, 그 차이(diff)와 커밋 목록을 한 페이지에 보여 줍니다. 그래서 PR을 만들기 전에 반드시 **내 기능 브랜치를 원격에 push**해 뒤야 합니다. 원격에 없는 브랜치는 GitHub가 비교할 수 없기 때문입니다.

또 하나, PR은 **살아 있는 페이지**입니다. PR을 만든 뒤에도 같은 브랜치에 커밋을 추가로 push하면, 그 커밋이 PR에 자동으로 따라붙어 diff가 갱신됩니다. 리뷰어가 "여기 고쳐 주세요"라고 반려하면, PR을 새로 만들 필요 없이 **같은 브랜치에 수정 커밋을 push**하기만 하면 됩니다. 이 성질이 협업을 매끄럽게 만듭니다.

### base와 head를 기억하세요

PR에는 두 브랜치가 등장합니다. **base**는 변경을 받을 쪽(거의 항상 `main`)이고, **head**는 변경을 보내는 쪽(내 기능 브랜치)입니다. "head를 base에 합쳐 달라"가 PR 한 문장입니다. 방향을 거꾸로 잡으면 엉뚱한 비교가 나오니, PR을 만들 때 base가 `main`인지 꼭 확인하십시오.

## 13.3 이슈로 할 일·버그 추적과 할당·라벨

협업의 출발점은 코드가 아니라 **"무엇을 할 것인가"**를 글로 적는 일입니다. 그 글이 이슈입니다. 이슈에는 보통 다음이 담깁니다.

- **제목(title):** 한 줄 요약. 예) "연락처 섹션 추가"
- **본문(body):** 배경·재현 절차·기대 결과. 버그라면 "어떤 상황에서 무엇이 잘못되는지".
- **담당자(assignee):** 이 일을 맡은 사람.
- **라벨(label):** bug, enhancement, documentation 같은 분류 꼬리표.
- **번호(#N):** GitHub가 자동으로 매기는 고유 번호. 커밋·PR에서 #3 처럼 참조합니다.

이슈를 GitHub 웹에서도 만들 수 있지만, 터미널에서 손을 떼지 않고 만드는 gh CLI가 협업에서 특히 편합니다. gh 는 GitHub가 공식 제공하는 명령줄 도구로, 한 번 gh auth login 으로 로그인해 두면 됩니다.

```
# 이슈 만들기: 제목·본문·담당자·라벨을 한 번에
gh issue create \
  --title "연락처 섹션 추가" \
  --body "푸터에 이메일과 GitHub 링크를 담은 연락처 섹션을 추가한다." \
  --assignee @me \
  --label enhancement

# 지금 저장소의 열린 이슈 목록 보기
gh issue list
```

```
Creating issue in myteam/todo-app

https://github.com/myteam/todo-app/issues/3

Showing 1 of 1 open issue in myteam/todo-app

#3 연락처 섹션 추가   enhancement   about 5 seconds ago
```

- gh issue create 의 --assignee @me 는 "나 자신을 담당자로"라는 뜻입니다. @me 는 로그인한 사용자를 가리키는 GitHub의 약속된 표현입니다.
- --label enhancement 로 분류 꼬리표를 붙였습니다. 라벨은 나중에 "기능 추가만 모아 보기"처럼 필터링할 때 쓰입니다.
- 출력의 마지막 줄에 https://.../issues/3 이 보입니다. 이 이슈의 번호는 3번이고, 앞으로 커밋 메시지나 PR에서 #3 으로 가리킵니다.

이슈 번호는 협업의 접착제입니다. 커밋 메시지에 #3 을 적으면 그 커밋이 이슈 3번과 연결되고, PR 본문에 closes #3 이라고 적으면 그 PR이 머지되는 순간 이슈 3번이 자동으로 닫힙니다. 손으로 이슈를 닫지 않아도 되니, 할 일과 코드가 끊김 없이 이어집니다.

## 13.4 worked example: 이슈 → 브랜치 → gh pr create → 머지

이제 협업 한 바퀴를 **상황 → 명령/조작 → 진단 → 결과**의 흐름으로 끝까지 돌아 보겠습니다. 무대는 작은 To-Do 웹앱 저장소( `todo-app` )이고, 방금 만든 이슈 3번("연락처 섹션 추가")을 처리합니다.

**상황 1 — 이슈에서 출발해 기능 브랜치를 판다.** 이슈 번호를 이름에 녹여 브랜치를 만들면 추적이 쉽습니다.

```
# main을 최신으로 맞추고 시작 (동료 변경을 먼저 받기)
git switch main
git pull

# 이슈 #3을 위한 기능 브랜치 생성 + 이동
git switch -c feature/3-contact-section
```

```
Already on 'main'
Already up to date.
Switched to a new branch 'feature/3-contact-section'
```

**상황 2 — 구현하고 커밋한다.** `index.html` 에 연락처 섹션을 추가했다고 합시다. 커밋 메시지에 #3 을 넣어 이슈와 잇습니다.

```
git add index.html
git commit -m "연락처 섹션 추가 (#3)"

# 브랜치를 원격에 올림 — PR을 만들려면 원격에 브랜치가 있어야 한다
git push -u origin feature/3-contact-section
```

```
[feature/3-contact-section 9a1c2e0] 연락처 섹션 추가 (#3)
1 file changed, 8 insertions(+)
remote:
remote: Create a pull request for 'feature/3-contact-section' on GitHub by visiting:
remote:   https://github.com/myteam/todo-app/pull/new/feature/3-contact-section
remote:
branch 'feature/3-contact-section' set up to track 'origin/feature/3-contact-section'.
```

여기서 `git` 은 이미 친절하게 "이 브랜치로 PR을 만들려면 이 주소로 가세요"라고 안내합니다. 우리는 웹에 가는 대신 터미널에서 바로 PR을 만듭니다.



**상황 3 — gh pr create 로 PR을 만든다.** 본문에 `Closes #3` 을 넣어, 이 PR이 머지되면 이슈가 자동으로 닫히도록 합니다.

```
gh pr create \
  --base main \
  --head feature/3-contact-section \
  --title "연락처 섹션 추가" \
  --body "푸터에 이메일·GitHub 링크 연락처 섹션을 추가한다. Closes #3"
```

Creating pull request for feature/3-contact-section into main in myteam/todo-app

<https://github.com/myteam/todo-app/pull/12>

**진단 — PR 상태를 확인한다.** 머지해도 되는 상태인지 `gh pr view` 로 점검합니다.

```
gh pr view 12
```

연락처 섹션 추가 #12

Open • alice wants to merge 1 commit into main from feature/3-contact-section

푸터에 이메일·GitHub 링크 연락처 섹션을 추가한다. Closes #3

Reviewers: (none)

Checks: (none)

**상황 4 — 리뷰를 받고 머지한다.** 리뷰는 13.5에서 자세히 다루므로, 여기서는 승인이 끝났다고 보고 머지합니다. `squash` 머지로 커밋을 하나로 합치고, 머지 후 기능 브랜치를 자동 삭제합니다.

```
gh pr merge 12 --squash --delete-branch
```

Merged pull request #12 (연락처 섹션 추가)

Deleted branch feature/3-contact-section and switched to branch main

**결과 검증 — 이슈가 자동으로 닫혔는지 본다.** PR 본문의 `Closes #3` 덕분에 이슈 3번이 닫혔어야 합니다. 다음 명령의 실행 결과로 확인합니다.

```
gh issue list --state closed
```

Showing 1 of 1 closed issue in myteam/todo-app

#3 연락처 섹션 추가 enhancement about 1 minute ago

- 이슈 → 브랜치 → 커밋(#3) → push → PR(Closes #3) → 머지 → 이슈 자동 종료까지, 할 일 하나가 코드로 끊김 없이 이어졌습니다. 이것이 GitHub 협업의 기본 한 바퀴입니다.
- `gh pr merge --squash` 는 기능 브랜치의 여러 커밋을 **하나로 합쳐** `main` 에 올립니다. `--delete-branch` 는 역할을 다한 브랜치를 정리해 목록을 깨끗하게 유지합니다.
- 머지 직후 `gh` 가 자동으로 `main` 으로 `switch` 해 줍니다. 이어서 로컬에서 `git pull` 로 머지된 결과를 받아 두면 다음 작업을 최신 상태에서 시작할 수 있습니다.

## 13.5 코드 리뷰: 라인 코멘트·Approve·Request changes

PR의 핵심 가치는 **코드 리뷰**에 있습니다. 리뷰어는 PR의 "Files changed" 탭에서 변경된 코드를 줄 단위로 보며 세 가지 행동을 합니다.

| 리뷰 행동           | 의미                    | 결과               |
|-----------------|-----------------------|------------------|
| Comment         | 의견·질문만 남김(판정 보류)      | 머지를 막지도 허용하지도 않음 |
| Approve         | "합쳐도 좋다"고 승인          | 머지 가능 표시         |
| Request changes | "이대로는 안 된다, 고쳐 달라" 반려 | 머지 차단(수정 필요)     |

리뷰어가 특정 줄에 다는 의견을 **라인 코멘트(line comment)**라고 합니다. "이 변수명이 모호합니다", "여기 들여쓰기가 어긋났습니다"처럼 정확히 그 줄에 붙어, 작성자가 어디를 고쳐야 할지 분명히 압니다. 여러 라인 코멘트를 모아 한 번에 제출하면서 Approve 또는 Request changes로 **종합 판정**을 내립니다.

`gh` 로도 리뷰할 수 있습니다. 팀 시뮬레이션에서 역할을 바꿔 리뷰어가 되어 보겠습니다.

# PR 12번을 승인

```
gh pr review 12 --approve --body "연락처 섹션 잘 동작합니다. LGTM!"
```

# 또는 반려: 고쳐야 할 점을 본문에 적어 변경 요청

```
gh pr review 12 --request-changes --body "이메일 링크의 mailto가 빠졌습니다. 보완 부탁드립니다."
```

Approving pull request #12

Requesting changes on pull request #12

- `gh pr review --approve` 는 승인입니다. 흔히 본문에 LGTM ("Looks Good To Me", 좋아 보입니다)이라고 적습니다.
- `--request-changes` 는 반려입니다. 무엇을 왜 고쳐야 하는지 구체적으로 적는 것이 핵심입니다. "고쳐 주세요"만으로는 작성자가 막막합니다.
- 리뷰는 비난이 아니라 품질을 함께 책임지는 협업입니다. 반려를 받았다고 위축될 필요 없이, 다음 절에서 보듯 같은 브랜치에 수정 push만 하면 됩니다.

### 자기 PR은 보통 자기가 승인하지 않습니다

코드 리뷰의 목적은 "다른 눈"으로 검증하는 것입니다. 그래서 팀 규칙상 PR 작성자 본인은 Approve할 수 없는 경우가 많습니다. 이 교재의 팀 협업 시뮬레이션에서는 혼자 두 역할(dev-alice·dev-bob)을 번갈아 연기하므로, 한 역할이 만든 PR을 다른 역할로 전환해 리뷰합니다. 종합 프로젝트(ch\_15~18)에서 이 방식을 본격적으로 씁니다.

## 13.6 반려 후 같은 브랜치 수정 push로 PR 갱신, 머지 방식 개요

리뷰어가 Request changes로 반려했습니다. 13.5의 예에서 "mailto가 빠졌다"는 지적을 받았다고 합시다. 이때 **PR을 새로 만들면 안 됩니다**. PR은 살아 있는 페이지라, 같은 브랜치에 수정 커밋을 push하면 그 PR이 자동으로 갱신됩니다.

```
# 반려받은 그 기능 브랜치로 돌아간다
git switch feature/3-contact-section

# 지적 사항(mailto 누락)을 수정하고 커밋
git add index.html
git commit -m "연락처 이메일에 mailto 링크 추가 (#3)"

# 같은 브랜치에 push — PR이 자동으로 갱신된다
git push
```

```
Switched to branch 'feature/3-contact-section'
[feature/3-contact-section 4d7f0b9] 연락처 이메일에 mailto 링크 추가 (#3)
1 file changed, 1 insertion(+), 1 deletion(-)
To https://github.com/myteam/todo-app.git
9a1c2e0..4d7f0b9 feature/3-contact-section -> feature/3-contact-section
```

push가 끝나면 PR 페이지의 커밋 목록과 diff에 새 커밋이 더해집니다. 리뷰어는 추가된 부분만 다시 보고, 만족하면 Request changes를 거두고 Approve로 바꿉니다. 이렇게 **반려 → 같은 브랜치 수정 push → 재리뷰 → 승인**의 작은 순환을 돌며 코드를 다듬습니다.

이제 머지 단계를 봅니다. PR을 `main`에 합칠 때 GitHub는 세 가지 머지 방식을 제공합니다. 셋의 결과 이력이 다르므로 개요를 알아 둡니다.

| 머지 방식                 | 이력에 남는 모습                                            | 언제 쓰나                                |
|-----------------------|------------------------------------------------------|--------------------------------------|
| Create a merge commit | 기능 커밋들 + 병합 커밋(두 부모) 모두 보존                           | 기능 단위와 통합 시점을 이력에 드러내고 싶을 때          |
| Squash and merge      | 기능의 여러 커밋을 <b>하나로 합쳐</b> <code>main</code> 에 한 커밋만   | 'wip'·'오타' 같은 지저분한 커밋을 깔끔히 정리하고 싶을 때 |
| Rebase and merge      | 기능 커밋들을 병합 커밋 없이 <code>main</code> 끝에 <b>직선으로</b> 엮음 | 병합 커밋 없는 직선 이력을 선호할 때                |

```
# 세 방식 중 하나를 골라 머지 (--merge / --squash / --rebase)
gh pr merge 12 --squash --delete-branch
```

Merged pull request #12 (연락처 섹션 추가)

- 반려 대응의 핵심은 "**같은 브랜치에 push**"입니다. `git push`만으로 PR이 갱신되니, 새 PR을 만들면 오히려 이력이 흩어집니다.
- 머지 방식의 차이는 ch\_06(병합)과 ch\_14(rebase)에서 배운 개념의 GitHub 버튼 버전입니다. squash는 "여러 커밋을 한 줄로", rebase는 "병합 커밋 없이 직선으로"라고 기억하면 됩니다.
- 입문 단계에서는 **squash 머지**를 기본으로 권합니다. 기능 하나가 `main`에 깔끔한 커밋 하나로 남아 이력을 읽기 쉽기 때문입니다.

## 13.7 흔한 문제·해결: PR 충돌로 머지 막힘 → 로컬 병합·해결·push

협업에서 가장 자주 만나는 벽입니다. 내가 PR을 만드는 동안 다른 동료의 PR이 먼저 `main`에 머지되어, 두 변경이 같은 파일의 같은 부분을 건드리면 GitHub가 머지 버튼을 잠그고 이렇게 알립니다.

```
This branch has conflicts that must be resolved
Conflicting files: index.html
```

GitHub는 간단한 충돌은 웹 편집기로도 풀게 해 주지만, 확실하고 안전한 방법은 로컬에서 직접 푸는 것입니다. 흐름은 ch\_07에서 배운 충돌 해결과 똑같습니다. 다만 합칠 대상이 "방금 앞서간 `main`"이라는 점만 다릅니다.

**상황 — PR 충돌로 머지가 막혔다.** 먼저 최신 `main`을 받습니다.

```
# 내 기능 브랜치에서, 최신 main을 받아 와 합친다
git switch feature/3-contact-section
git fetch origin
git merge origin/main
```

```
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.
```

**진단 — 어느 파일이 충돌했는지 status로 확인한다.**

```
git status
```

```
On branch feature/3-contact-section
You have unmerged paths.
  (fix conflicts and run "git commit")

Unmerged paths:
  (use "git add <file>..." to mark resolution)
    both modified:   index.html
```

**해결 — 충돌 표시를 편집해 최종본을 만든다.** 충돌 파일을 열면 ch\_07에서 본 익숙한 표시가 있습니다.

```
<<<<<< HEAD
<footer>연락처: contact@todo.app</footer>
=====
<footer>문의: help@todo.app</footer>
>>>>>> origin/main
```

<<<<<< HEAD 와 ===== 사이는 **내 변경**, ===== 와 >>>>>> origin/main 사이는 **먼저 머지된 동료의 변경**입니다. 두 의도를 모두 살려 최종본으로 직접 고치고, **충돌 표시 세 줄을 빠짐없이** 지웁니다.

```
<footer>연락처: contact@todo.app / 문의: help@todo.app</footer>
```

**마무리 — add·commit으로 병합을 완료하고 push해 PR을 다시 살린다.**

```
git add index.html
git commit -m "main 변경 병합: 연락처/문의 푸터 통합 (#3)"
git push
```

```
[feature/3-contact-section b2e8c41] main 변경 병합: 연락처/문의 푸터 통합 (#3)
To https://github.com/myteam/todo-app.git
 4d7f0b9..b2e8c41 feature/3-contact-section -> feature/3-contact-section
```

push가 끝나면 GitHub가 충돌이 사라졌음을 인식해 머지 버튼이 다시 열립니다. 이제 평소처럼 `gh pr merge` 로 머지하면 됩니다.

- PR 충돌은 에러가 아니라 "원격 `main` 이 내 브랜치보다 앞서갔다"는 신호입니다. ch\_07의 충돌 해결과 ch\_11의 동기화가 PR 위에서 만나는 지점입니다.
- 핵심 순서는 **fetch → merge origin/main → 충돌 해결 → commit → push**입니다. 로컬에서 미리 합쳐 두면, GitHub는 충돌 없는 깨끗한 PR을 보게 됩니다.
- 충돌 표시(<<<<<< · ===== · >>>>>>)는 한 줄도 남기면 안 됩니다. 남기면 HTML이 깨져 페이지가 망가집니다. `git diff` 로 표시 잔존을 한 번 더 확인하는 습관이 좋습니다.

## 13.8 즉시 연습(2종 1세트): PR 한 바퀴 채우기

이슈에서 머지까지 협업 한 바퀴를 직접 돌아 보는 연습입니다. 모범 흐름, 작성형 빈칸, 자가체크의 2종 1세트입니다.

### 모범 흐름

# 1) 이슈 생성 (담당자=나, 라벨=enhancement)

```
gh issue create --title "푸터에 저작권 표시 추가" --assignee @me --label enhancement
```

# 2) 최신 main에서 기능 브랜치 생성

```
git switch main
```

```
git pull
```

```
git switch -c feature/5-footer-copyright
```

# 3) 구현·커밋 (이슈 번호 #5 연결)

```
git add index.html
```

```
git commit -m "푸터에 저작권 표시 추가 (#5)"
```

# 4) push 후 PR 생성 (머지 시 이슈 자동 종료)

```
git push -u origin feature/5-footer-copyright
```

```
gh pr create --base main --title "푸터 저작권 표시" --body "Closes #5"
```

# 5) 리뷰 승인 후 squash 머지 + 브랜치 정리

```
gh pr merge --squash --delete-branch
```

```
https://github.com/myteam/todo-app/issues/5
```

```
Switched to a new branch 'feature/5-footer-copyright'
```

```
https://github.com/myteam/todo-app/pull/13
```

```
Merged pull request #13 (푸터 저작권 표시)
```

## 직접 작성: PR 한 바퀴 (빈칸 채우기)

아래 빈칸(\_\_\_\_)을 채워 협업 한 바퀴를 완성해 보십시오. 모범 흐름을 덮어 두고 도전하는 것이 효과적입니다.

```

# 1) 이슈 생성: 나를 담당자로
gh issue _____ --title "README에 사용법 추가" --assignee _____ --label documentation

# 2) 최신 main 받기 → 기능 브랜치 생성
git switch main
git _____
git switch _____ feature/7-readme-usage

# 3) 구현 후 커밋 (이슈 #7 연결)
git add README.md
git commit -m "README 사용법 섹션 추가 (_____)"

# 4) 원격에 올리고 PR 생성 (머지 시 이슈 자동 종료 문구 포함)
git push _____ origin feature/7-readme-usage
gh pr _____ --base main --title "README 사용법" --body "_____ #7"

# 5) 리뷰어로 전환해 승인
gh pr review --_____ --body "LGTM"

# 6) squash 머지 + 브랜치 삭제
gh pr merge --_____ --_____

```

**확장 과제:** PR을 만든 뒤 리뷰어가 "예시 코드가 빠졌다"고 Request changes 했다고 가정하고, 같은 브랜치에 수정 커밋을 **push**해 PR을 갱신하는 단계를 위 흐름에 끼워 넣어 보십시오.

## 자가체크 체크리스트

스스로 점검해 보십시오.

- [ ] 이슈를 만들고 담당자·라벨을 지정했는가( `gh issue create --assignee @me --label ...` )
- [ ] 기능 브랜치를 **최신 main** 에서 파고 push했는가(`git pull` 후 `switch -c` )
- [ ] 커밋 메시지에 이슈 번호( #7 )를 넣어 연결했는가
- [ ] PR 본문에 `closes #7` 을 넣어 머지 시 이슈가 자동으로 닫히게 했는가
- [ ] 반려를 받으면 새 PR이 아니라 **같은 브랜치에 수정 push**로 갱신했는가
- [ ] 머지 후 기능 브랜치를 정리( `--delete-branch` )했는가

## 13.9 오개념 교정: PR은 GitHub 기능·반려는 과정·머지 후 정리

PR 협업에서 입문자가 자주 부딪히는 함정들을 바로잡습니다.

**오개념 1: PR은 `git` 명령이다**



PR은 `git` 명령이 아니라 **GitHub가 제공하는 기능**입니다. `git`에는 `git pull-request` 같은 명령이 없습니다. `git`의 역할은 `push` 까지이고, "합쳐 달라는 제안과 리뷰"는 GitHub(또는 GitLab 등 호스팅 서비스)라는 협업 층이 그 위에 얹은 것입니다. 그래서 PR은 GitHub 웹이나 `gh` CLI로 만듭니다.

```
# 이런 git 명령은 없습니다 (오류)
# git pull-request

# 올바른 방법: GitHub의 gh CLI
gh pr create --base main --title "... " --body "... "
```

## 오개념 2: 반려(Request changes)를 받으면 PR을 새로 만들어야 한다

반려는 비난이 아니라 코드 품질을 함께 책임지는 **정상 과정**이고, PR을 새로 만들 필요가 전혀 없습니다. **같은 브랜치에 수정 커밋을 push**하면 그 PR이 자동으로 갱신됩니다. PR을 새로 만들면 리뷰 맥락(이전 코멘트·이력)이 끊겨 오히려 협업이 흩어집니다.

```
# 반려받은 그 브랜치로 돌아가 수정 후 push — PR이 갱신됨
git switch feature/3-contact-section
git add index.html
git commit -m "리뷰 반영: mailto 추가 (#3)"
git push
```

## 오개념 3: 머지가 끝나면 기능 브랜치를 그대로 뒀도 된다

PR을 머지하면 그 기능 브랜치는 **역할을 다한 것**입니다. 그대로 두면 브랜치 목록이 끝낸 기능들로 점점 지저분해집니다. 머지할 때 `--delete-branch`로 정리하거나, 머지 후 로컬·원격 브랜치를 삭제해 깨끗하게 유지하는 것이 좋습니다.

```
# 머지하면서 원격 브랜치 자동 삭제
gh pr merge 12 --squash --delete-branch

# 로컬에 남은 브랜치도 정리
git switch main
git branch -d feature/3-contact-section
```

## 그 밖의 주의점

- PR을 만들기 **전에 반드시 브랜치를 push**해야 합니다. 원격에 없는 브랜치는 GitHub가 비교할 수 없어 PR을 만들 수 없습니다.
- `Closes #3` · `Fixes #3` · `Resolves #3` 은 모두 머지 시 이슈를 자동으로 닫는 키워드입니다. 단, PR을 **머지해야** 닫히고, 그냥 닫으면(Close) 이슈는 열린 채로 남습니다.

- `gh auth login` 으로 한 번 로그인해 두지 않으면 `gh` 명령이 인증 오류를 냅니다. `gh auth status` 로 로그인 상태를 확인할 수 있습니다.

## 13.10 자주 묻는 질문(FAQ)

### Q. PR과 merge는 무엇이 다른가요?

`git merge (ch_06)`는 내 컴퓨터에서 브랜치를 합치는 `git` 명령이고, PR은 GitHub에서 "합쳐도 될지 동료에게 묻고 검토받는" 절차입니다. PR을 머지하면 결국 내부적으로 병합이 일어나지만, 그 앞에 리뷰·승인이라는 관문이 있다는 점이 핵심 차이입니다.

### Q. 이슈를 꼭 만들어야 PR을 만들 수 있나요?

아닙니다. 이슈 없이도 브랜치를 push하고 PR을 만들 수 있습니다. 다만 이슈로 할 일을 먼저 정의하면 "무엇을 왜 하는지"가 기록되고, `Closes #N` 으로 코드와 할 일이 자동으로 이어져 협업이 훨씬 깔끔해집니다.

### Q. squash·merge·rebase 머지 중 무엇을 골라야 하나요?

입문 단계에서는 **squash** 머지를 기본으로 권합니다. 기능 하나가 `main` 에 깔끔한 커밋 하나로 남아 이력을 읽기 쉽기 때문입니다. 팀마다 정책이 다르니, 실무에서는 합류한 팀의 규칙을 따르면 됩니다.

### Q. gh CLI 없이 웹으로만 해도 되나요?

됩니다. 이슈·PR·리뷰·머지는 모두 GitHub 웹에서 버튼으로 할 수 있습니다. `gh` 는 터미널에서 손을 떼지 않고 같은 일을 빠르게 하려는 도구일 뿐입니다. 둘 중 편한 쪽을 쓰되, 이 교재는 흐름을 명확히 보여 주려 `gh` 를 함께 씁니다.

### Q. 리뷰어가 없으면 내 PR을 어떻게 머지하나요?

혼자 연습할 때는 본인이 리뷰·머지를 검할 수 있습니다(개인 저장소는 보통 셀프 머지 허용). 팀에서는 브랜치 보호 규칙으로 "최소 1명 승인"을 강제하기도 합니다. 이 교재의 종합 프로젝트는 두 역할(dev-alice·dev-bob)을 번갈아 연기해 리뷰를 실습합니다.

### Q. PR을 머지하면 기능 브랜치의 커밋 메시지는 어떻게 되나요?

머지 방식에 따라 다릅니다. merge commit은 기능 커밋들을 그대로 보존하고, squash는 여러 메시지를 하나로 합치며(머지 화면에서 최종 메시지를 편집), rebase는 각 커밋을 `main` 끝에 직

선으로 엮습니다.

## 정리

이번 챕터에서는 **GitHub 협업 — PR·코드 리뷰·이슈** 개념을 깊이 익혔습니다.

- 이슈(Issue)로 할 일·버그를 기록하고 담당자·라벨로 추적하며, `#N` 과 `Closes #N` 으로 코드와 연결했습니다.
- 기능 브랜치를 push하고 풀 리퀘스트(PR)로 변경을 제안하는 **브랜치 push → PR → 리뷰 → 머지** 흐름을 끝까지 돌았습니다.
- 코드 리뷰의 라인 코멘트·Approve·Request changes를 구분하고, 반려 후 **같은 브랜치 수정 push**로 PR을 갱신했습니다.
- 머지 방식(merge-squash-rebase)의 차이를 개요로 익히고, `gh` CLI로 터미널에서 협업했습니다.
- PR 충돌로 머지가 막힐 때 **fetch → merge origin/main → 해결 → push**로 푸는 법을 연습했습니다.

이번 챕터에서 squash 머지로 "여러 커밋을 하나로 합치는" 정리를 잠깐 맛봤습니다. 다음 챕터(ch\_14)에서는 그 정리를 내 손으로 직접 하는 **git rebase와 황금룰**을 배웁니다. 'wip'·'오타' 커밋을 깔끔히 합치는 법과, "공유한 이력은 절대 rebase 하지 않는다"는 협업의 황금룰을 함께 익힙니다.



# 이력 정리: git rebase와 황금룰

## 이 챕터에서 배우는 것

- rebase가 커밋을 다시 쌓아 직선 이력을 만든다는 것을 merge와 비교해 설명할 수 있습니다.
- `git rebase main` 으로 기능 브랜치를 최신 `main` 위로 옮길 수 있습니다.
- `git rebase -i` 로 'wip'·'오타' 커밋을 squash해 의미 있는 커밋으로 정리할 수 있습니다.
- rebase 도중 충돌을 `git add` 후 `--continue` 로 이어 가거나 `--abort` 로 안전하게 되돌릴 수 있습니다.
- "공유한 이력은 rebase-force push 하지 않는다"는 황금룰과 그 이유를 설명할 수 있습니다.
- 공유 브랜치를 rebase한 뒤 push가 거부되는 상황을 진단하고 올바르게 대응할 수 있습니다.

이번 챕터에서 가르치는 개념은 **이력 정리 — rebase(git rebase)** 한 가지입니다. 선수 개념은 두 가지입니다. 하나는 ch\_13의 **GitHub 협업(PR·코드 리뷰)**입니다. rebase로 커밋을 정리하는 일은 보통 "PR을 머지하기 전에 지저분한 커밋을 깔끔하게 만드는" 맥락에서 일어납니다. 다른 하나는 ch\_08의 **되돌리기(reset·revert)**입니다. ch\_08에서 "공유한 이력은 reset이 아니라 revert로 되돌린다"고 배운 그 원칙이, 이번 챕터의 **황금룰**로 그대로 이어집니다. 이력을 바꾸는 도구는 강력한 만큼 위험하다는 감각을 이미 갖추셨다면, rebase를 안전하게 다룰 준비가 된 것입니다.

## 14.1 도입: 어수선한 커밋을 한 줄로 정리하기

기능 하나를 만드는 동안 우리의 커밋 이력은 생각보다 지저분합니다. 실제로 작업하다 보면 이런 커밋이 쌓입니다.

```
* 7f3a1b2  오타 수정
* 9c2e4d8  wip
* 4a8b0c1  버튼 색 다시 바꿈
* e1f5a93  로그인 버튼 추가
```

로그인 버튼 추가 하나면 충분한데, 중간에 wip ("작업 중"이라는 임시 커밋), 오타 수정, 버튼 색 다시 바꿈 같은 부스러기가 끼어 있습니다. 이 상태로 PR을 올리면 리뷰어가 의미 없는 커밋 네 개를 일일이 봐야 하고, 나중에 이력을 읽는 사람도 혼란스럽습니다. **네 커밋을 의미 있는 한 커밋으로 합쳐** 깔끔하게 정리하고 싶습니다.

또 다른 상황도 있습니다. 내가 기능 브랜치에서 며칠 작업하는 동안 main 이 한참 앞서갔습니다. 내 브랜치는 옛날 main 에서 갈라진 채라, PR을 올리면 "오래된 기준에서 만든 변경"처럼 보입니다. 내 커밋들을 **최신 main 위로 옮겨** 마치 방금 만든 것처럼 깔끔하게 엮고 싶습니다.

이 두 가지 — **여러 커밋을 정리(squash)하기**와 **최신 기준 위로 옮기기** — 를 해 주는 도구가 바로 **git rebase**입니다. rebase는 "커밋들을 다른 기준(base) 위로 다시(re) 쌓는다"는 뜻 그대로, 이력을 한 줄로 깔끔하게 다시 만듭니다.

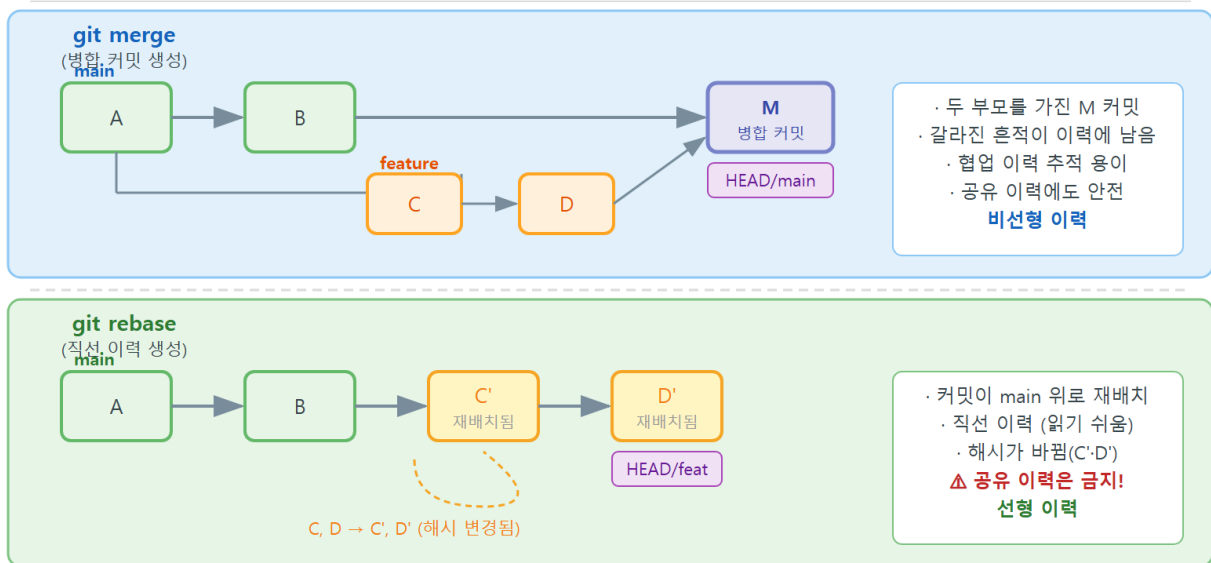
다만 rebase는 강력한 만큼 **위험한 도구**입니다. 이력을 "고쳐 쓰는" 일이라, 잘못 쓰면 동료의 작업과 어긋납니다. 그래서 이 챕터는 rebase 사용법과 함께, 반드시 지켜야 하는 **황금률**을 한 묶음으로 가르칩니다.

### 한눈에 보는 이력 정리 — rebase

git rebase 는 브랜치의 커밋들을 다른 기준 커밋 위로 "옮겨 다시 쌓아" 이력을 한 줄(직선)로 깔끔하게 만드는 명령입니다. merge 가 합친 흔적(병합 커밋)을 남기는 반면 rebase는 병합 커밋 없는 직선 이력을 만들고, 대화형 rebase(-i)로 커밋을 합치기(squash)·메시지 수정(reword)·삭제(drop)할 수 있습니다. 단, **이미 공유(push)한 이력은 rebase 하지 않는다**는 황금률을 지켜야 동료의 이력이 어긋나지 않습니다.

## 14.2 동작 원리: merge vs rebase(병합 커밋 vs 직선 이력)

## merge vs rebase — 이력 형태 비교



rebase를 이해하는 가장 빠른 길은 ch\_06의 **merge**와 나란히 놓고 비교하는 것입니다. 같은 상황 — main 이 앞서가고 내 feature 브랜치도 따로 진행된 상황 — 을 두 방식으로 합쳐 보겠습니다.

상황을 글자로 그리면 이렇습니다. C2 에서 갈라져, main 은 C3·C4 로, feature 는 F1·F2 로 각각 나아갔습니다.

```

      F1 --- F2      (feature)
      /
C1 --- C2 --- C3 --- C4  (main)

```

**merge 방식.** feature 에서 git merge main 을 하면, 두 갈래를 합치는 **병합 커밋 M** 이 새로 생깁니다. 두 갈래가 만난 흔적이 이력에 남습니다.

```

      F1 --- F2 --- M  (feature)
      /               /
C1 --- C2 --- C3 --- C4  (main)

```

**rebase 방식.** feature 에서 git rebase main 을 하면, F1·F2 를 떼어 내 최신 main (C4) 끝에 다시 쌓습니다. 병합 커밋 없이 한 줄로 곧게 이어집니다(F1'·F2' 는 내용은 같지만 부모가 바뀌어 새로 계산된 커밋입니다).

```

C1 --- C2 --- C3 --- C4 --- F1' --- F2'  (feature)

```

두 방식의 차이를 표로 정리합니다.

| 구분       | merge                 | rebase                   |
|----------|-----------------------|--------------------------|
| 이력 모양    | 갈라졌다 만난 그래프(병합 커밋 있음) | 한 줄 직선(병합 커밋 없음)         |
| 커밋 해시    | 기존 커밋 그대로 유지          | 옮겨진 커밋은 <b>해시가 새로 바뀜</b> |
| 통합 시점 정보 | 병합 커밋으로 "언제 합쳤는지" 남음  | 사라짐(직선이라 깔끔하지만 정보 손실)    |
| 안전성      | 공유 이력에도 안전            | 공유 이력엔 위험(황금률)           |

여기서 가장 중요한 한 가지는 "**옮겨진 커밋은 해시가 바뀐다**"입니다. rebase는 커밋을 그 자리에서 옮기는 게 아니라, 같은 변경을 새 부모 위에 **새 커밋으로 다시 만듭니다**. 커밋 해시는 내용과 부모로 계산되는 지문(ch\_04)이라, 부모가 달라지면 해시도 달라집니다. F1 이 F1' 이 되는 것입니다. 이 성질이 바로 14.6의 황금률을 낳는 핵심 원인입니다.

### 언제 merge, 언제 rebase?

정답은 "상황에 따라"입니다. **공동 작업한 main 통합처럼** 합친 시점이 중요하면 merge가 어울리고, **내 PR 브랜치를 깔끔하게 정리할 때는** rebase가 어울립니다. 입문 단계의 안전한 기준은 이렇습니다. "내가 아직 공유하지 않은(또는 나만 쓰는 PR) 커밋은 rebase로 정리해도 되지만, 남과 공유한 이력은 절대 rebase 하지 않는다."

## 14.3 git rebase main으로 기능 브랜치 최신화

먼저 가장 실용적인 쓰임 — **기능 브랜치를 최신 main 위로 옮기기** — 를 명령으로 익힙니다. PR을 올리기 전에 "내 변경을 최신 기준에서 다시 검증"하는 흔한 작업입니다.

**상황** — 내 feature 브랜치가 며칠 묵어 main 이 앞서갔다. 먼저 최신 main 을 받습니다.

```
# main을 최신으로 갱신
git switch main
git pull

# 내 기능 브랜치로 돌아가 최신 main 위로 다시 쌓는다
git switch feature/login-button
git rebase main
```



```
Switched to branch 'main'
Already up to date.
Switched to branch 'feature/login-button'
Successfully rebased and updated refs/heads/feature/login-button.
```

**진단 — 이력이 직선이 됐는지 확인한다.** `git log --oneline --graph` 로 모양을 봅니다.

```
git log --oneline --graph --all -5
```

```
* 2b9f4a1 (HEAD -> feature/login-button) 로그인 버튼 추가
* c4d7e88 (main, origin/main) 헤더 레이아웃 정리
* a13f902 푸터 저작권 표시
* 88e0b5c 초기 To-Do 앱 구조
```

병합 커밋 없이 로그인 버튼 추가 가 최신 main (헤더 레이아웃 정리) 바로 위에 직선으로 엮혔습니다. 이제 PR을 올리면 "최신 기준에서 만든 깔끔한 변경"으로 보입니다.

- `git rebase main` 은 "현재 브랜치( feature/login-button )의 커밋들을 main 위로 다시 쌓아라"는 뜻입니다. 옮길 대상은 현재 브랜치, 기준(base)은 인자로 준 main 입니다.
- 충돌이 없으면 Successfully rebased 로 한 번에 끝납니다. 충돌이 나면 14.5에서 다루는 `-continue / --abort` 로 처리합니다.
- 결과 이력이 `--graph` 에서 갈라짐 없이 한 줄이면 rebase가 잘 된 것입니다. merge 였다면 여기에 병합 커밋과 갈라진 선이 보였을 것입니다.

### rebase 전에는 작업트리를 깨끗이

rebase는 커밋을 다시 쌓는 작업이라, 커밋하지 않은 변경이 작업트리에 남아 있으면 시작을 거부합니다. `git status` 로 깨끗한지(working tree clean) 확인하고, 미완성 변경이 있으면 먼저 커밋하거나 `git stash (ch_09)`로 치워 두고 rebase하십시오.

## 14.4 worked example: rebase -i로 wip·오타 커밋 squash

이번에는 도입에서 본 지저분한 커밋들을 하나로 합치는(squash) 대화형 rebase를 끝까지 해보겠습니다. 무대는 PR을 올리기 직전, 아직 push하지 않은(또는 나만 쓰는) feature/login-button 브랜치입니다.

**상황 — 정리할 커밋이 네 개 쌓였다.** 먼저 이력을 확인합니다.

```
git log --oneline -4
```

```
7f3a1b2 (HEAD -> feature/login-button) 오타 수정
9c2e4d8 wip
4a8b0c1 버튼 색 다시 바꿈
e1f5a93 로그인 버튼 추가
```

**조작 — 최근 4개 커밋을 대화형으로 연다.** HEAD~4 는 "현재에서 4칸 뒤 커밋"이라, 그 뒤의 4개 커밋을 정리 대상으로 엽니다.

```
git rebase -i HEAD~4
```

그러면 편집기에 다음과 같은 **할 일 목록(todo list)**이 뜹니다. 위가 오래된 커밋, 아래가 최신입니다.

```
pick e1f5a93 로그인 버튼 추가
pick 4a8b0c1 버튼 색 다시 바꿈
pick 9c2e4d8 wip
pick 7f3a1b2 오타 수정

# Rebase ... onto ...
# Commands:
# p, pick    = 커밋을 그대로 사용
# r, reword  = 커밋을 사용하되 메시지를 수정
# s, squash  = 커밋을 사용하되 바로 위 커밋에 합침(메시지 합침)
# f, fixup   = squash와 같되 이 커밋 메시지는 버림
# d, drop    = 커밋을 제거
```

**편집 — 첫 커밋만 남기고 나머지를 합친다.** 맨 위 로그인 버튼 추가 를 기준으로, 아래 셋을 그 위에 합치도록 pick 을 squash (또는 메시지를 버릴 fixup )로 바꿉니다.

```
pick e1f5a93 로그인 버튼 추가
fixup 4a8b0c1 버튼 색 다시 바꿈
fixup 9c2e4d8 wip
fixup 7f3a1b2 오타 수정
```

저장하고 닫으면, Git이 네 커밋을 하나로 합쳐 다시 씁습니다. fixup 은 합치되 그 커밋들의 저분한 메시지(wip 등)를 버리므로, 최종 메시지는 로그인 버튼 추가 하나로 깔끔합니다.

```
Successfully rebased and updated refs/heads/feature/login-button.
```

**결과 검증 — 한 커밋으로 정리됐는지 확인한다.** 다음 명령의 실행 결과를 봅니다.

```
git log --oneline -3
```

```
b8d1c50 (HEAD -> feature/login-button) 로그인 버튼 추가
c4d7e88 (main) 헤더 레이아웃 정리
a13f902 푸터 저작권 표시
```

오타 수정 · wip · 버튼 색 다시 바꿈 이 사라지고, 로그인 버튼 추가 한 커밋으로 합쳐졌습니다. 이제 이 깔끔한 커밋으로 PR을 올리면 리뷰어가 변경을 한눈에 읽습니다.

- `git rebase -i HEAD~4` 의 `-i` 는 interactive(대화형)입니다. 어느 커밋을 어떻게 다룰지 todo 목록에서 직접 지정합니다.
- `squash` 는 합치면서 **메시지도 모아** 편집하게 하고, `fixup` 은 합치되 **메시지를 버립니다**. `wip` 같은 임시 메시지를 없애려면 `fixup` 이 편합니다.
- 정리 후 커밋 해시가 `7f3a1b2` 에서 `b8d1c50` 으로 바뀐 점에 주목하십시오. rebase는 새 커밋을 만드는 작업이라 해시가 달라집니다 — 이것이 14.6 황금률의 근거입니다.

**todo 목록의 순서가 곧 결과 순서입니다**

대화형 rebase의 todo 목록은 줄 순서를 바꾸면 **커밋 순서가 바뀌고**, 줄을 지우면 그 커밋이 drop 됩니다(매우 위험하니 주의). 처음에는 욕심내지 말고 "맨 위 pick 하나 + 나머지 fixup"으로 합치는 가장 단순한 패턴부터 익히는 것이 안전합니다.

## 14.5 rebase 충돌과 --continue / --abort

rebase는 커밋을 **한 개씩 차례로 다시 적용**합니다. 그 과정에서 옮기는 커밋이 새 기준과 같은 부분을 건드리면 충돌이 납니다. rebase 충돌은 merge 충돌(ch\_07)과 해결 방식이 거의 같지만, **마무리 명령이 다릅니다**. merge는 `git commit` 으로 끝내지만, rebase는 `git rebase --continue` 로 다음 커밋 적용을 이어 갑니다.

**상황 — rebase 도중 충돌이 났다.**

```
git rebase main
```

```
Auto-merging app.js
CONFLICT (content): Merge conflict in app.js
error: could not apply 9c2e4d8... 로그인 검증 추가
hint: Resolve all conflicts manually, mark them as resolved with
hint: "git add/rm <conflicted_files>", then run "git rebase --continue".
```

**진단 — 어디서 멈췄는지 status로 본다.**

```
git status
```

```
interactive rebase in progress; onto c4d7e88
Last command done (1 command done):
  pick 9c2e4d8 로그인 검증 추가
Unmerged paths:
  (use "git add <file>..." to mark resolution)
    both modified:   app.js
```

**해결 — 충돌 표시를 편집해 최종본을 만들고, add로 표시한 뒤 continue로 이어 간다.** 충돌 파일을 열면 ch\_07에서 본 익숙한 표시가 있습니다.

```
<<<<<<< HEAD
function login(id) { return id.length > 0; }
=====
function login(id, pw) { return id && pw; }
>>>>>>> 9c2e4d8 (로그인 검증 추가)
```

두 의도를 살려 최종본으로 고치고 표시를 지운 뒤, `add` 로 "해결됨"을 표시하고 `--continue` 로 다음 커밋 적용을 이어 갑니다.

```
git add app.js
git rebase --continue
```

```
[detached HEAD a7c93f1] 로그인 검증 추가
Successfully rebased and updated refs/heads/feature/login-button.
```

**되돌리고 싶다면 — --abort로 rebase 전으로 안전 복귀.** 충돌이 너무 복잡해 포기하고 싶으면, 언제든지 시작 전 상태로 되돌릴 수 있습니다.

```
git rebase --abort
```

```
# (출력 없음) rebase 시작 전 상태로 완전히 복구됨
```

- rebase 충돌 해결의 핵심 3단계는 **편집** → `git add` → `git rebase --continue` 입니다. merge처럼 `git commit` 을 하면 안 됩니다(rebase가 자동으로 커밋을 이어 만듭니다).
- 커밋이 여러 개면 충돌이 **여러 번** 날 수 있습니다. 한 번 해결하고 `--continue` 했는데 또 멈추면, 다음 커밋에서 또 충돌한 것이니 같은 절차를 반복합니다.
- 막히면 `git rebase --abort` 가 안전망입니다. rebase 시작 전 상태로 깨끗이 되돌리니, 부담 없이 다시 시도할 수 있습니다.

## 14.6 황금률: 공유한 이력은 rebase·force push 금물

이제 이 챕터에서 가장 중요한 규칙입니다. rebase는 강력하지만 한 가지 절대 원칙이 있습니다.

**황금률: 이미 공유(push)해 동료가 받아 간 이력은 rebase 하지 않는다.**

왜일까요? 14.2·14.4에서 보았듯 **rebase는 커밋의 해시를 바꿉니다**. 내가 push해 둔 F1 (해시 9c2e4d8)을 동료가 이미 받아 갔다고 합시다. 그 뒤 내가 그 브랜치를 rebase하면 F1은 F1' (해시 a7c93f1)이라는 **다른 커밋**이 됩니다. 이제 내 이력과 원격·동료의 이력에는 "같은 변경인데 해시가 다른 두 커밋"이 생겨 어긋납니다. 동료가 다음에 pull하면 Git은 둘을 별개로 보아 충돌·중복·혼란이 폭발합니다.

이 어긋남을 억지로 밀어붙이려면 **force push(`git push --force`)**가 필요한데, 이는 원격의 이력을 내 것으로 **강제로 덮어쓰는** 것이라, 그 사이 동료가 올린 커밋이 흔적도 없이 사라질 수 있습니다. 그래서 황금률은 둘을 한 묶음으로 말합니다 — **공유 이력은 rebase도, force push도 금물**.

| 상황                                  | 안전한가   | 이유                                        |
|-------------------------------------|--------|-------------------------------------------|
| 아직 push 안 한 로컬 커밋 rebase            | 안전     | 나만 가진 이력이라 해시가 바뀌어도 영향 없음                 |
| 나만 쓰는 PR 브랜치 rebase                 | 조건부 허용 | 협의를 경우 <code>--force-with-lease</code> 로만 |
| 동료와 공유한 <code>main</code> 등 rebase  | 금지     | 해시 변경으로 동료 이력과 어긋남                        |
| 공유 이력 <code>git push --force</code> | 금지     | 동료 커밋을 덮어 영구 소실 위험                        |

ch\_08을 떠올리면 같은 정신입니다. "공유한 이력을 되돌릴 땐 reset이 아니라 revert"였습니다. 여기서도 "공유한 이력은 rebase로 고쳐 쓰지 않는다"입니다. 둘 다 **"공유한 과거는 바꾸지 말고, 새 커밋으로 앞에 덧붙여 바로잡는다"**는 협업의 근본 원칙입니다.

**--force** 보다 **--force-with-lease**

피치 못해 나만 쓰는 PR 브랜치를 rebase한 뒤 push해야 한다면, **--force** 대신 **--force-with-lease** 를 쓰십시오. **--force-with-lease** 는 "내가 마지막으로 본 이후 원격이 바뀌지 않았을 때만" 덮어쓰므로, 그 사이 동료가 올린 커밋이 있으면 push를 거부해 사고를 막아 줍니다. 그래도 **공유 브랜치에는 절대 쓰지 않는다**는 원칙이 우선입니다.

```
# 위험: 무조건 덮어쓰 (동료 커밋 소실 가능)
# git push --force

# 그나마 안전: 그 사이 원격이 바뀌었으면 거부
git push --force-with-lease
```

## 14.7 흔한 문제·해결: 공유 브랜치 rebase 후 push 거부

황금률을 어겼을 때 실제로 무슨 일이 일어나는지, 그리고 어떻게 대응하는지 끝까지 봅시다.

**상황 — 이미 push한 브랜치를 rebase하고 그냥 push했다.**

```
# (이미 origin에 push해 둔 브랜치를) 정리하려고 rebase
git rebase -i HEAD~3
# ... squash로 커밋을 합침 (해시가 모두 바뀜) ...

git push
```

**문제 발생 — push가 거부된다.**

```
To https://github.com/myteam/todo-app.git
! [rejected]          feature/login-button -> feature/login-button (non-fast-forward)
error: failed to push some refs to 'https://github.com/myteam/todo-app.git'
hint: Updates were rejected because the tip of your current branch is behind
hint: its remote counterpart. Integrate the remote changes (e.g. 'git pull ...')
hint: before pushing again.
```

**진단 — 왜 거부됐는지 읽는다.** non-fast-forward 는 "원격의 이력과 내 이력이 같은 줄기로 이어지지 않는다"는 뜻입니다. rebase로 내 커밋 해시가 전부 바뀌어, 원격이 가진 옛 커밋들과 직선

으로 연결되지 않기 때문입니다. Git은 동료 작업이 사라질까 봐 push를 막은 것입니다.

## 해결 — 상황에 따라 갑니다.

먼저 이 브랜치를 나만 쓰는가를 판단합니다.

- 나만 쓰는 PR 브랜치이고, 동료와 협의했다면: 정리된 이력을 올리기 위해 `--force-with-lease` 로 푸시할 수 있습니다.

```
git push --force-with-lease
```

```
To https://github.com/myteam/todo-app.git
+ 7f3a1b2...b8d1c50 feature/login-button -> feature/login-button (forced update)
```

- 동료와 공유하는 브랜치(예: `main` 이나 함께 쓰는 기능 브랜치)라면: force push는 금지입니다. 가장 안전한 길은 **rebase**를 취소하고(아직 force push 전이라면 로컬만 영향), 공유 이력은 건드리지 않은 채 새 커밋으로 정리하는 것입니다. 이미 force push로 사고를 냈다면, ch\_08에서 배운 `git reflog` 로 덮이기 전 커밋을 찾아 복구합니다.

```
# 사고 복구: reflog로 force push 직전 상태를 찾아 되돌리기
git reflog
# ... 'before rebase' 시점의 해시(예: 7f3a1b2)를 확인 ...
git reset --hard 7f3a1b2
```

- `non-fast-forward` 거부는 ch\_11에서도 만났지만, 그때는 "pull 후 push"로 풀었습니다. rebase로 인한 거부는 **해시가 통째로 바뀌어** 생긴 것이라, pull로 합치면 오히려 옛 커밋과 새 커밋이 섞여 더 꼬입니다.
- 그래서 판단 기준은 단 하나, "**이 브랜치가 공유 이력인가**"입니다. 나만 쓰면 `--force-with-lease`, 공유면 금지하고 reflog로 복구합니다.
- 가장 좋은 해결은 사후 대응이 아니라 **예방**입니다. push하기 전에 정리(rebase)를 끝내는 습관을 들이면, 공유 이력을 고쳐 쓸 일 자체가 없습니다.

## 14.8 즉시 연습(2종 1세트): 커밋 정리 채우기

지저분한 커밋 세 개를 하나로 정리하는 대화형 rebase를 직접 해 보는 연습입니다. 모범 흐름, 작성형 빈칸, 자가체크의 2종 1세트입니다.

## 모범 흐름

```
# 1) 정리할 커밋 확인 (아직 push 전, 나만 쓰는 브랜치)
git log --oneline -3
# 3a1b2c4 오타
# 9d8e7f6 wip
# 1c2b3a4 검색 기능 추가

# 2) 최근 3개를 대화형 rebase로 연다
git rebase -i HEAD~3

# 3) 편집기에서 맨 위는 pick, 나머지는 fixup으로 변경 후 저장
# pick 1c2b3a4 검색 기능 추가
# fixup 9d8e7f6 wip
# fixup 3a1b2c4 오타

# 4) 결과 확인 — 한 커밋으로 합쳐졌는지
git log --oneline -2
```

```
Successfully rebased and updated refs/heads/feature/search.
e5f6a7b 검색 기능 추가
c4d7e88 헤더 레이아웃 정리
```

## 직접 작성: 커밋 정리 (빈칸 채우기)

아래 빈칸(\_\_\_\_)을 채워 커밋 정리 흐름을 완성해 보십시오. 모범 흐름을 덮어 두고 도전하는 것이 효과적입니다.



```
# 1) 정리할 커밋 4개 확인
git log --oneline -4

# 2) 최근 4개 커밋을 대화형으로 연다
git rebase ____ HEAD~____

# 3) 편집기 todo 목록: 맨 위만 pick, 나머지 셋은 메시지를 버리며 합치기
# pick ... 연락처 폼 추가
# ____ ... 버튼 위치 조정
# ____ ... wip
# ____ ... 오타

# 4) rebase 도중 충돌이 났다면: 편집 → add → 이어가기
git add styles.css
git rebase _____

# 5) 포기하고 처음으로 되돌리려면
git rebase _____

# 6) 결과: 한 커밋으로 정리됐는지 확인
git log --oneline -2
```

**확장 과제:** 위에서 정리한 브랜치가 이미 **push된 상태**라고 가정해 보십시오. 그냥 `git push` 하면 어떤 메시지로 거부될지 적고, 이 브랜치가 (a) 나만 쓰는 PR 브랜치일 때와 (b) 동료와 공유하는 브랜치일 때 각각 어떻게 대응해야 하는지 한 줄씩 써 보십시오.

## 자가체크 체크리스트

스스로 점검해 보십시오.

- ☐ `git rebase -i HEAD~N` 으로 정리 대상 커밋 N개를 올바르게 열었는가
- ☐ todo 목록에서 맨 위는 `pick`, 합칠 커밋은 `squash / fixup` 으로 바꿨는가
- ☐ 충돌이 나면 `git commit` 이 아니라 `git add` 후 `git rebase --continue` 로 이어 갔는가
- ☐ 막혔을 때 `git rebase --abort` 로 안전하게 되돌릴 수 있음을 확인했는가
- ☐ 정리한 브랜치가 **공유 이력이 아닌지**(아직 `push` 전이거나 나만 쓰는지) 확인했는가
- ☐ 정리 후 `git log --oneline` 이 의미 있는 커밋으로 깔끔해졌는가

## 14.9 오개념 교정: 공유 이력 금지·rebase가 만능 아님·충돌 후 continue

rebase에서 입문자가 자주 부딪히는 함정들을 바로잡습니다.

## 오개념 1: 공유한(push해 동료가 받은) 이력도 rebase로 정리하면 된다

공유한 이력을 rebase하면 같은 변경의 **해시가 바뀌어** 동료 이력과 어긋나고, 이를 밀어붙이려면 force push가 필요해 동료 커밋이 사라질 수 있습니다. 이것이 "공개된 이력은 rebase 금지"가 황금률인 이유입니다. 공유 이력을 바로잡아야 하면 rebase가 아니라 **새 커밋**(`revert` 등, **ch\_08**)으로 앞에 덧붙입니다.

```
# 공유 브랜치에 이런 행동은 금지
# git rebase -i HEAD~3 (공유 이력이면 X)
# git push --force (동료 커밋 소실 위험 X)
```

## 오개념 2: rebase는 merge보다 무조건 좋다

rebase는 이력을 직선으로 깔끔하게 만들지만, **언제 합쳐졌는지 같은 통합 정보를 지웁니다**. 또 공유 이력에서는 위험합니다. rebase는 "내 PR 브랜치 정리"처럼 적절한 자리에서 쓰는 도구이지, 모든 merge를 대체하는 만능이 아닙니다. 상황에 맞게 골라야 합니다.

## 오개념 3: rebase 중 충돌이 나면 git commit으로 끝낸다

merge 충돌은 `git commit` 으로 마무리하지만, **rebase 충돌은 다릅니다**. 충돌을 편집하고 `git add` 로 표시한 뒤 `git rebase --continue` 로 다음 커밋 적용을 이어 가야 합니다. `git commit` 을 하면 rebase 흐름이 꼬입니다. 포기하려면 `git rebase --abort` 입니다.

```
# rebase 충돌 마무리 — commit이 아니라 continue
git add app.js
git rebase --continue
```

## 그 밖의 주의점

- rebase는 작업트리가 깨끗해야 시작됩니다. 미커밋 변경이 있으면 먼저 커밋하거나 `git stash` (ch\_09)로 치우십시오.
- `HEAD~N` 의 N을 잘못 세면 의도보다 많은(또는 적은) 커밋을 건드립니다. `git log --oneline` 으로 정확한 개수를 먼저 확인하십시오.
- todo 목록에서 줄을 **지우면** 그 커밋이 `drop` 되어 사라집니다. 합칠 때는 줄을 지우지 말고 `pick` 을 `fixup` / `squash` 로 **바꾸기만** 하십시오.

# 14.10 단원 미니 프로젝트: PR 하나를 끝까지(2종 1세트)

이 단원(u6: 팀처럼 협업하기)에서 배운 **PR·코드 리뷰·이슈(ch\_13)**와 **rebase·황금률(ch\_14)**을 모두 묶어, 협업 한 바퀴를 이슈에서 머지까지 끝까지 돌아 보는 미니 프로젝트입니다. 무대는 작은 To-Do 웹앱 저장소이고, "연락처 섹션 추가"라는 작업 하나를 끝냅니다.

### 미니 프로젝트 목표

이슈로 작업을 정의하고, 기능 브랜치에서 구현하며 일부러 'wip'·'오타' 커밋을 쌓은 뒤, PR을 만들어 리뷰 반력을 받고 같은 브랜치 수정 push로 갱신하고, 머지 전에 `rebase -i`로 커밋을 하나로 정리해 머지합니다. **달힌 이슈 1개 + 머지된 깔끔한 PR 1개**가 산출물입니다. 적용 개념: c13(PR·리뷰·이슈), c14(rebase·황금률).

### 마일스톤(중분)

| 마일스톤 | 할 일                                                                                                                | 적용 개념           |
|------|--------------------------------------------------------------------------------------------------------------------|-----------------|
| M1   | <code>gh issue create</code> 로 이슈 작성·할당 → 기능 브랜치에서 구현(여러 커밋) → push                                                | c13(이슈·push)    |
| M2   | <code>gh pr create</code> 로 PR 생성 → 리뷰어가 Request changes 반력 → 같은 브랜치 수정 push로 PR 갱신                                | c13(PR·리뷰·반력)   |
| M3   | <code>git rebase -i</code> 로 wip·오타 커밋을 squash해 정리(push 전, 나만 쓰는 PR 브랜치) → 머지( <code>closes #N</code> 으로 이슈 자동 종료) | c14(rebase·황금률) |

### 모범 — 협업 한 바퀴

```
# === M1: 이슈 → 브랜치 → 구현(지저분한 커밋 포함) → push ===
gh issue create --title "연락처 섹션 추가" --assignee @me --label enhancement
git switch main
git pull
git switch -c feature/3-contact-section
git add index.html
git commit -m "연락처 섹션 추가 (#3)"
git commit -am "wip"
git commit -am "오타 수정"
git push -u origin feature/3-contact-section

# === M2: PR 생성 → 반려 → 같은 브랜치 수정 push로 갱신 ===
gh pr create --base main --title "연락처 섹션 추가" --body "Closes #3"
# 리뷰어로 전환: 변경 요청
gh pr review --request-changes --body "mailto 링크가 빠졌습니다."
# 작성자로 전환: 같은 브랜치에서 수정 후 push (PR 자동 갱신)
git add index.html
git commit -m "리뷰 반영: mailto 링크 추가 (#3)"
git push

# === M3: 머지 전 rebase -i로 커밋 정리 → push → 승인 → 머지 ===
git rebase -i HEAD~4 # 맨 위 pick, 나머지 fixup으로 한 커밋 정리
git push --force-with-lease # 나만 쓰는 PR 브랜치라 협의 하에 허용
gh pr review --approve --body "LGTM"
gh pr merge 12 --squash --delete-branch
```

```
https://github.com/myteam/todo-app/issues/3
Switched to a new branch 'feature/3-contact-section'
https://github.com/myteam/todo-app/pull/12
Successfully rebased and updated refs/heads/feature/3-contact-section.
Merged pull request #12 (연락처 섹션 추가)
Deleted branch feature/3-contact-section
```

- M1에서 일부러 wip · 오타 커밋을 쌓아, M3에서 rebase -i 로 정리할 거리를 만들었습니다. 실제 작업이 그렇게 지저분하게 진행되기 때문입니다.
- M2의 핵심은 반려 후 같은 브랜치 수정 push로 PR 갱신(c13)입니다. 새 PR을 만들지 않습니다.
- M3의 rebase -i 는 push 전 또는 나만 쓰는 PR 브랜치에서만 합니다(c14 황금률). 정리 후 push가 거부되면 나만 쓰는 브랜치임을 확인하고 --force-with-lease 로만 올립니다. 공유 이력이면 절대 force push하지 않습니다.

## 직접 작성: 내 PR 한 바퀴 (빈 스타터)

아래 빈칸을 채워, "README에 설치 안내 추가"라는 작업으로 협업 한 바퀴를 끝까지 돌아 보십시오. 모범을 덮어 두고 도전하는 것이 효과적입니다.

```
# M1: 이슈 → 브랜치 → 구현(여러 커밋) → push
gh issue _____ --title "README에 설치 안내 추가" --assignee _____ --label documentation
git switch main
git _____
git switch _____ feature/8-install-guide
git add README.md
git commit -m "설치 안내 추가 (_____)"
git commit -am "wip"
git push _____ origin feature/8-install-guide

# M2: PR 생성 → 반려 → 같은 브랜치 수정 push
gh pr _____ --base main --title "설치 안내" --body "_____ #8"
gh pr review --_____ --body "명령 예시가 빠졌습니다."
git add README.md
git commit -m "리뷰 반영: 명령 예시 추가 (#8)"
git push

# M3: rebase로 커밋 정리 → 승인 → squash 머지
git rebase _____ HEAD~3          # 맨 위 pick, 나머지 fixup
git push --_____          # 나만 쓰는 PR 브랜치라 협의 하에
gh pr review --_____ --body "LGTM"
gh pr merge --_____ --_____
```

## 자가체크 체크리스트

스스로 점검해 보십시오.

- [ ] 이슈를 만들고 담당자·라벨을 지정했는가(c13)
- [ ] 기능 브랜치를 최신 `main` 에서 파고 push했는가(c13)
- [ ] PR 본문에 `closes #N` 을 넣어 머지 시 이슈가 자동으로 닫히게 했는가(c13)
- [ ] 반려를 받았을 때 **새 PR이 아니라 같은 브랜치 수정 push**로 갱신했는가(c13)
- [ ] 머지 전 `rebase -i` 로 `wip` ·오타 커밋을 의미 단위로 정리했는가(c14)
- [ ] rebase한 브랜치가 **공유 이력이 아님**을 확인하고, 필요 시 `--force-with-lease` 만 썼는가(c14 황금률)
- [ ] 최종적으로 PR이 머지되고 이슈가 자동으로 닫혔으며 브랜치가 정리됐는가

## 14.11 자주 묻는 질문(FAQ)

### Q. rebase와 merge 중 결국 무엇을 써야 하나요?

내가 아직 공유하지 않은(또는 나만 쓰는 PR) 커밋을 **정리**할 때는 rebase, 공동 작업한 갈래를 **통합**할 때는 merge가 안전한 기본입니다. 핵심은 "공유한 이력은 rebase로 고쳐 쓰지 않는다"는

황금률을 지키는 것입니다.

#### Q. squash 와 fixup 은 무엇이 다른가요?

둘 다 커밋을 바로 위 커밋에 합칩니다. squash 는 합치면서 두 메시지를 모아 편집하게 하고, fixup 은 합쳐지는 커밋의 메시지를 버립니다. wip 같은 임시 메시지를 없앨 때는 fixup 이 편합니다.

#### Q. rebase하다 완전히 꼬였습니다. 되돌릴 수 있나요?

네. 진행 중이면 git rebase --abort 로 시작 전 상태로 돌아갑니다. 이미 rebase를 끝냈는데 망했다면 git reflog (ch\_08)로 rebase 직전 커밋 해시를 찾아 git reset --hard <해시> 로 복구할 수 있습니다.

#### Q. --force-with-lease 와 --force 는 무엇이 다른가요?

--force 는 원격을 무조건 덮어써 그 사이 동료가 올린 커밋을 날릴 수 있습니다. --force-with-lease 는 "내가 마지막으로 본 이후 원격이 안 바뀌었을 때만" 덮어쓰므로 그런 사고를 막아 줍니다. 그래도 공유 브랜치에는 둘 다 쓰지 않는 것이 원칙입니다.

#### Q. PR을 squash 머지하면 굳이 rebase로 정리할 필요가 없지 않나요?

squash 머지는 main 에 합칠 때 한 커밋으로 합쳐 줍니다. 그래도 PR 리뷰 단계에서는 지저분한 커밋이 그대로 보이므로, 리뷰어가 변경을 읽기 쉽게 미리 rebase -i 로 정리해 두면 협업이 매끄럽습니다. 팀 정책에 따라 둘 중 하나만 써도 됩니다.

#### Q. 황금률은 main 에만 적용되나요?

아닙니다. 동료가 받아 간 모든 브랜치에 적용됩니다. main 이든 함께 쓰는 기능 브랜치든, 누군가 그 이력을 pull해 갔다면 rebase-force push로 고쳐 쓰면 안 됩니다. "나만 가진 이력이냐"가 판단 기준입니다.

## 정리

이번 챕터에서는 **이력 정리 — rebase** 개념을 깊이 익혔습니다.

- rebase가 커밋을 다른 기준 위로 다시 쌓아 **직선 이력**을 만든다는 것을 merge(병합 커밋)와 비교해 이해했습니다.

- `git rebase main` 으로 기능 브랜치를 최신화하고, `git rebase -i` 로 wip · 오타 커밋을 squash·fixup으로 정리했습니다.
- rebase 충돌은 편집 → `git add` → `git rebase --continue` 로 이어 가고, 막히면 `--abort` 로 되돌렸습니다.
- "공유한 이력은 **rebase·force push** 하지 않는다"는 황금률과, 해시가 바뀐다는 그 근거를 익혔습니다.
- 단원 미니 프로젝트로 이슈에서 머지까지 협업 한 바퀴(PR·리뷰·반려·rebase 정리)를 끝까지 돌았습니다.

이로써 개념 단원(u1~u6)을 모두 마쳤습니다. 설치·커밋부터 브랜치·병합·충돌·되돌리기·원격·태그·PR·rebase까지, 협업에 필요한 도구를 갖췄습니다. 다음 챕터(ch\_15)부터는 이 모든 개념을 하나로 묶는 **종합 프로젝트 — 팀 협업 시뮬레이션**이 시작됩니다. 두 역할(dev-alice·dev-bob)을 번갈아 연기하며, 지금까지 배운 명령이 실제 협업의 사고 복구로 어떻게 이어지는지 직접 만들고 풀어 봅니다.





# 종합 프로젝트: 팀 협업 시뮬레이션 — 분석 (요구사항·범위)

## 이 챕터에서 배우는 것

- 종합 프로젝트의 드라이빙 퀘스천과 POV(관점 진술)를 요구사항으로 구체화할 수 있습니다.
- 협업·충돌·사고 복구·릴리스를 기능 요구사항(FR) 9개로 명세할 수 있습니다.
- 결정론적 재현·황금률·진단 단계 같은 품질 조건을 비기능 요구사항(NFR) 5개로 정리할 수 있습니다.
- 협업 사용자 스토리와 유스케이스(UC1~UC3)로 사고 상황의 흐름을 표현할 수 있습니다.
- 범위(In/Out)를 확정해 프로젝트가 한없이 커지는 것을 통제할 수 있습니다.
- 각 요구사항이 c01~c14의 어느 단원 개념으로 구현되는지 추적성 표로 매핑할 수 있습니다.

이번 챕터부터는 **종합 프로젝트**입니다. ch\_01부터 ch\_14까지 14개 개념(c01~c14)을 하나씩 배워 왔다면, 이제는 그 모두를 한자리에 모아 **"여럿이 함께 개발하는" 협업 시나리오**를 직접 만들 차례입니다. 종합 프로젝트는 새로운 명령을 가르치지 않습니다. 대신 지금까지 배운 명령들이 실제 팀 협업에서 **언제·왜·어떻게** 쓰이는지를, 일부러 만든 사고 상황을 통해 통합 적용합니다.

이 챕터(ch\_15)는 그 첫 단계인 **분석(analysis)**입니다. 무엇을 만들지 막연한 아이디어로 두지 않고, **요구사항·범위로 분명하게 못 박는** 일을 합니다. 설계(ch\_16)·구현(ch\_17)·발표(ch\_18)는 모두 이 분석 결과를 토대로 진행됩니다. 코드를 한 줄도 짜지 않지만, 협업 프로젝트의 성패는 여기서 절반이 갈립니다.

## 15.1 드라이빙 퀘스천과 POV 다시 보기

종합 프로젝트는 하나의 큰 질문에서 출발합니다. 이 질문을 **드라이빙 퀘스천(driving question)**이라고 부르며, 프로젝트 전체가 이 질문에 답하기 위해 움직입니다.

### 드라이빙 퀘스천

여러 기여자가 하나의 GitHub 저장소에서 동시에 작업할 때, 브랜치·풀 리퀘스트·코드 리뷰로 협업하면서 **동시 수정 충돌·잘못된 머지·force push 사고·핫픽스·리뷰 반려** 같은 다양한 상황의 문제를 어떻게 안전하게 진단하고 해결할까?

이 질문의 핵심은 "협업을 매끄럽게 하는 법"이 아니라 **"사고가 났을 때 침착하게 복구하는 법"**입니다. 실제 팀 협업에서 충돌과 사고는 피할 수 없습니다. 실력은 사고를 안 내는 데 있지 않고, 사고가 났을 때 `git status`·`git log`·`git reflog`로 침착하게 진단해 복구하는 데 있습니다.

드라이빙 퀘스트를 누구의, 어떤 필요에서 출발했는지로 풀어 쓴 것이 **POV(Point of View, 관점 진술)**입니다. POV는 **"사용자 + 필요 + 통찰"**의 세 조각으로 이루어집니다.

| POV 조각          | 내용                                                                                              |
|-----------------|-------------------------------------------------------------------------------------------------|
| 사용자<br>(user)   | 버전관리는 익혔지만 '여럿이 함께' 개발해 본 적은 없는 입문 개발자                                                          |
| 필요<br>(need)    | 하나의 저장소를 여러 기여자가 동시에 건드릴 때 생기는 충돌·사고를 두려움 없이 진단·복구하며, 브랜치·PR·리뷰로 안전하게 통합하는 협업 워크플로우를 몸에 익히는 것   |
| 통찰<br>(insight) | 협업 실력은 '충돌을 안 내는 것'이 아니라 '충돌·사고가 났을 때 침착하게 진단해 복구하는 것'이며, 이는 두 역할을 오가며 사고 상황을 직접 만들고 풀어 봐야 체득된다 |

이 통찰이 프로젝트의 형태를 결정합니다. 우리는 **GitHub To-Do 정적 웹앱**을 소재로, 혼자서 **dev-alice**와 **dev-bob** 두 기여자를 번갈아 연기합니다. 두 사람이 같은 파일을 동시에 고치고, 잘못 머지하고, force push로 사고를 내고, 그것을 복구하는 과정을 결정론적으로(매번 똑같이) 재현하는 것이 목표입니다.

## 15.2 기능 요구사항(FR): 협업·충돌·사고 복구·릴리스

**기능 요구사항(Functional Requirement, FR)**은 "시스템이 무엇을 할 수 있어야 하는가"를 한 문장으로 적은 명세입니다. 드라이빙 퀘스트를 실행 가능한 작업 단위로 쪼개면 아래 9개의 FR이 나옵니다. 각 FR에는 **그 기능이 어느 단원의 어느 개념으로 구현되는지**를 함께 적어, 커리큘럼과 프로젝트가 어긋나지 않게 합니다.

| ID  | 기능 요구사항(무엇을 한다)                           | 적용 단<br>원 | 적용 개념       |
|-----|-------------------------------------------|-----------|-------------|
| FR1 | 공유 저장소를 GitHub에 두고 두 기여자가 각자 clone 해 작업한다 | u1·u5     | c02·c03·c10 |
| FR2 | 기능마다 별도 브랜치를 파고 push해 PR로 제안한다            | u2·u5·u6  | c05·c11·c13 |
| FR3 | PR을 코드 리뷰(승인/반려)하고 반려 시 재작업해 갱신한다         | u6        | c13         |
| FR4 | 두 기여자가 같은 파일을 동시 수정해 생긴 충돌을 진단·해결한다       | u3        | c06·c07     |
| FR5 | 잘못 머지된 변경을 안전하게 되돌린다(공유 이력)               | u4        | c08         |
| FR6 | force push로 원격 이력이 덮인 사고를 reflog로 복구한다    | u4·u5     | c08·c11     |
| FR7 | 긴급 버그를 핫픽스 브랜치로 빠르게 수정·배포한다               | u3·u4·u5  | c06·c09·c11 |
| FR8 | 머지 전 PR 커밋을 rebase로 정리한다                  | u6        | c14         |
| FR9 | 안정 지점에 태그를 달고 GitHub 릴리스를 만든다             | u5        | c12         |

이 9개의 FR은 단순히 "기능 목록"이 아니라, **협업에서 실제로 벌어지는 다섯 가지 사고 상황**을 정면으로 겨냥합니다. 협업 사고를 요구사항으로 분명히 못 박아 두면, 구현 단계(ch\_17)에서 "이 사고를 일부러 재현하고 복구한다"는 목표가 또렷해집니다.

### FR이 겨냥하는 5대 협업 사고 상황

- **동시 수정 충돌(FR4):** alice와 bob이 `app.js`의 같은 함수를 각자 고친 뒤 둘 다 머지하려 할 때, 나중에 머지하는 쪽에서 충돌이 납니다. 한쪽을 통째로 버리지 않고 **양쪽 의도를 모두 살려** 해결해야 합니다.
- **잘못된 머지(FR5):** 리뷰가 끝나지 않은 PR이나 깨진 변경을 실수로 `main`에 머지했습니다. 이미 push되어 **공유된** 이력이므로 `reset`이 아니라 `git revert`로 되돌립니다.
- **force push 사고(FR6):** bob이 자기 브랜치를 rebase한 뒤 `--force`로 밀어, alice가 올려둔 커밋이 원격에서 덮였습니다. `git reflog`로 잃은 커밋을 되찾아 복구합니다.
- **핫픽스(FR7):** 릴리스된 버전에서 긴급 버그가 발견됐습니다. 정규 기능 개발을 멈추지 않고 `hotfix/*` 브랜치로 빠르게 고쳐 배포합니다.

- **리뷰 반려(FR3):** 리뷰어가 Request changes로 PR을 반려했습니다. 같은 브랜치에 수정 커밋을 push해 PR을 갱신하고 다시 리뷰를 받습니다.

각 사고는 결국 "문제 발생 → 진단 → 해결 → 검증"의 같은 골격을 따릅니다. 이 골격을 모든 사고에 일관되게 적용하는 것이 이 종합 프로젝트의 학습 핵심입니다.

## 15.3 비기능 요구사항(NFR): 결정론적 재현·황금률·진단 단계

**비기능 요구사항(Non-Functional Requirement, NFR)**은 "무엇을 한다"가 아니라 "어떻게/얼마나 잘 해야 하는가"를 정하는 품질·제약 조건입니다. 기능이 같아도 NFR을 지키느냐에 따라 협업의 안전성이 갈립니다.

| ID   | 비기능 요구사항(품질·제약)                                          | 적용 개념   |
|------|----------------------------------------------------------|---------|
| NFR1 | 모든 사고 시나리오는 혼자 두 역할로 결정론적으로 재현 가능해야 한다(두 로컬 클론)          | c10     |
| NFR2 | 공유한(push한) 이력은 reset/rebase로 덮지 않고 revert로 되돌린다(황금률)     | c08·c14 |
| NFR3 | 각 사고는 '문제 발생 → status/log/reflog 진단 → 해결 → 검증'의 단계로 기록한다 | c04·c08 |
| NFR4 | main은 항상 동작하는 상태를 유지한다(직접 push 대신 PR 경유)                 | c13     |
| NFR5 | 커밋 메시지와 PR 설명이 '무엇을 왜'를 분명히 드러낸다                         | c03     |

이 다섯 NFR 중 **NFR2(황금률)**가 가장 중요합니다. ch\_08과 ch\_14에서 반복해 배운 "**공유한 이력은 rebase·force push 하지 않는다**"는 원칙이, 종합 프로젝트의 모든 사고 복구를 관통하는 안전 규칙입니다. 이미 push되어 동료가 받은 커밋을 `reset` 이나 `rebase` 로 덮으면 같은 변경의 해시가 바뀌어 동료 이력과 어긋나고, 이것이 바로 force push 사고의 씨앗입니다.

**NFR2를 지키면 무슨 일이 일어나는가**

bob이 이미 push한 `feature/x` 브랜치를 `rebase` 로 정리한 뒤 그냥 push하면 거부됩니다(non-fast-forward). 여기서 `git push --force` 로 밀어버리면, 그 사이 alice가 같은 브랜치에 올린 커밋이 원격에서 **사라집니다**. 이것이 FR6의 force push 사고입니다. 황금률은 이 사고를 **예방**하는 규칙이고, `reflog` (FR6)는 사고가 났을 때의 **복구** 수단입니다. 예방이 복구보다 언제나 낫습니다.

NFR3은 사고를 다루는 **일관된 절차**를 강제합니다. 어떤 사고든 추측으로 명령을 던지지 않고, 반드시 `git status` (지금 어떤 상태인가) → `git log` (이력이 어떻게 됐나) → 필요하면 `git reflog` (잃은 것처럼 보이는 커밋은 어디 있나)로 **먼저 진단**한 뒤 해결합니다. 진단 없이 손이 먼저 나가는 습관이 작은 실수를 큰 사고로 키웁니다.

## 15.4 협업 사용자 스토리와 유스케이스

요구사항을 사람의 시선으로 다시 쓰면 **사용자 스토리(user story)**가 됩니다. "(역할)로서, (목적)을 위해, (무엇)을 하고 싶다"의 형식으로, 누가 왜 이 기능을 원하는지를 드러냅니다.

| #   | 사용자 스토리                                           |
|-----|---------------------------------------------------|
| US1 | 기여자로서, 내 기능을 별도 브랜치에서 만들고 PR로 제안해 동료 리뷰를 받고 싶다    |
| US2 | 리뷰어로서, 변경을 라인 단위로 검토하고 미흡하면 반려해 품질을 지키고 싶다        |
| US3 | 기여자로서, 동료와 같은 파일을 고쳐 충돌이 나도 침착하게 진단·해결하고 싶다       |
| US4 | 메인테이너로서, 잘못 머지되거나 force push로 덮인 사고를 안전하게 복구하고 싶다 |
| US5 | 팀으로서, 긴급 버그를 핫픽스로 빠르게 배포하고 안정 지점을 릴리스로 남기고 싶다     |

사용자 스토리가 "누가 무엇을 원하는가"라면, **유스케이스(use case)**는 그 목적을 이루기까지의 **단계별 흐름**입니다. 이 프로젝트의 협업은 세 개의 유스케이스로 압축됩니다.

### UC1 기능 추가 — 정상 협업 한 바퀴

1. 기여자가 GitHub에 **이슈**를 만들어 할 일을 정의·할당한다(c13).
2. 이슈 번호에 맞춰 `feature/*` **브랜치**를 판다(c05).
3. 기능을 구현하고 의미 있는 커밋을 쌓아(c03) 원격에 **push**한다(c11).
4. **PR**을 생성해 변경을 제안한다(c13).

5. 상대 역할이 **코드 리뷰**로 승인 또는 반려한다(c13).
6. 승인되면 **머지**하고, `closes #N` 으로 이슈가 자동 종료된다.

### UC2 동시 수정 충돌 — 같은 파일을 둘이 고칠 때

1. dev-alice와 dev-bob이 각자 클론에서 `app.js` 의 **같은 부분**을 다르게 수정한다(c05).
2. alice의 PR이 **먼저 머지**되어 `origin/main` 이 앞선다(c11~c13).
3. bob이 자기 브랜치를 최신화하려 `pull` (또는 머지)하는 순간 **충돌**이 난다(c06~c07).
4. `git status` 로 `Unmerged paths` 를 확인하고(c04~c07) 충돌 표시를 읽는다.
5. 양쪽 의도를 모두 살려 **최종본을 편집**한 뒤 `add · commit` 으로 해결한다(c07).
6. 실행해서 두 변경이 모두 동작하는지 **검증**한다.

### UC3 사고 복구 — 잘못된 머지·force push·핫픽스

1. **잘못된 머지**: 깨진 변경이 `main` 에 머지됐다 → 공유 이력이므로 `git revert` 로 되돌리는 새 커밋을 만든다(c08).
2. **force push 사고**: bob의 `--force` 로 alice의 커밋이 덮였다 → alice가 `git reflog` 로 잃은 커밋을 찾아 복구한다(c08~c11).
3. **핫픽스**: 릴리스 버전에 긴급 버그 → `main` 에서 `hotfix/*` 를 파 빠르게 고치고(필요 시 진행 중 작업은 `stash` 로 치움, c09) PR·태그로 마무리한다(c12).

세 유스케이스는 각각 ch\_17의 마일스톤 M2(UC1)·M3(UC2)·M4·M5(UC3)로 그대로 이어집니다. 지금 흐름을 분명히 그려 두면 구현 단계가 훨씬 수월해집니다.

## 15.5 범위 In/Out 확정(Actions는 다음 학습)

요구사항을 다 적고 나면 욕심이 생깁니다. "CI도 붙이고, 권한 관리도 하고, 무거운 브랜치 전략도..." 하지만 입문 단계의 종합 프로젝트는 **6개 단원으로 완결**되어야 하므로, **범위(scope)**를 분명히 그어 "이번엔 하지 않을 것"을 먼저 정합니다. 이것이 프로젝트가 통제 불능으로 커지는 것을 막는 가장 효과적인 장치입니다.

| 범위 In(이번에 한다)                          | 범위 Out(이번엔 하지 않는다)                      |
|----------------------------------------|-----------------------------------------|
| 기능별 브랜치 전략과 PR·코드 리뷰·이슈                | GitHub Actions(CI/CD) 자동화 — 다음 학습으로만 언급 |
| 동시 수정 충돌 재현과 해결                        | 복잡한 앱 기능·프레임워크(코드는 의도적으로 가볍게)           |
| 잘못된 머지 revert, force push 사고 reflog 복구 | submodule·LFS·monorepo 등 고급 저장소 구조      |
| 핫픽스 브랜치와 리뷰 반려 후 재작업                   | 조직(Org)·권한·브랜치 보호 규칙의 심화 설정             |
| rebase로 PR 커밋 정리, 태그·릴리스               | GitFlow 등 무거운 브랜치 모델(간단한 기능 브랜치로 한정)    |
| 혼자 두 역할(dev-alice·dev-bob)로 결정론적 재현    | —                                       |

특히 **GitHub Actions(자동화된 CI/CD)**는 이번 범위에서 의도적으로 제외합니다. 협업의 사고·진단·복구라는 핵심에 온전히 집중하기 위해서이며, Actions는 ch\_18의 성찰에서 "다음 학습"으로만 짚습니다. 범위를 좁히는 것은 후퇴가 아니라, 한정된 시간에 **깊이 있는 학습**을 보장하는 전략입니다.

### MVP(최소 기능 제품)를 먼저 본다

범위 안의 모든 것을 한꺼번에 하려 하지 말고, 먼저 **MVP**를 정의합니다. 이 프로젝트의 MVP는 "두 기여자가 기능 브랜치·PR로 To-Do 앱에 기능 2개를 추가하고, 한 번의 동시 수정 충돌을 해결해 머지하며, v1.0.0 릴리스까지 남기는 최소 협업 한 바퀴"입니다. force push 사고 복구·핫픽스·rebase 정리는 그 위에 얹는 확장 시나리오입니다. MVP가 먼저 동작해야 확장도 의미가 있습니다.

## 15.6 추적성: 요구사항 ↔ 단원 개념 매핑

분석의 마지막 단계는 **추적성(traceability)** 확인입니다. "우리가 적은 요구사항들이 정말로 c01~c14 개념을 빠짐없이 쓰는가, 반대로 배운 개념이 어딘가에 쓰이는가"를 한 표로 점검합니

다. 이 표가 채워지면 **커리큘럼(배운 것)과 프로젝트(만들 것)가 정렬되어** 있음을 증명할 수 있습니다.

| 개념  | 개념 이름                       | 이 개념을 쓰는 요구사항     |
|-----|-----------------------------|-------------------|
| c01 | Git 소개·설치·config            | (환경 전제) 모든 작업의 토대 |
| c02 | init-status                 | FR1               |
| c03 | add·commit·3영역·gitignore    | FR1·NFR5          |
| c04 | log-diff                    | NFR3(진단)          |
| c05 | branch-switch               | FR2·UC1·UC2       |
| c06 | merge                       | FR4·FR7           |
| c07 | 충돌 해결                       | FR4               |
| c08 | restore·reset·revert·reflog | FR5·FR6·NFR2·NFR3 |
| c09 | stash                       | FR7(핫픽스 전환)       |
| c10 | remote-clone                | FR1·NFR1          |
| c11 | push·pull·fetch             | FR2·FR6·FR7       |
| c12 | tag·릴리스                     | FR9               |
| c13 | PR·코드 리뷰·이슈                 | FR2·FR3·NFR4·UC1  |
| c14 | rebase·황금룰                  | FR8·NFR2          |

c01부터 c14까지 **14개 개념이 모두** 적어도 하나의 요구사항·유스케이스에 매핑됩니다. 빠진 개념이 없다는 것은, 이 종합 프로젝트가 6개 단원의 학습을 **고르게 통합**한다는 뜻입니다. 만약 어떤 개념이 어디에도 매핑되지 않았다면, 그 개념은 복습할 기회를 잃은 것이고, 반대로 어떤 요구사항이 배우지 않은 개념을 필요로 한다면 범위를 넘은 것입니다. 추적성 표는 이 두 가지 위험을 한눈에 잡아 줍니다.



## 15.7 작성형 연습(2종 1세트): 요구사항 표 채우기·자가체크

이제 직접 분석 문서를 작성해 볼 차례입니다. 아래 **모범**을 참고해, **빈 템플릿**을 여러분의 손으로 채우고 **자가체크**로 점검하세요.

### 연습 A 모범 — 새 요구사항 한 줄 명세하기

협업 시나리오에 "리뷰 반려 후 재작업으로 PR을 갱신한다"는 요구를 FR로 적는다면 다음과 같습니다.

| ID  | 기능 요구사항                                       | 적용<br>단원 | 적용<br>개념 | 거당하는<br>사고 |
|-----|-----------------------------------------------|----------|----------|------------|
| FR3 | PR을 코드 리뷰(승인/반려)하고 반려 시 같은 브랜치에 수정 push로 갱신한다 | u6       | c13      | 리뷰 반려      |

핵심은 네 가지를 빠짐없이 적는 것입니다. ① 무엇을 하는가(동사로), ② 어느 단원, ③ 어느 개념, ④ 어떤 협업 사고를 거당하는가.

### 연습 B 작성형 빈 템플릿

#### 직접 작성 1 — 사고 요구사항을 FR로 명세하기

아래 두 협업 사고를 각각 FR 한 줄로 직접 작성하세요(빈칸을 채웁니다).

- 사고 ㄱ: "alice와 bob이 `style.css`의 같은 색상 값을 다르게 고쳐 충돌이 났다"

| ID   | 기능 요구사항 | 적용<br>단원 | 적용<br>개념 | 거당하는 사고 |
|------|---------|----------|----------|---------|
| FR__ | _____   | u__      | C__·C__  | _____   |

- 사고 ㄴ: "릴리스된 v1.0.0에서 긴급 버그가 발견되어 빠르게 고쳐 배포해야 한다"

| ID   | 기능 요구사항 | 적용<br>단원 | 적용<br>개념 | 거당하는 사고 |
|------|---------|----------|----------|---------|
| FR__ | _____   | u__      | C__·C__  | _____   |

## 직접 작성 2 — 유스케이스 흐름 빈칸 채우기

UC2(동시 수정 충돌)의 단계를 순서대로 채우세요.

1. 두 클론이 \_\_\_\_\_ 파일의 같은 부분을 다르게 수정한다(개념: c\_)
2. \_\_\_\_\_ 의 PR이 먼저 머지되어 `origin/main` 이 앞선다
3. 둘째가 \_\_\_\_\_ 하는 순간 충돌이 난다(개념: c\_·c\_)
4. `git _____` 로 `Unmerged paths` 를 확인한다(개념: c\_)
5. 양쪽 의도를 살려 편집한 뒤 `git _____` → `git _____` 으로 해결한다(개념: c\_)
6. 실행해서 두 변경이 모두 동작하는지 \_\_\_\_\_ 한다

## 직접 작성 3 — 범위 한 줄 결정

여러분이라면 다음 항목을 In/Out 중 어디에 넣겠습니까? 이유 한 줄과 함께 적으세요.

- "PR마다 자동으로 테스트를 돌리는 GitHub Actions" → ( In / Out ), 이유: \_\_\_\_\_

## 자가체크

### 스스로 점검 체크리스트

- [ ] 작성한 FR이 **동사**로 시작해 '무엇을 하는가'가 분명한가요?
- [ ] 각 FR에 **적용 단위·적용 개념**을 빠짐없이 매핑했나요?
- [ ] 각 FR이 5대 협업 사고(동시 수정 충돌·잘못된 머지·force push·핫픽스·리뷰 반려) 중 무엇을 겨냥하는지 적었나요?
- [ ] UC2 흐름이 '문제 발생 → 진단(status) → 해결(add-commit) → 검증'의 골격을 따르나요?
- [ ] In/Out 결정에 **이유**를 적었나요? (이유 없는 범위 결정은 나중에 흔들립니다)
- [ ] c01~c14 중 어디에도 매핑되지 않은 개념이 없는지 추적성 표로 확인했나요?

## 15.8 분석 한 장 요약과 자주 묻는 질문

지금까지의 분석을 한 장으로 압축하면 다음과 같습니다. 이 표는 ch\_16(설계)·ch\_17(구현)의 출발점이 됩니다.

| 항목       | 결론                                                    |
|----------|-------------------------------------------------------|
| 드라이빙 퀘스천 | 동시 작업의 충돌·사고를 어떻게 안전하게 진단·복구할까                        |
| 산출물      | GitHub To-Do 웹앱 + 두 클론(dev-alice·dev-bob)의 완성 협업 시나리오 |
| 기능 요구사항  | FR1~FR9 (협업·충돌·잘못된 머지·force push·핫픽스·rebase·릴리스)      |
| 비기능 요구사항 | NFR1~NFR5 (결정론적 재현·황금률·진단 단계·main 보호·메시지 품질)          |
| 유스케이스    | UC1 기능 추가 / UC2 동시 수정 충돌 / UC3 사고 복구                  |
| 범위 Out   | GitHub Actions·프레임워크·고급 저장소 구조(다음 학습)                 |
| 개념 커버리지  | c01~c14 전부가 적어도 하나의 요구사항에 매핑됨                         |

### Q. 혼자 하는데 왜 두 사람(dev-alice·dev-bob)으로 나누나요?

협업의 핵심 사고인 '동시 수정 충돌'과 'force push로 동료 커밋이 덮이는 사고'는 **두 사람이 같은 origin을 따로 건드릴 때만** 생깁니다. 한 폴더에서 브랜치만 나누면 이 동기화 사고를 재현할 수 없습니다(ADR1). 두 클론으로 1인 2역을 하면 혼자서도 실제 팀의 push/pull 동기화 흐름을 그대로 체험할 수 있습니다.

### Q. 요구사항을 FR과 NFR로 굳이 나누는 이유는 무엇인가요?

FR은 '무엇을 하는가'(기능), NFR은 '어떻게 잘 하는가'(품질·제약)입니다. 둘을 섞으면 "충돌을 해결한다"(FR)와 "공유 이력은 revert로 되돌린다"(NFR)가 뒤엉켜 우선순위가 흐려집니다. 특히 NFR2(황금률)는 모든 FR에 걸리는 안전 규칙이라, 따로 떼어 분명히 적어 두는 것이 중요합니다.

### Q. 충돌을 줄이려면 애초에 같은 파일을 안 건드리면 되지 않나요?

이상적으로는 그렇지만, 실제 협업에서 같은 파일을 동시에 고치는 일은 피할 수 없습니다. 이 프로젝트의 목적은 충돌을 '회피'하는 것이 아니라 충돌이 났을 때 '침착하게 복구'하는 능력을 기르는 것(POV의 통찰)이므로, 충돌을 일부러 결정론적으로 재현합니다. 충돌을 줄이는 습관(작은 커밋·찾은 통합)은 ch\_18 성찰에서 다룹니다.

분석이 끝났습니다. 우리는 막연한 "협업해 보자"를 **9개 FR·5개 NFR·3개 유스케이스·명확한 범위로 구체화**했고, 모든 요구사항을 배운 개념과 매핑했습니다. 다음 챕터(ch\_16)에서는 이 요구사항을 **어떻게** 만들지 — 저장소 모델·협업 워크플로우·협업 정책(ADR) — 를 설계합니다.



# 종합 프로젝트: 팀 협업 시뮬레이션 — 설계 (저장소·워크플로우·ADR)

---

## 이 챕터에서 배우는 것

- 단일 원격(origin) + 두 로컬 클론(dev-alice·dev-bob)의 저장소 모델을 설계할 수 있습니다.
- 간단한 기능 브랜치 전략과 협업 워크플로우를 단계로 분해할 수 있습니다.
- 협업 정책 결정(재현 방식·main 통합·되돌리기 수단·force push 대응·핫픽스)을 ADR 5필드로 기록할 수 있습니다.
- 잘못된 머지·force push·충돌의 사고 복구 경로를 분기도로 표현할 수 있습니다.
- 협업·동기화·복구 흐름을 다이어그램으로 나타내 팀과 합의할 수 있습니다.

ch\_15에서 **무엇을** 만들지(요구사항·범위)를 못 박았습니다. 이번 챕터(ch\_16)는 그것을 **어떻게** 만들지 — **설계(design)**입니다. 설계는 코드를 짜기 전에 "구조와 규칙"을 먼저 정하는 일입니다. 협업 프로젝트에서 구조는 **저장소 모델**(누가 어디서 작업하는가)이고, 규칙은 **워크플로우와 정책**(어떤 순서로, 어떤 약속 아래 협업하는가)입니다.

새 명령은 여전히 등장하지 않습니다. c01~c14에서 배운 개념들을 **어떻게 조합해 협업 구조를 세울지**를 결정하고, 그 결정의 근거를 **ADR(설계 결정 기록)**로 남깁니다. 잘 설계된 구조는 사고를 줄이고, 잘 기록된 결정은 나중에 "왜 이렇게 했지?"라는 혼란을 막습니다.

## 16.1 설계 개요: 무엇을 어떻게 협업으로 나눌까

---

설계에서 가장 먼저 답해야 할 질문은 "**혼자서 어떻게 여럿의 협업을 재현할 것인가**"입니다.

ch\_15의 통찰대로, 협업 실력은 두 역할을 오가며 사고를 직접 만들고 풀어 봐야 체득됩니다. 그러려면 "두 사람이 같은 저장소를 동시에 건드리는 상황"을 혼자서도 **결정론적으로** 만들 수 있어야 합니다.

이 종합 프로젝트의 설계는 세 가지 결정으로 요약됩니다.

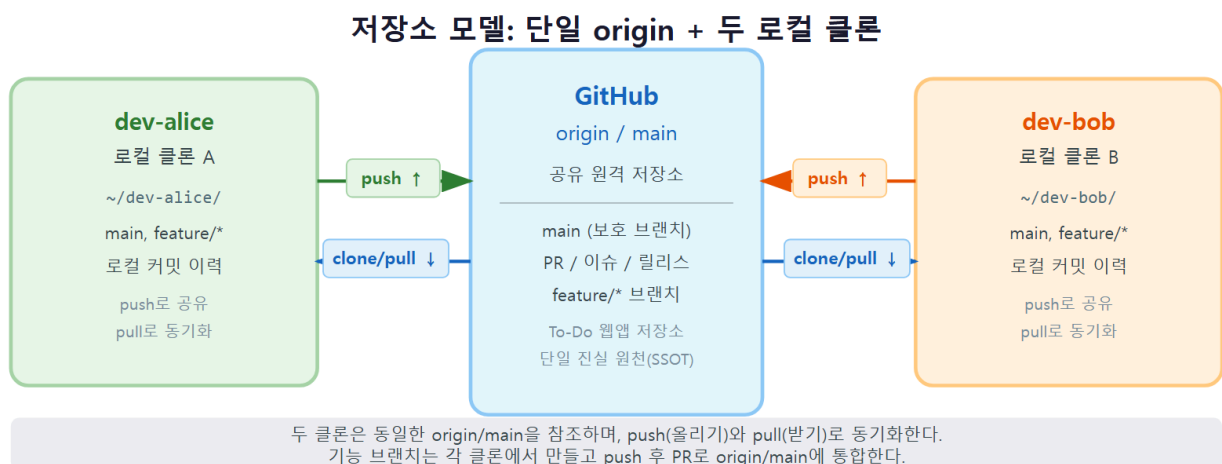
| 설계 영역         | 결정                                                    | 관련 개념       |
|---------------|-------------------------------------------------------|-------------|
| 저장소 모델(구조)    | GitHub 단일 원격(origin) 1개 + 로컬 클론 2개(dev-alice-dev-bob) | c10·c11     |
| 협업 워크플로우(규칙)  | 이슈 → 기능 브랜치 → push → PR → 리뷰 → (충돌 시 해결) → 머지 → 정리    | c05·c07·c13 |
| 사고 복구 경로(안전망) | 잘못된 머지 revert / force push reflog / 충돌 편집·해결          | c08·c11·c07 |

소재인 **To-Do 정적 웹앱**은 일부러 가볍게 둡니다. `index.html` · `app.js` · `style.css` 세 파일이면 충분합니다. 코드가 가벼워야 git 협업 흐름에 온전히 집중할 수 있고, "같은 파일을 둘이 동시에 손대는" 충돌도 자연스럽게 만들 수 있습니다. 설계의 초점은 앱 기능이 아니라 **협업 구조와 사고 복구**입니다.

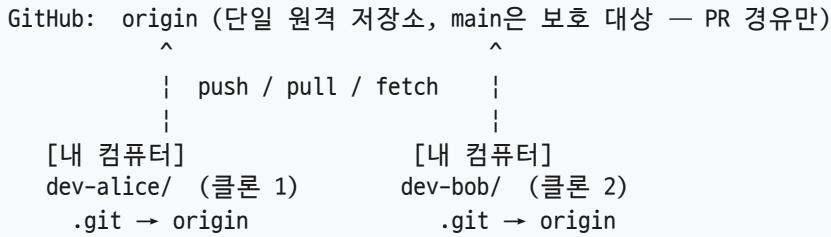
**설계 = '구조 + 규칙 + 안전망'**

좋은 협업 설계는 세 가지를 갖춥니다. ① 누가 어디서 작업하는지의 **구조**(저장소 모델), ② 변경을 어떤 순서로 통합하는지의 **규칙**(워크플로우), ③ 사고가 났을 때 어떻게 되돌리는지의 **안전망**(복구 경로)입니다. 셋 중 하나라도 빠지면 협업이 흔들립니다. 이 챕터는 셋을 차례로 설계합니다.

## 16.2 저장소 모델: 단일 origin + 두 로컬 클론



협업의 무대는 **하나의 GitHub 원격 저장소**입니다. 이 원격지를 모두가 `origin`이라는 관례적 이름(c10)으로 부르며, 두 기여자는 각자 이 `origin`을 **clone**해 자기 로컬 저장소를 가집니다. 폴더 구조로 그리면 다음과 같습니다.



핵심은 **두 클론이 같은 origin을 가리킨다**는 점입니다. dev-alice 폴더에서 push한 커밋은 origin에 올라가고, dev-bob 폴더에서 pull하면 그 커밋을 받습니다. 이렇게 폴더 두 개를 오가며 한 사람이 두 기여자를 번갈아 연기하면, 실제 팀의 **push/pull 동기화·충돌·force push** 사고를 그대로 재현할 수 있습니다(NFR1).

저장소 모델을 명세로 정리하면 이렇습니다.

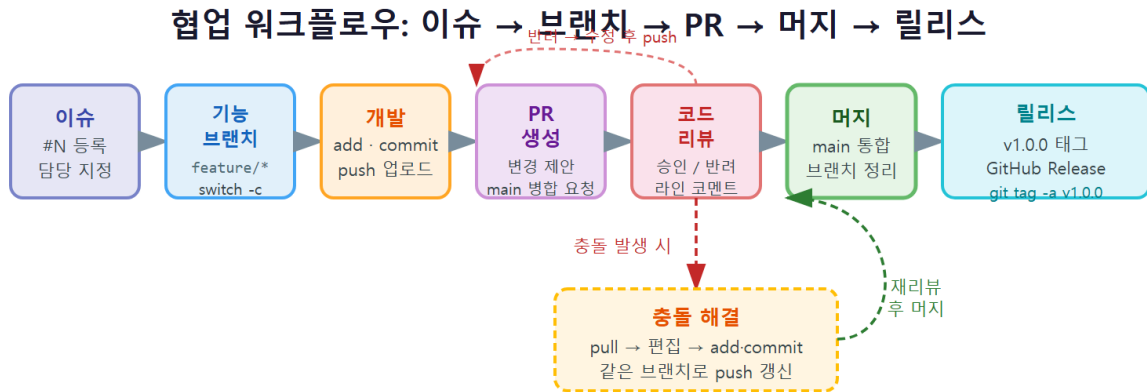
| 구성요소              | 설계                                                                                       | 근거 개념   |
|-------------------|------------------------------------------------------------------------------------------|---------|
| 원격(remote)        | GitHub 단일 원격 <code>origin</code> , <code>main</code> 은 보호 대상(직접 push 금지)                 | c10·c13 |
| 클론(clone)         | <code>dev-alice/</code> · <code>dev-bob/</code> 두 로컬 클론으로 두 기여자를 결론적 연기                  | c10     |
| 브랜치<br>(branches) | <code>main</code> (통합) + <code>feature/*</code> (기능) + <code>hotfix/*</code> (긴급)        | c05·c06 |
| 앱(app)            | <code>index.html</code> · <code>app.js</code> · <code>style.css</code> — 가벼운 To-Do 정적 웹앱 | (소재)    |

## 두 클론을 헛갈리면 사고가 난다 — 진단 습관

dev-alice와 dev-bob 폴더를 오가다 보면 "지금 내가 누구인지"를 놓치기 쉽습니다. 엉뚱한 폴더에서 push하면 의도치 않은 사고가 됩니다. 그래서 매 작업 전에 `git status`로 **어느 폴더·어느 브랜치**인지 먼저 확인하는 습관(NFR3)이 필요합니다. 프롬프트에 현재 폴더 이름을 표시해 두는 것도 좋은 방법입니다.



## 16.3 브랜치 전략과 협업 워크플로우 분해



정상 흐름: 이슈 → 기능 브랜치 → 개발·push → PR → 코드 리뷰(승인) → 머지 → 릴리스  
 충돌 시: pull로 최신 반영 → 편집으로 해결 → add-commit → push(PR 자동 갱신) → 재리뷰 → 머지

구조를 세웠으니 이제 **규칙**입니다. 두 기여자가 같은 origin을 공유할 때, 변경을 어떤 순서로 만들어 통합할지를 정합니다. 우리는 입문 범위에 맞춰 **간단한 기능 브랜치 전략**을 택합니다 (GitFlow 같은 무거운 모델은 범위 Out).

브랜치의 역할은 셋입니다.

| 브랜치       | 역할                      | 규칙                              |
|-----------|-------------------------|---------------------------------|
| main      | 통합·배포 기준. 항상 동작하는 상태 유지 | 직접 push 금지, PR 경유만(NFR4·c13)    |
| feature/* | 기능 하나를 개발하는 브랜치         | 이슈 단위로 파고, 머지 후 삭제(c05·c06)     |
| hotfix/*  | 릴리스 버전의 긴급 버그 수정        | main 에서 분기, PR·태그로 마무리(c06·c12) |

이 전략 위에서 **정상 협업 한 바퀴**는 다음 단계로 분해됩니다. 이것이 ch\_15의 UC1을 명령 수준으로 구체화한 것입니다.

|              |                         |       |                  |
|--------------|-------------------------|-------|------------------|
| 1. 이슈 생성     | gh issue create         | (c13) | 할 일 정의·할당        |
| 2. 브랜치 분기    | git switch -c feature/x | (c05) | main에서 갈래 만들기    |
| 3. 구현·커밋     | git add / git commit    | (c03) | 의미 단위 커밋         |
| 4. 원격 push   | git push -u origin ...  | (c11) | upstream 설정      |
| 5. PR 생성     | gh pr create            | (c13) | 변경 제안            |
| 6. 코드 리뷰     | (Approve / Request)     | (c13) | 상대 역할이 검토        |
| 7. (충돌 시) 해결 | status→편집→add→commit    | (c07) | 필요 시에만           |
| 8. 머지        | PR Merge                | (c13) | Closes #N로 이슈 종료 |
| 9. 브랜치 정리    | git branch -d feature/x | (c06) | 끝난 갈래 삭제         |

이 9단계 중 7번(충돌 해결)은 **항상 일어나지는 않습니다**. 동시 수정이 없으면 건너뛰고, 두 기여자가 같은 부분을 고쳤을 때만 발동됩니다. 충돌이 났을 때의 동기화 흐름은 다음과 같습니다.

```
[동시 수정 충돌 흐름]
alice: feature/a 머지됨 → origin/main 전진
bob:   feature/b 작업 중 → git pull(=fetch+merge) → 충돌!
      git status (Unmerged paths 진단)
      충돌 표시 편집(양쪽 의도 모두 살림)
      git add → git commit (충돌 해결 완료)
      git push (이제 origin과 합쳐진 상태로 올림)
```

### 왜 main 직접 push를 금지하는가

main에 누구나 바로 push할 수 있으면, 리뷰받지 않은 깨진 코드가 통합 브랜치에 섞여 "항상 동작하는 main"이 무너집니다(NFR4 위반). 모든 변경을 **PR이라는 관문**으로 통과시키면, 리뷰라는 한 겹의 점검을 거치게 되어 품질이 유지됩니다. 작은 변경에도 PR을 거치는 번거로움이 있지만, 그 번거로움이 곧 협업 규율입니다.

## 16.4 협업 정책 결정 기록: ADR 5건

설계에서 가장 가치 있는 산출물은 **결정의 근거**입니다. "왜 두 계정이 아니라 두 클론인가", "왜 reset이 아니라 revert인가" 같은 선택은 시간이 지나면 잊혀, 나중에 "그냥 이렇게 하자"는 즉흥적 결정으로 흔들립니다. 이를 막는 도구가 **ADR(Architecture Decision Record, 설계 결정 기록)**입니다.

ADR은 다섯 필드로 한 결정을 기록합니다. ① **제목**, ② **맥락(왜 결정이 필요한가)**, ③ **결정(무엇을 정했나)**, ④ **근거(왜 그 선택인가, 대안과 비교)**, ⑤ **결과(이 결정의 득과 실)**입니다. 이 프로젝트의 협업 정책은 다섯 개의 ADR로 기록됩니다.

### ADR1 — 팀 협업 재현 방식

| 필드 | 내용                                                                                                  |
|----|-----------------------------------------------------------------------------------------------------|
| 맥락 | 혼자서도 다수 기여자 협업과 충돌을 결정론적으로 재현해야 한다                                                                  |
| 결정 | 두 로컬 클론(dev-alice-dev-bob)으로 1인 2역을 한다                                                              |
| 대안 | ① 두 로컬 클론 ② 두 GitHub 계정 ③ 한 클론에서 브랜치만 분리                                                            |
| 근거 | 두 계정은 준비 부담이 크고, 한 클론은 '동시 수정'의 동기화 사고를 못 살린다. 두 클론이 push/pull 동기화·충돌·force push 사고까지 가장 사실적으로 재현한다 |
| 결과 | 폴더 두 개를 오가야 하나, 실제 협업 동기화 흐름을 그대로 체험한다                                                              |

## ADR2 — main 통합 정책

| 필드 | 내용                                          |
|----|---------------------------------------------|
| 맥락 | main은 항상 동작하는 상태를 유지해야 한다                   |
| 결정 | main 직접 push를 금지하고 PR 경유만 허용한다              |
| 대안 | ① PR 경유만 ② 직접 push 허용 ③ 혼합                  |
| 근거 | 리뷰 관문을 거쳐야 품질이 유지된다. PR 경유로 통일하는 것이 실무 표준이다 |
| 결과 | 작은 변경도 PR을 거쳐 번거로우나, 협업 규율을 학습한다            |

## ADR3 — 공유 이력 되돌리기 수단

| 필드 | 내용                                                                  |
|----|---------------------------------------------------------------------|
| 맥락 | 잘못된 머지·커밋을 되돌리되 동료 이력을 깨면 안 된다                                      |
| 결정 | 공유(push)한 커밋은 revert, 로컬 미공유만 reset/rebase로 정리한다                    |
| 대안 | ① 항상 revert ② 항상 reset ③ 공유 여부로 상황별 구분                              |
| 근거 | 이미 push된 이력을 reset/rebase로 덮으면 동료와 어긋난다. 공유 여부로 구분하는 것이 황금률(NFR2)이다 |
| 결과 | revert는 되돌림 커밋이 이력에 남지만, 협업 안전성이 보장된다                               |

## ADR4 — force push 사고 대응

| 필드 | 내용                                                                                                                             |
|----|--------------------------------------------------------------------------------------------------------------------------------|
| 맥락 | force push로 원격 이력이 덮이는 사고를 안전하게 다뤄야 한다                                                                                         |
| 결정 | 사고는 reflog로 복구하고, 평상시엔 <code>--force-with-lease</code> 만 제한적으로 허용한다                                                            |
| 대안 | ① force 전면 금지 ② <code>--force-with-lease</code> 제한 허용 ③ <code>--force</code> 자유                                                |
| 근거 | <code>--force</code> 는 동료 커밋을 무조건 덮어 위험하다. <code>--force-with-lease</code> 는 원격이 내가 아는 상태일 때만 밀어 안전장치가 되며, 사고는 reflog로 복구 훈련한다 |
| 결과 | 복구는 가능하나, 예방(공유 이력 불변)이 더 중요함을 학습한다                                                                                            |

## ADR5 — 긴급 수정 처리

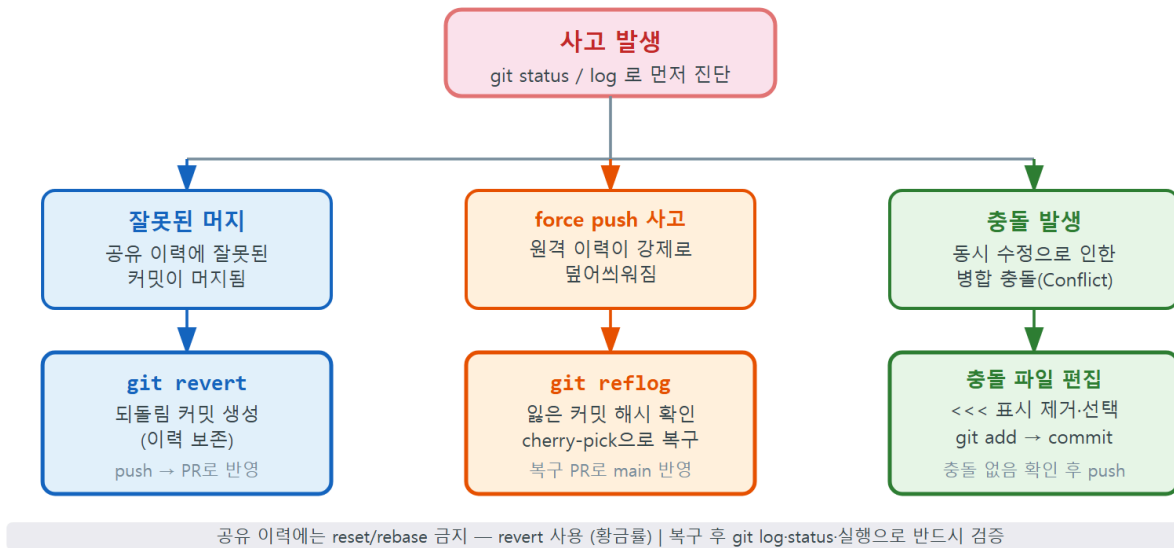
| 필드 | 내용                                                           |
|----|--------------------------------------------------------------|
| 맥락 | 릴리스된 버전의 긴급 버그를 빠르게 고쳐야 한다                                   |
| 결정 | main에서 분기한 hotfix/* 브랜치로 처리한 뒤 PR 머지·태그한다                    |
| 대안 | ① hotfix 브랜치 ② main 직접 수정 ③ 다음 릴리스까지 대기                      |
| 근거 | main 직접 수정은 규율을 깨고, 대기는 위험하다. hotfix 브랜치로 빠르되 PR·태그 규율은 유지한다 |
| 결과 | 핫픽스도 PR을 거쳐 다소 느리나, 이력·릴리스 추적이 명확하다                          |

**ADR은 '결정'보다 '근거와 대안'이 핵심이다**

ADR의 가치는 "무엇을 정했다"가 아니라 "어떤 대안들 중에 왜 이것을 골랐고, 무엇을 포기했는가"에 있습니다. 대안과 결과까지 적어 두면, 나중에 상황이 바뀌어 결정을 뒤집을 때도 "그때 왜 그랬는지"를 알고 합리적으로 바꿀 수 있습니다. 근거 없는 결정은 미래의 혼란이 됩니다.

## 16.5 사고 복구 분기도(revert·reflog·충돌)

## 사고 복구 분기도: 상황별 진단과 해결 경로



설계의 마지막은 **안전망** — 사고가 났을 때 어느 길로 복구할지를 미리 그려 두는 **사고 복구 분기도**입니다. 협업에서 마주치는 사고는 종류가 다르고, 종류마다 올바른 복구 명령이 다릅니다. 잘못된 명령을 고르면 사고가 더 커지므로, "어떤 사고인지 진단 → 알맞은 길 선택"의 분기를 분명히 해 둡니다.

## 사고 발생

- ▼ 먼저 git status / git log로 진단 (NFR3)
  - [충돌] 같은 파일 같은 부분이 다르게 바뀜
    - git status로 Unmerged paths 확인 (c07)
    - 표시 편집(양쪽 의도 살림) → git add → git commit
    - (포기하려면) git merge --abort
  - [잘못된 머지] 깨진 변경이 이미 main에 push됨 (공유 이력)
    - git revert <merge> (c08, 되돌리는 새 커밋)
    - reset 금지! (공유 이력 — 황금률 NFR2)
  - [force push 사고] 내 커밋이 원격에서 덮임
    - git reflog로 잃은 커밋의 해시 찾기 (c08)
    - git branch 복구용 <해시> 로 되살리기
    - 복구 PR로 main에 안전하게 재통합 (c11·c13)

이 분기도의 첫 갈림목은 항상 **진단**입니다. 어떤 사고든 손이 먼저 나가지 않고 git status·git log·필요하면 git reflog 로 "지금 무엇이 어떻게 됐는지"를 먼저 읽습니다(NFR3). 그다음에야 사고 종류에 맞는 길을 고릅니다.

세 갈래의 복구 원칙을 한 표로 정리합니다.

| 사고 종류         | 진단 도구                                    | 복구 명령                                             | 절대 하지 말 것                      |
|---------------|------------------------------------------|---------------------------------------------------|--------------------------------|
| 충돌            | <code>git status</code> (Unmerged paths) | 편집 → <code>add</code> → <code>commit</code>       | 충돌 표시를 남긴 채 커밋                 |
| 잘못된 머지 (공유)   | <code>git log --graph</code>             | <code>git revert &lt;merge&gt;</code>             | <code>reset</code> 으로 공유 이력 뺏기 |
| force push 사고 | <code>git reflog</code>                  | <code>reflog</code> 해시 → <code>branch</code> 로 복구 | <code>--force</code> 로 다시 덮기   |

### 복구 명령을 잘못 고르면 사고가 두 배가 된다

가장 흔한 2차 사고는 "공유된 잘못된 머지를 `reset`으로 지우는 것"입니다. `reset`은 내 로컬 이력만 되감을 뿐, 이미 origin에 올라간 머지는 그대로입니다. 이 상태로 `push`하면 거부되고(non-fast-forward), 여기서 `--force` 로 밀면 그 사이 동료가 올린 커밋까지 사라져 **force push 사고로 번집니다**. 공유 이력은 반드시 `git revert` 로 — 이 한 줄이 협업 안전의 핵심입니다.

## 16.6 작성형 연습(2종 1세트): ADR 작성·자가체크

설계의 산출물인 ADR을 직접 작성해 봅니다. 아래 모범을 본떠 빈 템플릿을 채우고 자가체크로 점검하세요.

### 연습 A 모범 — 새 협업 정책을 ADR로 기록하기

"기능 브랜치 이름을 어떻게 지을 것인가"를 ADR로 기록한 예시입니다.

| 필드 | 내용                                                                                                |
|----|---------------------------------------------------------------------------------------------------|
| 제목 | 기능 브랜치 이름 규칙                                                                                      |
| 맥락 | 두 기여자가 만드는 브랜치 이름이 제각각이면 어떤 이슈의 작업인지 알기 어렵다                                                       |
| 결정 | <code>feature/&lt;이슈번호&gt;-&lt;짧은-설명&gt;</code> 형식으로 통일한다(예: <code>feature/12-add-filter</code> ) |
| 대안 | ① 이슈번호 포함 규칙 ② 자유 명명 ③ 작성자 이름 접두                                                                  |
| 근거 | 이슈번호를 넣으면 브랜치만 봐도 어떤 작업인지 추적되고 PR·이슈 연결이 쉽다                                                       |
| 결과 | 이름이 다소 길지만, 이력과 이슈의 추적성이 좋아진다                                                                     |

## 연습 B 작성형 빈 템플릿

### 직접 작성 1 — ADR 한 건 완성하기

"PR을 머지할 때 `merge commit-squash-rebase` 중 무엇을 쓸 것인가"를 ADR 5필드로 직접 작성하세요(빈칸을 채웁니다).

| 필드 | 내용(직접 작성)      |
|----|----------------|
| 제목 | PR 머지 방식       |
| 맥락 | _____          |
| 결정 | _____          |
| 대안 | ① _ ② _ ③ ____ |
| 근거 | _____          |
| 결과 | _____          |

### 직접 작성 2 — 저장소 모델 그림 채우기

아래 빈칸을 채워 우리 저장소 모델을 완성하세요.



```

GitHub:  _____ (단일 원격, main은 _____ 경유만)
          ^               ^
          |               |
          | _____ / _____ / fetch |
dev-alice/ (클론 _)   dev-____/ (클론 _)
.git → _____   .git → _____

```

### 직접 작성 3 — 사고 복구 길 고르기

다음 세 사고에 알맞은 복구 명령을 빈칸에 적으세요(reset은 정답이 아닐 수 있습니다).

- ㄱ. 이미 push한 머지가 잘못됐다 → `git _____`
- ㄴ. bob의 force push로 내 커밋이 원격에서 사라졌다 → `git _____` 로 해시를 찾는다
- ㄷ. 같은 파일을 둘이 다르게 고쳐 충돌이 났다 → `git status` 로 진단 후 편집 → `git _____` → `git _____`

## 자가체크

### 스스로 점검 체크리스트

- [ ] 작성한 ADR에 다섯 필드(제목·맥락·결정·대안·근거·결과)가 모두 있나요?
- [ ] 대안을 둘 이상 적고, 왜 그것을 포기했는지 근거에 드러냈나요?
- [ ] 저장소 모델 그림에서 두 클론이 같은 origin을 가리키게 그렸나요?
- [ ] 사고 복구에서 공유 이력에는 reset이 아니라 revert를 골랐나요?(황금률 NFR2)
- [ ] force push 사고 복구의 첫 단계가 reflog 진단인가요?
- [ ] 모든 복구 경로가 '진단(status·log·reflog) → 해결'의 순서를 지키나요?(NFR3)

## 16.7 설계를 구현 마일스톤으로 잇기

설계가 추상적인 그림에 머물지 않으려면, 다음 챕터(ch\_17)의 구현 마일스톤과 어떻게 연결되는지를 미리 짚어 두어야 합니다. 저장소 모델·워크플로우·ADR·복구 분기도가 각각 어느 마일스톤에서 실제 명령으로 살아나는지 표로 정리합니다.

| 구현 마일스톤<br>(ch_17) | 이 설계에서 가져오는 것                                  | 적용 개념       |
|--------------------|------------------------------------------------|-------------|
| M1 협업 환경 구성        | 저장소 모델(origin + 두 클론), 브랜치 전략                  | c03·c10·c11 |
| M2 정상 협업 한 바퀴      | 워크플로우 9단계(이슈→PR→리뷰→머지)                         | c05·c13     |
| M3 동시 수정 충돌 해결     | 충돌 흐름·복구 분기도의 [충돌] 갈래                          | c06·c07     |
| M4 사고 발생과 복구       | 복구 분기도의 [잘못된 머지]·[force push] 갈래,<br>ADR3·ADR4 | c08·c11     |
| M5 핫픽스·정리·릴리스      | ADR5(hotfix 정책), rebase 정리, 태그                 | c09·c12·c14 |

설계가 잘 되었는지는 이 연결이 빈틈없는가로 판단합니다. 다섯 마일스톤이 모두 설계 산출물에 뿌리를 두고 있고, 거꾸로 설계 산출물(저장소 모델·워크플로우·ADR 5건·복구 분기도)이 모두 어느 마일스톤엔가 쓰입니다. 분석(ch\_15)의 추적성 표가 '요구사항 ↔ 개념'을 정렬했다면, 이 표는 '설계 ↔ 구현'을 정렬합니다.

#### Q. 왜 GitFlow 같은 정식 브랜치 모델을 쓰지 않나요?

GitFlow는 `develop`·`release`·`hotfix` 등 여러 영구 브랜치를 두는 무거운 모델로, 큰 팀·정기 릴리스에 어울립니다. 입문 단계에서는 오히려 학습 부담만 키우므로 범위 Out으로 두었습니다(ch\_15). 우리는 `main` + `feature/*` + `hotfix/*`의 간단한 기능 브랜치 전략으로 협업의 본질에 집중합니다.

#### Q. `--force` 와 `--force-with-lease` 는 어떻게 다른가요?

`--force` 는 원격 상태를 보지 않고 무조건 내 이력으로 덮어, 그 사이 동료가 올린 커밋을 지웁니다. `--force-with-lease` 는 '내가 마지막으로 본 원격 상태가 그대로일 때만' 밀어, 동료의 새 커밋이 있으면 거부됩니다(ADR4의 안전장치). 다만 둘 다 공유 이력에는 쓰지 않는 것이 원칙이고, 사고는 `reflog`로 복구합니다.

#### Q. ADR을 꼭 표로 써야 하나요?

형식은 자유입니다. 핵심은 다섯 필드(제목·맥락·결정·대안·근거·결과)가 빠지지 않는 것입니다. 표·목록·문단 어느 형식이든 '어떤 대안 중에 왜 이것을 골랐고 무엇을 포기했는가'가 드러나면 좋은 ADR입니다.

**Q. 설계 단계에서 코드를 한 줄도 안 짜도 괜찮나요?**

괜찮습니다. 설계는 '구조·규칙·안전망'을 합의하는 단계이고, 실제 명령 실행은 구현(ch\_17)의 몫입니다. 오히려 설계 없이 바로 코드부터 짜면 두 클론을 헛갈리거나 공유 이력을 reset으로 덮는 사고가 잦아집니다. 충분히 설계해 둘수록 구현이 빠르고 안전해집니다.

설계가 끝났습니다. 우리는 **저장소 모델(구조)·협업 워크플로우(규칙)·ADR 5건(근거)·사고 복구 분기도(안전망)**를 갖추었고, 이를 구현 마일스톤과 연결했습니다. 이제 무엇을·어떻게 만들지가 모두 분명해졌으니, 다음 챕터(ch\_17)에서는 이 설계대로 **실제로 협업하고 사고를 내고 복구하는** 구현 단계로 들어갑니다.



# 종합 프로젝트: 팀 협업 시뮬레이션 — 구현 (협업·충돌·사고·릴리스)

설계도가 준비됐습니다. 이제 실제로 두 기여자가 하나의 저장소를 함께 개발하는 협업을 **손으로 돌려 볼** 차례입니다. 이번 챕터는 종합 프로젝트의 세 번째 단계인 **구현 (implementation)**입니다. 협업 전체를 한 번에 쏟아내지 않고, **마일스톤(milestone)**이라는 작은 단계로 나누어 **증분 (incremental)**으로 쌓아 올립니다. M1에서 협업 환경을 세우고, M2에서 정상 협업 한 바퀴를 돌며, M3에서 충돌을, M4에서 사고를 일으켜 복구하고, M5에서 핫픽스와 릴리스로 마무리합니다.

이 챕터에서는 **새 명령을 배우지 않습니다**. ch\_01부터 ch\_14까지 배운 명령들을 협업이라는 실전 무대에 통합해 적용할 뿐입니다. 가장 중요한 약속이 하나 있습니다. 모든 사고를 **문제 발생 → status·log·reflog 진단 → 해결 → 검증**의 네 단계로 기록한다는 것입니다(NFR3). 협업의 실력은 사고를 안 내는 데 있지 않고, 사고가 났을 때 침착하게 진단해 복구하는 데 있기 때문입니다.

## 이 챕터의 학습 목표

- 정상 협업부터 사고 복구까지 다섯 개의 증분 마일스톤으로 수행할 수 있습니다.
- 각 마일스톤에서 어떤 단원 개념을 적용했는지 설명할 수 있습니다.
- 각 사고를 '문제 발생 → 진단 → 해결 → 검증'의 네 단계로 기록할 수 있습니다.
- 마일스톤을 통합해 하나의 완성 협업 시나리오를 만들 수 있습니다.

## ch\_17\_s01. 구현 전략과 마일스톤 개요

우리는 협업 시나리오를 다섯 개의 마일스톤으로 나눠 만듭니다. 각 마일스톤은 앞 단계가 만든 저장소 상태 위에 새 사건을 하나씩 엮는 방식이라, M1이 끝나면 두 클론이 같은 원격을 바라보고, M2가 끝나면 정상 협업 한 바퀴가 돌며, M5가 끝나면 충돌·사고·핫픽스를 모두 겪고 복구한 완성 이력이 됩니다.

| 마일스톤 | 무엇을 하나                                 | 사용단원       | 사용 개념                                           |
|------|----------------------------------------|------------|-------------------------------------------------|
| M1   | 협업 환경 구성(원격·두 클론·브랜치 전략)               | u1, u5     | c03 add-commit, c10 remote-clone, c11 push-pull |
| M2   | 정상 협업 한 바퀴(이슈→PR→리뷰→머지)                | u2, u6     | c05 branch-switch, c13 PR·리뷰                    |
| M3   | 동시 수정 충돌 발생·진단·해결·검증                   | u3         | c06 merge, c07 충돌 해결                            |
| M4   | 사고 복구(잘못된 머지 revert·force push reflog) | u4, u5     | c08 revert-reflog, c11 push                     |
| M5   | 핫픽스·rebase 정리·v1.0.0 릴리스               | u4, u5, u6 | c09 stash, c12 tag, c14 rebase                  |

이 표는 ch\_15의 추적성 매핑과 ch\_16의 워크플로우 분해가 실제 구현 순서로 펼쳐진 것입니다. 각 마일스톤이 어느 개념을 복습·적용하는지 명시되어 있어, 협업을 돌리면서 배운 명령을 다시 확인하게 됩니다.

### 드라이빙 퀘스천을 다시 떠올립니다

"여러 기여자가 하나의 저장소에서 동시에 작업할 때, 동시 수정 충돌·잘못된 머지·force push 사고·핫픽스·리뷰 반려 같은 문제를 어떻게 안전하게 진단하고 해결할까?" 이번 챕터의 다섯 마일스톤이 이 질문에 하나씩 답합니다.

## 재현 환경 — 로컬 bare 원격 + 두 클론

이 모든 시나리오를 혼자서도 결정론적으로 재현하려고(NFR1), 우리는 GitHub 서버 대신 **로컬 bare 저장소**를 원격(origin) 역할로 쓰고, 그 원격을 두 번 clone 해 dev-alice·dev-bob 두 기여자를 1인 2역으로 연기합니다(ADR1). ch\_17/project/ 폴더에 완성된 To-Do 웹앱과 이 시나리오를 단계별로 재현하는 스크립트 모음이 들어 있습니다. 아래 본문의 명령과 출력은 그 스크립트를 실제로 실행했을 때 나오는 것과 일치합니다. GitHub 서버가 없으므로, GitHub 웹에서 누르는 PR 머지 버튼은 로컬에서 `git merge --no-ff` 후 `git push`로 대신 재현합니다. 실제 GitHub에서는 같은 일을 `gh pr merge`가 서버에서 해 줍니다.

## ch\_17\_s02. M1 협업 환경 구성(원격·두 클론·브랜치 전략)

사용 개념: c03 add-commit, c10 remote-clone, c11 push-pull

가장 먼저 만들 것은 협업의 무대입니다. 두 기여자가 공유할 원격 저장소를 두고, 각자 clone 해 자기 작업 공간을 갖춘 뒤, 첫 To-Do 앱을 origin/main에 올립니다.

### 빈 원격(bare)과 첫 클론 만들기

bare 저장소는 작업트리 없이 이력만 담는 저장소로, GitHub의 원격이 하는 일을 로컬에서 그대로 합니다.

```
# 1) 공유 원격 역할을 할 bare 저장소 생성(작업트리 없이 이력만 보관)
git init --bare origin.git
```

```
# 2) dev-alice가 원격을 clone(아직 비어 있음)
git clone origin.git dev-alice
```

```
# 3) dev-alice에서 초기 To-Do 앱 만들기
git -C dev-alice add index.html app.js style.css README.md
git -C dev-alice commit -m "chore: To-Do 앱 초기 구조 추가"
git -C dev-alice push -u origin main
```

```
$ git init --bare origin.git
Initialized empty Git repository in /work/origin.git/

$ git clone origin.git dev-alice
Cloning into 'dev-alice'...
warning: You appear to have cloned an empty repository.
done.

$ git -C dev-alice push -u origin main
Enumerating objects: 6, done.
Writing objects: 100% (6/6), 1.21 KiB | 1.21 MiB/s, done.
To /work/origin.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

- `git init --bare origin.git`: 작업트리 없는 원격 저장소를 만듭니다(c10). GitHub 저장소를 새로 만드는 것과 같은 역할입니다.
- `git clone origin.git dev-alice`: 원격을 통째로 복제합니다(c10). 비어 있어 경고가 뜨지만 정상입니다.

- `git -C dev-alice ...`: `-C` 옵션은 "그 폴더에 들어간 쉘 치고" 명령을 실행합니다. 두 클론을 오갈 때 폴더 이동 실수를 막아 줍니다.
- `git push -u origin main`: 첫 커밋을 원격 main에 올리고 upstream을 맺습니다(c11). 이후 alice는 `git push` 만으로 동기화됩니다.

## 둘째 기여자 dev-bob 합류

alice가 초기 앱을 올렸으니, 이제 bob이 clone 하면 그 내용이 그대로 따라옵니다.

```
# dev-bob이 원격을 clone(이번엔 내용이 들어 있음)
git clone origin.git dev-bob
git -C dev-bob log --oneline
```

```
$ git clone origin.git dev-bob
Cloning into 'dev-bob'...
done.

$ git -C dev-bob log --oneline
a1b2c3d (HEAD -> main, origin/main) chore: To-Do 앱 초기 구조 추가
```

**체크포인트:** 두 클론이 같은 origin/main을 가리키고 각자 push·pull이 되는지 확인합니다.

- 두 클론에서 `git remote -v` 가 같은 origin.git을 가리키는가
- 두 클론의 `git log --oneline` 최신 커밋 해시가 같은가(같은 origin/main)
- alice가 push한 커밋이 bob의 clone에 들어 있는가

세 가지가 확인되면 M1 통과입니다. 협업의 무대가 준비됐습니다.

## ch\_17\_s03. M2 정상 협업 한 바퀴(이슈→PR→리뷰→머지)

**사용 개념:** c05 branch-switch, c13 PR-리뷰

무대가 섰으니 표준 협업 한 바퀴를 돌립니다(UC1). dev-alice가 "완료 토글" 기능을 이슈로 정의하고, 기능 브랜치에서 구현해 PR을 올리면, dev-bob이 리뷰합니다. main은 직접 push 하지 않고 반드시 PR을 거칩니다(ADR2·NFR4).



## 이슈 → 기능 브랜치 → 구현 → push

```
# dev-alice: 기능 브랜치를 만들며 전환(main은 그대로 둠)
git -C dev-alice switch -c feature/toggle-done

# app.js에 완료 토글 함수를 추가하고 커밋
git -C dev-alice add app.js
git -C dev-alice commit -m "feat: 할 일 완료 토글 기능 추가 (#1)"
git -C dev-alice push -u origin feature/toggle-done
```

```
$ git -C dev-alice switch -c feature/toggle-done
Switched to a new branch 'feature/toggle-done'

$ git -C dev-alice push -u origin feature/toggle-done
To /work/origin.git
 * [new branch]      feature/toggle-done -> feature/toggle-done
branch 'feature/toggle-done' set up to track 'origin/feature/toggle-done'.
```

- `git switch -c feature/toggle-done`: main을 안전하게 둔 채 기능 브랜치를 만들며 전환합니다(c05). main을 직접 고치지 않는 것이 협업의 기본 규율입니다.
- 커밋 메시지의 (#1): 이 변경이 이슈 1번과 연결됨을 드러냅니다(NFR5). 실제 GitHub에서는 `gh issue create` 로 만든 이슈 번호입니다.
- `git push -u origin feature/toggle-done`: 기능 브랜치를 원격에 올려 PR 대상으로 만듭니다(c11). 실제 GitHub에서는 이 직후 `gh pr create` 로 PR을 엽니다.

## 코드 리뷰 → 반려 → 재작업 → 머지

GitHub에서라면 dev-bob이 PR을 열어 라인 코멘트를 달고 Request changes로 반려합니다(c13). alice는 같은 브랜치에 수정 커밋을 push 하면 PR이 자동 갱신됩니다. 로컬 재현에서는 리뷰 반려를 "수정 커밋 한 개 더"로, 머지 버튼을 maintainer 클론의 `git merge --no-ff` 로 재현합니다.

```
# 리뷰 반려 반영: alice가 빈 항목 방지 코드를 더해 같은 브랜치에 push(PR 자동 갱신)
git -C dev-alice add app.js
git -C dev-alice commit -m "fix: 완료 토글 시 빈 항목 무시 (리뷰 반영)"
git -C dev-alice push
```

```
# 리뷰 승인 후 머지(GitHub 머지 버튼 = no-ff 병합 + push)
git -C dev-alice switch main
git -C dev-alice pull --ff-only
git -C dev-alice merge --no-ff feature/toggle-done -m "Merge PR #1: 완료 토글 기능"
git -C dev-alice push origin main
git -C dev-alice branch -d feature/toggle-done
git -C dev-alice push origin --delete feature/toggle-done
```

```
$ git -C dev-alice merge --no-ff feature/toggle-done -m "Merge PR #1: 완료 토글 기능"
Merge made by the 'ort' strategy.
 app.js | 14 +++++
 1 file changed, 14 insertions(+)

$ git -C dev-alice push origin main
To /work/origin.git
 a1b2c3d..e4f5a6b  main -> main
```

`--no-ff` 는 병합 커밋을 명시적으로 남겨(c06), "이 변경 묶음이 PR 하나로 들어왔다"는 사실을 이력에 보존합니다. 머지 후에는 다 쓴 기능 브랜치를 로컬·원격 모두에서 정리합니다.

**체크포인트:** 리뷰 승인 후 머지되고 main이 동작하는지, 이슈가 자동 종료되는지 확인합니다.

- main에 완료 토글 기능이 들어가 페이지가 정상 동작하는가
- 병합 커밋(Merge PR #1)이 `git log --graph` 에 보이는가
- 커밋 메시지의 `closes #1` 로 GitHub 이슈가 자동으로 닫혔는가(실제 GitHub 기준)

이 세 가지가 확인되면 M2 통과입니다. dev-bob은 `git -C dev-bob pull` 로 머지된 main을 받아 두 클론을 다시 같은 상태로 맞춥니다.

## ch\_17\_s04. M3 동시 수정 충돌 발생·진단·해결·검증

**사용 개념:** c06 merge, c07 충돌 해결

이제 협업에서 가장 흔한 사건, **동시 수정 충돌**을 일부러 일으킵니다(UC2-FR4). 두 기여자가 같은 파일의 같은 부분을 서로 다르게 고치면, 먼저 머지된 쪽 다음에 둘째가 합칠 때 충돌이 납니다. 충돌은 실패가 아니라 정상 과정이며(c07), 핵심은 침착한 진단입니다.

## 문제 발생 — 두 클론이 app.js의 같은 함수를 수정

alice와 bob 모두 같은 시점의 main에서 분기해, app.js의 `renderTasks` 부분을 각자 다르게 고칩니다. alice는 정렬을, bob은 완료 항목 흐리기를 같은 줄 근처에 넣습니다.

```
# alice의 브랜치가 먼저 머지됨(정렬 기능)
git -C dev-alice switch main
git -C dev-alice merge --no-ff feature/sort-tasks -m "Merge PR #2: 목록 정렬"
git -C dev-alice push origin main

# bob은 옛 main에서 분기해 작업했음 → 최신 main을 받아 합치려다 충돌
git -C dev-bob switch feature/dim-done
git -C dev-bob fetch origin
git -C dev-bob merge origin/main
```

```
$ git -C dev-bob merge origin/main
Auto-merging app.js
CONFLICT (content): Merge conflict in app.js
Automatic merge failed; fix conflicts and then commit the result.
```

bob이 자기 기능 브랜치에 최신 main을 합치려 하자, 같은 `renderTasks` 부분을 둘 다 고쳤기 때문에 Git이 자동으로 합치지 못하고 멈췄습니다.

## 진단 — status·log로 무엇이 어떻게 부딪혔는지 읽기

당황하지 말고 먼저 `git status`로 진행 상태를 읽습니다. 진단이 복구의 출발점입니다(NFR3).

```
git -C dev-bob status
git -C dev-bob log --oneline --all --graph -5
```

```
$ git -C dev-bob status
On branch feature/dim-done
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
    both modified:   app.js

no changes added to commit (use "git add" and/or "git commit -a")
```

Unmerged paths에 `both modified: app.js`가 보입니다. 두 갈래가 같은 파일을 고쳤다는 신호입니다. 이제 충돌 표시를 직접 봅니다.

```
function renderTasks() {
  const list = document.getElementById("task-list");
  list.innerHTML = "";
  <<<<<<< HEAD
  const view = [...tasks].sort((a, b) => a.text.localeCompare(b.text));
  for (const task of view) {
    =====
    for (const task of tasks) {
      const cls = task.done ? "task done" : "task";
  >>>>>>> origin/main
      const li = document.createElement("li");
      list.appendChild(li);
    }
  }
}
```

<<<<<<< HEAD 와 ===== 사이는 bob의 변경(정렬한 목록 순회), ===== 와 >>>>>>> origin/main 사이는 main의 변경(완료 항목에 done 클래스)입니다. 어느 한쪽을 버리는 것이 아니라, 두 의도를 모두 살리는 것이 옳은 해결입니다(루브릭 충돌 해결 '우수' 기준).

## 해결 — 양쪽 의도를 모두 살려 편집

충돌 표시를 지우고, 정렬과 완료 흐리기를 함께 담은 최종본으로 편집합니다.

```
function renderTasks() {
  const list = document.getElementById("task-list");
  list.innerHTML = "";
  const view = [...tasks].sort((a, b) => a.text.localeCompare(b.text));
  for (const task of view) {
    const cls = task.done ? "task done" : "task";
    const li = document.createElement("li");
    li.className = cls;
    list.appendChild(li);
  }
}
```

```
# 표시를 모두 지우고 두 변경을 합친 뒤 해결을 표시하고 병합 커밋
git -C dev-bob add app.js
git -C dev-bob commit -m "merge: 정렬과 완료 흐리기 동시 반영, 충돌 해결"
```

git add 가 "이 충돌을 해결했다"는 표시이고, git commit 이 병합을 마무리합니다(c07).

## 검증 — 충돌 표시가 남지 않았는지, 동작하는지

해결을 끝냈다고 믿지 말고 반드시 검증합니다.

```
git -C dev-bob diff --check
git -C dev-bob log --oneline --graph -4
```

```
$ git -C dev-bob diff --check

$ git -C dev-bob log --oneline --graph -4
* 7c8d9e0 (HEAD -> feature/dim-done) merge: 정렬과 완료 흐리기 동시 반영, 충돌 해결
|\
| * b2c3d4e (origin/main, main) Merge PR #2: 목록 정렬
* | 5a6b7c8 feat: 완료 항목 흐리게 표시
```

git diff --check 는 충돌 표시(<<<<, =====, >>>>)가 남아 있으면 잡아냅니다. 출력이 비어 있으면 표시가 깨끗이 제거됐습니다. 마지막으로 브라우저에서 페이지를 열어 정렬과 완료 흐리기가 함께 동작하는지 눈으로 확인합니다.

**체크포인트:** 충돌 표시 0, 두 변경의 의도가 모두 반영됐는지 실행으로 검증합니다.

- git diff --check 가 아무것도 보고하지 않는가(표시 잔존 0)
- 페이지에서 정렬과 완료 흐리기가 둘 다 동작하는가
- 병합 커밋이 두 부모를 갖는가(git log --graph 의 갈라진 선)

**흔한 실수: 충돌이 막막해 처음부터 다시 하고 싶을 때**

충돌이 너무 복잡해 보이면 git merge --abort 로 병합 직전 상태로 안전하게 되돌릴 수 있습니다(c07). 단, 이미 git add 로 일부를 해결 표시한 상태에서도 abort는 동작하지만, 편집한 내용은 사라집니다. 진짜 위험한 실수는 ===== 를 깜빡 남긴 채 커밋하는 것입니다. 이때는 코드가 깨지므로, 커밋 전 git diff --check 를 습관화해야 합니다.

## ch\_17\_s05. M4 사고 복구: 잘못된 머지 revert·force push reflog

**사용 개념:** c08 revert·reflog, c11 push

충돌은 정상 과정이지만, 이번에는 진짜 **사고**를 일으킵니다(UC3·FR5·FR6). 잘못된 변경이 main 에 머지돼 버린 사고와, force push로 동료의 커밋이 원격에서 덮인 사고입니다. 두 사고 모두 **공유된 이력**을 건드리므로, 황금률(NFR2·ADR3)에 따라 reset이 아니라 revert와 reflog로 복구합니다.

## 사고 A: 잘못된 머지를 revert로 되돌리기

### 문제 발생 — 버그가 있는 PR이 main에 머지됨

검토가 소홀해, 페이지를 깨뜨리는 `feature/bulk-clear` 브랜치가 main에 머지되고 push까지 됐습니다. 이미 공유된 이력이라 dev-bob도 그 main을 받았습니다.

```
$ git -C dev-alice log --oneline --graph -3
* d9e0f1a (HEAD -> main, origin/main) Merge PR #3: 전체 삭제 버튼
|\
| * c0d1e2f feat: 전체 삭제 (목록 초기화 버그 포함)
|/
* b2c3d4e Merge PR #2: 목록 정렬
```

### 진단 — log로 잘못된 병합 커밋을 특정

`git log --graph` 로 어느 커밋이 사고인지 찾습니다. Merge PR #3 (해시 `d9e0f1a`)이 두 부모를 가진 병합 커밋입니다. 병합 커밋을 되돌릴 때는 어느 부모를 본류로 볼지 알려 줘야 합니다.

```
git -C dev-alice show d9e0f1a --stat
```

### 해결 — git revert -m 1 로 되돌림 커밋 만들기

이미 push된 공유 이력이므로 reset으로 지우면 동료와 어긋납니다(황금률). 대신 revert로 되돌리는 새 커밋을 씁습니다(c08). 병합 커밋이라 `-m 1` 로 본류(첫 부모, main 쪽)를 지정합니다.

```
# -m 1: 병합 커밋의 첫 번째 부모(main)를 본류로 삼아 그 변경을 되돌림
git -C dev-alice revert -m 1 d9e0f1a -m "revert: PR #3 머지 되돌림 (목록 초기화 버그)"
git -C dev-alice push origin main
```

```
$ git -C dev-alice revert -m 1 d9e0f1a -m "revert: PR #3 머지 되돌림 (목록 초기화 버그)"
[main 2f3a4b5] revert: PR #3 머지 되돌림 (목록 초기화 버그)
1 file changed, 8 deletions(-)
```

### 왜 reset이 아니라 revert인가

`git reset --hard b2c3d4e` 로 사고 커밋을 지우면 내 로컬에서는 깔끔해 보이지만, 원격은 여전히 사고 커밋을 갖고 있어 push가 거부되고, 강제로 밀면 동료 dev-bob의 이력과 어긋납니다. 공유한(push한) 이력은 지우지 말고 되돌립니다(c08·황금률). revert는 사고와 복구가 모두 이력에 남아, 나중에 "무슨 일이 있었는지" 추적할 수 있습니다.

## 검증 — main 정상화 확인

```
git -C dev-alice log --oneline -3
git -C dev-alice diff d9e0f1a~1 HEAD -- app.js
```

```
$ git -C dev-alice log --oneline -3
2f3a4b5 (HEAD -> main, origin/main) revert: PR #3 머지 되돌림 (목록 초기화 버그)
d9e0f1a Merge PR #3: 전체 삭제 버튼
b2c3d4e Merge PR #2: 목록 정렬
```

revert 커밋이 쌓여 페이지가 다시 동작합니다. 사고 커밋은 이력에 남되 그 효과만 취소됐습니다.

## 사고 B: force push로 덮인 커밋을 reflog로 복구

### 문제 발생 — bob의 force push가 alice의 커밋을 덮음

공유 브랜치 `feature/labels`에 alice가 커밋 `feat: 라벨 색상`을 push 했습니다. bob은 그 사실을 모른 채 자기 로컬에서 rebase 한 뒤 `git push --force`로 밀어, 원격의 alice 커밋이 사라졌습니다(ADR4가 막으려는 바로 그 사고).

```
$ git -C dev-bob push --force origin feature/labels
To /work/origin.git
+ 9a8b7c6...1f2e3d4 feature/labels -> feature/labels (forced update)
```

forced update 와 + 기호가 위험 신호입니다. 원격에서 alice의 커밋 `9a8b7c6`가 강제로 덮였습니다.

### 진단 — reflog로 잃은 것처럼 보이는 커밋 찾기

alice의 로컬에는 그 커밋이 아직 reflog에 남아 있습니다. reflog는 HEAD가 거쳐 간 모든 지점을 기록하는 안전망입니다(c08).

```
git -C dev-alice reflog
```

```
$ git -C dev-alice reflog
1f2e3d4 (origin/feature/labels) HEAD@{0}: pull: forced-update
9a8b7c6 HEAD@{1}: commit: feat: 라벨 색상
b2c3d4e HEAD@{2}: checkout: moving to feature/labels
```

HEAD@{1} 의 9a8b7c6 feat: 라벨 색상 이 force push로 원격에서 사라진 그 커밋입니다. 해시만 알면 되살릴 수 있습니다.

## 해결 — 잃은 커밋을 복구 브랜치로 살려 PR로 반영

```
# 잃은 커밋 해시로 복구 브랜치를 만들어 그 지점을 되살림
git -C dev-alice switch -c recover/labels 9a8b7c6
git -C dev-alice push -u origin recover/labels
# 이후 복구 PR로 main(또는 feature/labels)에 다시 머지해 정상화
```

```
$ git -C dev-alice switch -c recover/labels 9a8b7c6
Switched to a new branch 'recover/labels'

$ git -C dev-alice push -u origin recover/labels
To /work/origin.git
 * [new branch]      recover/labels -> recover/labels
```

## 검증 — 잃은 변경이 되돌아왔는지 확인

```
git -C dev-alice log --oneline -1 recover/labels
```

```
$ git -C dev-alice log --oneline -1 recover/labels
9a8b7c6 (HEAD -> recover/labels, origin/recover/labels) feat: 라벨 색상
```

라벨 색상 커밋이 복구 브랜치로 되살아났습니다.

**체크포인트:** revert 후 main 정상화, reflog로 잃은 커밋을 되찾아 복구 PR로 반영합니다.

- 잘못된 머지가 revert 커밋으로 되돌려져 main이 동작하는가
- 사고 이력이 reset으로 지워지지 않고 그대로 남아 추적 가능한가
- force push로 덮인 커밋을 reflog에서 찾아 복구 브랜치로 되살렸는가

## 예방이 복구보다 낫습니다

reflog 복구는 강력하지만, 애초에 공유 브랜치에 `--force` 를 쓰지 않는 것이 정답입니다. 불가피하게 강제 push가 필요하면 `git push --force-with-lease` 를 씁니다. 이는 "내가 본 원격 상태가 그대로일 때만" 밀어, 그 사이 동료가 올린 커밋이 있으면 거부해 사고를 막습니다(ADR4). 또한 reflog는 그 커밋이 로컬에 한 번이라도 있었던 사람만 복구할 수 있습니다 — 한 번도 받지 않은 사람은 복구하지 못하므로, 예방이 최선입니다.



## ch\_17\_s06. M5 핫픽스·rebase 정리·v1.0.0 릴리스

**사용 개념: c09 stash, c12 tag, c14 rebase**

마지막 마일스톤에서는 릴리스 직전의 세 가지 실무를 묶습니다(FR7·FR8·FR9). 작업 중 들어온 긴급 버그를 핫픽스로 처리하고, 머지 전 지저분한 PR 커밋을 rebase로 정리하며, 안정 지점에 v1.0.0 태그·릴리스를 남깁니다.

### 핫픽스 — stash로 하던 일을 치우고 급한 수정

**문제 발생 — 작업 중 긴급 버그 신고**

alice가 새 기능을 작업하던 중, 입력창에 스크립트를 넣으면 그대로 실행되는 보안 버그가 배포된 main에서 발견됐습니다. 지금 작업은 아직 커밋하기 이릅니다.

```
# 하던 작업을 선반에 잠시 치워 작업트리를 깨끗이(c09)
git -C dev-alice stash push -m "작업 중: 마감일 기능"
git -C dev-alice switch -c hotfix/escape-input main
```

```
$ git -C dev-alice stash push -m "작업 중: 마감일 기능"
Saved working directory and index state On feature/due-date: 작업 중: 마감일 기능

$ git -C dev-alice switch -c hotfix/escape-input main
Switched to a new branch 'hotfix/escape-input'
```

git stash로 미완성 변경을 선반에 치우면 작업트리가 깨끗해져, 안전하게 핫픽스 브랜치로 전환할 수 있습니다(c09).

**해결과 검증 — 핫픽스 PR 머지 후 돌아와 pop**

```
# 입력을 이스케이프하도록 수정·커밋·push 후 PR 머지(ADR5: 핫픽스도 PR 경유)
git -C dev-alice add app.js
git -C dev-alice commit -m "fix: 입력값 이스케이프로 스크립트 주입 차단"
git -C dev-alice push -u origin hotfix/escape-input
git -C dev-alice switch main
git -C dev-alice merge --no-ff hotfix/escape-input -m "Merge PR #5: 입력 이스케이프 핫픽스"
git -C dev-alice push origin main

# 하던 작업으로 복귀해 선반에서 다시 꺼내 이어 가기
git -C dev-alice switch feature/due-date
git -C dev-alice stash pop
```

```
$ git -C dev-alice stash pop
On branch feature/due-date
Changes not staged for commit:
  modified:   app.js
Dropped refs/stash@{0} (a1c2e3f...)
```

핫픽스가 머지되고, 치워 뒀던 마감일 작업이 그대로 돌아왔습니다. stash가 "급한 전환 → 복귀"를 안전하게 이어 주었습니다.

## rebase로 PR 커밋 정리

### 문제 발생 — wip·오타 커밋이 지저분하게 쌓임

alice의 feature/due-date 브랜치에는 wip, 오타 수정, 진짜 수정 같은 의미 없는 커밋이 쌓였습니다. 머지 전에 하나의 의미 단위로 정리하고 싶습니다.

```
$ git -C dev-alice log --oneline feature/due-date -4
3c4d5e6 (HEAD -> feature/due-date) 진짜 수정
2b3c4d5 오타 수정
1a2b3c4 wip
e4f5a6b (origin/main) Merge PR #5: 입력 이스케이프 핫픽스
```

### 해결 — rebase -i로 squash

이 브랜치는 아직 PR로 공유되지 않은 본인 전용 브랜치이므로 rebase가 안전합니다(c14. 황금률 영역 밖).

```
# main 위로 다시 쌓으며 wip·오타 커밋을 하나로 합치기(squash)
git -C dev-alice rebase -i origin/main
```

편집기가 열리면 첫 줄만 pick 으로 두고 나머지를 squash (또는 s)로 바꿔 세 커밋을 하나로 합칩니다.

```
pick    1a2b3c4 wip
squash  2b3c4d5 오타 수정
squash  3c4d5e6 진짜 수정

# 합쳐진 커밋의 메시지를 다음으로 정리:
# feat: 할 일 마감일 입력·표시 기능 추가 (#6)
```

### 검증 — 직선 이력으로 정리됐는지 확인

```
$ git -C dev-alice log --oneline feature/due-date -2
7f8a9b0 (HEAD -> feature/due-date) feat: 할 일 마감일 입력·표시 기능 추가 (#6)
e4f5a6b (origin/main) Merge PR #5: 입력 이스케이프 핫픽스
```

세 개의 지저분한 커밋이 의미 있는 한 커밋으로 정리됐습니다. 이 깔끔한 브랜치를 push 해 PR 을 올리고 머지합니다.

## v1.0.0 태그와 릴리스

모든 기능이 들어가고 핫픽스까지 끝난 안정된 main에 릴리스 지점을 표시합니다(c12·FR9).

```
# 안정 지점에 주석 태그를 달고 따로 push(태그는 기본 미전송)
git -C dev-alice switch main
git -C dev-alice pull --ff-only
git -C dev-alice tag -a v1.0.0 -m "첫 정식 릴리스: 협업 To-Do 앱"
git -C dev-alice push origin v1.0.0
```

```
$ git -C dev-alice tag -a v1.0.0 -m "첫 정식 릴리스: 협업 To-Do 앱"

$ git -C dev-alice push origin v1.0.0
To /work/origin.git
* [new tag]          v1.0.0 -> v1.0.0
```

실제 GitHub에서는 이 태그를 토대로 Releases 페이지에서 릴리스 노트를 붙여 공개합니다.

**체크포인트:** 핫픽스가 머지·태그되고, 정리된 커밋 이력과 릴리스가 보이는지 확인합니다.

- 핫픽스가 PR을 거쳐 머지되고 stash로 치웠던 작업이 복구됐는가
- rebase -i로 wip·오타 커밋이 의미 단위로 정리됐는가(직선 이력)
- v1.0.0 태그가 원격에 올라가 릴리스 기반이 됐는가

## ch\_17\_s07. 통합 시나리오 실행과 결과 확인

다섯 마일스톤을 통합하면 '정상 협업 → 충돌 → 사고 복구 → 핫픽스 → 릴리스'의 전체 협업 시나리오가 하나의 저장소 이력으로 완성됩니다(integration). ch\_17/project/ 의 재현 스크립트 모음을 순서대로 실행하면, 본문에서 단계별로 본 명령들이 한 번에 돌아 같은 결과를 만듭니다.

```
# 재현 스크립트를 순서대로 실행해 전체 시나리오를 한 번에 재현
bash ch_17/project/run_all.sh
```

실행이 끝나면 원격에는 여러 머지된 PR, 해결된 충돌 병합 커밋, revert 복구 커밋, force push 사고의 복구 브랜치, 핫픽스 머지, v1.0.0 태그가 모두 담깁니다. 전체 그림을 한 번에 봅니다.

```
$ git -C dev-alice log --oneline --graph --all -12
* a0b1c2d (HEAD -> main, tag: v1.0.0, origin/main) Merge PR #6: 마감일 기능
|\
| * 7f8a9b0 feat: 할 일 마감일 입력·표시 기능 추가 (#6)
|/
* e4f5a6b Merge PR #5: 입력 이스케이프 핫픽스
* 2f3a4b5 revert: PR #3 머지 되돌림 (목록 초기화 버그)
* d9e0f1a Merge PR #3: 전체 삭제 버튼
|\
| * c0d1e2f feat: 전체 삭제 (목록 초기화 버그 포함)
|/
* b2c3d4e Merge PR #2: 목록 정렬
* e4f5a6b Merge PR #1: 완료 토글 기능
* a1b2c3d chore: To-Do 앱 초기 구조 추가
```

이 한 화면이 협업의 모든 사건을 증명합니다. PR 머지(병합 커밋), 충돌 해결, 사고의 revert 복구, 그리고 v1.0.0 태그까지 — 흩어진 명령들이 하나의 협업 이력으로 모였습니다. 각 사고의 '문제 발생 → 진단 → 해결 → 검증' 과정을 캡처로 함께 남기면, 다음 챕터의 발표·평가에 그대로 쓸 수 있습니다.

## ch\_17\_s08. 작성형 연습(2종 1세트): 사고 복구 단계 채우기·자가체크

이번 챕터의 핵심은 "사고를 네 단계로 기록하는 힘"입니다. 직접 사고 하나를 골라 단계를 채워 봅니다.

### 모범 예시 — 사고 기록 한 건

**사고:** 버그가 있는 PR #3이 main에 머지·push됐다. **진단:** `git log --oneline --graph`로 두 부모를 가진 병합 커밋 `d9e0f1a`를 특정했다. **해결:** 공유 이력이므로 `reset` 대신 `git revert -m 1 d9e0f1a`로 되돌림 커밋을 만들어 push했다(-m 1은 main을 본류로 지정). **검증:** `git log`에 revert 커밋이 쌓이고 페이지가 다시 동작함을 확인했다. 사고 커밋은 이력에 남아 추적 가능하다.

이처럼 사고 기록은 "무엇이 터졌고, 무엇으로 진단했으며, 어떤 명령으로 왜 그렇게 해결했고, 어떻게 검증했는지"를 구체적으로 담을 때 가치가 있습니다. "그냥 고쳤다" 같은 막연한 기록은 다음에 도움이 되지 않습니다.

## 직접 작성 1 — 사고 복구 4단계 채우기 (빈 템플릿)

force push로 덮인 커밋을 reflog로 복구하는 사고를, 네 단계로 직접 적어 봅니다. 빈칸을 실제 명령과 함께 채웁니다.

- **문제 발생:** dev-bob의 `git push` \_\_\_\_\_ 로 원격의 dev-alice 커밋이 덮였다.
- **진단:** dev-alice의 로컬에서 `git` \_\_\_\_\_ 로 잃은 커밋의 해시(`HEAD@{__}`)를 찾는다.
- **해결:** 찾은 해시로 `git switch -c recover/labels` \_\_\_\_\_ 후 `git` \_\_\_\_\_ 로 복구 브랜치를 올린다.
- **검증:** `git log --oneline -1 recover/labels` 로 잃은 커밋( \_\_\_\_\_ )이 되살아났는지 확인한다.
- **예방 한 줄:** 다음부터는 `git push` \_\_\_\_\_ 를 써서 동료 커밋을 덮지 않게 한다.

작성 힌트:

- 진단에는 반드시 `reflog` 가 들어가야 합니다(잃은 것처럼 보이는 커밋은 reflog에 남습니다).
- 예방 한 줄에는 `--force-with-lease` 가 들어가야 합니다.

## 직접 작성 2 — 충돌 해결 최종본 직접 작성 (빈 템플릿)

아래 충돌 조각을 보고, 양쪽 의도를 모두 살린 최종본을 직접 작성한 뒤 해결 명령을 적어 봅니다.

```
<<<<<<< HEAD
const view = [...tasks].filter((t) => !t.done);    // 미완료만 보기
=====
const view = [...tasks].sort((a, b) => a.text.localeCompare(b.text)); // 이름순 정렬
>>>>>>> feature/sort
```

- 두 의도(미완료만 보기 + 이름순 정렬)를 모두 살린 최종 한 줄: `const view =`  
\_\_\_\_\_;
- 해결을 표시하는 명령: `git` \_\_\_\_\_ `app.js`

- 병합을 마무리하는 명령: `git _____ -m "merge: 필터와 정렬 동시 반영"`
- 표시 잔존을 잡는 검증 명령: `git _____`

작성 힌트:

- 두 의도를 한 줄에 담으려면 `filter` 결과에 `sort` 를 이어 붙입니다(메서드 체이닝).
- 검증 명령은 `diff --check` 입니다.

## 자가체크

- [ ] 두 클론(dev-alice·dev-bob)이 같은 origin/main을 가리키고 push·pull이 되는가(M1)
- [ ] 정상 협업 한 바퀴(브랜치→PR→리뷰→머지)를 main 직접 push 없이 돌렸는가(M2)
- [ ] 동시 수정 충돌에서 양쪽 의도를 모두 살려 해결하고 `git diff --check` 로 검증했는가(M3)
- [ ] 잘못된 머지를 reset이 아닌 `git revert -m 1` 로 되돌렸는가(M4·황금률)
- [ ] force push로 덮인 커밋을 `git reflog` 로 찾아 복구 브랜치로 되살렸는가(M4)
- [ ] 핫픽스를 stash·hotfix 브랜치로 처리하고, rebase -i로 커밋을 정리했는가(M5)
- [ ] v1.0.0 태그를 달아 origin에 push했는가(M5)
- [ ] 각 사고를 '문제 발생 → 진단 → 해결 → 검증' 네 단계로 기록했는가(NFR3)

## 자주 묻는 질문(FAQ)

### Q. 로컬 bare 원격으로 연습한 것이 실제 GitHub 협업과 같은가요?

네, 핵심 git 메커니즘은 완전히 같습니다. clone·push·pull·merge·revert·reflog는 원격이 GitHub든 로컬 bare든 동일하게 동작합니다. 차이는 GitHub가 PR·코드 리뷰·릴리스 같은 협업 UI를 서버에서 제공한다는 점뿐입니다. 그래서 본문에서는 머지 버튼을 `git merge --no-ff + git push` 로 재현했습니다. 로컬에서 메커니즘을 몸에 익히면, GitHub에서는 같은 일을 버튼과 `gh` 명령으로 더 편하게 할 뿐입니다.

### Q. 병합 커밋을 revert 할 때 -m 1과 -m 2는 어떻게 다른가요?

병합 커밋은 부모가 둘입니다. `-m 1` 은 첫 번째 부모(보통 머지를 받은 main 쪽)를 본류로 삼아, 머지로 들어온 변경을 되돌립니다. `-m 2` 는 두 번째 부모(머지된 기능 브랜치 쪽)를 본류로 삼습니다. 우리가 원하는 것은 "main에 잘못 들어온 기능 변경을 취소"이므로 거의 항상 `-m 1` 입니다. 부모 번호를 잘못 지정하면 엉뚱한 방향으로 되돌려지니, revert 후 반드시 `git diff` 로 의도한 변경이 취소됐는지 검증합니다.

**Q. reset --hard로 미커밋 변경을 날렸는데 reflog로 복구되나요?**

아니요. reflog는 커밋된 지점만 복구합니다. 한 번도 커밋되지 않은 작업트리 변경은 reset --hard로 사라지면 reflog에 흔적이 없어 복구할 수 없습니다. 그래서 위험한 명령 전에는 commit이나 stash로 안전 지점을 만드는 습관이 중요합니다. 반대로 commit까지 됐던 지점은, 브랜치를 옮겨 잃은 것처럼 보여도 reflog의 해시로 거의 항상 되살릴 수 있습니다.

**Q. PR 브랜치를 rebase 했는데 push가 거부됩니다.**

rebase는 커밋 해시를 새로 만들기 때문에, 이미 push한 브랜치를 rebase 하면 원격과 갈라져 일반 push가 거부됩니다. 그 브랜치가 **본인만 쓰는 PR 브랜치**임이 확실하면 `git push --force-with-lease`로 안전하게 갱신할 수 있습니다(ADR4). 하지만 동료도 함께 쓰는 공유 브랜치라면 절대 force 하지 말고, 정리는 merge로 하거나 새 커밋으로 처리합니다(황금률).

## 마무리: 다섯 마일스톤을 완주하며

여러분은 빈 원격 하나에서 시작해, 두 기여자가 함께 개발하며 **충돌을 해결하고, 사고를 복구하고, 핫픽스를 배포하고, v1.0.0을 릴리스**하는 완성된 협업 시나리오를 손수 돌렸습니다. 이 과정에서 ch\_01부터 ch\_14까지 배운 명령들이 협업이라는 실전 무대에서 어떻게 협력하는지 직접 보았습니다.

가장 값진 것은 완성된 이력 그래프가 아니라, **사고가 났을 때 status·log·reflog로 먼저 진단하고 침착하게 복구하는 태도**입니다. 충돌은 정상 과정이고, 잘못된 머지는 revert로 되돌리며, 덮인 커밋은 reflog로 되살린다는 것을 — 외운 것이 아니라 직접 사고를 일으켜 풀어 봤기에 — 이제 몸으로 압니다. 다음 챕터에서는 이 완성된 협업 시나리오를 발표하고, 루브릭으로 평가하며, 어떤 명령이 어느 사고 복구로 이어졌는지 성찰합니다.





# 종합 프로젝트: 팀 협업 시뮬레이션 — 발표·평가·성찰

협업 시나리오를 완주했습니다. 하지만 프로젝트는 "동작하는 이력"으로 끝나지 않습니다. 마지막 단계는 **발표·평가·성찰(present & reflect)**입니다. 만든 것을 남에게 보여 주고 (발표), 정해진 기준으로 평가하며(평가), 어떤 명령이 어느 사고 복구로 이어졌는지 돌아봅니다 (성찰). 이 단계가 있어야 흠어진 협업 경험이 진짜 협업 실력으로 굳어집니다.

협업을 해 보기만 하고 돌아보지 않으면, 다음에 같은 사고를 만나도 또 당황합니다. 반대로 "어떤 명령이 어느 사고를 복구했고, 왜 reset이 아니라 revert였으며, 다시 한다면 무엇을 바꿀지"를 정리해 두면, 그 깨달음이 다음 협업의 자산이 됩니다. 이번 챕터에서도 새 명령은 없습니다. 대신 지난 열일곱 챕터의 여정, 특히 ch\_17의 다섯 마일스톤을 정리하고 평가합니다.

## 이 챕터의 학습 목표

- 완성한 협업 시나리오를 이력·머지된 PR·릴리스로 시연할 수 있습니다.
- 여섯 기준 평가 루브릭으로 자신과 동료의 작업을 객관적으로 평가할 수 있습니다.
- 배운 명령이 어느 사고 복구로 이어졌는지 구체적으로 성찰할 수 있습니다.
- 프로젝트의 한계를 인식하고 다음 학습(GitHub Actions·CI)을 제시할 수 있습니다.

## ch\_18\_s01. 산출물 시연: 협업 이력·머지된 PR·릴리스 데모

발표의 핵심은 "이 협업에서 무슨 일이 있었는지"를 짧은 시간에 보여 주는 것입니다. 코드 프로젝트라면 실행 화면을 보여 주지만, 협업 프로젝트의 진짜 산출물은 **이력 그래프**입니다. 좋은 데모는 대표적인 협업 사건을 이력 위에서 차례로 짚습니다.

대표 시연 흐름은 다음과 같습니다.

1. 두 클론이 같은 origin을 공유함을 보여 준다(M1 — `git remote -v`).
2. 머지된 PR들을 이력 그래프로 보여 준다(M2 — 정상 협업 한 바퀴).
3. 충돌이 두 부모를 가진 병합 커밋으로 해결된 지점을 짚는다(M3).

4. 잘못된 머지가 revert 커밋으로 되돌려진 지점을 짚는다(M4-A).
5. force push로 덮인 커밋이 reflog로 복구된 브랜치를 보여 준다(M4-B).
6. 핫픽스 머지와 v1.0.0 태그를 보여 준다(M5).

## 데모 스크립트 예시 — 이력 한 장으로 협업 증명

발표 때는 다음처럼 이력 그래프를 띄워 놓고 어떤 사건이 어디 있는지 손으로 짚어 줍니다.

```
$ git -C dev-alice log --oneline --graph --all -12
* a0b1c2d (HEAD -> main, tag: v1.0.0, origin/main) Merge PR #6: 마감일 기능
|\
| * 7f8a9b0 feat: 할 일 마감일 입력·표시 기능 추가 (#6)
|/
* e4f5a6b Merge PR #5: 입력 이스케이프 핫픽스
* 2f3a4b5 revert: PR #3 머지 되돌림 (목록 초기화 버그)
* d9e0f1a Merge PR #3: 전체 삭제 버튼
|\
| * c0d1e2f feat: 전체 삭제 (목록 초기화 버그 포함)
|/
* b2c3d4e Merge PR #2: 목록 정렬
* e4f5a6b Merge PR #1: 완료 토글 기능
* a1b2c3d chore: To-Do 앱 초기 구조 추가
```

이 한 화면이 협업의 모든 사건을 증명합니다. 발표에서는 "여기 두 부모로 갈라진 곳이 M3 충돌 해결입니다", "이 revert 커밋이 M4에서 잘못된 머지를 되돌린 곳입니다"처럼 **이력 위치와 사건을 이어 말하면 설득력이 큼**니다.

## 릴리스 시연

```
$ git -C dev-alice tag -n
v1.0.0          첫 정식 릴리스: 협업 To-Do 앱

$ git -C dev-alice show v1.0.0 --stat -s
tag v1.0.0
Tagger: dev-alice <alice@example.com>

첫 정식 릴리스: 협업 To-Do 앱
```

실제 GitHub에서는 이 태그를 토대로 Releases 페이지에서 릴리스 노트와 함께 공개합니다. 태그가 "이 지점이 v1.0.0 배포 버전"임을 고정해 주므로, 언제든지 그 시점으로 정확히 돌아갈 수 있습니다.

## 좋은 협업 데모의 3원칙

- **이력으로 말하기:** 코드 한 줄이 아니라 `git log --graph`의 사건들로 협업을 증명합니다.
- **사고를 자랑하기:** 사고를 숨기지 말고 "여기서 잘못 머지됐고 revert로 복구했습니다"라고 보여 주는 것이 오히려 복구 실력의 증거입니다.
- **연결해서 말하기:** "이 revert는 ch\_08에서 배운 명령입니다"처럼 이력과 배운 개념을 이어 말합니다.

---

## ch\_18\_s02. 평가 루브릭

---

데모를 봤다면 이제 **루브릭(rubric)**으로 평가합니다. 루브릭은 "무엇을 어떤 기준으로 평가하는가"를 표로 정한 평가 기준입니다. 점수를 막연히 매기지 않고, 미흡·보통·우수 세 수준의 구체적 설명에 따라 객관적으로 평가하게 해 줍니다.

팀 협업 시뮬레이션의 평가 루브릭은 여섯 개 기준으로 구성됩니다.

| 기준                      | 가중치  | 미흡                     | 보통                | 우수                                         |
|-------------------------|------|------------------------|-------------------|--------------------------------------------|
| 협업 워크플로우 운영             | 0.25 | main 직접 push로 진행       | 브랜치·PR은 쓰나 리뷰 형식적 | 이슈→브랜치→PR→리뷰→머지가 일관되게 운영됨                  |
| 충돌 해결                   | 0.20 | 충돌에서 한 쪽을 통째로 버림       | 충돌은 해결하나 의도 일부 누락 | 양쪽 의도를 모두 살려 해결·검증                         |
| 사고 복구 (revert·reflog)   | 0.25 | 사고를 복구 못 하거나 force로 덮음 | 한 종류 사고는 복구       | 잘못된 머지·force push 사고를 revert·reflog로 안전 복구 |
| 이력 품질(커밋·rebase·태그)     | 0.15 | 의미 없는 커밋·태그 없음         | 메시지는 양호하나 정리 안 됨  | rebase로 정리된 의미 단위 커밋과 릴리스 태그               |
| 진단 능력 (status·log·diff) | 0.10 | 추측으로 명령 실행             | 가끔 status 확인      | 매 사고를 status·log·reflog로 먼저 진단             |
| 발표·성찰                   | 0.05 | 결과만 보여줌                | 흐름 설명             | 각 사고의 원인·복구·예방까지 회고                        |

## 루브릭으로 점수 매기는 법

각 기준에서 자신이 어느 수준인지 고르고, 수준에 점수(미흡 1·보통 2·우수 3)를 매긴 뒤 가중치를 곱해 더합니다. 예를 들어 사고 복구가 우수(3), 충돌 해결이 보통(2)이라면 사고 복구는  $3 \times 0.25 = 0.75$ , 충돌 해결은  $2 \times 0.20 = 0.40$ 처럼 계산해 합산합니다. 만점은 3.0입니다.

이 루브릭은 가중치가 말해 주듯 **협업 워크플로우 운영(0.25)**과 **사고 복구(0.25)**에 가장 큰 무게를 둡니다. 이는 이 프로젝트의 POV — "협업의 실력은 충돌을 안 내는 것이 아니라 사고가 났을 때 침착하게 진단해 복구하는 것" — 을 그대로 반영한 것입니다.

## 루브릭은 목표 지도이기도 합니다

루브릭의 "우수" 칸은 곧 "여기까지 가면 잘한 것"이라는 목표입니다. 아직 보통 수준이라면 우수 칸을 보고 "무엇을 더 해야 우수가 되는가"를 알 수 있습니다. 예컨대 사고 복구가 보통(한 중

류만 복구)이라면, 잘못된 머지 revert와 force push relog 복구를 모두 해내면 우수가 됩니다. 루브릭은 평가 기준이자 다음 행동의 안내입니다.

## ch\_18\_s03. 동료 피드백(갤러리 워크)

혼자 평가하면 자기 이력의 약점이 잘 안 보입니다. 그래서 **갤러리 워크(gallery walk)**로 서로의 협업 이력을 둘러보며 피드백을 주고받습니다. 각자의 `git log --graph`와 PR·릴리스를 돌아가며 보고 의견을 남기는 방식입니다.

좋은 피드백은 막연한 칭찬("잘했어요")이 아니라 **구체적이고 실행 가능한** 것입니다. 다음 Keep-Change-Question 형식이 도움이 됩니다.

- **좋은 점(Keep):** "잘못된 머지를 revert로 되돌려 사고 이력이 추적 가능하게 남았습니다."
- **개선점(Change):** "충돌 해결 커밋 메시지가 'merge'뿐이라, 무엇을 어떻게 합쳤는지 드러나지 않습니다. '정렬과 완료 흐리기 동시 반영'처럼 적으면 좋겠습니다."
- **질문(Question):** "force push 사고를 relog로 복구했는데, 애초에 --force-with-lease를 썼다면 어떻게 막혔을까요?"

### 피드백 주고받기 예시

실제로 동료의 협업 이력을 보고 피드백을 주는 장면을 그려 봅니다. 동료가 잘못된 머지를 다 음과 같이 처리했다고 합니다.

```
$ git -C dev-bob log --oneline -3
9f8e7d6 (HEAD -> main) 다시 정상화
1a2b3c4 main
8e7d6c5 작업
```

이 이력을 보고 줄 수 있는 좋은 피드백은 다음과 같습니다.

- **좋은 점(Keep):** 사고 후 main을 동작 상태로 되돌리려 한 시도는 좋습니다.
- **개선점(Change):** 커밋 메시지가 "main", "작업", "다시 정상화"처럼 무엇을 왜 했는지 드러나지 않습니다(NFR5 위반). "revert: PR #3 머지 되돌림 (초기화 버그)"처럼 사고와 복구를 명시해야 다음에 추적됩니다.

- **질문(Question):** 공유된 사고 커밋을 reset으로 지운 흔적이 보이는데, revert를 썼다면 동료 이력과 어긋나지 않았을 텐데 어땠나요?

이렇게 구체적인 이력을 짚어 주는 피드백은 막연한 "잘했어요"보다 훨씬 도움이 됩니다. 받는 사람은 정확히 어디를 어떻게 고치면 되는지 알게 됩니다. 피드백을 줄 때는 늘 **무엇을(이력의 어느 부분), 왜(어떤 문제), 어떻게(개선 방향)**의 세 가지를 함께 담는 것이 좋습니다.

## 동료 피드백 표

피드백을 받을 때는 루브릭 기준에 맞춰 정리하면 다음에 무엇을 고칠지 분명해집니다.

| 기준          | 동료가 본 내 수준 | 구체적 피드백                        |
|-------------|------------|--------------------------------|
| 협업 워크플로우 운영 | 우수         | 모든 변경이 PR을 거쳤음                 |
| 사고 복구       | 보통         | revert는 했으나 force push 사고는 미복구 |
| 이력 품질       | 보통         | 커밋 메시지가 일부 모호함                 |

## ch\_18\_s04. 성찰 회고: 명령 ↔ 사고 복구의 연결

이제 가장 중요한 단계, **성찰(reflection)**입니다. 협업 시나리오를 돌리는 동안 여섯 개 단원의 명령이 어느 사고 복구로 이어졌는지를 돌아봅니다. 아래 표는 이 종합 프로젝트가 어떻게 모든 배움을 통합했는지 한눈에 보여 줍니다.

| 단<br>원 | 명령                                       | 협업 시나리오의 어느 사건으로 이어졌나                                 |
|--------|------------------------------------------|-------------------------------------------------------|
| u1     | c03 add-commit, .gitignore               | 초기 To-Do 앱 첫 커밋(M1), 의미 있는 커밋 메시지(NFR5)               |
| u2     | c04 log-diff, c05 branch-switch          | 사고 진단의 log-diff, 기능 브랜치 분기(M2-M3)                     |
| u3     | c06 merge, c07 충돌 해결                     | --no-ff PR 머지, 동시 수정 충돌 해결(M3)                        |
| u4     | c08 revert-reflog, c09 stash             | 잘못된 머지 revert, force push reflog 복구, 핫픽스 stash(M4-M5) |
| u5     | c10 remote-clone, c11 push-pull, c12 tag | 두 클론·원격 동기화, v1.0.0 릴리스(M1-M5)                        |
| u6     | c13 PR-리뷰, c14 rebase                    | PR-코드 리뷰 흐름, rebase -i 커밋 정리(M2-M5)                   |

이 표를 보면 협업 시나리오가 **어느 명령도 빠뜨리지 않고** 통합했음을 알 수 있습니다. 처음 ch\_01에서 git을 설치하던 일이, 마지막에는 두 기여자의 사고를 진단·복구하고 릴리스를 남기는 협업으로 자라났습니다.

## 명령 ↔ 사고 복구 깊이 회고

성찰을 한 걸음 더 깊게 합니다. 단순히 "어디에 쓰였나"를 넘어, 각 명령이 **왜 그 사고에 꼭 필요했는지**를 짚으면 배움이 단단해집니다.

- **c08 revert**는 공유된 사고를 안전하게 되돌리는 유일한 길이었습니다. reset으로 지웠다면 동료 dev-bob의 이력과 어긋났을 텐데, revert는 사고와 복구를 모두 이력에 남겨 추적을 지켰습니다(황금률).
- **c08 reflog**는 force push로 사라진 것처럼 보인 커밋을 되살린 안전망이었습니다. "지워진 게 아니라 보이지 않을 뿐"이라는 사실이, 사고 앞에서 침착할 수 있게 한 자신감의 근거였습니다.
- **c07 충돌 해결**은 협업에서 충돌이 실패가 아니라 정상임을 몸으로 익히게 했습니다. 양쪽 의도를 모두 살리는 편집과 `git diff --check` 검증이 핵심이었습니다.
- **c09 stash**는 "급한 핫픽스 요청"이라는 현실적 상황을 매끄럽게 처리했습니다. 하던 작업을 잃지 않고 잠시 치웠다가 그대로 이어 갈 수 있었습니다.

- **c14 rebase**는 지저분한 wip-오타 커밋을 의미 단위로 정리해 이력 품질을 높였습니다. 단, 본인 전용 브랜치에서만 — 공유 이력엔 손대지 않는다는 황금률을 지켜 가며.
- **c04 status-log-diff**는 모든 사고의 출발점이었습니다. 추측으로 명령을 던지지 않고, 먼저 진단해 무엇이 어떻게 잘못됐는지 읽은 뒤에야 복구 명령을 골랐습니다(NFR3).

이렇게 한 줄씩 회고하면, 각 단원에서 "이걸 언제 쓰지?" 싶었던 명령들이 협업 사고 앞에서 어떤 역할을 했는지가 분명해집니다. 명령은 따로 떨어진 지식이 아니라, 협업이라는 무대에서 서로 맞물리는 도구였습니다.

## 성찰 프롬프트

다음 네 가지 질문에 스스로 답하며 회고합니다.

1. 6개 단원의 어떤 명령이 협업 시나리오의 어느 사고 복구로 이어졌는지 구체적으로 적어 보자.
2. 공유한 이력을 reset/rebase로 덮으면 왜 위험한지, 직접 만든 force push 사고로 설명해 보자.
3. 충돌·사고를 줄이려면 협업 습관(커밋 단위·통합 주기·PR 크기)을 어떻게 바꾸면 좋을까.
4. 다음 단계로 GitHub Actions(CI)를 도입한다면 어디에 어떤 검증을 자동화하고 싶은가.

## 한계와 다음 학습

완성한 협업 시나리오에도 한계가 있습니다. 이를 솔직히 인식하는 것이 성숙한 개발자의 태도입니다.

| 한계                   | 왜 생겼나              | 다음 학습 방향                          |
|----------------------|--------------------|-----------------------------------|
| 머지 전 자동 검증이 없음       | 범위 Out(Actions 제외) | GitHub Actions로 PR마다 테스트·린트 자동 실행 |
| main 보호가 규율(약속)에만 의존 | 조직 권한 설정은 범위 밖     | 브랜치 보호 규칙으로 직접 push·미승인 머지 차단     |
| 두 역할을 혼자 연기          | 결정론적 재현을 우선(ADR1)  | 실제 팀원과 두 계정으로 협업                  |
| 핫픽스도 수동 배포           | CI/CD는 다음 학습       | Actions로 태그 push 시 자동 배포          |



이 한계들은 "실패"가 아니라 **의도적 선택의 결과**입니다. 범위(scope\_out)에서 GitHub Actions를 다음 학습으로 미뤄 두었기에, 지금은 협업의 핵심인 진단·복구에 온전히 집중할 수 있었습니다.

### 다음 학습: GitHub Actions로 자동 검증

이번 프로젝트에서 main은 "직접 push 하지 않는다"는 **약속**으로만 보호됐습니다. 다음 단계인 GitHub Actions(CI)를 도입하면, PR이 올라올 때마다 테스트·린트를 **자동으로** 돌려 통과해야만 머지할 수 있게 강제할 수 있습니다. "사람이 잊지 않도록 구조로 강제"하는 것 — 이것이 협업을 한 단계 더 견고하게 만드는 다음 여정입니다.

## ch\_18\_s05. 작성형 연습(2종 1세트): 성찰 템플릿·자가체크

마지막으로 자신의 협업 시나리오를 직접 평가하고 성찰해 봅니다.

### 모범 예시 — 성찰 한 줄

"force push 사고(M4-B)는 dev-bob이 pull 없이 rebase 후 `git push --force` 한 것이 원인이었다. dev-alice의 `git reflog` 에서 `HEAD@{1}` 의 해시로 잃은 커밋을 찾아 `git switch -c recover/labels <해시>` 로 복구 브랜치를 만들어 되살렸다. 처음엔 커밋이 영영 사라진 줄 알고 당황했지만, reflog가 로컬에 남아 있다는 걸 알고 안심했다. 다시 한다면 애초에 `--force-with-lease` 를 써서 동료 커밋을 덮지 않게 하겠다."

이처럼 성찰은 "어떤 명령으로, 무슨 사고를 겪고, 어떻게 진단·복구했으며, 다음엔 무엇을 바꿀지"를 구체적으로 담을 때 가치가 있습니다. "그냥 잘 됐다", "어려웠다" 같은 막연한 감상은 다음에 도움이 되지 않습니다. 구체적인 명령(`git reflog`), 구체적인 위치(`HEAD@{1}`), 구체적인 예방책(`--force-with-lease`)을 적을수록 성찰은 다음 협업의 진짜 자산이 됩니다.

### 직접 작성 1 — 나의 루브릭 자기 평가 (빈 템플릿)

내 협업 시나리오를 여섯 기준으로 스스로 평가하고, 각 점수의 근거를 한 줄씩 적어 봅니다.

| 기준                     | 내 수준(미흡/보통/우수) | 근거 한 줄 |
|------------------------|----------------|--------|
| 협업 워크플로우 운영            | _____          | —      |
| 충돌 해결                  | _____          | —      |
| 사고 복구(revert·reflog)   | _____          | —      |
| 이력 품질(커밋·rebase·태그)    | _____          | —      |
| 진단 능력(status·log·diff) | _____          | —      |
| 발표·성찰                  | _____          | —      |

- 가중 합계(미흡 1·보통 2·우수 3 × 가중치): \_\_\_\_\_ / 3.0

작성 힌트:

- "우수"라고 적으려면 루브릭 우수 칸의 조건을 실제로 만족해야 합니다(예: 사고 복구 우수 = 잘못된 머지 revert와 force push reflog 복구를 모두 해냄).

## 직접 작성 2 — 명령 ↔ 사고 복구 성찰 (빈 템플릿)

네 가지 성찰 프롬프트에 직접 답하며 회고를 완성해 봅니다.

- 가장 인상 깊었던 사고 복구와 사용한 명령: \_\_\_\_\_
- 공유 이력을 reset/rebase로 덮으면 위험한 이유(내 force push 사고로 설명): \_\_\_\_\_
- 충돌·사고를 줄일 협업 습관(커밋 단위·통합 주기·PR 크기): \_\_\_\_\_
- GitHub Actions(CI)를 도입한다면 어디에 어떤 검증을 자동화하고 싶은가: \_\_\_\_\_
- 다시 협업한다면 가장 먼저 바꾸고 싶은 한 가지: \_\_\_\_\_

작성 힌트:

- "위험한 이유"에는 실제로 만난 사고(force push로 동료 커밋이 덮임)와 그 복구(reflog)를 적으면 생생합니다.

## 자가체크

- [ ] 협업 이력(`git log --graph`)으로 머지된 PR·충돌·revert·릴리스를 시연할 수 있는가
- [ ] 여섯 기준 루브릭으로 내 작업을 미흡/보통/우수로 평가했는가
- [ ] 각 평가에 구체적 근거를 한 줄씩 적었는가

- [ ] 여섯 단원의 명령이 어느 사고 복구로 이어졌는지 표로 정리했는가
- [ ] 공유 이력을 덮으면 안 되는 이유를 내 force push 사고로 설명했는가
- [ ] 프로젝트의 한계를 최소 두 가지 인식하고 다음 학습(Actions·CI)을 제시했는가

## 자주 묻는 질문(FAQ)

### Q. 발표 때 코드를 전부 보여 줘야 하나요?

아니요. 협업 프로젝트의 산출물은 코드보다 **이력**입니다. To-Do 앱 코드는 짧으니 화면을 한 번 보여 주되, 발표의 중심은 `git log --oneline --graph --all`로 협업 사건들을 짚는 것입니다. "여기가 충돌 해결, 여기가 revert 복구, 여기가 v1.0.0"처럼 이력 위에서 사건을 이어 말하는 편이 훨씬 효과적입니다.

### Q. 루브릭에서 "보통"이 나왔는데 실패한 건가요?

전혀 아닙니다. 루브릭은 "어디까지 왔고 다음에 무엇을 더하면 되는지"를 보여 주는 지도입니다. 사고 복구가 보통(한 종류만 복구)이라면, 못 해 본 다른 사고 복구를 연습해 우수로 올리는 것이 다음 목표가 됩니다. 보통은 실패가 아니라 "성장 여지가 보이는 좋은 출발점"입니다.

### Q. 혼자 두 역할로 한 협업이 진짜 협업 경험이 되나요?

핵심 역량은 그대로 길러집니다. 동시 수정 충돌, 잘못된 머지, force push 사고, 핫픽스는 역할이 한 명이든 두 명이든 git 메커니즘이 동일합니다(ADR1). 1인 2역의 장점은 오히려 **결정론적 재현**입니다 — 같은 사고를 원하는 만큼 반복해 복구를 연습할 수 있습니다. 실제 팀 협업에서는 사람 사이 소통이 더해질 뿐, 진단·복구의 토대는 지금 익힌 그대로입니다.

### Q. 성찰에 꼭 한계와 다음 학습을 적어야 하나요?

네, 그것이 성찰의 핵심입니다. 잘된 점만 적으면 다음에 나아질 길이 보이지 않습니다. 한계를 솔직히 인식하고(예: 머지 전 자동 검증이 없음) 다음 방향을 제시하는 것(GitHub Actions로 자동 테스트)이야말로 루브릭의 발표·성찰 항목에서 "우수"를 받는 조건이며, 다음 학습의 지도가 됩니다.

## 마무리: 협업 시뮬레이션을 마치며

축하합니다. 여러분은 git 명령을 하나씩 외우던 입문자에서 시작해, 두 기여자가 하나의 저장소를 함께 개발하며 **충돌을 해결하고, 잘못된 머지를 revert로 되돌리고, 덮인 커밋을 reflog로 되살리고, 핫픽스를 배포하고, v1.0.0을 릴리스하는** 완성된 협업 시나리오를 만들어 냈습니다. 이 과정에서 ch\_01부터 ch\_14까지 배운 명령을 하나도 빠짐없이 통합했습니다.

무엇보다 중요한 것은, 여러분이 **협업의 진짜 실력**을 경험했다는 점입니다. 협업의 실력은 충돌을 안 내는 데 있지 않고, 사고가 났을 때 status·log·reflog로 먼저 진단하고 침착하게 복구하는 데 있습니다. 무엇을 만들지 분석하고(ch\_15), 어떻게 협업할지 설계하며(ch\_16), 다섯 마일스톤으로 증분 구현하고(ch\_17), 평가하고 성찰하는(ch\_18) 이 네 단계는 이 프로젝트뿐 아니라 앞으로 만날 모든 협업에 그대로 쓰입니다.

성찰에서 적은 한계와 다음 학습은 다음 여정의 출발점입니다. 머지 전 자동 검증이 아쉬웠다면 GitHub Actions(CI)를, 더 큰 협업이 궁금하다면 실제 팀과 두 계정으로 — 이렇게 배움은 이어집니다. 하지만 그 모든 것의 토대는 지금 여러분이 손수 일으키고 복구한 이 협업 시나리오에 이미 단단히 놓였습니다. 사고 앞에서 당황하지 않고 "먼저 진단하자"고 말할 수 있는 개발자 — 그것이 이 종합 프로젝트가 여러분에게 남긴 가장 값진 선물입니다.